R para análisis de datos

Aplicación en la comunidad CLIO (Coopera, Lidera, Innova y Orienta)

ZONA CLIO

Coopera • lidera • innova • orienta

R para análisis de datos

Aplicación en la comunidad CLIO (Coopera, Lidera, Innova y Orienta)

Luis Francisco Gomez Lopez

2026-11-12

Tabla de contenidos

Bienvenidos **1**
Uso asistido de inteligencia artificial 1

1. Prefacio **3**

I. Primeros pasos con R **5**

2. Bienvenidos a R **9**
2.1. ¿Qué es R y Positron? 9
2.2. ¿Cómo instalar R y Positron? 10
2.2.1. R 10
2.2.2. Positron 11
2.3. Posit Workbench 12
2.4. ¿Qué es Quarto? 12
2.5. Interfaz de Positron 13
2.5.1. Configuración recomendada para principiantes 15

3. Flujo de trabajo orientado a proyectos **17**
3.1. Organizar tu trabajo por proyectos 17
3.2. Flujo de trabajo por proyectos en Positron para R 19
3.3. ¿Cómo nombrar archivos y carpetas? 19

4. Paquetes de R **23**
4.1. ¿Qué son los paquetes en R? 23
4.2. Instalación de paquetes 24
4.3. Carga de paquetes 25
4.4. Uso de paquetes 26

5. ¿Cómo escribir código en R? **33**
5.1. Operaciones aritméticas 33
5.2. Vectores atómicos en R 34
5.3. Mensajes (messages), advertencias (warnings) y errores (errors) 37
5.3.1. Mensajes 37
5.3.2. Advertencias 39
5.3.3. Errores 40
5.4. Tibbles 40

5.5. Operadores relacionales 43
5.6. Operadores lógicos vectorizados 45

II. Visualización de datos **47**

6. Cantidades **51**
6.1. Gramática de gráficos en capas 51
6.2. Gráfico de barras mediante conteo automático 54
6.2.1. Datos 54
6.2.2. Mapeo 59
6.2.3. Capas 62
6.2.4. Escalas 66

7. Distribuciones **71**
7.1. Histograma 71
7.2. Datos, mapeo y capas 72
7.2.1. Aplicación de colores 75
7.3. Asociación de colores a una variable categórica 77
7.4. Escalas 79
7.5. Tema 83

8. Proporciones **85**
8.1. Gráfico de barras apiladas y normalizadas 85
8.2. Datos, mapeo, capas y escalas 87
8.2.1. Gráfico de barras apiladas 87
8.2.2. Normalización 88
8.2.3. Ordenamiento de etiquetas 89
8.2.4. Modificación de etiquetas 92
8.3. Tema 96

9. Relaciones x-y **99**
9.1. Gráfico de puntos 100
9.1.1. Datos, mapeo y capas 100
9.1.2. Facetas 104
9.1.3. Escalas 106
9.2. Gráfico de líneas 108
9.2.1. Datos, mapeo y capas 108
9.2.2. Descripción de variables 110
9.2.3. Mapeo y capas 111
9.2.4. Capas y escalas 113
9.2.5. Escalas y tema 116

III. Manipulación de datos **123**

10. Verbos de una sola tabla **127**
10.1. Gramática de manipulación de datos en `dplyr` . . . 127
10.1.1. Verbos . . . 128
10.2. Caso de estudio: obtención de datos de interés del Saber-Pro para FAEDIS 146
10.2.1. Datos . . . 147
10.2.2. Filtrar (`filter()`) . . . 150
10.2.3. Seleccionar (`select()`) . . . 151
10.2.4. Reubicar (`relocate()`) . . . 152
10.2.5. Modificar (`mutate()`) . . . 153
10.2.6. Colapsar, agrupar y desagrupar (`summarise()`, `group_by()` y `ungroup()`) . . . 156

11. Verbos de dos tablas **167**
11.1. Diagrama Entidad-Relación . . . 168
11.2. Entidades . . . 168
11.3. Atributos . . . 170
11.3.1. Clave primaria . . . 170
11.3.2. Clave foránea . . . 171
11.4. Relaciones . . . 172
11.4.1. Notación Crow's Foot . . . 173
11.5. Verbos . . . 174
11.5.1. Filtering joins . . . 175
11.6. Mutating joins . . . 179
11.6.1. Inner joins . . . 180
11.6.2. Left joins . . . 181
11.6.3. Right joins . . . 183
11.6.4. Full joins . . . 183
11.7. Caso de estudio: consolidación de resultados Saber-Pro para FAEDIS . . . 185
11.7.1. Descripción de variables . . . 186
11.7.2. Ajuste de datos . . . 188
11.7.3. Consolidación de tablas . . . 191

IV. Importación de datos **197**

12. Serialización de datos en R **201**
12.1. El formato `.rds` . . . 202
12.2. Ventajas y desventajas de utilizar archivos `.rds` . . . 202
12.3. Importación de datos en formato `.rds` . . . 202
12.4. Exportación de datos en formato `.rds` . . . 205

13. Texto plano **207**
13.1. Archivos de valores separados por comas (`.csv`) . . . 207
13.1.1. Parámetros comunes al importar datos . . . 209
13.2. Otros formatos de archivos de texto plano . . . 213
13.3. Control de tipos de columnas y resolución de problemas . . . 216
13.3.1. Especificación explícita de tipos con `col_types` . . . 216
13.3.2. Diagnóstico de errores con `problems()` . . . 217
13.4. Exportación de archivos de texto plano . . . 219
13.5. Caso de estudio: importación de datos del Saber-Pro para el año 2021 en Colombia . . . 220
13.5.1. Importación inicial e inspección de tipos . . . 220
13.5.2. Naturaleza de la variable y formato de fecha . . . 223
13.5.3. Intento inicial con `col_date()` y diagnóstico con `problems()` . . . 224
13.5.4. Solución con el parámetro `format` en `col_date()` . . . 225
13.5.5. Especificaciones comunes de formato para fechas en `readr` . . . 227

14. Hojas de cálculo **229**
14.1. Archivos con extensión `.xls` y `.xlsx` . . . 229
14.2. Importación de datos en formato `.xlsx` . . . 230
14.3. Estructura de archivos con extensión `.xlsx` . . . 231
14.4. Importar datos de una hoja de trabajo . . . 232
14.5. Importar datos de parte de una hoja de trabajo . . . 235

15. GeoPackage (`.gpkg`) **237**
15.1. Mapa coroplético . . . 238
15.2. El formato `.gpkg` . . . 238
15.3. Importación de datos en formato `.gpkg` . . . 240
15.3.1. Estructura de un objeto `sf` . . . 241
15.3.2. Importación de variables no espaciales y manipulación de datos . . . 243
15.4. Visualización de datos geoespaciales . . . 248
15.4.1. Datos, mapeo y capas . . . 248
15.4.2. Escalas . . . 250
15.4.3. Tema . . . 257

V. Organización y limpieza de datos **261**

16. Datos ordenados **265**
16.1. Definición de datos ordenados . . . 265
16.1.1. Estructura de datos . . . 265
16.1.2. Semántica de datos . . . 266
16.1.3. Características de datos ordenados . . . 267
16.2. Problemas más comunes con datos no ordenados . . . 268
16.2.1. Nombres de columnas que son valores, no variables . . . 268

16.2.2. Múltiples variables en una sola columna . . . 271
16.2.3. Variables almacenadas tanto en filas como en columnas . . . 273
16.2.4. Diferentes unidades observacionales en una sola tabla . . . 275
16.2.5. Una misma unidad observacional almacenada en múltiples tablas . 276

17. Pivoteo **277**
17.1. Pirámides poblacionales . . . 278
17.2. Formato largo . . . 278
17.3. Formato ancho . . . 280
17.4. Caso de estudio: estructurar datos en formato largo para generar pirámides poblacionales de Colombia . . . 281
17.4.1. Datos . . . 281
17.4.2. Limpieza de nombres . . . 283
17.4.3. Filtrar categoría de interés . . . 284
17.4.4. Seleccionar variables de interés . . . 285
17.4.5. Ordenar datos en formato largo . . . 286
17.4.6. Exportar datos en formato largo . . . 288

18. Separación y unión de caracteres en columnas **289**
18.1. Separar caracteres en columnas . . . 290
18.2. Unir múltiples columnas mediante concatenación de caracteres . . . 291
18.3. Caso de estudio: separar caracteres en columnas, transformar y preparar datos para generar pirámides poblacionales de Colombia . . . 292
18.3.1. Separar caracteres en columnas . . . 293
18.3.2. Transformar variables . . . 294
18.3.3. Preparar datos para visualizarlos . . . 295
18.4. Caso de estudio: visualizar datos a través de pirámides poblacionales de Colombia . . . 297
18.4.1. Creación de variables auxiliares para selección de años de interés . 297
18.4.2. Creación de pirámides poblacionales . . . 299

VI. Programación funcional **311**

19. Funciones anónimas y `purrr` **315**
19.1. `purrr` . . . 315
19.2. Vectores . . . 316
19.2.1. Vectores atómicos . . . 316
19.2.2. Listas . . . 317
19.3. Funciones . . . 319
19.4. Funciones anónimas . . . 323
19.5. `map()` . . . 324
19.6. `reduce()` . . . 327
19.7. `list_rbind()` . . . 329

20. Funciones `map()`, `reduce()` y `list_rbind()` **331**
20.1. Importación . . . 332
20.2. Unión por filas . . . 337
20.3. Separación de caracteres en columnas . . . 338
20.4. Corrección códigos departamentos y municipios . . . 339
20.5. Caso de estudio: consolidación de resultados Saber Pro para UMNG . . . 342
20.5.1. Limpiar y filtrar . . . 343
20.5.2. Transformar . . . 344
20.5.3. Exportar . . . 344

VII. Comunicación con Quarto **349**

Referencias **353**

Bienvenidos

Este es el sitio web de **R para análisis de datos** aplicado para la comunidad CLIO (Coopera, Lidera, Innova y Orienta). El libro tiene como objetivo introducir a los participantes de la comunidad de aprendizaje CLIO al uso del lenguaje de programación R a través del entorno integrado de desarrollo Positron para realizar análisis de datos y comunicar los resultados utilizando Quarto.

A través del libro darás los primeros pasos en el uso de R y aprenderás a visualizar, manipular, organizar, limpiar e importar datos así como programar en este lenguaje y comunicar resultados en Quarto.

El acceso a este sitio web es libre y siempre será libre, bajo los términos de la licencia CC BY-NC 4.0. Las imágenes, figuras, diagramas e íconos dentro del libro se generaron utilizando Gemini, el paquete `ggplot2`, fueron creados con Inkscape o Lucidchart, provienen de Fontawesome o son de dominio público. En particular, la portada del libro corresponde a la pintura del artista Christian Bernhard Rode (1725 - 1797) titulada Die Muse Klio, una obra que personifica a Clío, la musa de la historia en la mitología griega.

Uso asistido de inteligencia artificial

Con base en los lineamientos para el uso de inteligencia artificial generativa en la educación superior (IA-Uniandes, 2024), la elaboración y desarrollo técnico de este libro contó con la asistencia de Antigravity CLI. En particular, se utilizó en los siguientes ámbitos:

- **Desarrollo de ideas y estructuración**: Apoyo en la generación de ideas, planificación y diseño de la estructura pedagógica de los capítulos.
- **Exploración conceptual y argumentativa**: Sugerencia de explicaciones alternativas, revisiones conceptuales y refinamiento de la narrativa técnica.
- **Asistencia en programación y código reproducible**: Apoyo en la depuración, verificación y optimización de ejemplos de código en R y configuraciones técnicas de Quarto.
- **Revisión y edición de estilo**: Asistencia en la claridad de redacción, consistencia terminológica y corrección ortotipográfica.

En concordancia con los principios éticos e institucionales de autoría y transparencia académica, el autor asume la plena y exclusiva responsabilidad por el contenido, la veracidad de los análisis, los códigos y los materiales presentados en este libro.

1. Prefacio

El presente libro tiene como objetivo introducir a la comunidad CLIO (Coopera, Lidera, Innova y Orienta) y en general a los miembros de la Universidad Militar Nueva Granada al lenguaje de programación R, el entorno integrado de desarrollo Positron y el sistema de publicación científica y técnica de código abierto Quarto, para realizar análisis de datos.

El análisis de datos se enmarca dentro del concepto de *Ciencia de datos* (ver Figura 1.1) donde se requiere un conjunto de habilidades que abarcan tener un grado de habilidades en programación, conocimientos en matemáticas y estadística así como una experticia en un determinado campo. El libro adopta una visión pragmática donde el enfoque es el desarrollo de habilidades en programación utilizando microdatos anonimizados y abiertos de Data Icfes relacionados con los resultados del examen Saber Pro del grupo de competencias genéricas para programas de pregrado así como proyecciones y retroproyecciones de población a nivel nacional para Colombia elaboradas con base en los resultados del Censo Nacional de Población y Vivienda (CNPV) 2018 y actualizadas con información posterior a la pandemia de COVID-19 por parte del Departamento Administrativo Nacional de Estadística (DANE).

El uso de este conjunto de datos se fundamenta en 2 aspectos. En primera instancia, utilizar datos abiertos de tal manera que sean de fácil acceso. Segundo, emplear información relacionada con la Universidad Militar Nueva Granada y en particular con fenómenos que afectan el sector de educación como lo es la transición demográfica y el envejecimiento poblacional en Colombia. Debido a la heterogeneidad respecto a los campos de experticia que puede tener la comunidad CLIO (Coopera, Lidera, Innova y Orienta) y en general los miembros de la Universidad Militar Nueva Granada se buscó aprovechar el uso de datos relacionados con la institución educativa o que afectaran de alguna manera al sector de educación superior.

El libro se construyó inicialmente utilizando las mismas herramientas que se buscan introducir (R, Positron y Quarto) más el uso de Git y GitHub[1] donde se estructuró siguiendo las recomendaciones señaladas en (Halmos, 1973) respecto a escribir cada parte de forma iterativa o en espiral así como en elaborar el contenido de acuerdo a lo indicado en (Anderson & Benedetti, 2009). Además, el código fuente del libro se encuentra disponible en un repositorio de GitHub de libre acceso donde estará en constante cambio

[1] El uso de Git y GitHub para análisis de datos no se abarca en este libro pero el lector interesado puede más adelante, cuando tenga cierta familiaridad con R, Positron y Quarto, consultar (Bryan & Hester, 2025).

Figura 1.1.: El diagrama de Venn de la ciencia de datos (Basado en (Conway, 2010) y (H2O.ai, 2015))

dependiendo de las necesidades que surjan y las sugerencias que se reciban por parte de la comunidad CLIO (Coopera, Lidera, Innova y Orienta) así como en general de los miembros de la Universidad Militar Nueva Granada.

Luis Francisco Gómez López

Parte I.

Primeros pasos con R

El objetivo de esta parte, es presentar los conceptos básicos para el análisis de datos en R a través de los siguientes 4 capítulos:

- Capítulo 2: introduce las herramientas fundamentales para el análisis de datos, definiendo la naturaleza de R y el rol de componentes adicionales tales como Positron, Posit Workbench y Quarto.
- Capítulo 3: introduce los elementos básicos para adoptar un flujo de trabajo por proyectos, así como recomendaciones para mantener el espacio de trabajo debidamente ordenado.
- Capítulo 4: introduce el concepto de paquetes así como el proceso para instalarlos y cargarlos para poder utilizarlos.
- Capítulo 5: introduce los conceptos básicos del lenguaje de programación R ilustrando estos aspectos a través de un conjunto de datos.

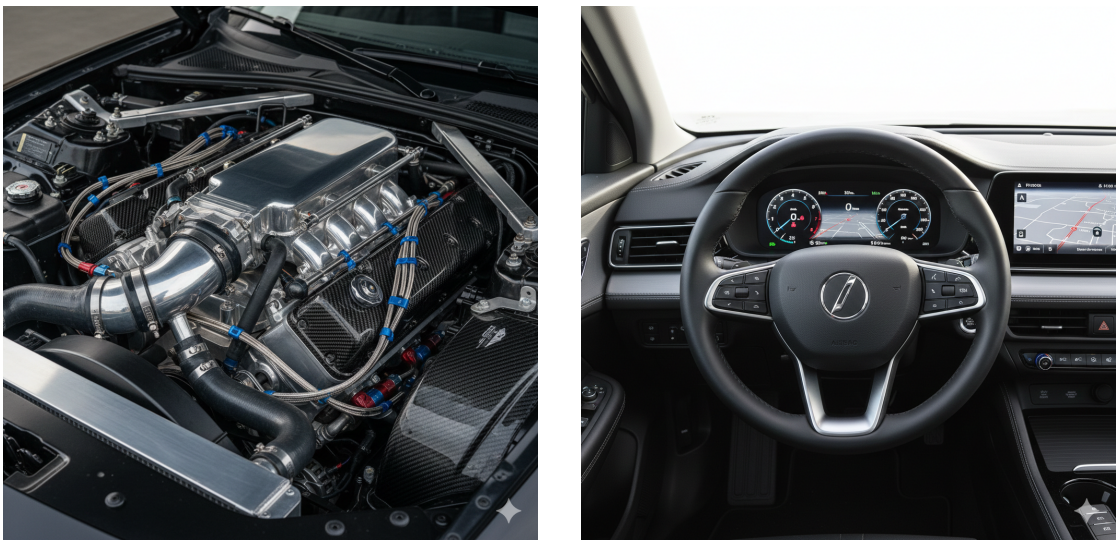
2. Bienvenidos a R

Este capítulo se basa en (Ismay, 2025, Capítulo 1) y tiene como propósito introducir las herramientas fundamentales para el análisis de datos con R. Exploraremos los siguientes aspectos:

1. ¿Qué es R y Positron?
2. ¿Cómo instalar R y Positron o utilizar Posit Workbench?
3. ¿Qué es Quarto?
4. Interfaz de Positron

2.1. ¿Qué es R y Positron?

En términos simples R es un entorno gratuito para realizar análisis estadísticos y crear gráficos. A lo largo del libro utilizaremos R a través de Positron. R se puede entender como el motor de un automóvil y Positron el tablero de control del automóvil que queremos manejar (ver Figura 2.1).



(a) R: Motor

(b) Positron: Tablero de control

Figura 2.1.: Analogía para entender la diferencia entre R y Positron

Si abandonamos la analogía anterior y somos más específicos, R es un lenguaje de programación y Positron es un *Entorno de Desarrollo Integrado (IDE)* que incluye un conjunto de elementos en su interfaz para utilizar de manera mucho más fácil R cuando queremos llevar a cabo análisis estadísticos y explorar datos.

2.2. ¿Cómo instalar R y Positron?

> **!** Importante 4
>
> Si formas parte de una institución educativa, verifica si tienes acceso a **Positron Pro** a través de **Posit Workbench** (ver Sección 2.3). En caso de contar con este acceso, no es necesario realizar una instalación local. Por lo tanto, puedes saltarte los pasos de la Sección 2.2.1 y la Sección 2.2.2, ya que tendrás acceso directo a estas herramientas.
> Sin embargo, una vez que te sientas familiarizado con estas herramientas, consulta esas secciones para instalar R y Positron en tu propio computador personal.

2.2.1. R

Para utilizar R con Positron es necesario primero tener instalada la versión 4.2 de R o una superior. Para instalar R, puedes seguir las instrucciones que aplican para tu sistema operativo en la página de descarga oficial de CRAN.

2.2.1.1. Windows

Selecciona el enlace **Download R for Windows**, haz clic en **base** y, finalmente, pulsa en el enlace **Download R X.X.X for Windows**. Ten en cuenta que **X.X.X** representa la versión más reciente disponible al momento de la descarga. Una vez que el archivo `.exe` se haya descargado, haz doble clic en él para ejecutarlo, esto abrirá un asistente con instrucciones en pantalla que te guiará paso a paso hasta completar la instalación.

2.2.1.2. Mac

Selecciona el enlace **Download R for macOS** y dirígete a la sección **Latest release**. Haz clic en el paquete adecuado para la arquitectura de tu Mac (por ejemplo, **R-X.X.X-arm64.pkg** para chips Apple M1/M2/M3, o **R-X.X.X-x86_64.pkg** para procesadores Intel) para descargar el archivo `.pkg`. Ten en cuenta que **X.X.X** representa la versión más reciente disponible al momento de la descarga. Una vez que el archivo se haya descargado, haz doble clic en él para ejecutarlo, esto abrirá un asistente con instrucciones en pantalla que te guiará paso a paso hasta completar la instalación.

2.2.1.3. Linux

Selecciona el enlace **Download R for Linux** y elige la distribución que utilizas (como Ubuntu, Debian, Fedora o Red Hat). A diferencia de otros sistemas operativos, la instalación en Linux se realiza generalmente a través del terminal siguiendo las instrucciones específicas de cada página.

2.2.1.4. Verificando la instalación de R

Una vez instales R es importante que verifiques la instalación. La forma más fácil de hacerlo es dar doble click en el ícono de R (ver Figura 2.2) para el caso de Windows o Mac o en Linux abrir el terminal y digita `R` en la línea de comandos.

Figura 2.2.: Ícono de R

Si observas que se abre una ventana con la consola de R (en los casos de Windows y Mac) o que en el terminal aparece el texto de bienvenida de la versión 4.2 o superior (en el caso de Linux), la instalación ha sido exitosa. Para finalizar la verificación, simplemente cierra la ventana de la aplicación en Windows/Mac, o en el caso de Linux, escribe el comando `q()` y presiona la tecla `Return` (mac), `Enter` (linux), `Enter` (windows) para salir de la sesión de R en el terminal.

2.2.2. Positron

Para utilizar R con Positron, es necesario que primero revises los requisitos de instalación para asegurarte que el programa funcione bien en tu computador. Luego, dirígete a la página de descarga oficial de Positron, acepta el contrato de licencia, descarga el instalador que corresponda a tu sistema operativo (Windows, Mac o Linux) y completa la instalación.

2.3. Posit Workbench

> **!** Importante 2
>
> Si realizaste una instalación local de acuerdo a los pasos descritos en la Sección 2.2.1 y la Sección 2.2.2 puedes saltarte esta sección.

Posit Workbench es un entorno de desarrollo basado en la nube diseñado para ciencia de datos que puede ser utilizado por las instituciones educativas para enseñanza. Posit Workbench reúne en un solo lugar las herramientas de trabajo más populares para ciencia de datos incluido Positron, ofreciendo un entorno seguro y listo para usar.

Para utilizar esta herramienta, es indispensable contar con una cuenta activa que incluya un usuario y una contraseña proporcionados por tu institución. Una vez que tengas tus credenciales, debes ingresar a la dirección web asignada por tu institución educativa para acceder a la plataforma. Por ejemplo, en la Universidad Militar Nueva Granada (UMNG), el acceso se realiza a través del portal de ingreso de Posit Workbench de la UMNG, donde podrás visualizar la interfaz típica para iniciar sesión y comenzar a trabajar.

Antes de comenzar a utilizar R con Positron a través de este entorno, te recomendamos revisar los requisitos del sistema. Esta verificación previa te permitirá asegurar que tu computador cumpla con las especificaciones necesarias para tener una experiencia adecuada.

2.4. ¿Qué es Quarto?

Quarto es un sistema de publicación técnica y científica de código abierto que combina código, resultados y narrativa para generar documentos reproducibles en diferentes formatos (p. ej., HTML, PDF, MS Word), presentaciones, sitios web y proyectos como libros o manuscritos.

Al utilizar Positron, no es necesario instalar Quarto por separado, ya que se integra automáticamente de dos formas:

1. **Como *"motor"* de procesamiento (Interfaz de línea de comandos)**: es la herramienta interna que transforma tus archivos de texto y código en los reportes y documentos finales.
2. **Como asistente de edición (Extensión denominada quarto-vscode)**: proporciona las herramientas visuales para trabajar cómodamente, incluyendo el autocompletado de comandos, el resaltado de colores, la ejecución interactiva de código y una vista previa en tiempo real de tus resultados.

> **Nota 1**
>
> Más adelante profundizaremos en el uso de Quarto utilizando R y Positron. Por ahora te recomendamos familiarizarte primero con la interfaz de Positron (ver Sección 2.5) y R. No obstante, si tienes experiencia previa, puedes consultar la Guía de inicio rápido de Quarto con Positron en la documentación oficial.

2.5. Interfaz de Positron

Si realizaste una instalación local siguiendo los pasos de la Sección 2.2.1 y la Sección 2.2.2 podrás abrir 2 programas. Sin embargo, a lo largo del libro siempre utilizaremos Positron y no la aplicación de R directamente. En la Figura 2.3 se indica que ícono debes dar doble click o que programa abrir en tu computador.

(a) R: **No** dar click sobre este ícono

(b) Positron: Dar click sobre este ícono

Figura 2.3.: Íconos de R y Positron en tu computador

En caso que tengas acceso a Posit Workbench, de acuerdo a lo señalado en la Sección 2.3, sigue los pasos descritos en Primeros pasos con Positron Pro para abrir de manera directa Positron.

Al utilizar Positron, encontrarás la interfaz organizada como se muestra en la Figura 2.4, donde se resalta cada sección de la interfaz. No te preocupes si encuentras pequeñas diferencias o un entorno visual con diferente color dado que el equipo de desarrollo de Positron está constantemente actualizando la aplicación o puede que tengas una configuración por defecto diferente en cuanto al entorno visual de la interfaz.

> **Nota 2**
>
> Si abres Positron por primera vez, verás una pantalla de bienvenida en lugar del **Editor** (ver Figura 2.4). Esto es normal, ya que este espacio se activará automáticamente una vez que abras tu primer archivo de R, un proceso que se señalará más adelante.

1. **Barra de actividades**: Es tu menú principal. Funciona como un selector de herramientas mediante diferentes iconos. Al hacer clic en cada icono, eliges qué contenido deseas ver en la columna de al lado, permitiéndote cambiar rápidamente entre tus archivos, el buscador, el asistente o cualquier otra funcionalidad que se añada al instalar nuevas extensiones.

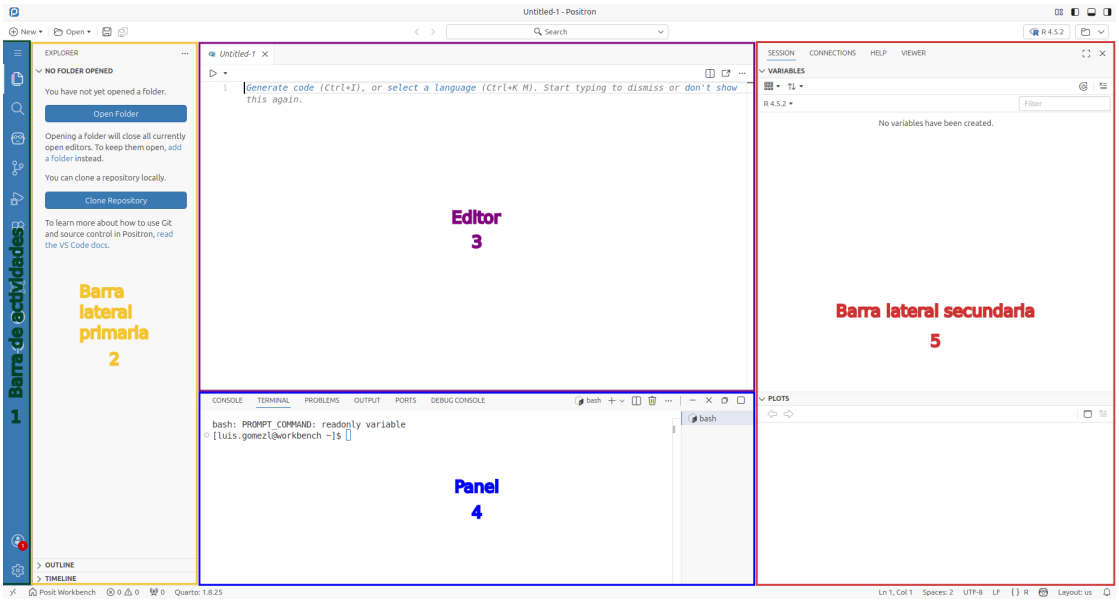


Figura 2.4.: Interfaz de Positron resaltando cada sección

2. **Barra lateral primaria**: Es tu ventana de navegación principal y su contenido cambia según el icono que hayas pulsado en la **Barra de actividades**. Por ejemplo, al seleccionar el icono de archivos , este espacio te permitirá abrir una carpeta desde tu computador para visualizar y gestionar todos los documentos y datos con los que vas a trabajar.

3. **Editor**: Es el área central donde abres, escribes y modificas tus archivos. Aquí es donde principalmente redactarás instrucciones en el lenguaje R, permitiéndote crear y organizar el código necesario para tus análisis antes de ejecutarlo.

4. **Panel**: Es el espacio donde se encuentra la consola, que actúa como la vía de comunicación directa con el lenguaje R. Aquí es donde el *"motor"* procesa las instrucciones que envías desde el editor tras ejecutar tu código.

5. **Barra lateral secundaria**: Es tu panel de apoyo y consulta. En este espacio puedes examinar la lista de variables que has creado, visualizar los gráficos generados y acceder a los manuales de ayuda. También sirve para mostrar visualizaciones interactivas o conexiones externas que complementan tu trabajo sin interferir con el código que estás escribiendo.

> **Tip 1**
>
> Positron dispone de una guía oficial que detalla cada aspecto de la aplicación. Sin embargo, dominar una aplicación es un proceso gradual que se logra con la

práctica constante. En esta etapa inicial, no es necesario conocer todas sus funciones. Conforme ganes confianza con la interfaz y según lo requieran tus análisis, podrás consultarla para profundizar en aspectos específicos que requieras.

2.5.1. Configuración recomendada para principiantes

Existen diferentes recomendaciones para principiantes sobre cómo configurar Positron. Para personalizar tu entorno, debes modificar el archivo de configuración de usuario `settings.json`. Para abrirlo, utiliza la combinación de teclas `Command-Shift-P` (mac), `Ctrl-Shift-P` (linux), `Ctrl-Shift-P` (windows) dentro de Positron para desplegar la paleta de comandos y escribe: **Preferences: Open User Settings (JSON)**. Luego podrás seleccionar esta opción para abrir el archivo `settings.json`. En este libro, te recomendamos añadir y guardar las siguientes opciones dentro de este archivo:

```
{
    "workbench.colorTheme": "Default Positron Dark",
    "[r]": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "Posit.air-vscode"
    },
    "[quarto]": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "quarto.quarto"
    },
    "positron.r.defaultRepositories": "posit-ppm"
}
```

Con estos ajustes lograrás 3 objetivos:

1. **Tema de color:** aplicar la interfaz oscura `Default Positron Dark` para reducir la fatiga visual.
2. **Autoformateo al guardar:** aplicar automáticamente un estilo visual uniforme a tus archivos de R y Quarto utilizando las extensiones Air y quarto-vscode. Esto permite que el código se reorganice solo, manteniendo una estructura limpia y profesional sin aplicar un esfuerzo manual.
3. **Instalación rápida de paquetes**: configurar Posit Public Package Manager (PPM) como el repositorio por defecto. Esto garantiza que, al instalar paquetes de R, se descarguen versiones binarias en lugar de compilar el código fuente desde cero. Como resultado, las instalaciones serán drásticamente más rápidas y estables.

> **!** Importante 3
>
> Una vez que hayas guardado los cambios en tu archivo `settings.json`, es necesario reiniciar Positron (cerrar y volver a abrir la aplicación). Esto asegura que se activen de inmediato todas las configuraciones recomendadas.

En caso que hayas seguido las instrucciones descritas en la Sección 2.3 es posible que el administrador de la herramienta previamente haya fijado por defecto estos aspectos. En caso contrario, puedes agregar esta configuración.

3. Flujo de trabajo orientado a proyectos

Este capítulo se basa en (Bryan et al., 2025, Capítulos 3, 5) y tiene como propósito introducir los elementos básicos para trabajar por proyectos en Positron para R. Exploraremos los siguientes aspectos:

1. ¿Qué significa organizar tu trabajo por proyectos?
2. ¿Cómo adoptar un flujo de trabajo por proyectos en Positron para R?
3. ¿Qué recomendaciones adoptar para nombrar archivos y carpetas para mantener un proyecto organizado?

3.1. Organizar tu trabajo por proyectos

Imagina que eres un carpintero profesional y quieres construir una silla y una cama. Tienes 2 formas de trabajar (ver Figura 3.1):

1. El taller del caos

Trabajas con ambos proyectos mezclados sobre la misma mesa de trabajo. Las piezas de la silla están amontonadas sobre las de la cama y los tornillos están revueltos. Para avanzar en la silla, primero debes apartar lo que te estorba de la cama. Si decides mudarte de taller, tienes que identificar y recoger cada pieza una por una, con el riesgo de olvidar algo o llevarte una parte que no corresponde.

2. El taller por proyectos

Utilizas una caja independiente para cada mueble. Sobre la mesa de trabajo solo pones la caja de la silla o la de la cama, según lo que necesites. Cada caja contiene únicamente las piezas y los planos de ese proyecto. Si decides trabajar en la silla, la mesa queda libre de cualquier parte de la cama. Si tienes que mudarte de taller, simplemente cierras la caja y te la llevas: cada proyecto es autónomo y esta listo para continuar en cualquier otro lugar.

Cuando utilices R con Positron para realizar un análisis de datos, es importante adoptar un flujo de trabajo orientado a proyectos. Esto significa:

- **Establecer un orden y estructura**: consiste en que coloques todos los archivos relacionados con un mismo proyecto en una carpeta específica. Si el proyecto crece en complejidad, utiliza subcarpetas para mantener la claridad. Asimismo, adopta un sistema coherente para nombrar tanto tus carpetas como los archivos.

(a) El taller del caos: sin un flujo de trabajo definido

(b) El taller por proyectos: flujo de trabajo orientado a proyectos

Figura 3.1.: Analogía para entender la importancia de adoptar un flujo de trabajo orientado a proyectos

- **Saber dónde estás parado**: Al trabajar en un proyecto, Positron debe estar situado exactamente en la carpeta específica del proyecto. Esto se conoce como directorio de trabajo. Si Positron intenta buscar tus archivos en el lugar equivocado, no podrá abrirlos. Lo ideal es que Positron se sitúe en la carpeta correcta al abrir tu proyecto.
- **Adoptar una disciplina de rutas**: todas las rutas de tus archivos deben ser en lo posible relativas. Esto significa que, como ya sabes dónde estás parado, solo necesitas indicar el camino desde la carpeta del proyecto hacia el archivo que quieres usar. De esta forma, evitas escribir rutas largas y fijas que solo funcionan en tu computador.

Estas prácticas permiten que el proyecto sea portátil y comprensible. Al implementarlas, podrás mover la carpeta del proyecto de un lugar a otro en tu computador, e incluso enviarla a otras personas y que el proyecto simplemente "funcione" (Bryan, 2017). Además, permites que otras personas puedan explorar el proyecto y entender la estructura de tu trabajo.

> **Nota 3**
>
> En la Sección 3.2 abordaremos de manera práctica en Positron cómo se materializan los aspectos mencionados en esta sección. Si algunos elementos no se entienden a profundidad con la generación de un proyecto de R en Positron quedarán más claros

estos aspectos.

3.2. Flujo de trabajo por proyectos en Positron para R

En Positron, un proyecto es un conjunto de archivos que se encuentran en una carpeta específica así como un conjunto de configuraciones que se aplican por defecto para realizar una tarea determinada. Al abrir esta carpeta, Positron asume automáticamente que todo lo que necesitas buscar o guardar se encuentra dentro de ella. Organizar tu trabajo de esta manera permite la reproducibilidad, eficiencia, automatización y colaboración con otras personas.

Para generar un proyecto de R en Positron, utiliza la combinación de teclas `Command-Shift-P` (mac), `Ctrl-Shift-P` (linux), `Ctrl-Shift-P` (windows) para desplegar la paleta de comandos y escribe: **Workspaces: New Folder from Template...**. Luego, podrás presionar esta opción y elegir a continuación **R project** para realizar las configuraciones respectivas de tu proyecto.

Primero, deberás definir el nombre de la carpeta donde estará tu proyecto y la ubicación dentro de tu computador o equipo. En el caso del nombre, para que sigas el contenido del libro te recomendamos utilizar `r-analisis-datos`.

> **Nota 4**
>
> En la Sección 3.3 abarcaremos con más detalle la forma recomendada de nombrar archivos y carpetas adoptando un conjunto de principios, de modo que puedas mantener tu proyecto organizado.

Segundo, aunque es posible aplicar ajustes avanzados, te sugerimos mantener la configuración inicial lo más simple posible. Para el desarrollo de este libro, solo es necesario que selecciones la versión de R que utilizarás. Otras opciones como **Initialize Git repository** o **Use `renv` to create a reproducible environment** no las abarcaremos.

> **Nota 5**
>
> Si deseas profundizar en estas opciones más adelante, cuando estés familiarizado con R y Positron, puedes consultar (Bryan & Hester, 2025) y la documentación oficial de `renv`.

3.3. ¿Cómo nombrar archivos y carpetas?

Existen 3 principios para nombrar archivos y carpetas (Bryan et al., 2025, Capítulo 5):

1. **Legible por máquinas:** los nombres deben estar optimizados para el procesamiento automatizado, basándose en 2 pilares.

   - **Compatibilidad con búsquedas y automatización**: un nombre debe prescindir de espacios en blanco, signos de puntuación (p. ej., `.`, `,`, `;`, `:`)[1] y caracteres acentuados (p. ej., `á`, `ñ`, `ü`) así como usar preferiblemente minúsculas.

   - **Facilidad de cómputo**: es necesario utilizar de manera deliberada delimitadores como `_` o `-`, facilitando que el sistema operativo o R fragmente el nombre y recupere datos automáticamente.

2. **Legible por humanos:** un nombre debe permitir deducir el contenido de un archivo o carpeta sin tener que abrirlo, basándose en 2 pilares.

   - **Nombres con carga informativa**: el nombre debe funcionar como un resumen del contenido mediante el uso de palabras claves.

   - **Semántica del slug**: este concepto proviene de las URLs semánticas, que reemplazan códigos ilegibles por palabras que explican la jerarquía y el tema de una página (p. ej., `dominio.com/cursos/r-basico` en lugar de `dominio.com/?id=123`). Al aplicar esta lógica a tus archivos, transformas un nombre genérico en una ruta con significado (p. ej., `001_limpieza-datos.R` en lugar de `documento_v1.R` donde el primero comunica qué contiene y en qué etapa del proyecto se encuentra).

3. **Compatible con el ordenamiento por defecto:** un nombre debe permitir que aparezca en la secuencia lógica deseada de forma automática, basándose en 3 pilares.

   - **Priorizar el uso de números al inicio:** colocar un identificador numérico al comienzo del nombre garantiza que los archivos o carpetas se listen siguiendo una jerarquía del proyecto, en lugar de un orden alfabético aleatorio.

   - **Utilizar el estándar ISO 8601** para fechas: si el nombre incluye fechas utilizar el formato **AAAA-MM-DD** (Año-Mes-Día).

   - **Rellenar números con ceros a la izquierda:** para mantener el orden se deben usar ceros (p. ej., `000`, `001`, `002`, … , `010`, …) en lugar de números simples (`0`, `1`, `2`, … , `10`, …). Sin los ceros iniciales, un sistema operativo situará el `10` inmediatamente después del `1`, rompiendo la secuencia lógica.

[1] El punto que precede a la extensión de un archivo (p. ej., `.html`, `.pdf`, `.docx`), bajo este contexto, se excluye en esta indicación debido a que un sistema operativo requiere identificar el programa por defecto para abrirlo.

Tip 4

¿Cuándo usar guión bajo (`_`) vs guión medio (`-`) para nombrar archivos y carpetas? Para que tus archivos sean fáciles de fragmentar por R o el sistema operativo, utiliza esta regla sencilla:

- **Guion bajo (`_`) para separar "metadatos"**: Úsalo para dividir bloques de información distinta, como el número de orden del tema.
- **Guion medio (`-`) para separar palabras**: Úsalo para unir palabras que pertenecen a un mismo concepto o sección.

Por ejemplo, en `001_limpieza-datos.R` el `_` separa el orden (`001`) del tema (`limpieza-datos`). El `-` simplemente une las palabras `limpieza` y `datos` para que se lean como una sola idea.

4. Paquetes de R

Este capítulo se basa en (Ismay, 2025, Capítulo 1) y tiene como propósito introducir el concepto de paquetes de R. Exploraremos los siguientes aspectos:

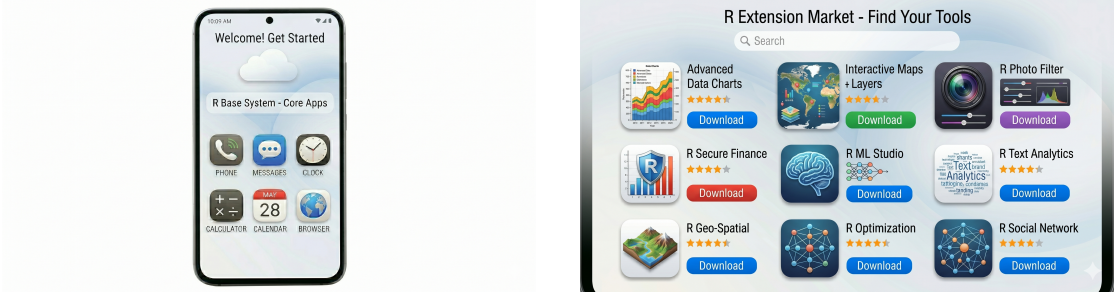
1. ¿Qué son los paquetes en R?
2. ¿Cómo instalar paquetes?
3. ¿Cómo cargar paquetes?
4. Una demostración de cómo utilizar un paquete

4.1. ¿Qué son los paquetes en R?

En R un paquete, en términos simples, es una carpeta organizada de cierta forma que alguien más creó que te brinda herramientas listas para usar con el objetivo de realizar una tarea específica o extender la funcionalidad base de R. De hecho, cuando instalas R por primera vez, este ya viene con un grupo de paquetes estándar (o de base) por defecto que le permiten funcionar y realizar ciertas tareas. Independientemente de si un paquete está incluido en R por defecto o tiene como propósito extender su funcionalidad, su estructura siempre consta de 3 elementos (Wickham & Bryan, 2023):

- **Funciones**: son instrucciones predefinidas que reciben una información, la procesan y entregan un resultado para realizar una tarea específica de manera automática.
- **Documentación**: es la guía que explica qué hace cada función, qué requiere para funcionar y qué resultado genera.
- **Datos de muestra**: son conjuntos de datos para ilustrar el funcionamiento de las funciones.

Para entender cómo funcionan los paquetes en R se puede utilizar una analogía con las aplicaciones de un celular (Ismay, 2025, Capítulo 1). Cuando compras un celular nuevo, este viene por defecto con aplicaciones para que se pueda utilizar. Sin embargo, de acuerdo a las necesidades de cada usuario se instalan otras aplicaciones adicionales para extender la funcionalidad del celular de tal manera que se cubran los requerimientos que cada individuo necesita (ver Figura 5.1).



(a) Paquetes por defecto en R: Celular nuevo

(b) Paquetes que extienden la funcionalidad de R: Tienda de aplicaciones

Figura 4.1.: Analogía para entender la diferencia entre paquetes que vienen incorporados por defecto en R y aquellos que se pueden instalar para extender su funcionalidad base.

4.2. Instalación de paquetes

Para instalar un paquete que no viene por defecto en R, retomaremos la analogía de la Sección 4.1. En un celular, primero es necesario descargar la aplicación de una tienda una sola vez. Luego, basta con abrirla cada vez que se requiera. Aunque ocasionalmente deba actualizarse, no es necesario reinstalarla nuevamente.

En R, el proceso es similar. Primero instalas el paquete desde un repositorio (equivalente a la tienda de aplicaciones) y luego lo cargas, un paso análogo a abrir la aplicación en tu celular.

> **Nota 6**
>
> En caso de que estés utilizando Posit Workbench, de acuerdo a lo señalado en la Sección 2.3, es posible que el proceso descrito a continuación no debas realizarlo. En algunas ocasiones el administrador de esta herramienta puede haber instalado ciertos paquetes con anterioridad.
> Para el caso de Posit Workbench ubica la pestaña **Console** en el **Panel** (Ver la Figura 2.4) y, tras el símbolo `>` escribe el comando `library(tidyverse)`, luego presiona `Return` (mac), `Enter` (linux), `Enter` (windows). Si te aparece un mensaje similar al señalado en el Listado 4.1 no es necesario que realices alguna instalación y puedes dirigirte directamente a la Sección 4.3. En caso contrario, si se genera un mensaje igual o similar a:
>
> ```
> Error:
> there is no package called ‘tidyverse’
> ```
>
> puedes proceder con las instrucciones que se señalan a continuación.

Para practicar este proceso, instalaremos el paquete `tidyverse`. Este paquete es una colección de herramientas diseñadas para ciencia de datos que utilizaremos constantemente a lo largo del libro. Para iniciar la instalación, ubica la pestaña **Console** en el **Panel** (Ver la Figura 2.4) y, tras el símbolo `>`, escribe el siguiente comando:

```
install.packages("tidyverse")
```

Luego, presiona `Return` (mac), `Enter` (linux), `Enter` (windows). Ten paciencia hasta que el proceso finalice y no te preocupes por los mensajes que aparezcan en la consola. Al ser un conjunto robusto de herramientas, la instalación puede tardar. Sabrás que ha terminado el proceso cuando veas aparecer nuevamente el símbolo `>`.

> **!** Importante 5
>
> Al instalar un paquete en R, asegúrate de escribir su nombre entre comillas (`""`). De lo contrario, la instalación fallará. Por ahora, no te preocupes por la razón detrás de este aspecto. Lo importante es que te asegures que puedas saber cómo instalar un paquete en R.

4.3. Carga de paquetes

Cuando instalas una aplicación en tu celular es necesario abrirla para poder usarla, no basta con simplemente instalarla. En R para poder utilizar un paquete que se encuentre instalado previamente es necesario cargarlo (equivalente a abrir una aplicación que se encuentra instalada en un celular). En R para cargar un paquete se utiliza la función `library()`.

Por ejemplo, si deseas cargar el paquete `tidyverse` escribe el siguiente comando en la consola:

```
library(tidyverse)
```

Luego, presiona `Return` (mac), `Enter` (linux), `Enter` (windows). Si el paquete `tidyverse` fue instalado de manera correcta se generará un texto (ver Listado 4.1) igual o similar al que se muestra a continuación en la consola:

> **i** Nota 7
>
> En el Capítulo 5 podrás explorar más acerca de los tipos de texto que la consola puede generar y cómo interpretarlos cuando se genere algún contenido. Allí explicaremos en particular el texto del Listado 4.1.

Listado 4.1

```
 Attaching core tidyverse packages   tidyverse 2.0.0
dplyr     1.2.0       readr     2.2.0
forcats   1.0.1       stringr   1.6.0
ggplot2   4.0.2       tibble    3.3.1
lubridate 1.9.5       tidyr     1.3.2
purrr     1.2.1
 Conflicts             tidyverse_conflicts()
dplyr::filter() masks stats::filter()
dplyr::lag()    masks stats::lag()
Use the conflicted package to force all conflicts
to become errors
```

4.4. Uso de paquetes

Para entender cómo utilizar un paquete, explorarás tu primer conjunto de datos en R utilizando el paquete `tidyverse`. Primero, cierra la ventana de Positron si seguiste las instrucciones descritas en la Sección 2.2 y abre la carpeta `r-analisis-datos` (ver Sección 3.2) utilizando las opciones **File > Open Folder...** que podrás encontrar en el menú superior izquierdo de la interfaz. En caso de que estés utilizando Posit Workbench (ver Sección 2.3) abre una sesión con el proyecto `r-analisis-datos`, creado de acuerdo a lo señalado en la Sección 3.2.

Luego, en la **Barra de actividades** (ver Figura 2.4) selecciona el ícono y párate con el puntero del ratón en la **Barra lateral primaria** (ver Figura 2.4) donde al lado izquierdo de la etiqueta `R-ANALISIS-DATOS` podrás encontrar en el segundo ícono de izquierda a derecha la opción de crear una carpeta. Crea la carpeta con el nombre `004_paquetes-r` y dentro de la carpeta también en la **Barra lateral primaria** utiliza el primer ícono de izquierda a derecha para crear un archivo con el nombre `001_uso-paquetes.R`. De esa manera podrás abrirlo en el **Editor** (ver Figura 2.4) tal como se muestra en la Figura 4.2:

En el archivo `001_uso-paquetes.R` carga el paquete `tidyverse` escribiendo el contenido que se menciona a continuación:

Listado 4.2 Línea de código para cargar el paquete tidyverse

```
# Cargar paquetes ----
library(tidyverse)
```

Para que R procese la instrucción, sitúa el cursor (|) sobre la línea de código (ver

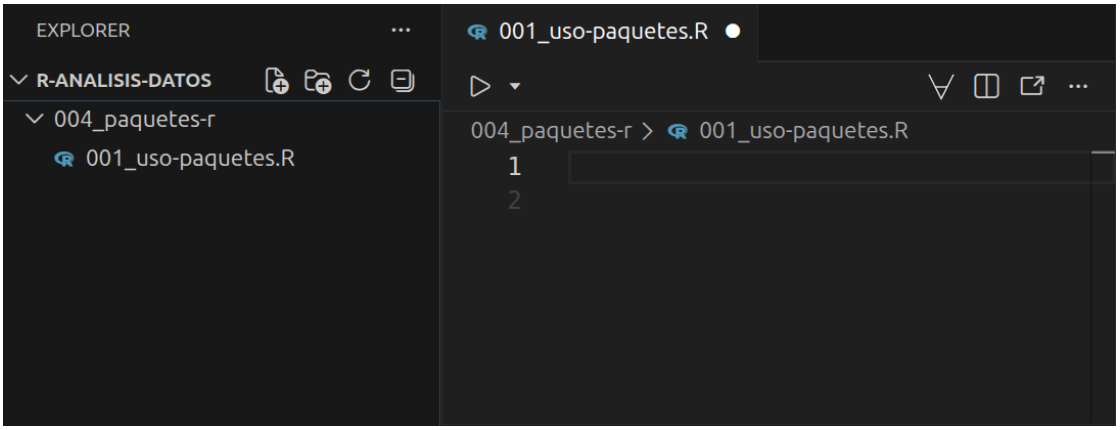


Figura 4.2.: Estructura de carpetas y archivos para explorar tu primer conjunto de datos

Listado 4.2) y presiona la combinación de teclas `Command-Return` (mac), `Ctrl-Enter` (linux), `Ctrl-Enter` (windows).

> **Nota 8**
>
> Cada vez que requieras ver el resultado de una línea de código que se encuentra dentro de un archivo con extensión `.R` sitúa el cursor (|) sobre la línea de código y presiona la combinación de teclas `Command-Return` (mac), `Ctrl-Enter` (linux), `Ctrl-Enter` (windows). De ahora en adelante nos referimos a este proceso como **ejecutar el código**.

En el Listado 4.2 el símbolo `#` permite agregar comentarios para entender los comandos que escribimos y `----` genera secciones para mantener organizado el contenido dentro de un archivo con extensión `.R` donde las señala en la **Barra lateral primaria** en la sección **OUTLINE** (ver Figura 4.3) en Positron.

Un archivo con extensión `.R` por sí solo no genera ningún resultado. Por ello, debes utilizar la combinación de teclas mencionada anteriormente para ejecutar los comandos en el **Editor** y mostrar el resultado señalado en el Listado 4.1 en el **Panel** (ver Figura 2.4) dentro de la pestaña **CONSOLE**.

Una vez cargado el paquete `tidyverse` puedes explorar el conjunto de datos `mpg` que contiene información de economía de combustible desde 1999 hasta 2008 para 38 modelos populares de automóviles. Agrega al archivo `001_uso-paquetes.R` el siguiente contenido y ejecuta el código (ver Nota 8) para que puedas visualizar parte de los datos en la consola donde el resultado puede variar dependiendo del tamaño de la pantalla de tu equipo:

```
# A tibble: 234 x 11
   manufacturer model    displ  year   cyl trans drv     cty
```

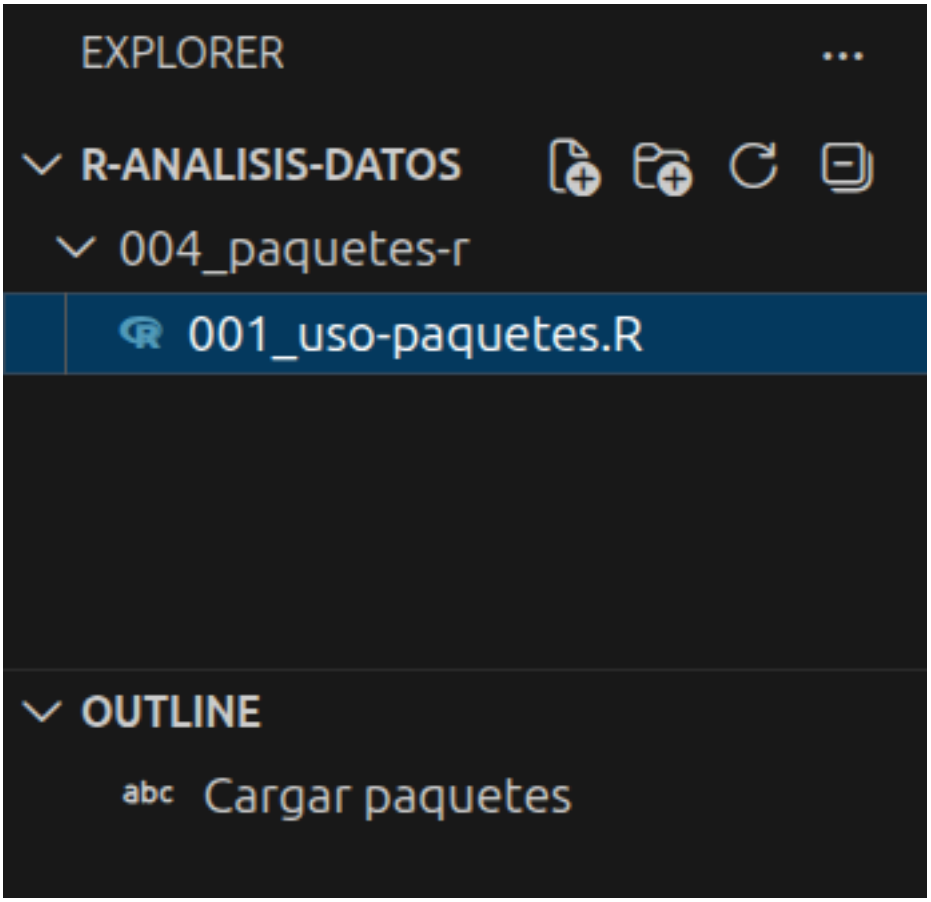


Figura 4.3.: Sección OUTLINE dentro de la Barra lateral primaria de Positron

Listado 4.3 Conjunto de datos `mpg`

```
# Explorar el conjunto de datos mpg ----
mpg
```

```
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int>
 1 audi         a4           1.8  1999     4 auto~ f        18
 2 audi         a4           1.8  1999     4 manu~ f        21
 3 audi         a4           2    2008     4 manu~ f        20
 4 audi         a4           2    2008     4 auto~ f        21
 5 audi         a4           2.8  1999     6 auto~ f        16
 6 audi         a4           2.8  1999     6 manu~ f        18
 7 audi         a4           3.1  2008     6 auto~ f        18
 8 audi         a4 quat~     1.8  1999     4 manu~ 4        18
 9 audi         a4 quat~     1.8  1999     4 auto~ 4        16
10 audi         a4 quat~     2    2008     4 manu~ 4        20
# i 224 more rows
# i 3 more variables: hwy <int>, fl <chr>, class <chr>
```

En el resultado del Listado 4.3 se incluye información para entender aspectos del conjunto de datos `mpg`:

- `# A tibble: 234 × 11` indica en primera instancia el tipo de objeto en R denominado `tibble` donde se compone de `234` filas y `11` columnas.
- Una `tibble` solo muestra la cantidad de columnas que caben en tu pantalla. Por ese motivo se señalan en la parte superior los nombres de las columnas `manufacturer`, `model`, `displ`, `year`, `cyl`, `trans`, `drv`, `cty` y luego en la parte inferior las variables restantes `hwy`, `fl` y `class` en el mensaje `# 3 more variables: hwy <int>, fl <chr>, class <chr>`.
- Debajo de los nombres de las columnas se indica el tipo de dato que corresponde a cada variable (p. ej., `<chr>`, `<dbl>`, `<int>`). En caso que no se puedan mostrar todas las columnas en la pantalla, en la parte inferior se señalan los tipos restantes de las demás columnas. Las diferentes opciones de tipos de columnas se pueden consultar en la documentación oficial del paquete `tibble`, que hace parte del paquete `tidyverse`, en Column types.
- Por defecto una `tibble` solo muestra las primeras 10 filas pero señala en la parte inferior con el mensaje `# 224 more rows` el número de filas restantes que componen el conjunto de datos.

Si el conjunto de datos pertenece a un paquete, como en el caso de `mpg`, es posible consultar la documentación acerca de la misma utilizando el símbolo `?` más su nombre, tal como se muestra a continuación:

Listado 4.4 Uso del símbolo `?` para consultar documentación en R

```
# Documentación del conjunto de datos mpg ----
?mpg
```

Si ejecutas este código, en la **Barra lateral secundaria** (ver Figura 2.4) dentro de la pestaña **HELP**, se desplegará la documentación relacionada con `mpg`.

En Positron también se puede utilizar una herramienta denominada Explorador de Datos para visualizar la información en una cuadrícula similar a una hoja de cálculo, así como filtrar y ordenar los datos temporalmente y obtener estadísticas de cada variable. Para utilizar esta herramienta ejecuta el siguiente código:

```
# Uso del explorador de datos ----
View(mpg)
```

De esa manera podrás utilizar el explorador de datos y examinar los elementos indicados anteriormente para `mpg` tal como se muestra en la Figura 4.4.

En ocasiones se hace necesario poder dar un "vistazo" a un conjunto de datos cuando se tiene un número considerable de columnas. En ese sentido, otra forma adicional de explorar un conjunto de datos, es mediante el uso de la función `glimpse()`. Esta función permite visualizar los nombres de las columnas verticalmente hacia abajo y los datos correspondientes a cada variable de manera horizontal. Para utilizar esta herramienta ejecuta el siguiente código:

Listado 4.5 Uso de la función `glimpse` para dar un "vistazo" a un conjunto de datos

```
# "Vistazo" del conjunto de datos ----
glimpse(mpg)
```

```
Rows: 234
Columns: 11
$ manufacturer <chr> "audi", "audi", "audi", "audi", "audi~
$ model        <chr> "a4", "a4", "a4", "a4", "a4", "a4", "~
$ displ        <dbl> 1.8, 1.8, 2.0, 2.0, 2.8, 2.8, 3.1, 1.~
$ year         <int> 1999, 1999, 2008, 2008, 1999, 1999, 2~
$ cyl          <int> 4, 4, 4, 4, 6, 6, 6, 4, 4, 4, 4, 6, 6~
```

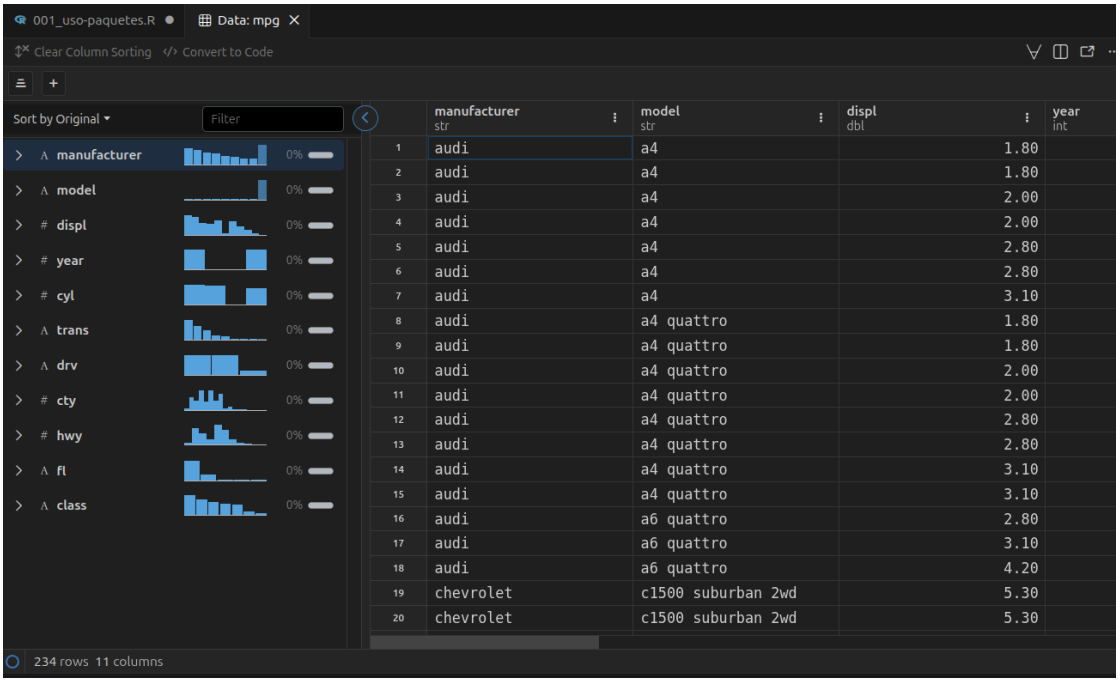


Figura 4.4.: Interfaz del explorador de datos aplicado a `mpg`

```
$ trans        <chr> "auto(l5)", "manual(m5)", "manual(m6)~
$ drv          <chr> "f", "f", "f", "f", "f", "f", "f", "4~
$ cty          <int> 18, 21, 20, 21, 16, 18, 18, 18, 16, 2~
$ hwy          <int> 29, 29, 31, 30, 26, 26, 27, 26, 25, 2~
$ fl           <chr> "p", "p", "p", "p", "p", "p", "p", "p~
$ class        <chr> "compact", "compact", "compact", "com~
```

De esa manera tal como se muestra anteriormente es posible visualizar en su totalidad los nombres de las columnas sin importar el ancho de la pantalla de tu equipo, así como el tipo de cada columna y una parte de los datos asociados a cada variable. Este aspecto es útil, particularmente cuando se tiene un conjunto de datos con una cantidad considerable de columnas.

Por último, en ciertos contextos puede ser útil extraer y explorar solo una variable dentro de un conjunto de datos. En R existen diversas formas de lograr esto. Sin embargo, señalaremos solo 2 alternativas, enfocándonos en el uso del operador `$`, que está incluido por defecto en R, y la función `pull()` del paquete `dplyr`, que hace parte del paquete `tidyverse`.

Si se buscara, por ejemplo, extraer la variable `manufacturer` de `mpg` con el operador `$` puedes ejecutar el siguiente código:

```
# Extracción de columnas ----
mpg$manufacturer
```

De manera alternativa puedes utilizar la función `pull()` del paquete `dplyr`, ejecutando el siguiente código:

```
pull(mpg, manufacturer)
```

5. ¿Cómo escribir código en R?

Este capítulo se basa en (Ismay, 2025, Capítulo 1), (Grolemund, 2014, Capítulo 1) y (Wickham, 2019, Capítulo 2) donde tiene como propósito introducir conceptos básicos del lenguaje de programación R utilizando datos de los programas de pregrado de la Facultad de Estudios a Distancia (FAEDIS) de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. Operaciones aritméticas
2. Vectores atómicos en R
3. Mensajes (messages), advertencias (warnings) y errores (errors)
4. Tibbles
5. Operadores relacionales

> **Nota 9**
>
> Para desarrollar el contenido de este capítulo recuerda trabajar por proyectos de acuerdo a lo señalado en el Capítulo 3. En ese sentido, genera una carpeta con el nombre `005_como-escribir-codigo-en-r` y dentro de la carpeta crea un archivo con el nombre `001_codigo-en-r-datos-faedis.R` siguiendo las indicaciones descritas en la Sección 4.4.

5.1. Operaciones aritméticas

Para el año 2026 en el primer semestre, la Facultad de Estudios a Distancia ofrecía 7 programas académicos de pregrado con su respectivo costo de matrícula en pesos colombianos (COP), tal como se señala en la siguiente lista:

- Administración de Empresas: $ 4,389,000 COP
- Administración de Riesgos, Seguridad y Salud en el Trabajo: $ 4,896,000 COP
- Contaduría Pública: $ 4,135,000 COP
- Ingeniería Civil: $ 4,940,000 COP
- Ingeniería Industrial: $ 5,638,000 COP
- Ingeniería Informática: $ 4,661,000 COP
- Relaciones Internacionales y Estudios Políticos: $ 6,238,000 COP

Si se buscara calcular el promedio del costo de matrícula dentro de la facultad para este periodo, es posible utilizar R como una calculadora ejecutando el siguiente código:

Listado 5.1 Costo promedio de mátricula en programas de pregrado de la Facultad de Estudios a Distancia para el año 2026 semestre 1

```
# Costo promedio de matrícula ----
(4389000 +
  4896000 +
  4135000 +
  4940000 +
  5638000 +
  4661000 +
  6238000) /
  7
```

```
[1] 4985286
```

En R podemos utilizar operadores aritméticos para llevar a cabo operaciones como en una calculadora (ver Tabla 5.1):

Tabla 5.1.: Operadores aritméticos en R

Operador	Nombre	Ejemplo	Resultado
`+`	Suma	`2 + 4`	`6`
`-`	Resta	`2 - 4`	`-2`
`*`	Multiplicación	`2 * 4`	`8`
`/`	División	`2 / 4`	`0.5`
`^`	Exponente	`2 ^ 4`	`16`

5.2. Vectores atómicos en R

En el Listado 5.1 se utilizó R para calcular el costo promedio de la matrícula. Sin embargo, si quisieramos reutilizar estos datos tendríamos que escribirlos nuevamente, donde este proceso no es eficiente. R permite guardar datos almacenándolos dentro de un objeto. Una forma de almacenar datos es utilizando vectores atómicos a través de la función `c()` y luego vinculando el objeto que se busque crear a través de un nombre para utilizarlo cuando lo requieras.

A continuación guardarás el costo de matrícula de cada programa de pregrado en un vector atómico mediante la función `c()` y luego vincularás ese objeto al nombre `costo_matricula`, tal como se señala a continuación:

Listado 5.2 Uso de un vector atómico para almacenar datos

```
# Vector atómico ----
costo_matricula <- c(
  4389000,
  4896000,
  4135000,
  4940000,
  5638000,
  4661000,
  6238000
)
costo_matricula
```

```
[1] 4389000 4896000 4135000 4940000 5638000 4661000 6238000
```

En el Listado 5.2 se utilizaron comas para combinar los valores en la función `c()` y el operador de asignación `<-` para vincular este objeto con el nombre `costo_matricula`. De esa manera, puedes reutilizar esta información si la requieres nuevamente.

Por ejemplo, si quisieras calcular la dispersión del costo de matrícula alrededor de la media a través de la desviación estandar muestral, (ver Ecuación 5.1), entonces es posible reutilizar los datos vinculados con el nombre `costo_matricula`.

$$\begin{aligned} s &= \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \\ &= \sqrt{\frac{1}{n-1}\left[(x_1 - \bar{x})^2 + \cdots + (x_n - \bar{x})^2\right]} \end{aligned} \tag{5.1}$$

Donde en la Ecuación 5.1, $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{1}{n}(x_1 + \cdots + x_n)$ corresponde a la media muestral[1].

A continuación podras utilizar R para calcular la desviación estandar muestral y examinar las ventajas de utilizar vectores atómicos. En primera instancia puedes utilizar la functión `mean()` para calcular el costo promedio de matrícula así como vincular este dato con el nombre `media_costo_matricula` de la siguiente forma:

```
# Cálculo de la media muestral ----
media_costo_matricula <- mean(costo_matricula)
media_costo_matricula
```

[1] Para nuestro caso $i = 1, \ldots, 7$, $x_1 = 4389000, \ldots, x_7 = 6238000$ y $\bar{x} = 4985286$.

```
[1] 4985286
```

Luego puedes utilizar parte de la Ecuación 5.1 para calcular la varianza muestral de la siguiente forma:

Listado 5.3 Uso de un vector atómico para calcular la varianza muestral

```
# Cálculo de la varianza muestral ----
varianza_costo_matricula <- sum(
  (costo_matricula - media_costo_matricula)^2
) /
  (7 - 1)
varianza_costo_matricula
```

```
[1] 531509238095
```

En el Listado 5.3 primero restamos a cada dato la media muestral, `costo_matricula - media_costo_matricula`, donde R utiliza operaciones vectorizadas. En ese sentido, R resta a cada elemento de `costo_matricula` el valor de la media muestral `media_costo_matricula`. Después, también a través de una operación vectorizada, R aplica el exponente `2` mediante `^` a cada elemento de `costo_matricula - media_costo_matricula`. Posteriormente, utilizando la función `sum()` R suma cada uno de los elementos en `(costo_matricula - media_costo_matricula)^2` para poder dividir el valor correspondiente por `7 - 1` y así obtener la varianza muestral.

Una vez se calcule la varianza muestral, de acuerdo a lo señalado en el Listado 5.3, puedes calcular la raiz cuadrara principal utilizando la función `sqrt()`, para obtener la desviacion estandar muestral, de la siguiente forma :

```
# Cálculo de la desviación estandar muestral ----
desv_estandar_costo_matricula <- sqrt(
  varianza_costo_matricula
)
desv_estandar_costo_matricula
```

```
[1] 729046.8
```

En la Figura 5.1 se señala una representación gráfica de la media y la desviación estandar muestral con los datos asociados donde el valor de $ 729,047 se representan a través de un intervalo alrededor de la media, $[\bar{x} - s, \bar{x} + s]$, con un segmento y los extremos señalados con flechas.

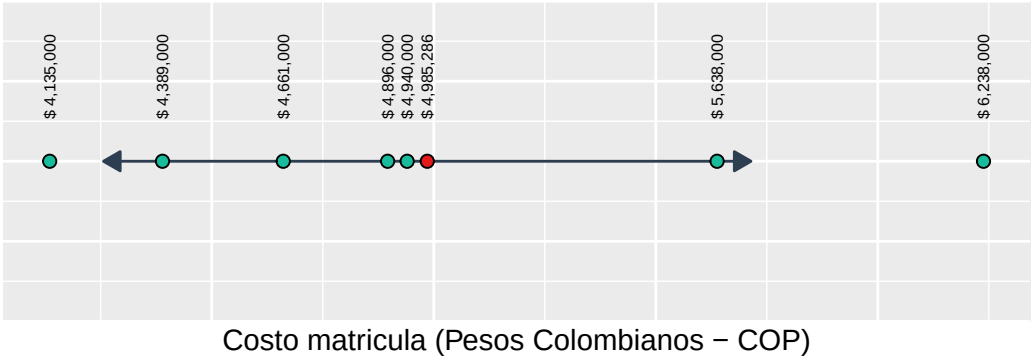


Figura 5.1.: Media (círculo rojo) y desviación estandar (segmento y flechas azules) muestral del costo de matrícula

De esa manera en la Figura 5.1 es posible observar aquellos valores que se encuentran 1 desviación estandar o menos alrededor de la media y que se encuentran dentro del intervalo $[\bar{x} - s, \bar{x} + s]$.

5.3. Mensajes (messages), advertencias (warnings) y errores (errors)

Cuando utilices R y ejecutes un código es posible que observes texto en la consola. Este texto puede ser el resultado generado al ejecutar el código como también mensajes, advertencias y errores.

Para entender que es un mensaje, advertencia o un error realiza el procedimiento descrito en el primer párrafo de la Sección 4.4. Luego, carga el paquete `tidyverse` siguiendo las indicaciones de esa misma sección, donde se generará un contenido igual o similar al señalado en el Listado 4.1.

5.3.1. Mensajes

El contenido del Listado 4.1 es un ejemplo de un **mensaje**. Un mensaje en R tiene como propósito informar al usuario sobre algún aspecto en la ejecución del código. En el caso del Listado 4.1 la primera parte indica el conjunto de paquetes que `tidyverse` carga al cargar este paquete, así como sus respectivas versiones:

En el Listado 5.4 se indica que los paquetes cargardos al cargar `tidyverse` son: `dplyr`, `readr`, `forcats`, `stringr`, `ggplot2`, `tibble`, `lubridate`, `tidyr` y `purrr`.

La segunda parte indica que existen 2 funciones en el paquete `dplyr` que tienen el mismo nombre que otras funciones en otro paquete estandar (o de base) (ver Sección 4.1)

Listado 5.4

```
 Attaching core tidyverse packages   tidyverse 2.0.0
dplyr     1.2.0       readr     2.2.0
forcats   1.0.1       stringr   1.6.0
ggplot2   4.0.2       tibble    3.3.1
lubridate 1.9.5       tidyr     1.3.2
purrr     1.2.1
```

llamado `stats` y que por lo tanto si se utilizan de manera directa, entonces R empleará las referentes al paquete `dplyr`:

Listado 5.5

```
 Conflicts             tidyverse_conflicts()
dplyr::filter() masks stats::filter()
dplyr::lag()    masks stats::lag()
Use the conflicted package to force all conflicts
to become errors
```

En el Listado 5.5 las funciones con el mismo nombre en `dplyr` y `stats` son `filter()` y `lag()`. Si utilizas de manera directa alguna de las funciones, el mensaje indica que estas funciones corresponderán al paquete `dplyr`. En caso de que quieras utilizar las funciones del paquete `stats`, deberás indicarlo de manera explícita utilizando la sintaxis `stats::filter()` o `stats::lag()` (ver Nota 10), según el caso.

> **Nota 10**
>
> La sintaxis `paquete::funcion()` utiliza el operador de 2 puntos dobles (`::`) para indicarle a R de forma explícita a qué paquete pertenece una función.
> Esto resulta útil en 2 casos:
>
> 1. **Resolver conflictos de nombres**: cuando 2 paquetes tienen una función llamada igual como `dplyr::filter()` y `stats::filter()`, escribir `paquete::funcion()` le dice a R con exactitud cuál de ellas utilizar.
> 2. **Consultar la ayuda**: permite consultar la documentación oficial de una función en un paquete específico escribiendo `?paquete::funcion` (por ejemplo, `?dplyr::filter` o `?stats::filter`) en la consola.

5.3.2. Advertencias

Una **advertencia** señala un posible problema en el código pero no detiene su ejecución. Para ilustrar este caso ejecuta nuevamente el código señalado en el Listado 5.2. Luego, asume que se quiere aumentar el costo de matrícula para cada uno de los programas académicos pero se comete el error de incluir un aumento sólo para 6 programas de la siguiente forma:

Listado 5.6 Advertencia al realizar un operación matemática entre vectores atómicos con un número diferente de elementos

```
aumento_costo_matricula <- c(
  100000,
  130000,
  250000,
  145000,
  210000,
  94000
)

costo_matricula + aumento_costo_matricula
```

```
Warning in costo_matricula + aumento_costo_matricula:
longer object length is not a multiple of shorter object
length
```

```
[1] 4489000 5026000 4385000 5085000 5848000 4755000 6338000
```

Debido a que R utiliza operaciones vectorizadas es posible llevar a cabo la operación señalada en el Listado 5.6, aún si cada vector atómico (`costo_matricula` y `aumento_costo_matricula`) tiene un número diferente de elementos mediante un proceso que se conocer como **reciclaje de vectores** y que se describe a continuación:

- R repite el vector más corto (`aumento_costo_matricula`) hasta que tenga la misma longitud que el vector más largo (`costo_matricula`)
- Luego realiza la operación matemática

Por ese motivo para el último costo de matrícula el resultado al generar el aumento es de $ 6,338,000. Sin embargo, R le indica una advertencia al usuario sobre este aspecto aún si lleva a cabo la respectiva suma.

5.3.3. Errores

Por último, un **error** se genera cuando al ejecutar un código existe una imposibilidad lógica de hacerlo o viola las reglas del lenguaje R. En ese caso se detiene la ejecución del código y se genera un texto que busca explicar la razón del problema. De nuevo, para ilustrar este caso asumamos que corregimos la advertencia agregando un nuevo valor de la siguiente manera:

Listado 5.7 Error por un problema de sintaxis al no separar de manera correcta los elementos de un vector atómico

```
aumento_costo_matricula <- c(
  100000,
  130000,
  250000,
  145000,
  210000,
  94000
  45000
)

costo_matricula + aumento_costo_matricula
```

```
Error in parse(text = input): <text>:8:3: unexpected numeric constant
7:   94000
8:   45000
     ^
```

En el Listado 5.7 no es posible ejecutar el código debido a que no se agregó una coma, `","`, al final del penúltimo valor en el vector atómico `aumento_costo_matricula`. R no entiende como proceder por qué no se esta separando este elemento del último número para que se definan en total 7 valores.

5.4. Tibbles

El uso de vectores atómicos es útil para almacenar datos de un mismo tipo pero tiene limitaciones si se buscan integrar variables de diferentes tipos en una misma estructura. Cuando se analizan datos es necesario agrupar distintos atributos para entender el contexto de la información y poder relacionar diferentes variables.

Una tibble permite agrupar datos en una estructura rectangular para entender la relación de variables de diferentes tipos. Para entender este aspecto construirás una tibble para

agrupar y darle un contexto adecuado a la información señalada al principio del Capítulo 5 utilizando la functión `tibble()`.

Para organizar la información de los programas académicos relacionados con el costos de matrícula definiremos en primera instancia tanto el año como el semestre al que se refieren los datos, luego asociaremos un identificador único para cada programa académico, su nombre, nivel académico y finalmente una variable del costo de matrícula.

Para construir una tibble que reuna las características señaladas anteriormente, ejecuta el código del Listado 5.8:

Listado 5.8 Construcción de una tibble a partir de información del costo de matrícula de programas académicos de pregrado

```
# Creación de tibbles ----
costo_matricula_pregrado <- tibble(
  ano = 2026,
  semestre = 1,
  codigo_snies = c(
    6527,
    108241,
    11428,
    11004,
    53703,
    105142,
    10963
  ),
  nombre_programa = c(
    "Administración de Empresas",
    "Administración de Riesgos, Seguridad y Salud en el Trabajo",
    "Contaduría Pública",
    "Ingeniería Civil",
    "Ingeniería Industrial",
    "Ingeniería Informática",
    "Relaciones Internacionales y Estudios Políticos"
  ),
  nivel_academico = "Pregrado",
  costo_matricula = costo_matricula
)

costo_matricula_pregrado
```

```
# A tibble: 7 x 6
```

```
    ano semestre codigo_snies nombre_programa
  <dbl>    <dbl>        <dbl> <chr>
1  2026        1         6527 Administración de Empresas
2  2026        1       108241 Administración de Riesgos, Se~
3  2026        1        11428 Contaduría Pública
4  2026        1        11004 Ingeniería Civil
5  2026        1        53703 Ingeniería Industrial
6  2026        1       105142 Ingeniería Informática
7  2026        1        10963 Relaciones Internacionales y ~
# i 2 more variables: nivel_academico <chr>,
#   costo_matricula <dbl>
```

En la tibble del Listado 5.8 respecto a las variables `ano`, `semestre` y `nivel_academico` R aplica el proceso de reciclaje de vectores (ver Sección 5.3.2) para ajustar el tamaño de las columnas. En el caso de la variable `codigo_snies` se fija un código establecido por el Sistema Nacional de Información de la Educación Superior (SNIES) para identificar y asociar de manera única a cada programa académico con el nombre señalado en la variable `nombre_programa`. Por último, es posible en una tibble utilizar un vector atómico previamente definido, como en el caso de `costo_matricula`, para generar una variable. En el Listado 5.8 se utiliza el mismo nombre asociado al vector atómico que contiene el costo de matrícula para la columna que incluye dicha información.

! Importante 6

En el Listado 5.8 y en el desarrollo del Capítulo 5, se utilizó una convención para nombrar variables similar a la descrita en la Sección 3.3. Sin embargo, existe una excepción fundamental: no se utilizó el carácter `-`.
En R, para que un nombre asociado a un objeto sea **sintácticamente válido**, debe cumplir con un conjunto de reglas que se pueden consultar con el comando `?make.names` en la sección **Details**:

1. **Caracteres permitidos**: Solo puede contener letras, números, puntos (`.`) y guiones bajos, (`_`). El guion medio, (`-`), no está permitido porque R lo interpreta como el operador de resta.
2. **Inicio del nombre**: Debe comenzar obligatoriamente con una letra o con un punto, (`.`).
3. **Restricción del punto**: Si un nombre inicia con punto, el siguiente carácter no puede ser un número.
4. **Palabras reservadas**: No pueden utilizarse términos del lenguaje de programación de R que están reservados, como por ejemplo `if`, `else`, `for`, `TRUE` o `FALSE`. La lista completa se puede consultar con el comando `?Reserved` en la sección **Details**.

Aunque R permite técnicamente nombres como `.mi_variable`, `.Mi_Variable`, `ñi_variablé`, en este libro seguiremos las prácticas señaladas en la Sección 3.3, pero ajustándolas para que generen nombres sintácticamente válidos.
Esto implica 2 cambios principales respecto a nombrar carpetas o archivos: no se utilizará el carácter `-` donde se reemplazará por `_` y nunca se iniciará un nombre con números. Esta diferencia radica en que un sistema operativo es más flexible al nombrar archivos o carpetas mientras que R, al ser un lenguaje de programación, requiere reglas estrictas para evitar ambigüedades al ejecutar código.

5.5. Operadores relacionales

Los operadores relacionales son operadores binarios que permiten la comparación de valores en vectores atómicos. Estos operadores son útiles porque establecen condiciones lógicas para filtrar filas de interés en una tibble. En la Tabla 5.2 se señalan algunos de los operadores relacionales utilizados por R.

Tabla 5.2.: Operadores relacionales en R

Operador	Nombre
`==`	Igual a
`!=`	Diferente de
`>`	Mayor que
`<`	Menor que
`>=`	Mayor o igual que
`<=`	Menor o igual que

Para ilustrar el uso de operadores relacionales utilizaremos inicialmente el operador `$`, el cual fue descrito en la última parte de la Sección 4.4. De esa manera será posible entender más adelante la utilidad de obtener condiciones lógicas para filtrar filas dentro de una tibble.

Si se buscara identificar aquellos valores del costo de matrícula en la tibble `costo_matricula_pregrado` respecto a la variable `costo_matricula` que son mayores a la media, se podría proceder de la siguiente manera utilizando el operador `$` y operadores relacionales:

```
[1] FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE
```

En el Listado 5.9 se utiliza el operador `$` para extraer de la tibble `costo_matricula_pregrado` los valores de la variable `costo_matricula` para identificar si son mayores a la media

Listado 5.9 Uso de operadores relacionales para identificar un conjunto de valores que cumplen una condición

```
costo_matricula_pregrado$costo_matricula >
  mean(costo_matricula_pregrado$costo_matricula)
```

del costo de matrícula, utilizando la función `mean()`. El resultado, es un vector atómico con valores lógicos (`FALSE` o `TRUE`) que identifica qué valores dentro de `costo_matricula_pregrado$costo_matricula` cumplen o no cumplen con la condición. En ese sentido, en el resultado del Listado 5.9 aquellas posiciones con valores `TRUE` corresponden a las posiciones de `costo_matricula_pregrado$costo_matricula` que cumplen con la condición, es decir que son mayores a la media del costo de matrícula.

Al principio no pareciera claro cómo se puede utilizar el resultado generado en el Listado 5.9. Sin embargo, a continuación se señalará cómo un resultado con valores lógicos de `TRUE` o `FALSE` permite ser el insumo para poder filtrar filas con base a una determinada condición lógica de interés.

A modo de ejemplo, si buscaras identificar los programas académicos con sus respectivos atributos que cumplen con la condición señalada en el Listado 5.9, es posible utilizar la función `filter()` del paquete `dplyr` de la siguiente forma:

Listado 5.10 Uso de la función `filter()` y operadores lógicos para filtrar filas en una tibble

```
filter(
  costo_matricula_pregrado,
  costo_matricula > mean(costo_matricula)
)
```

```
# A tibble: 2 x 6
    ano semestre codigo_snies nombre_programa
  <dbl>    <dbl>        <dbl> <chr>
1  2026        1        53703 Ingeniería Industrial
2  2026        1        10963 Relaciones Internacionales y ~
# i 2 more variables: nivel_academico <chr>,
#   costo_matricula <dbl>
```

En el Listado 5.10, la función `filter()` requiere como primer argumento una tibble (o un objeto similar) y una expresión que resulte en un vector lógico. Los paquetes del `tidyverse` facilitan este proceso mediante el **enmascaramiento de datos** (data masking). Este mecanismo permite que la expresión lógica (en este ejemplo,

`costo_matricula > mean(costo_matricula)`) haga referencia directa a las columnas del objeto `costo_matricula_pregrado` como si fueran variables "independientes", eliminando la necesidad de utilizar el operador `$`. De esa manera requieres escribir menos texto al escribir código en R y es posible referirse a las variables de la tibble de una manera más natural al utilizar funciones del `tidyverse`.

5.6. Operadores lógicos vectorizados

Los operadores lógicos vectorizados son operadores que realizan comparaciones elemento por elemento de manera similar a los operadores aritméticos (ver Tabla 5.1) pero que se refieren a la negación lógica NO (`NOT` en inglés), el operador lógico Y (`AND` en inglés) y el operador lógico O (`OR` en inglés). En la Tabla 5.3 se señalan algunos de los operadores lógicos vectorizados utilizados en R.

Tabla 5.3.: Operadores lógicos vectorizados en R

Operador	Nombre	Operación
`!`	Negación lógica	NO (`NOT`)
`&`	Conjunción lógica	Y (`AND`) elemento a elemento
`|`	Disyunción lógica	O (`OR`) elemento a elemento

A modo de ejemplo, si buscaras identificar los programas académicos con sus respectivos atributos que cumplen con la condición señalada en el Listado 5.9 y que además son menores al valor máximo respecto a la variable `costo_matricula`, es posible utilizar la función `filter()` del paquete `dplyr` de la siguiente forma:

Listado 5.11 Uso de la función `filter()` y operadores lógicos vectorizados para filtrar filas en una tibble

```
filter(
  costo_matricula_pregrado,
  costo_matricula > mean(costo_matricula) &
    costo_matricula < max(costo_matricula)
)
```

```
# A tibble: 1 x 6
    ano semestre codigo_snies nombre_programa
  <dbl>    <dbl>        <dbl> <chr>
1  2026        1        53703 Ingeniería Industrial
# i 2 more variables: nivel_academico <chr>,
#   costo_matricula <dbl>
```

Parte II.

Visualización de datos

El objetivo de esta parte, es presentar el modelo mental de **la gramática de gráficos en capas** (**layered grammar of graphics**, (Wickham, 2010)) para visualizar datos en R utilizando el paquete `ggplot2` a través de los siguientes 4 capítulos:

- Capítulo 6: introduce la forma de visualizar variables numéricas para un conjunto de categorías a través del uso de un gráfico de barras.
- Capítulo 7: introduce la forma de visualizar la distribución de una variable numérica a través del uso de un histograma.
- Capítulo 8: introduce la forma de visualizar proporciones para un conjunto determinado de grupos a través del uso de un gráfico de barras apiladas y normalizadas.
- Capítulo 9: introduce la forma de visualizar asociaciones entre variables numéricas y la evolución de una variable contínua a lo largo del tiempo a través del uso de un gráfico de puntos y un gráfico de líneas.

Una vez se entienda el funcionamiento del paquete `ggplot2` a través del modelo mental de **la gramática de gráficos en capas**, el lector interesado en profundizar y explorar otras formas de visualizar datos puede consultar el recurso The R Graph Gallery donde contiene ejemplos acompañados de código reproducible y una explicación de como se construyen estas visualizaciones.

Asimismo, quienes tengan experiencia previa en el lenguaje de programación Python podrán transferir fácilmente las habilidades de R desarrolladas en esta parte. Esto es posible gracias a que dicho lenguaje cuenta con una implementación fiel de **la gramática de gráficos en capas** mediante el paquete `plotnine`.

6. Cantidades

Este capítulo se basa en (Wickham, 2016) y (Wilke, 2019) así como en la documentación oficial de `ggplot2` donde tiene como propósito introducir el modelo mental de **la gramática de gráficos en capas** (**layered grammar of graphics**, (Wickham, 2010)) para visualizar variables numéricas a través de un conjunto de categorías utilizando el paquete `ggplot2` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de la Facultad de Estudios a Distancia (FAEDIS) de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. ¿Qué es la gramática de gráficos en capas?
2. ¿Qué elementos componen la gramática de gráficos en capas?
3. ¿Cómo visualizar variables numéricas para un conjunto de categorías?

> **Nota 11**
>
> Para desarrollar el contenido de este capítulo es necesario que descargues el archivo señalado a continuación:
>
> **2021-2024_saber-pro-genericas_faedis.rds**
>
> Luego, genera una carpeta con el nombre `data` y dentro de esta carpeta incluye el archivo `2021-2024_saber-pro-genericas_faedis.rds`. Finalmente, genera una carpeta con el nombre `006_visualizar-cantidades-en-r` y dentro de la carpeta crea un archivo con el nombre `001_visualizar-cantidades-datos-faedis.R`, siguiendo las indicaciones descritas en la Sección 4.4.

6.1. Gramática de gráficos en capas

El paquete `ggplot2`, incluido en el `tidyverse`, permite generar visualizaciones a partir de un conjunto de datos. Este paquete utiliza un marco conceptual basado en la gramática de gráficos desarrollada en (Wilkinson & Wills, 2005). La gramática de gráficos en capas es una adaptación de esta gramática para integrarla en el lenguaje de programación de R.

A medida que explores diferentes formas de visualizar un conjunto de datos entenderás en más detalle la gramática de gráficos en capas utilizando el paquete `ggplot2`. Por ahora se describirán de manera general las 7 capas que conforman este modelo mental (ver Figura 6.1) y que se utilizan en `ggplot2` como un conjunto de instrucciones para generar un gráfico.

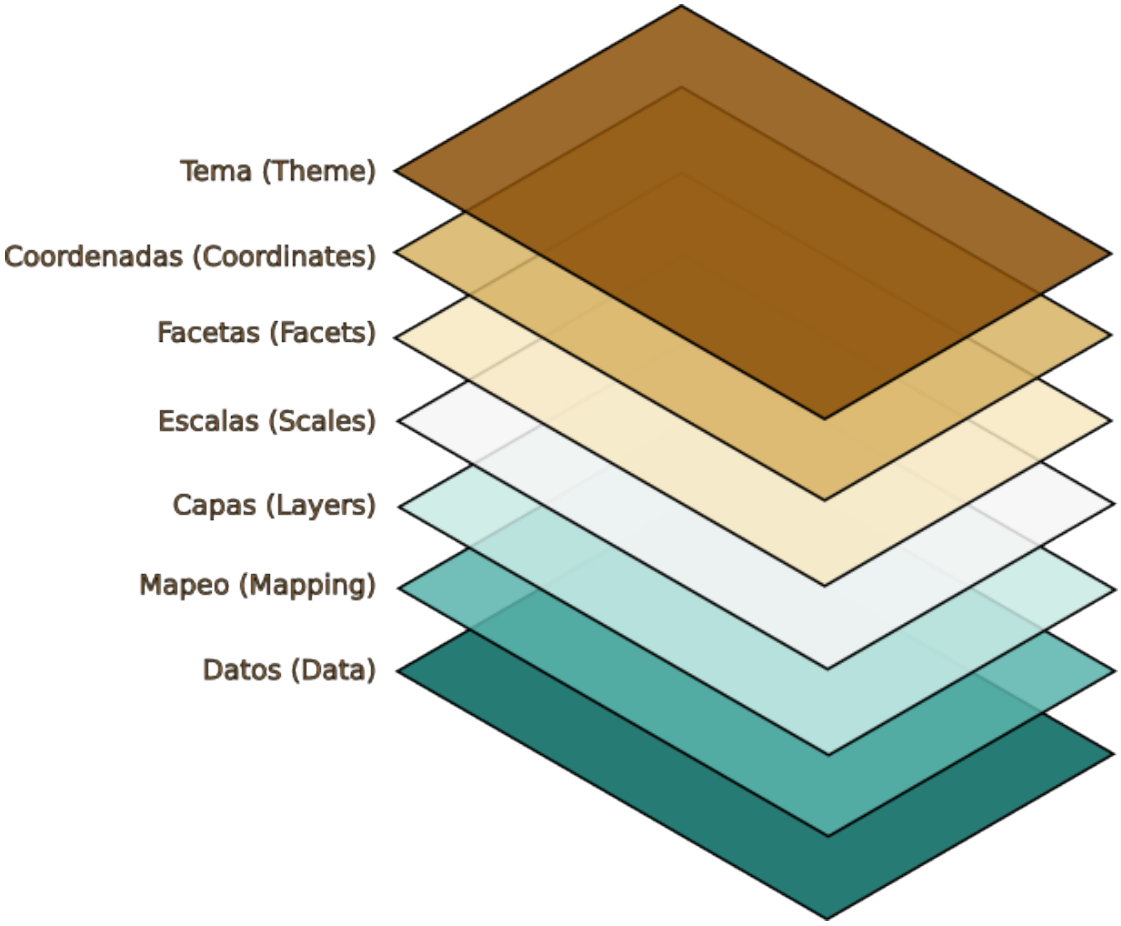


Figura 6.1.: Modelo mental de la gramática de gráficos en capas

Para que `ggplot2` genere un gráfico requiere como mínimo los siguientes 3 componentes: Datos, Mapeo y Capas. Los demás elementos restantes (Escalas, Facetas, Coordenadas y Tema) cuentan con valores predeterminados que eliminan inicialmente la necesidad de fijar configuraciones adicionales.

1. **Datos**: es el insumo principal que se requiere para generar un gráfico donde se refiere al conjunto de datos que se busca visualizar. `ggplot2` funciona mejor si el conjunto de datos es ordenado (tidy data en inglés, (Wickham, 2014)) donde este concepto se tratará en más detalle en el Capítulo 16.
2. **Mapeo**: es el conjunto de instrucciones que permiten mapear las variables de los

datos a propiedades visuales denominadas atributos estéticos. En la Figura 6.2 se indican algunos ejemplos de atributos estéticos.

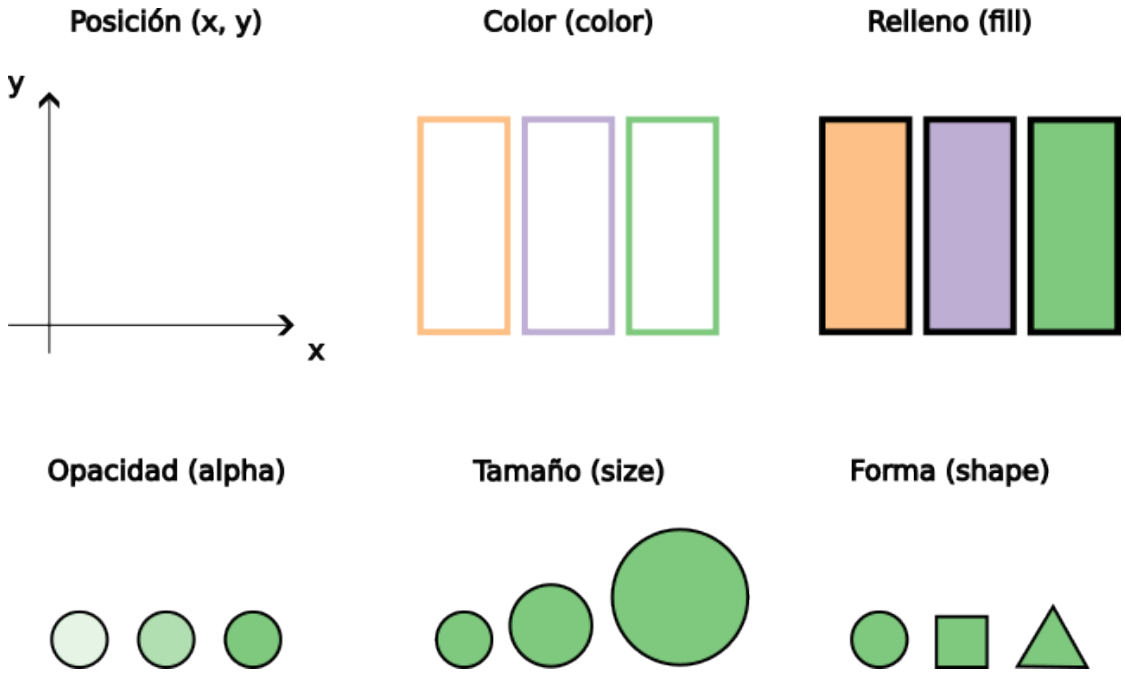


Figura 6.2.: Ejemplos de atributos estéticos

3. **Capas**: es el componente que toma los datos mapeados para transformarlos en una representación visual comprensible. Cada capa se compone de 3 elementos:

   a. Una **geometría** que determina cómo se muestran los datos a través de atributos estéticos.

   b. Una **transformación estadística** donde calcula nuevas variables a partir de los datos y afecta qué parte de ellos se muestra.

   c. Un **ajuste de posición** donde determina la ubicación en la que se presentará visualmente cada dato.

   Una capa en `ggplot2` se construye con funciones de la forma `geom_*()` o `stat_*()`.

4. **Escalas**: es el componente que traduce la información visual del gráfico de vuelta a los datos originales, utilizando los ejes y las leyendas como guías.

   En `ggplot2` las escalas se aplican usualmente mediante funciones que utilizan el patrón `scale_{aesthetic}_{type}()`, donde `{aesthetic}` corresponde a uno de los atributos estéticos utilizados en el mapeo y `{type}` una etiqueta descriptiva asignada a la escala.

Sin embargo, existen también funciones auxiliares como `labs()` y `lims()`que permiten realizar ajustes a las etiquetas y los límites de un gráfico.

5. **Facetas**: es el componente que permite visualizar diferentes subconjuntos de datos en paneles individuales, dividiendo el gráfico principal en función de variables categóricas.
6. **Coordenadas**: es el componente que determina cómo se combinan los atributos estéticos `x` y `y` para posicionar los elementos en el plano del gráfico. Por defecto, se utiliza el sistema de coordenadas cartesianas, pero existen otras opciones dentro de `ggplot2`.
7. **Tema**: es el componente que se encarga de controlar cualquier elemento visual del gráfico que sea ajeno a los datos. Su función principal es determinar la apariencia estética y el estilo general del diseño del gráfico.

6.2. Gráfico de barras mediante conteo automático

En esta sección realizarás un gráfico de barras señalando el número de inscritos al examen Saber Pro para cada programa de pregrado de la Facultad de Estudios a Distancia de la Universidad Militar en el periodo 2021-2024, tal como se indica en la Figura 6.3.

6.2.1. Datos

El primer componente de la gramática de gráficos en capas son los datos. En ese sentido explorarás el conjuntos de datos en `2021-2024_saber-pro-genericas_faedis.rds`. Primero, debes cargar el paquete `tidyverse` utilizando el archivo `001_visualizar-cantidades-datos-faedis.R`

```
# Cargar paquetes ----
library(tidyverse)
```

De esa manera puedes importar los datos para explorarlos en R como se señala a continuación teniendo en cuenta lo indicado en la Nota 11.

Listado 6.1 Importación de datos para obtener el insumo principal que se requiere para generar un gráfico

```
# Importar datos ----
saber_pro_genericas_faedis_2021_2024 <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis.rds"
)
```

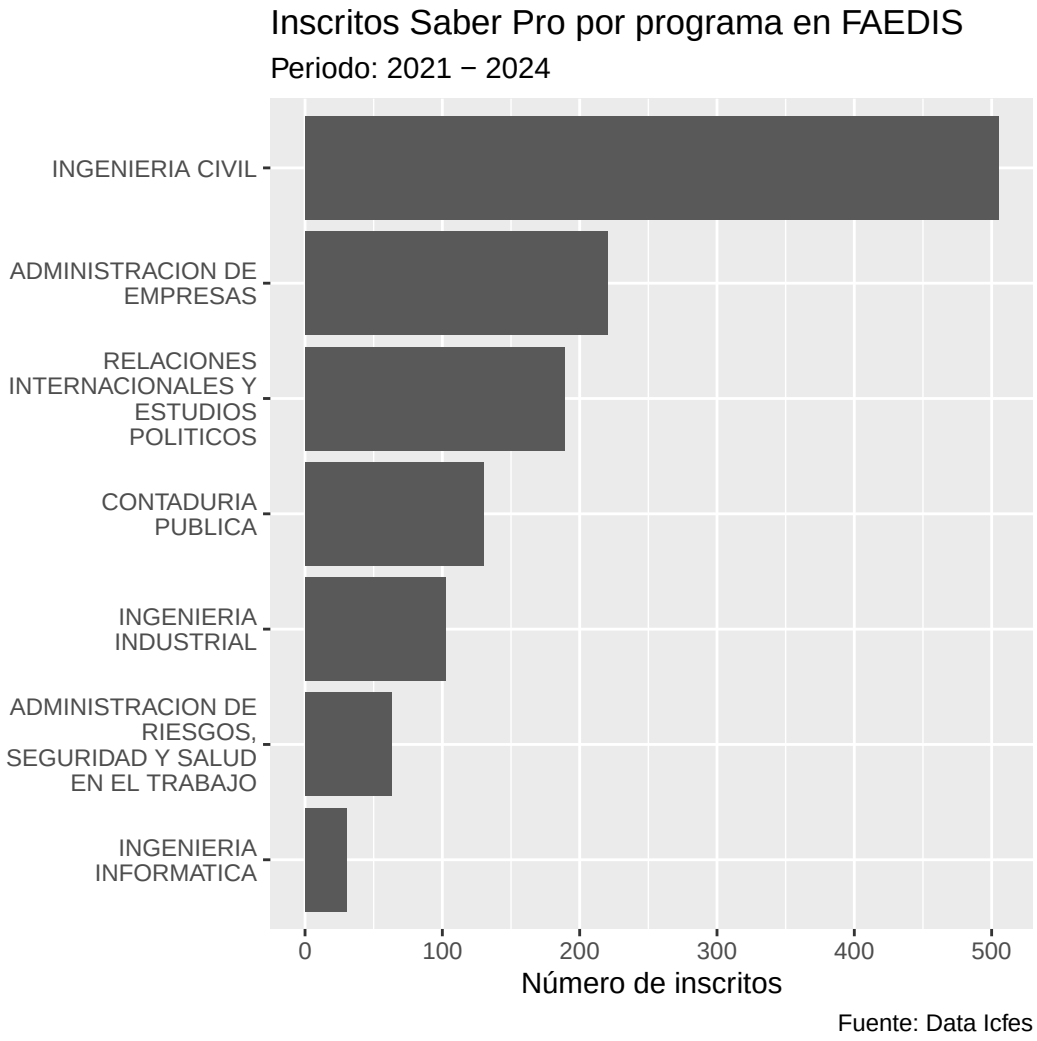


Figura 6.3.: Resultado final de un gráfico de barras utilizando `ggplot2`

> **Nota 12**
>
> En el Capítulo 12 se abarcará con más detalle la importación de datos para archivos con una extensión `.rds` mediante la función `read_rds()`, teniendo en cuenta lo indicado en el Listado 6.1. Por ahora, el enfoque del Capítulo 6 será la visualización de un conjunto de datos.

Una vez importes el conjunto de datos, podrás darle un "vistazo" utilizando la functión `glimpse()` (ver Listado 4.5) para identificar el número y los tipos de variables que lo componen, así como la cantidad de observaciones disponibles.

```
# Inspeccionar datos ----
glimpse(saber_pro_genericas_faedis_2021_2024)
```

```
Rows: 1,239
Columns: 21
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_genero                  <fct> M, M, M, M, F, F, F, F~
$ estu_semestrecursa           <ord> 09, 09, 09, 09, 09, 09~
$ fami_estratovivienda         <ord> Estrato 3, Estrato 4, ~
$ fami_educacionmadre          <ord> Técnica o tecnológica ~
$ fami_educacionpadre          <ord> Secundaria (Bachillera~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion  <chr> "11", "11", "11", "11"~
$ estu_depto_presentacion      <chr> "BOGOTÁ", "BOGOTÁ", "B~
$ estu_cod_mcpio_presentacion  <chr> "11001", "11001", "110~
$ estu_mcpio_presentacion      <chr> "BOGOTÁ D.C.", "BOGOTÁ~
$ estu_snies_prgmacademico     <int> 6527, 11428, 11428, 11~
$ estu_prgm_academico          <chr> "ADMINISTRACION DE EMP~
$ mod_competen_ciudada_punt    <dbl> 162, 190, 154, 179, 17~
$ mod_comuni_escrita_punt      <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt              <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem             <ord> A2, A2, A2, A2, A2, A2~
$ mod_lectura_critica_punt     <dbl> 114, 168, 139, 148, 18~
$ mod_razona_cuantitat_punt    <dbl> 135, 164, 113, 149, 16~
$ punt_global                  <dbl> 139, 170, 128, 155, 16~
```

6.2.1.1. Descripción de variables

Para examinar y en particular crear un gráfico a partir de un conjunto de datos es necesario entender a qué se refiere cada una de las variables que lo componen. En ese

sentido, a continuación se describen cada una de las variables en `2021-2024_saber-pro-genericas_faedis.rds`:

1. **ano**: año de presentación del examen Saber Pro
2. **estu_consecutivo**: identificador (id) público del estudiante (los microdatos se encuentran anonimizados)
3. **estu_genero**: género del estudiante (F para femenino y M para masculino)
4. **estu_semestrecursa**: semestre que cursa el estudiante en el año de presentación del examen
5. **fami_estratovivienda**: estrato socioeconómico de la vivienda donde habita el estudiante según recibo de energía eléctrica
6. **fami_educacionmadre**: nivel educativo más alto alcanzado por la madre del estudiante
7. **fami_educacionpadre**: nivel educativo más alto alcanzado por el padre del estudiante
8. **estu_exterior**: indica si el estudiante presenta la prueba Saber Pro en el exterior (`SI` ó `NO`)
9. **estu_cod_depto_presentacion**: código del departamento de presentación del examen según codificación DIVIPOLA[1]
10. **estu_depto_presentacion**: nombre del departamento de presentación del examen
11. **estu_cod_mcpio_presentacion**: código del municipio de presentación del examen según codificación DIVIPOLA
12. **estu_mcpio_presentacion**: nombre del municipio de presentación del examen
13. **estu_snies_prgmacademico**: código SNIES[2] del programa académico que cursa el estudiante y relacionado con el examen Saber Pro que presenta
14. **estu_prgm_academico**: nombre del programa académico que cursa el estudiante y relacionado con el examen Saber Pro que presenta
15. **mod_competen_ciudada_punt**: puntaje del módulo de competencias ciudadanas obtenido por el estudiante (el rango posible es $[0, 300]$)
16. **mod_comuni_escrita_punt**: puntaje del módulo de comunicación escrita obtenido por el estudiante (el rango posible es $[0, 300]$)
17. **mod_ingles_punt**: puntaje del módulo de inglés obtenido por el estudiante (el rango posible es $[0, 300]$)
18. **mod_ingles_desem**: nivel de desempeño del módulo de inglés obtenido por el estudiante (el rango posible es $[A_1, A_2, B_1, B_2]$)
19. **mod_lectura_critica_punt**: puntaje del módulo de lectura crítica obtenido por el estudiante (el rango posible es $[0, 300]$)

[1] DIVIPOLA es el acrónimo de **Divi**sión **Pol**ítico **A**dministrativa de Colombia donde es una nomenclatura estandarizada, diseñada por el Departamento Administrativo Nacional de Estadística (DANE) para la identificación de Entidades Territoriales (departamentos, distritos y municipios), Áreas No Municipalizadas y Centros Poblados.

[2] SNIES es el acrónimo de **S**istema **N**acional de **I**nformación de la **E**ducación **S**uperior donde el Ministerio de Educación de Colombia asígna un número de registro único a un programa académico para identificarlo (ver también Sección 5.4).

20. **mod_razona_cuantitat_punt**: puntaje del módulo de razonamiento cuantitativo obtenido por el estudiante (el rango posible es $[0, 300]$)
21. **punt_global**: puntaje total obtenido por el estudiante en el grupo de competencias genéricas (este grupo comprenden los módulos de competencias ciudadanas, comunicación escrita, inglés, lectura crítica y razonamiento cuantitativo y la variable se refiere al promedio simple del puntaje de estos módulos redondeado al entero más cercano)

6.2.1.2. `ggplot()`

Para construir la Figura 6.3 se requiere en primera instancia inicializar un objeto `ggplot`. Para llevar a cabo este proceso se utiliza la función `ggplot()` y el parámetro `data` de esta función con el conjunto de datos que se busca visualizar, en este caso `saber_pro_genericas_faedis_2021_2024`:

Listado 6.2 Inicialización de un objeto `ggplot`

```
ggplot(data = saber_pro_genericas_faedis_2021_2024)
```

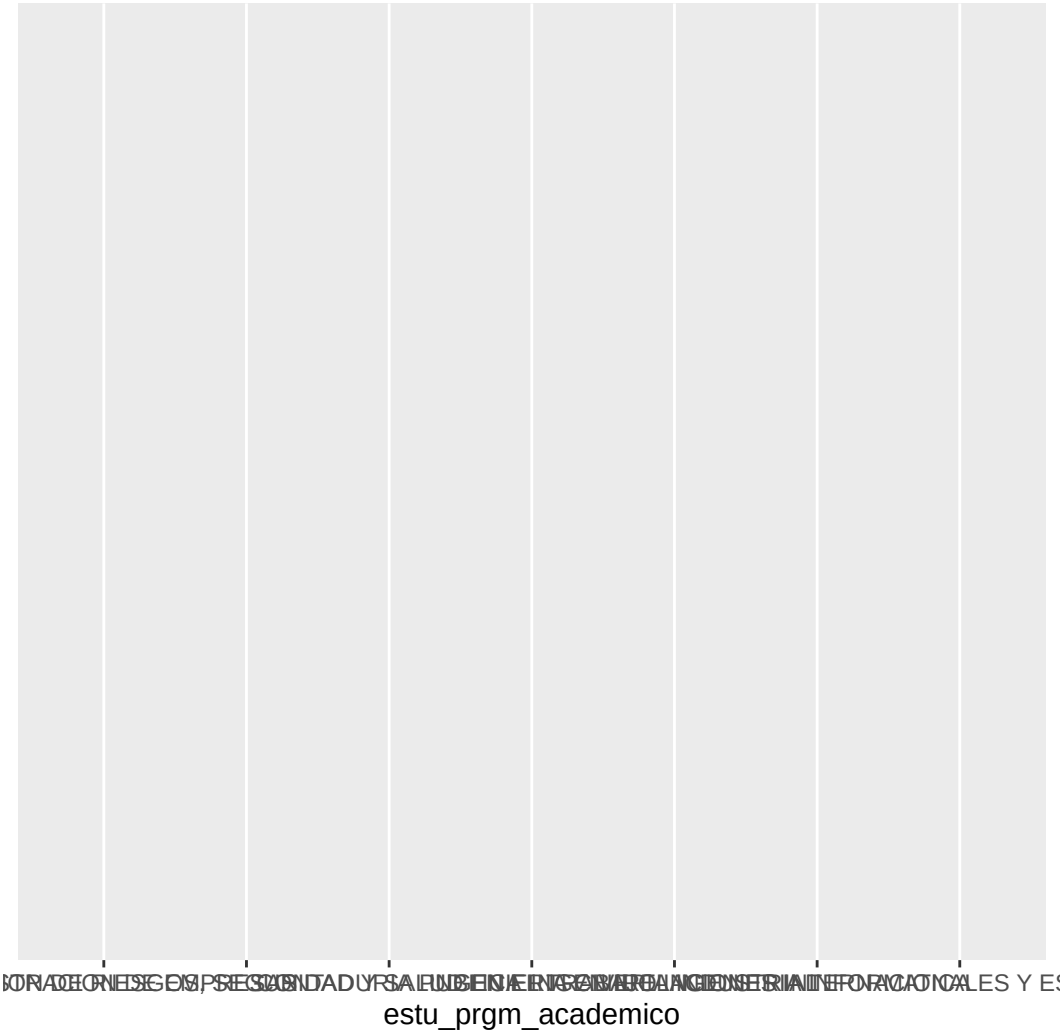
En este paso R no genera un gráfico, simplemente genera visualmente un "lienzo" completamente vacío. El propósito en el Listado 6.2 es almacenar los datos para que sean después utilizados por otros componentes.

6.2.2. Mapeo

Este componente tiene como propósito fijar un conjunto de instrucciones respecto a cómo los datos son asociados a los atributos estéticos. Para realizar el mapeo se utiliza la función `aes()` junto con el parámetro `mapping` dentro de la función `ggplot()`. Para ilustar este aspecto de manera concreta utilizaremos inicialmente la posición (ver Figura 6.2) asociando la coordenada `x` a la variable `estu_prgm_academico`, donde se encuentran los nombres de los programas académicos:

Listado 6.3 Mapeo de datos hacia atributos estéticos utilizando la coordenada `x`

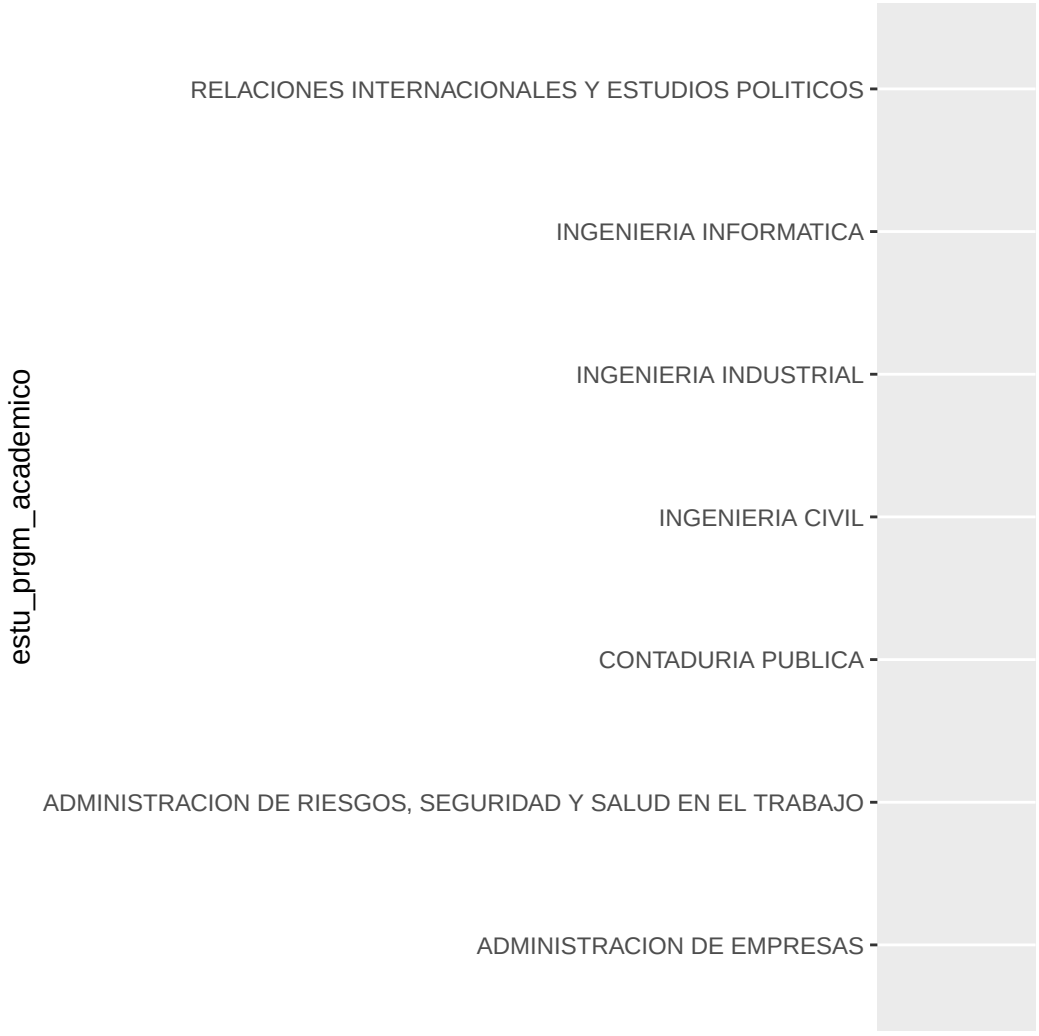
```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = estu_prgm_academico)
)
```



En el Listado 6.3 se fija una posición para cada nombre de los programas académicos en la coordenada `x`. Debido a la longitud de los nombres y la forma como se muestran de manera horizontal, el gráfico generado es confuso. Para solucionar este aspecto es posible utilizar de manera alternativa la coordena `y` asociándola a la variable `estu_prgm_academico`:

Listado 6.4 Mapeo de datos hacia atributos estéticos utilizando la coordenada `y`

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = estu_prgm_academico)
)
```



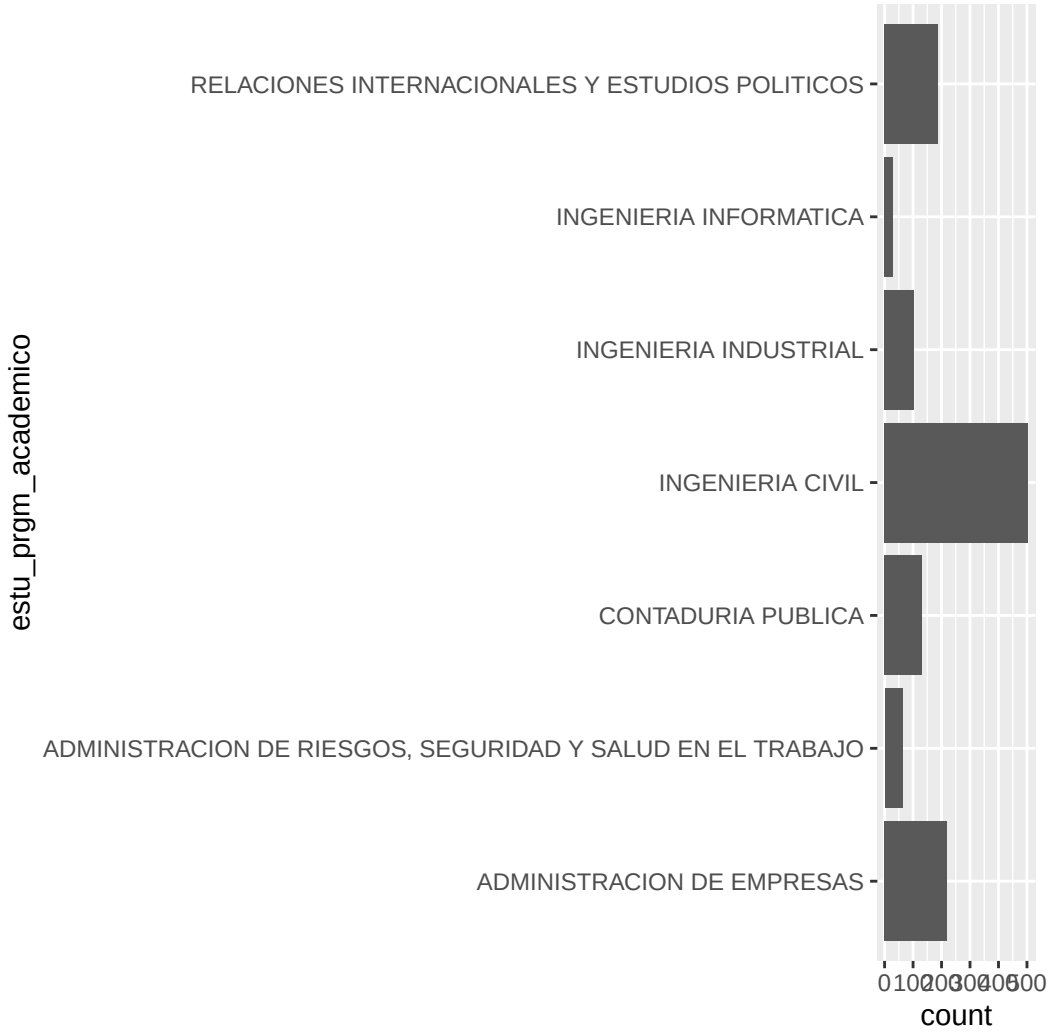
6.2.3. Capas

6.2.3.1. Geometrías

Para ilustrar el número de estudiantes inscritos por programa académico es posible utilizar la función `geom_bar()` donde a través de la longitud de un conjunto de barras y contando el número de casos para cada grupo se genera una representación gráfica de este aspecto. Para lograr lo descrito anteriormente agrega la capa `geom_bar()` utilizando el operador `+`, tal como se señala a continuación:

Listado 6.5 Uso de una geometría para ilustrar gráficamente datos a través de un atributo estético

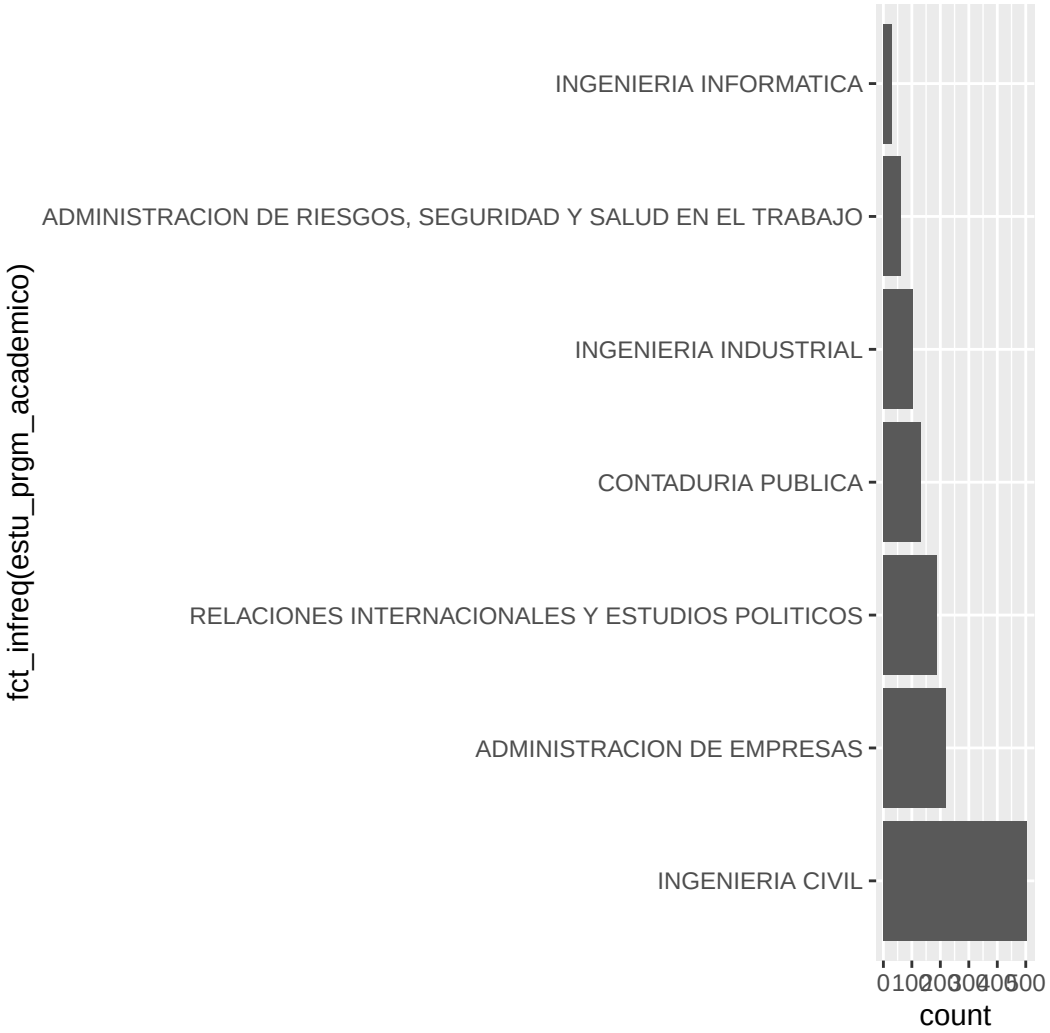
```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = estu_prgm_academico)
) +
  geom_bar()
```



En el Listado 6.5 los nombres de los programas académicos se ordenan internamente de manera alfabética y se representan al generar el gráfico de esa forma en la coordenada `y`. Sin embargo, para facilitar la comunicación respecto al número de inscritos en relación a la variable `estu_prgm_academico` es importante ordenarla por el número de casos en cada grupo dentro del gráfico. Para ordenar `estu_prgm_academico` por el número de casos se puede utilizar la función `fct_infreq()`:

Listado 6.6 Uso de la función `fct_infreq()` para ordenar datos categóricos de acuerdo al número de casos

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = fct_infreq(estu_prgm_academico))
) +
  geom_bar()
```

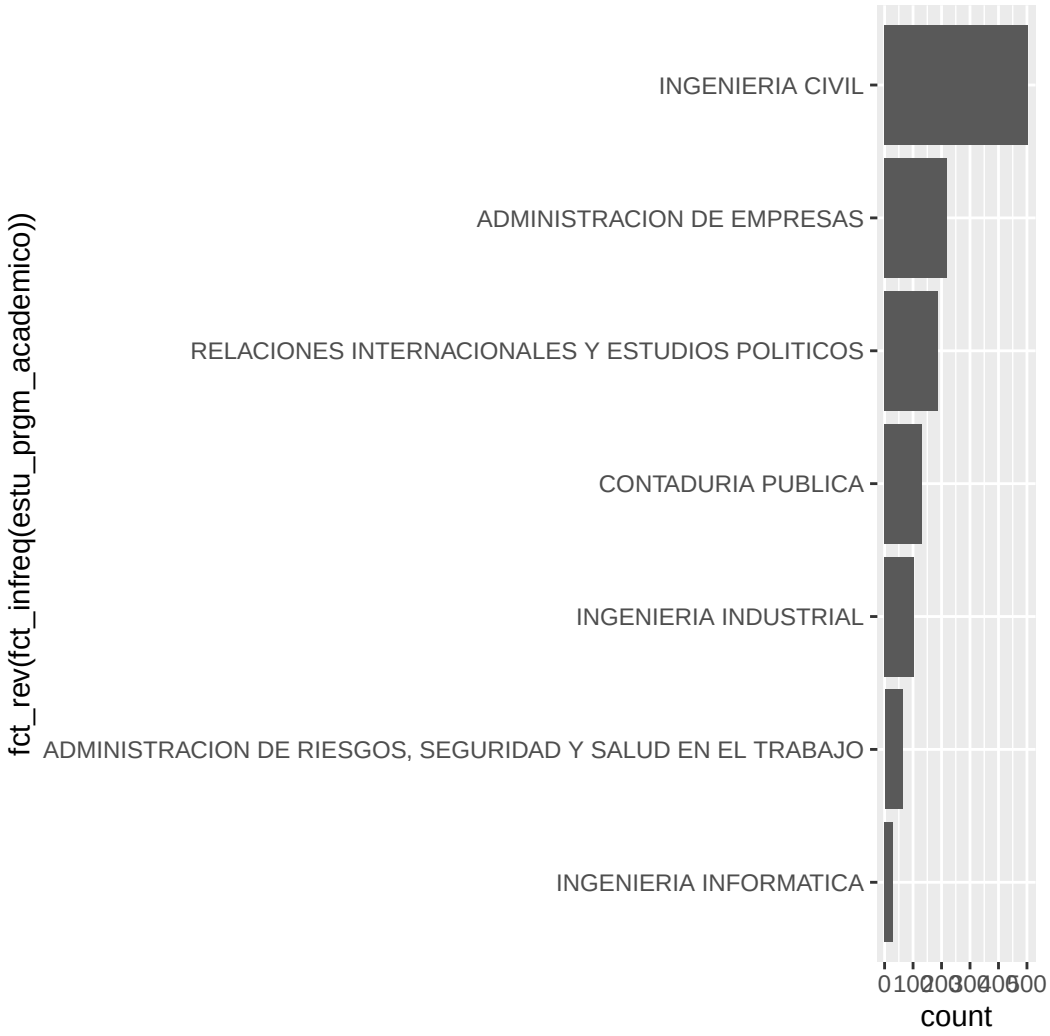


La función `fct_infreq()` ordena una variable categórica por el número de casos en cada grupo empezando por el más grande y terminando por el más pequeño. En caso que se busque ordenar `estu_prgm_academico` de forma descendente desde arriba hacia abajo, se

puede reversar el orden generado por la función `fct_infreq()` con la función `fct_rev()` utilizando como insumo el código del Listado 6.6:

Listado 6.7 Uso de la función `fct_rev()` para reversar el orden de datos categóricos previamente ordenados

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = fct_rev(fct_infreq(estu_prgm_academico)))
) +
  geom_bar()
```



6.2.3.2. Transformaciones estadísticas

En el Listado 4.4 se señaló la forma de consultar la documentación de un conjunto de datos que pertenece a un paquete utilizando el símbolo `?`. De la misma forma, es posible también realizar este proceso con una función. Si se examina la documentación de la función `geom_bar()` con `?geom_bar` en la sección **Usage** es posible verificar que se encuentra el parámetro `stat` con el argumento `"count"`, como `stat = "count"`. Esta opción es la que permite que la función `geom_bar()` realice un conteo del número de casos dentro de la variable `estu_prgm_academico` para generar la representación gráfica mediante la longitud de un conjunto de barras utilizando el código en el Listado 6.7.

6.2.3.3. Ajustes de posición

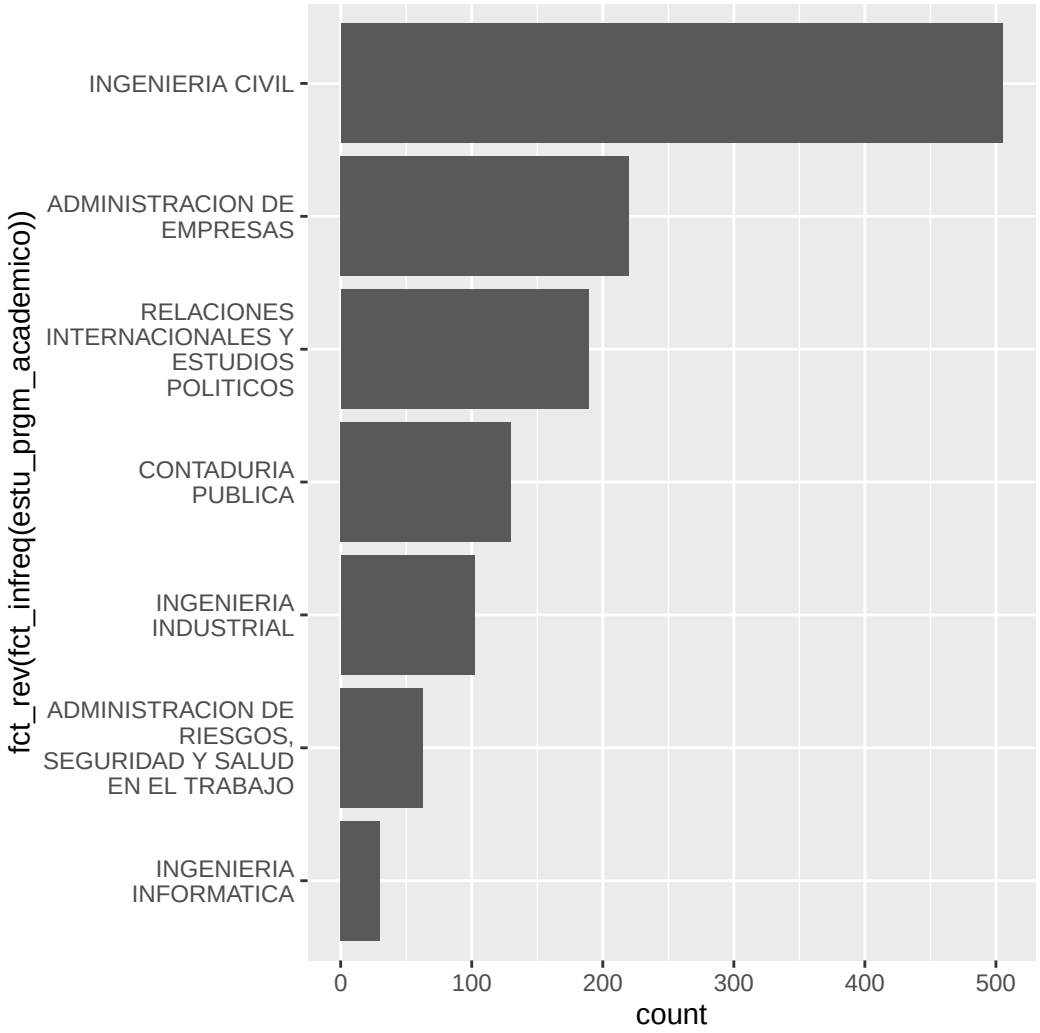
Al igual que en la Sección 6.2.3.2 si examinas la documentación de la función `geom_bar()` en la sección **Usage** se puede también encontrar el parámetro `position` con el argumento `"stack"`, como `position = "stack"`. En la Sección 7.3 se explorará, con otra tipo de geometría, cómo se puede utilizar este ajuste de posición para agregar información adicional dentro de un gráfico. Por ahora, en el Capítulo 6 no se profundizará respecto al uso de ajustes de posición.

6.2.4. Escalas

Este componente permite interpretar lo que se muestra en el gráfico a partir de etiquetas y leyendas. En el gráfico generado en el Listado 6.7 las etiquetas correspondientes a los nombres de los programas en la coordenada `y` tienen una longitud demasiado extensa donde abarcan la mayor área del gráfico, opacando la información que se busca resaltar respecto al número de inscritos por programa. Para corregir este aspecto, se puede utilizar la función `scale_y_discrete()` utilizando el parámetro `labels` y la función auxiliar `label_wrap_gen()` con el propósito de limitar el ancho de los nombres de los programas que se señalan en el gráfico. Utilizando de nuevo el operador `+`, como en el Listado 6.5, puedes agregar este componente de la siguiente manera:

Listado 6.8 Uso de una escala para modificar las etiquetas en la coordenada `y`

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = fct_rev(fct_infreq(estu_prgm_academico)))
) +
  geom_bar() +
  scale_y_discrete(
    labels = label_wrap_gen(width = 18)
  )
```



De esa manera, se puede limitar con el parámetro `width` y el argumento `18` la longitud

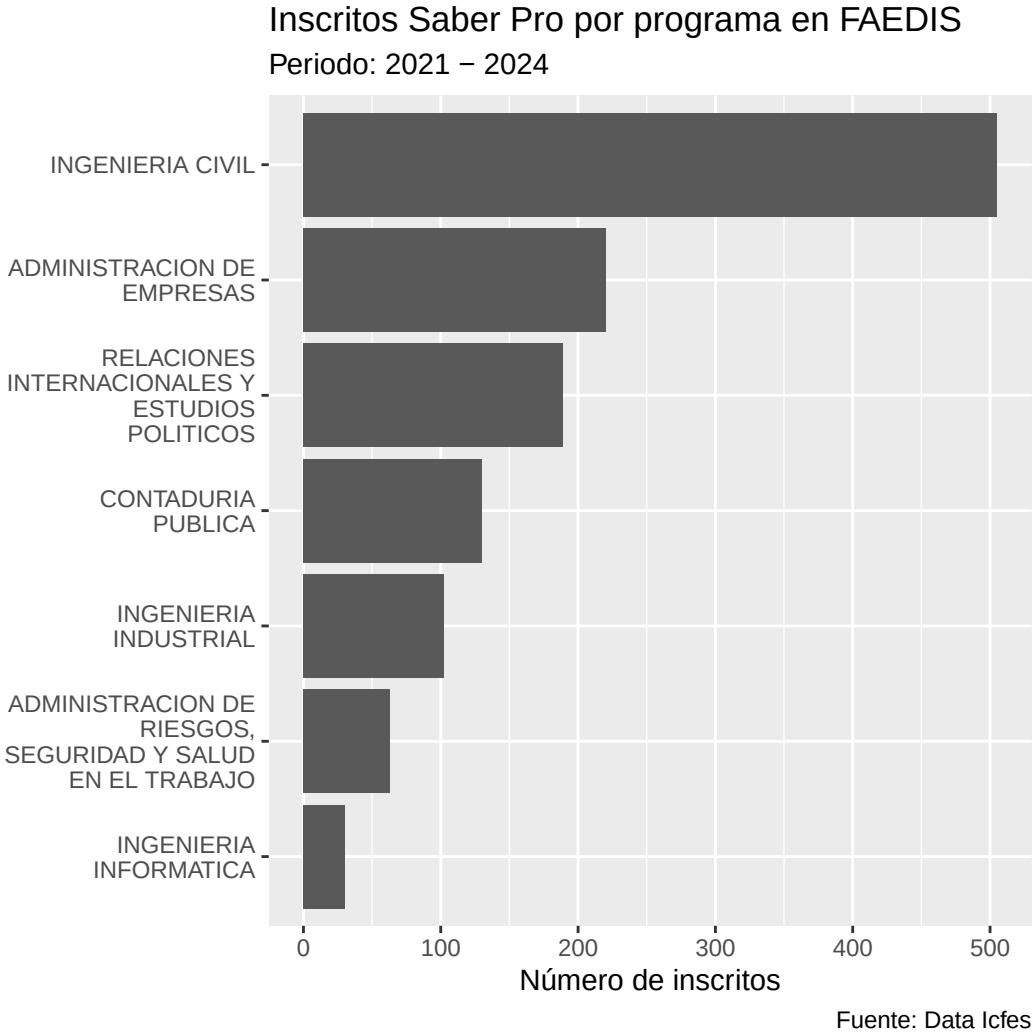
de las etiquetas en la coordenada `y` a un máximo de 18 caracteres, utilizando la función `label_wrap_gen()` dentro de `scale_y_discrete()`.

Adicionalmente, se pueden eliminar o modificar diferentes etiquetas en el gráfico para incluir información de contexto de tal manera que sea más comprensible. Para modificar estos elementos en el paquete `ggplot2` se utiliza la función `labs()` haciendo uso nuevamente del operador `+`.

Utilizando esta función se puede agregar un título, un subtítulo y una nota al pie así como señalar una descripción más adecuada en la etiqueta de la coordenada `x` y eliminar la etiqueta de la coordenada `y`, debido a que el nombre de los programas es lo suficiente claro para determinar a que se refiere esta información:

Listado 6.9 Uso de la función `labs()` para agregar, modificar o eliminar etiquetas en un gráfico

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = fct_rev(fct_infreq(estu_prgm_academico)))
) +
  geom_bar() +
  scale_y_discrete(
    labels = label_wrap_gen(width = 18)
  ) +
  labs(
    x = "Número de inscritos",
    y = NULL,
    title = "Inscritos Saber Pro por programa en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  )
```



En el Listado 6.9 se utilizan diferentes parámetros de la función `labs()` con unos argumentos para modificar las etiquetas de la siguiente manera:

1. `x = "Número de inscritos"`: permite modificar la etiqueta de la coordenada `x` para entender los valores que se presentan
2. `y = NULL`: permite eliminar la etiqueta de la coordenada `y`, utilizando `NULL` para señalar la ausencia de este elemento
3. `title = "Inscritos Saber Pro por programa en FAEDIS"`: permite señalar el título que acompaña la gráfica indicando un texto que describe el fenómeno que se busca representar
4. `subtitle = "Periodo: 2021-2024"`: permite señalar un subtítulo donde en este caso se indica el periodo al que se refieren los datos
5. `caption = "Fuente: Data Icfes"`: permite señalar una nota al pie indicando la fuente de donde se obtuvieron los datos

7. Distribuciones

Este capítulo se basa en (Wickham, 2016) y (Wilke, 2019) así como en la documentación oficial de `ggplot2` donde tiene como propósito utilizar el modelo mental de la gramática de gráficos en capas para visualizar la distribución de variables numéricas utilizando el paquete `ggplot2` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de la Facultad de Estudios a Distancia (FAEDIS) de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. ¿Cómo visualizar la distribución de una variable numérica?
2. ¿Cómo aplicar colores a un gráfico mediante los atributos estéticos `color` y `fill`?
3. ¿Cómo incorporar una variable categórica dentro de la visualización de una distribución?
4. ¿Cómo aplicar una paleta personalizada de colores?

> **Nota 13**
>
> Para desarrollar el contenido de este capítulo es necesario que hayas completado la descarga y ubicación del archivo `2021-2024_saber-pro-genericas_faedis.rds` dentro de la carpeta `data`, respecto a lo descrito en la Nota 11.
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `007_visualizar-distribuciones-en-r` y dentro de ella genera el archivo `001_visualizar-distribuciones-datos-faedis.R`, siguiendo las pautas de la Sección 4.4.

7.1. Histograma

En esta sección realizarás un histograma señalando la distribución del puntaje del módulo de inglés por nivel de desempeño para los programas de pregrado de la Facultad de Estudios a Distancia de la Universidad Militar en el periodo 2021-2024, tal como se indica en la Figura 7.1.

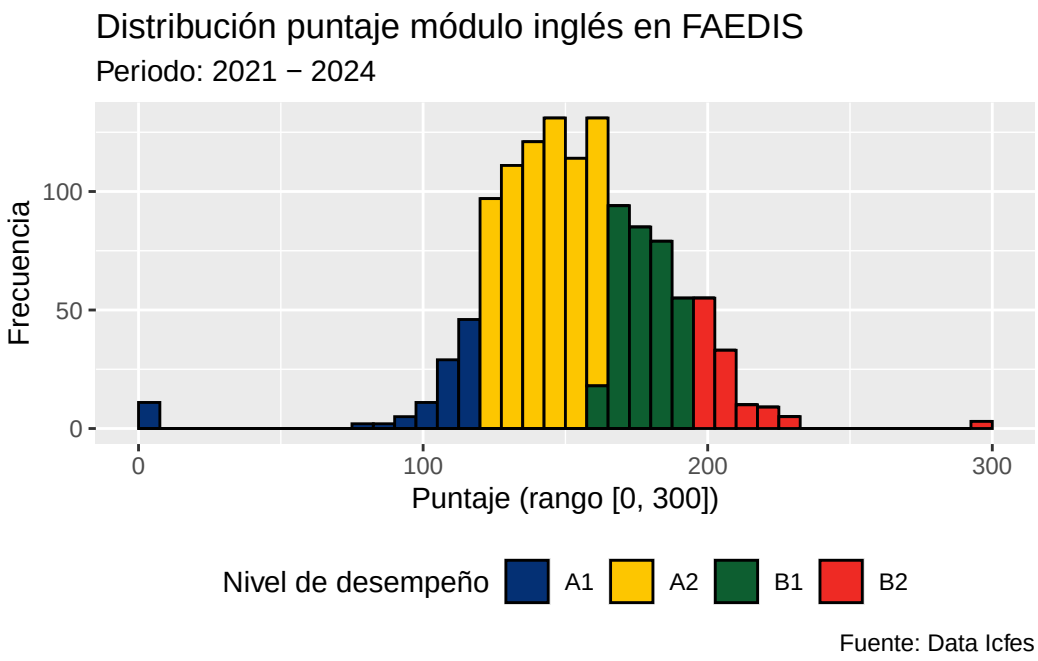


Figura 7.1.: Resultado final de un histograma utilizando `ggplot2`

7.2. Datos, mapeo y capas

Para generar la Figura 7.1 es necesario en primera instancia cargar los paquetes que se requieran utilizar y posteriormente importar los datos para explorarlos en R como se señala a continuación, teniendo en cuenta lo indicado en la Nota 13:

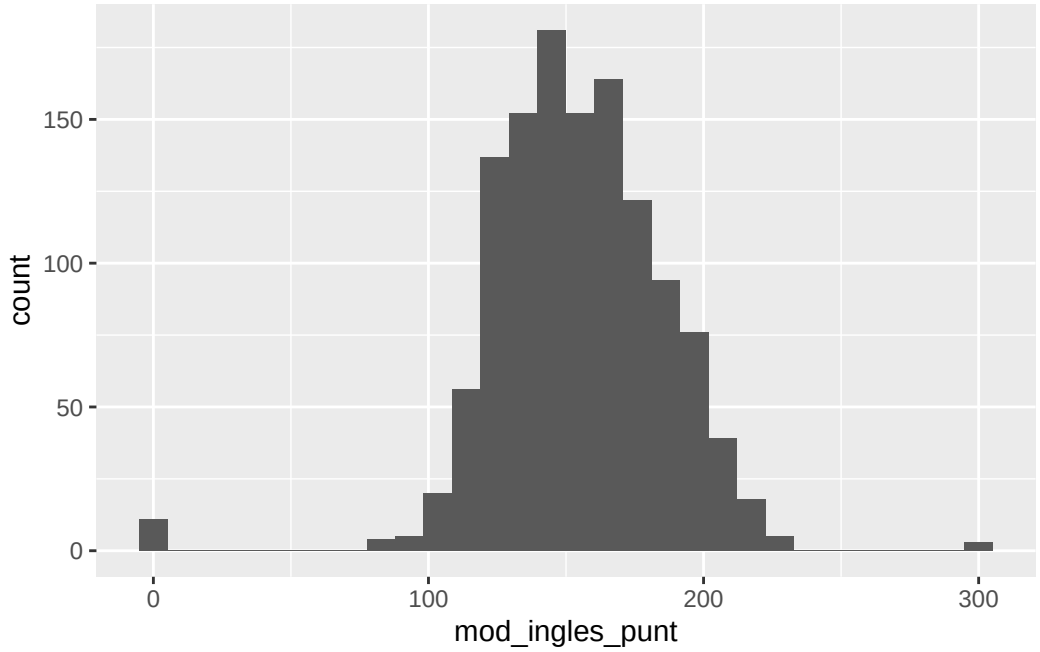
```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
saber_pro_genericas_faedis_2021_2024 <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis.rds"
)
```

Luego, se puede utilizar la función `ggplot()` y el parámetro `data`, asociando la coordenada `x` a la variable `mod_ingles_punt` para definir el mapeo así como el operador `+` junto con la función `geom_histogram()` para visualizar la distribución de una variable numérica, tal como se señala a continuación:

```
`stat_bin()` using `bins = 30`. Pick better value
`binwidth`.
```

Listado 7.1 Uso de datos, mapeo y una geometría para ilustrar gráficamente la distribución de una variable numérica

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram()
```



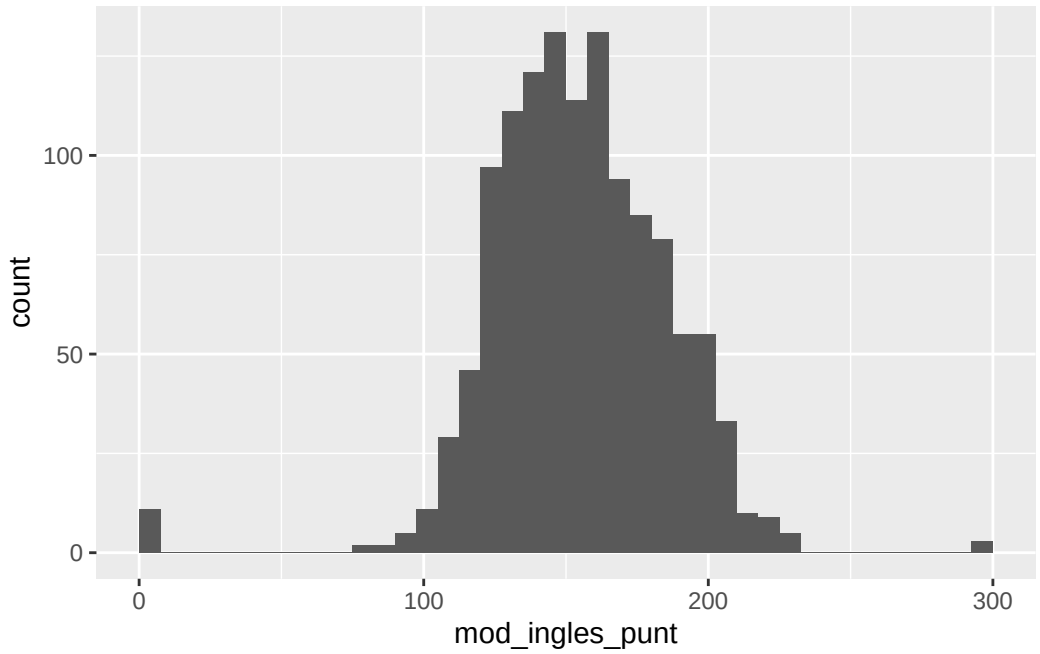
Para visualizar una distribución de una variable numérica se requieren definir, de manera ex-ante, determinados argumentos para un conjunto de parámetros. Por ejemplo, al ejecutar el código del Listado 7.1 se genera un mensaje indicando el número de contenedores (ver `stat_bin() using bins = 30`) que por defecto se utiliza para generar el histograma y solicitando al usuario seleccionar un determinado ancho del contenedor (ver `Pick better value binwidth`). En la literatura se han propuesto diferentes criterios (ver Nota 14) para determinar el número de contenedores o el ancho de cada contenedor pero en el caso del paquete `ggplot2` esta decisión se traslada al usuario respecto a qué valor finalmente desea fijar.

> **Nota 14**
>
> En la sección Number of bins and width del artículo de Wikipedia relacionado con el término histogram se puede consultar la lista de criterios más conocidos que se han propuesto para definir el número de contenedores o el ancho de cada contenedor en un histograma.

Para fines ilustrativos se fijará el parámetro `bins` con el argumento `40`, como `bins = 40`, con el propósito de recrear la Figura 7.1 donde en el Capítulo 7 no se ahondará en los criterios señalados en la Nota 14. También si se examina el gráfico generado por el Listado 7.1 el primer contenedor incluye valores negativos. Sin embargo, el rango de la variable `mod_ingles_punt` es $[0, 300]$. Para mejorar este aspecto existe el parámetro `boundary` que se utiliza para anclar el borde inicial a partir del cual el número de contenedores se generan de izquierda a derecha. En este caso podemos utilizar este parámetro con el argumento `0`, como `boundary = 0`. Aplicando estos aspectos como se muestra a continuación podemos mejorar el histograma relacionado con el Listado 7.1 y eliminar el mensaje que inicialmente se señalaba respecto al número de contenedores que se fijaba por defecto:

Listado 7.2 Definición del número de contenedores y anclaje inicial de bordes

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram(
    boundary = 0,
    bins = 40
  )
```



7.2.1. Aplicación de colores

En el gráfico generado como resultado del Listado 7.2 es difícil identificar de manera individual cada uno de los contenedores debido a que no existe un color aplicado a los bordes de cada uno de ellos dentro del histograma. El paquete `ggplot2` incluye la posibilidad de agregar colores a este tipo de geometría a través de los atributos estéticos `color` y `fill`. En `ggplot2` existen 2 maneras de agregar colores:

1. Fijar un valor a los atributos estéticos `color` o `fill` por fuera de `aes()` pero dentro de `geom_histogram()`
2. Asociar una variable a los atributos estéticos `color` o `fill` dentro de `aes()` en `geom_histogram()`

En el primer caso este valor se puede inicialmente especificar a través de un nombre donde R cuenta con 657 colores predefinidos, los cuales se pueden listar mediante la función `colors()` o `colours()`. También es posible utilizar una representación hexadecimal con un carácter de la forma `#rrggbb`, donde cada uno de los pares `rr`, `gg` y `bb` consta de 2 dígitos hexadecimales que representan un valor en el rango de `00` a `ff` (ver Nota 15).

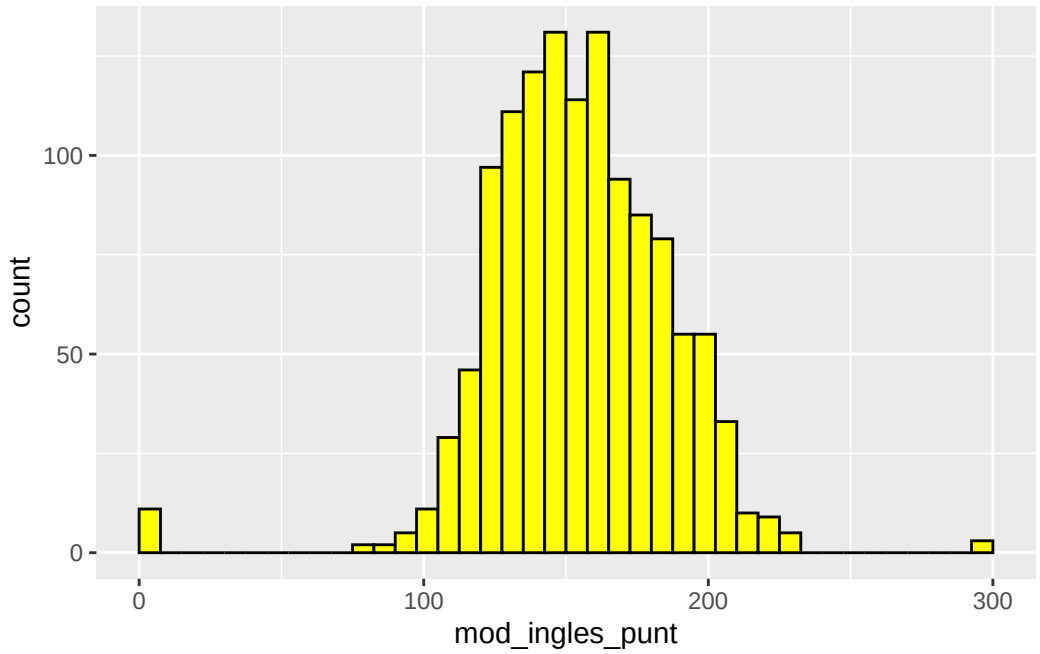
> **Nota 15**
>
> Una representación hexadecimal se especifica mediante el formato `#rrggbb`, donde los enteros hexadecimales `rr` (red o rojo), `gg` (green o verde) y `bb` (blue o azul)

definen los componentes del color. En el artículo de W3Schools titulado HTML HEX Colors el lector puede explorar con más detalle y de manera interactiva la representación de un color determinado utilizando un valor hexadecimal.
Debido a que los dígitos en un sistema de numeración hexadecimal corresponden a $0, 1, \ldots, 9$ más $a, b, \ldots, f$ y por el uso de 2 dígitos para cada par `rr`, `gg` y `bb`, existen en total $(16 \times 16) \times (16 \times 16) \times (16 \times 16) = 16,777,216$ posibles colores bajo esta representación. En particular `#000000` representa la ausencia total de luz en `rr`, `gg` y `bb` donde corresponde a un negro absoluto y `#ffffff` representa la intensidad máxima permitida en `rr`, `gg` y `bb` donde corresponde a un blanco absoluto.

Para ilustrar los anteriores aspectos utilizando como insumo el código del Listado 7.2, se hará uso de los nombres de los colores `black` y `yellow` como argumentos para los parámetros `color` y `fill` utilizando el primer caso, tal como se señala a continuación:

Listado 7.3 Asignación de colores por fuera de `aes()` para los atributos estéticos `color` y `fill` en una geometría asociada con un histograma

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram(
    boundary = 0,
    bins = 40,
    color = "black",
    fill = "yellow"
  )
```

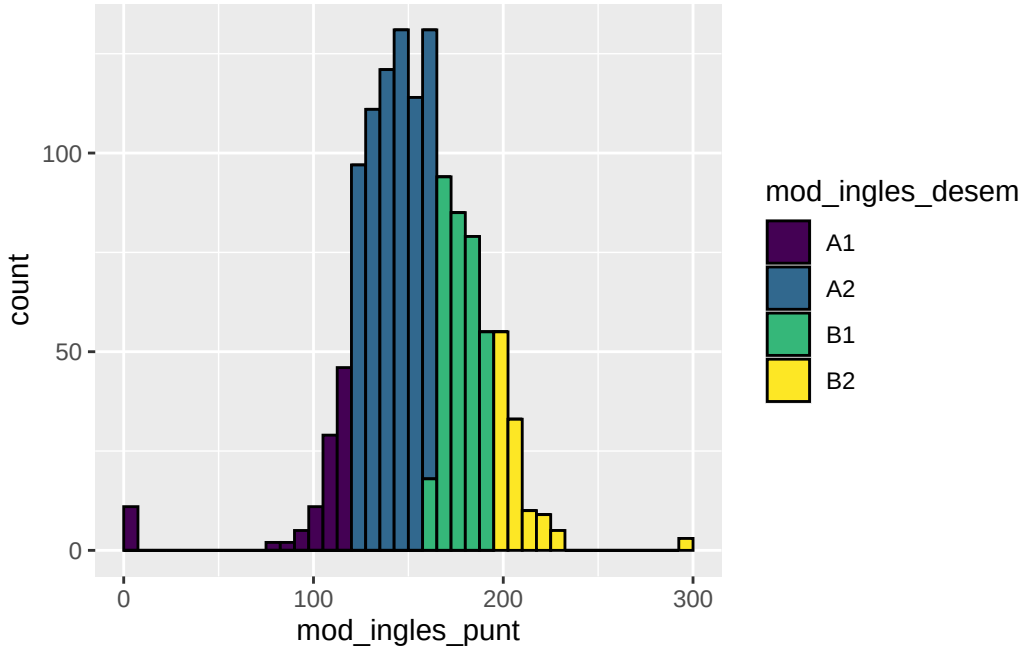


7.3. Asociación de colores a una variable categórica

Para entender el segundo caso señalado en la Sección 7.2.1 respecto a la asignación de colores, en esta sección se realizará una modificación en el código del Listado 7.3 para visualizar los niveles de desempeño del módulo de inglés utilizando la variable `mod_ingles_desem`. De esa manera se complementa la visualización del puntaje numérico obtenido con una descripción cualitativa de las habilidades y conocimientos en inglés. Para incluir este aspecto es necesario asociar el parámetro `fill` con el argumento `mod_ingles_desem` dentro de la función `geom_histogram()` a través de `aes()`, tal como se señala a continuación:

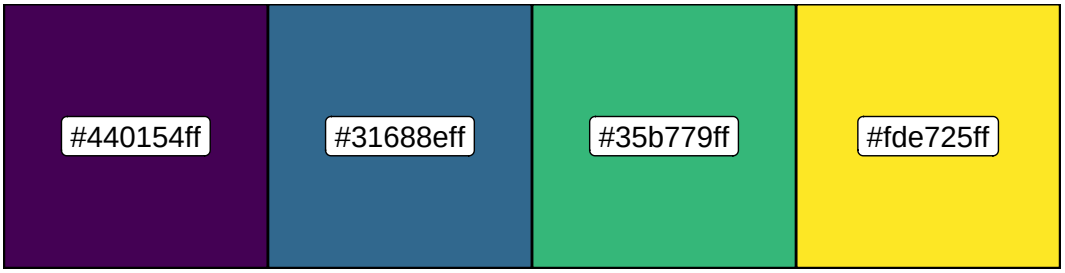
Listado 7.4 Asignación de colores dentro de `aes()` para el atributo estético `fill` en una geometría asociada con un histograma

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram(
    mapping = aes(fill = mod_ingles_desem),
    boundary = 0,
    bins = 40,
    color = "black"
  )
```



Como resultado de la ejecución del código del Listado 7.4 se genera una visualización tanto del puntaje numérico como de los niveles de desempeño del módulo de inglés. Este aspecto es posible incluirlo debido a que la función `geom_histogram()` contiene un ajuste de posición (ver Sección 6.2.3.3) a través del parámetro `position` con el argumento `"stack"`, como `position = "stack"`. Si se consulta la documentación de esta función (`?geom_histogram`) en la sección **Usage**, se encontrará que esta es la configuración por defecto que se utiliza. Es decir, el ajuste de posición permite apilar contenedores cuando se agrupan los valores por una variable categórica como `mod_ingles_desem` y existen diferentes grupos que están incluidos dentro de un mismo contenedor.

Adicionalmente, en el gráfico generado por el código del Listado 7.4 `ggplot2` utiliza una paleta de colores por defecto cuando se asocia el parámetro `fill` con el argumento `mod_ingles_desem`. En particular, la paleta de colores que utiliza `ggplot2` para una variable categórica ordenada con 4 valores como `mod_ingles_desem` es:



7.4. Escalas

La paleta de colores utilizada por defecto en `ggplot2` y señalada en la Sección 7.3 se puede modificar y personalizar haciendo uso de una representación hexadecimal (ver Nota 15) para el conjunto de colores alternativos que se busquen aplicar. En particular, en la Universidad Militar Nueva Granada (UMNG) a través del Manual de Identidad Visual se estableció una paleta de colores primaria que contiene 2 colores institucionales y 4 colores complementarios, donde se encuentra dada por:

Figura 7.2.: Paleta de colores primaria de la Universidad Militar Nueva Granada (UMNG) de acuerdo al Manual de Identidad Visual

De esa manera es posible definir un vector de caracteres que contenga la representación hexadecimal de cada color señalado en la Figura 7.2 como se señala a continuación y posteriormente seleccionar los primeros 4 colores:

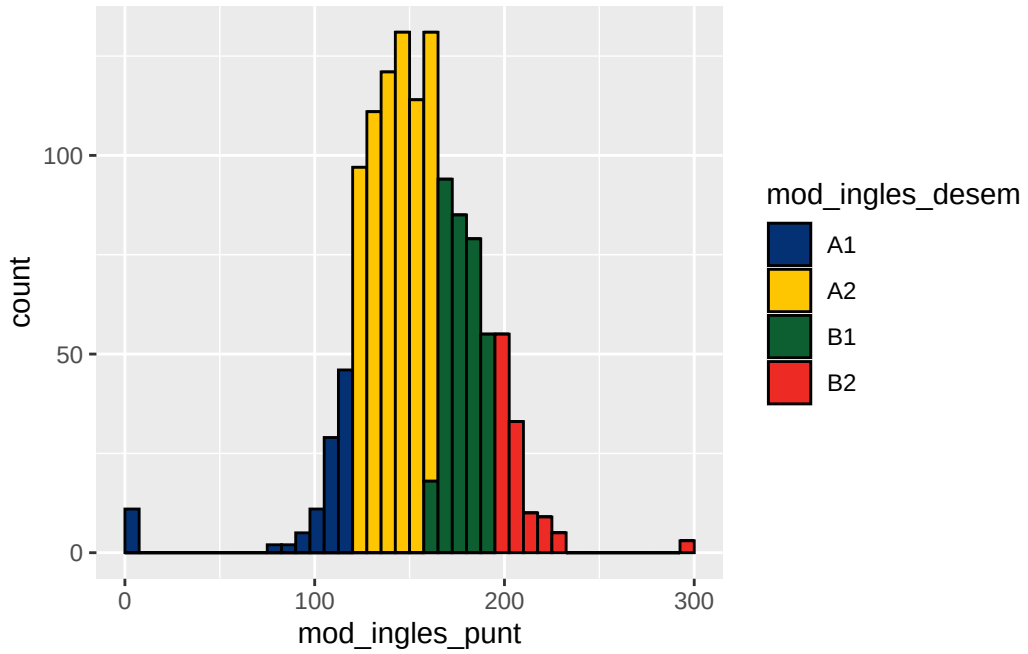
```
# Paleta primaria UMNG ----
paleta_primaria_umng <- c(
  "#043074",
  "#fdc600",
```

```
  "#0d5e30",
  "#ee2a24",
  "#000000",
  "#ffffff"
)
```

Para modificar la paleta de colores por defecto que utiliza `ggplot2` respecto al gráfico generado por el código señalado en el Listado 7.4 se puede utilizar el operador `+` junto con la función `scale_fill_manual()`. Esta función a través del parámetro `values` permite fijar un vector de caracteres con la representación hexadecimal de los colores que se busquen utilizar. De esa manera, se pueden seleccionar los primeros 4 colores de `paleta_primaria_umng` y agregar los aspectos mencionados al código del Listado 7.4, tal como se señala a continuación:

Listado 7.5 Personalización de la paleta de colores asociada al atributo estético `fill` dentro de `aes()` en una geometría

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram(
    mapping = aes(fill = mod_ingles_desem),
    boundary = 0,
    bins = 40,
    color = "black"
  ) +
  scale_fill_manual(
    values = paleta_primaria_umng[1:4]
  )
```

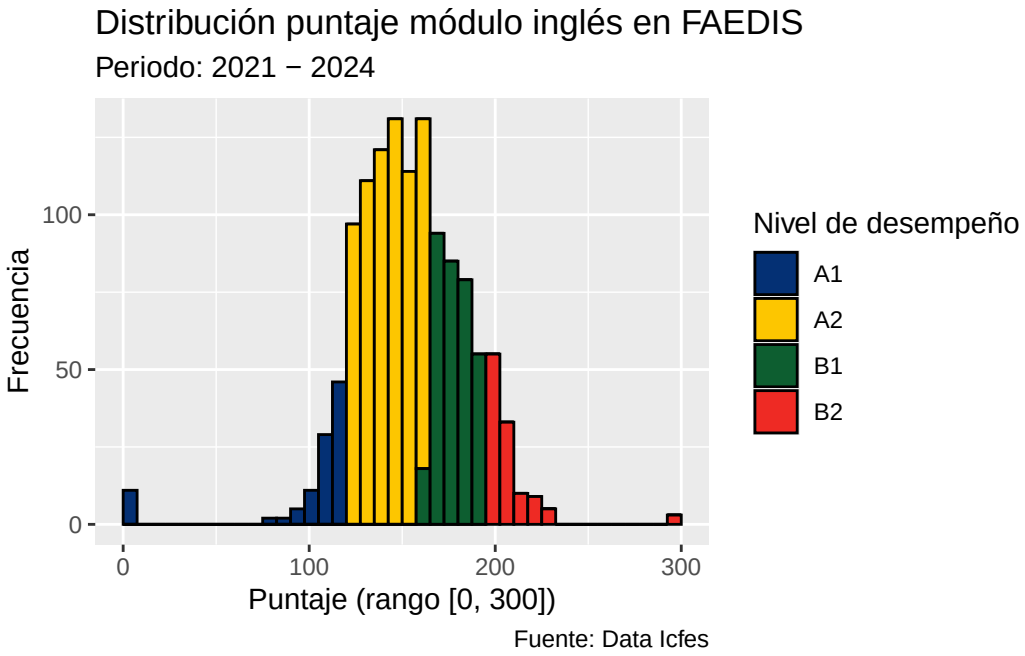


En el Listado 7.5, la expresión `1:4` genera un vector con la secuencia de números `1, 2, 3, 4`. Al combinar esta secuencia con los corchetes, `[]`, de la forma `paleta_primaria_umng[1:4]`, se realiza una extracción por posición que recupera los primeros 4 elementos del vector original para asignarlos directamente al parámetro `values` de la función `scale_fill_manual()`.

Adicionalmente, se pueden eliminar o modificar diferentes etiquetas en el gráfico para incluir información de contexto de tal manera que sea más comprensible mediante el operador `+` junto con la función `labs()`, tal como se señala a continuación:

Listado 7.6 Uso de la función `labs()` para agregar, modificar o eliminar etiquetas en un gráfico

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram(
    mapping = aes(fill = mod_ingles_desem),
    color = "black",
    boundary = 0,
    bins = 40
  ) +
  scale_fill_manual(
    values = paleta_primaria_umng[1:4]
  ) +
  labs(
    x = "Puntaje (rango [0, 300])",
    y = "Frecuencia",
    fill = "Nivel de desempeño",
    title = "Distribución puntaje módulo inglés en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  )
```



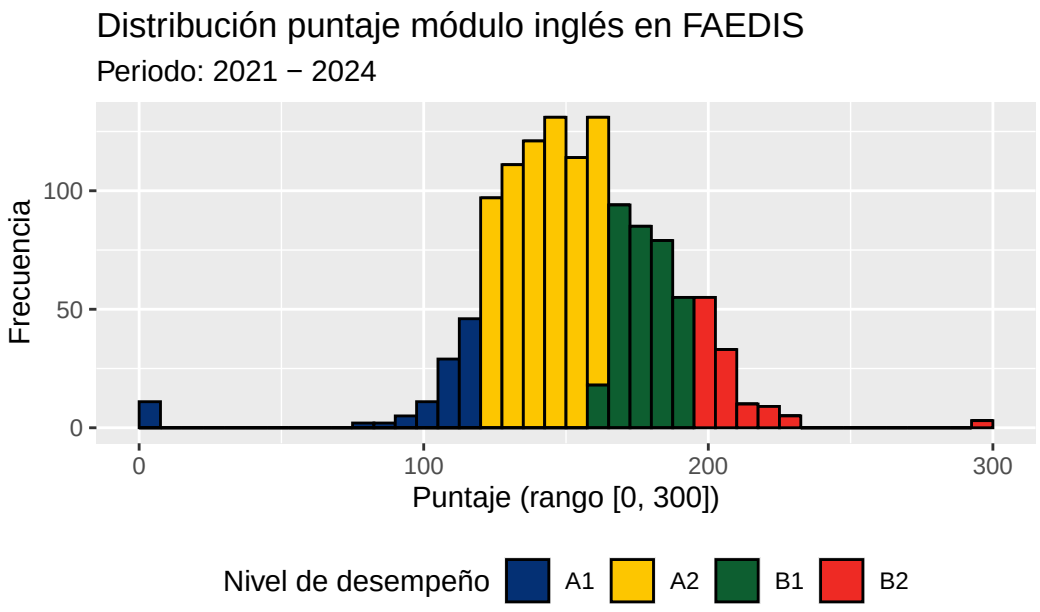
En el Listado 7.6 utilizando el parámetro `fill` dentro de la función `labs()` es posible modificar el título de la leyenda que señala los valores de la variable categórica `mod_ingles_desem` cuando se asocia al atributo estético `fill` dentro de la función `aes()` en la geometría `geom_histogram`. Respecto a los demás parámetros con sus respectivos argumentos que se incluyeron en la función `labs()`, son elementos que se aplicaron de manera similar a lo señalado en el Listado 6.9.

7.5. Tema

Un aspecto que es posible mejorar dentro del gráfico generado por el Listado 7.6 es el foco de atención hacia el histograma, sin perder la información generada por la leyenda que ilustra los niveles de desempeño a través de la variable `mod_ingles_desem`.

Para modificar este aspecto es necesario personalizar los componentes del gráfico que no están vinculados a los datos donde se utiliza la función `theme()`. Esta función tiene un conjunto amplio de parámetros que pueden modificar diferentes aspectos del gráfico. Sin embargo, el objetivo no es abarcar cada uno de ellos.

En esta última sección del Capítulo 7 modificarás la posición de la leyenda que contiene la información de la variable `mod_ingles_desem` de tal manera que se pueda extender el ancho del gráfico. Para aplicar este cambio se utilizará el parámetro `legend.position` junto con el argumento `"bottom"`, como `legend.position = "bottom"`, dentro de la función `theme()` agregando este aspecto con el operador `+`, tal como se señala a continuación:



Fuente: Data Icfes

Listado 7.7 Uso de la función `theme()` para modificar la posición de una leyenda

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(x = mod_ingles_punt)
) +
  geom_histogram(
    mapping = aes(fill = mod_ingles_desem),
    color = "black",
    boundary = 0,
    bins = 40
  ) +
  scale_fill_manual(
    values = paleta_primaria_umng[1:4]
  ) +
  labs(
    x = "Puntaje (rango [0, 300])",
    y = "Frecuencia",
    fill = "Nivel de desempeño",
    title = "Distribución puntaje módulo inglés en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  ) +
  theme(
    legend.position = "bottom"
  )
```

De esa manera con el código señalado en el Listado 7.7, es posible recrear finalmente la Figura 7.1 para ilustrar la distribución de la variable `mod_ingles_punt` junto con los niveles de desempeño representados en `mod_ingles_desem` para el módulo de inglés.

8. Proporciones

Este capítulo se basa en (Wickham, 2016) y (Wilke, 2019) así como en la documentación oficial de `ggplot2` y `scales` donde tiene como propósito utilizar el modelo mental de la gramática de gráficos en capas para visualizar proporciones para un conjunto determinado de grupos utilizando el paquete `ggplot2` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de la Facultad de Estudios a Distancia (FAEDIS) de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. ¿Cómo visualizar la composición de un determinado grupo donde cada parte representa una proporción?
2. ¿Cómo ordenar las etiquetas correspondientes al eje de un gráfico con base en otra variable?
3. ¿Cómo controlar el formato de las etiquetas de los ejes en un gráfico?

> **Nota 16**
>
> Para desarrollar el contenido de este capítulo es necesario que hayas completado la descarga y ubicación del archivo `2021-2024_saber-pro-genericas_faedis.rds` dentro de la carpeta `data`, respecto a lo descrito en la Nota 11.
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `008_visualizar-proporciones-en-r` y dentro de ella genera el archivo `001_visualizar-proporciones-datos-faedis.R`, siguiendo las pautas de la Sección 4.4.

8.1. Gráfico de barras apiladas y normalizadas

En esta sección realizarás un gráfico de barras apiladas y normalizadas en el rango $[0, 1]$, señalando la proporción de inscritos por género al examen Saber Pro para cada programa de pregrado de la Facultad de Estudios a Distancia de la Universidad Militar en el periodo 2021-2024, tal como se indica en la Figura 8.1.

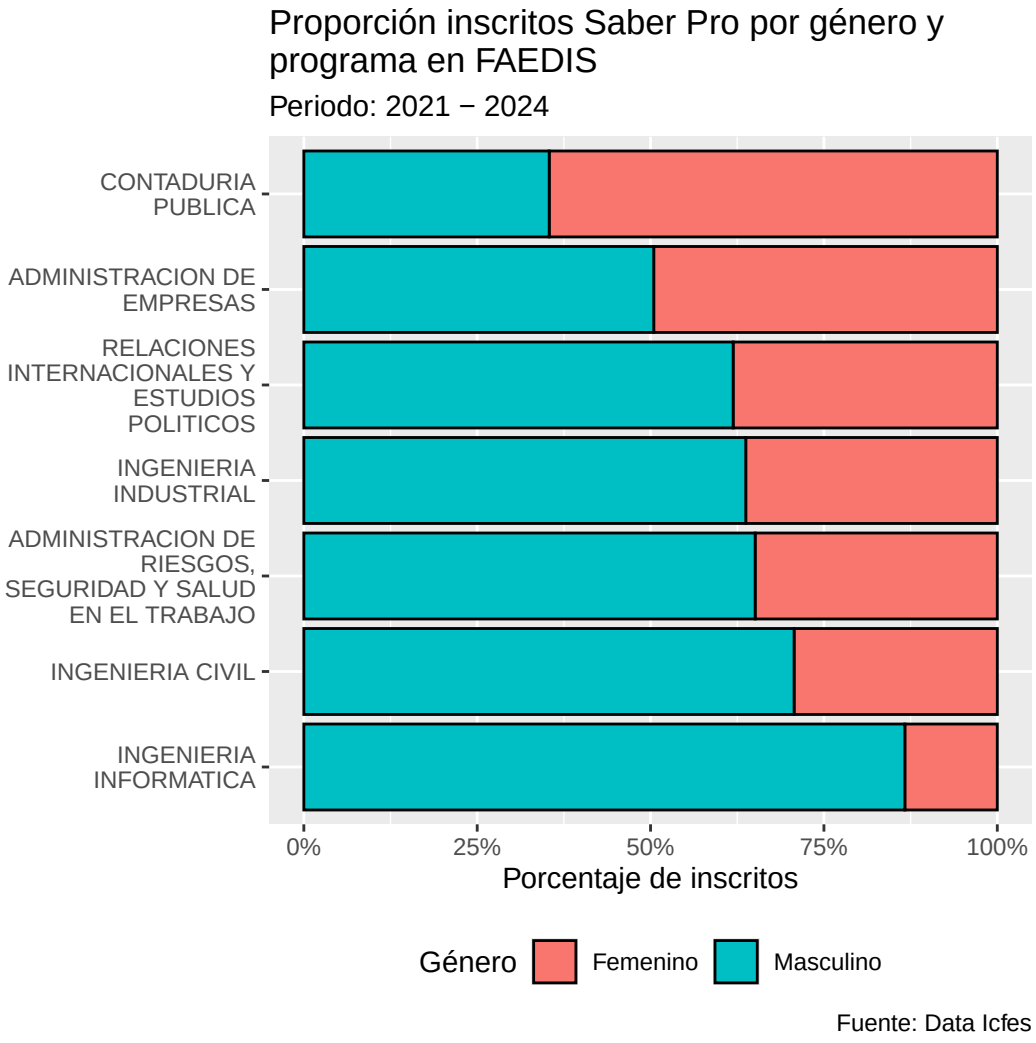


Figura 8.1.: Resultado final de un gráfico de barras apiladas y normalizadas en el rango $[0, 1]$ utilizando `ggplot2`

8.2. Datos, mapeo, capas y escalas

Para generar la Figura 8.1 es necesario en primera instancia cargar los paquetes que se requieran utilizar y posteriormente importar los datos para explorarlos en R como se señala a continuación, teniendo en cuenta lo indicado en la Nota 16:

```
# Cargar paquetes ----
library(tidyverse)
library(scales)

# Importar datos ----
saber_pro_genericas_faedis_2021_2024 <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis.rds"
)
```

En la Sección 5.3.1 respecto al Listado 4.1 se señaló que al utilizar `library(tidyverse)` se cargaba un conjunto de paquetes. Estos paquetes conforman el núcleo del `tidyverse`. Sin embargo, al instalar el paquete `tidyverse` (ver Sección 4.2), se incluye un conjunto adicional de paquetes con un uso más específico. Estos paquetes por fuera del núcleo del `tidyverse` no se cargan al ejecutar `library(tidyverse)`, donde en caso que se requieran utilizar se necesitan cargar por separado utilizando la función `library()`.

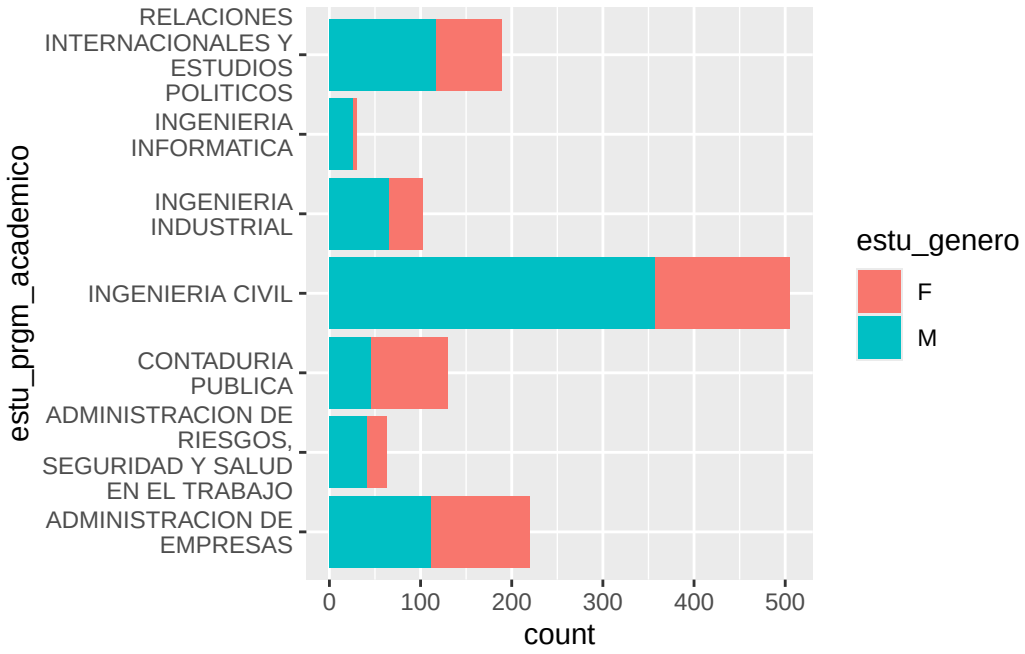
Uno de estos paquetes por fuera del núcleo del `tidyverse` es `scales`. Este paquete contiene herramientas para modificar los cortes, las etiquetas y aplicar transformaciones a los ejes de un gráfico así como especificar otras paletas de colores. En el Capítulo 8 utilizarás el paquete `scales` para controlar el formato de las etiquetas del eje asociado con la coordenada `x`.

8.2.1. Gráfico de barras apiladas

Una vez se carguen los paquetes que se utilizarán y se importen los datos, es posible utilizar las ideas desarrolladas en el Listado 6.5 así como el atributo estético `fill` junto con la variable `estu_genero` como `fill = estu_genero` dentro de `aes()` en la función `geom_bar()`. También se puede aplicar el ajuste mencionado en la Sección 6.2.4 respecto a la longitud de las etiquetas en el eje asociado con la coordenada `y`, tal como se señala a continuación:

Listado 8.1 Gráfico de barras apiladas

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = estu_prgm_academico)
) +
  geom_bar(mapping = aes(fill = estu_genero)) +
  scale_y_discrete(labels = label_wrap_gen(width = 18))
```



8.2.2. Normalización

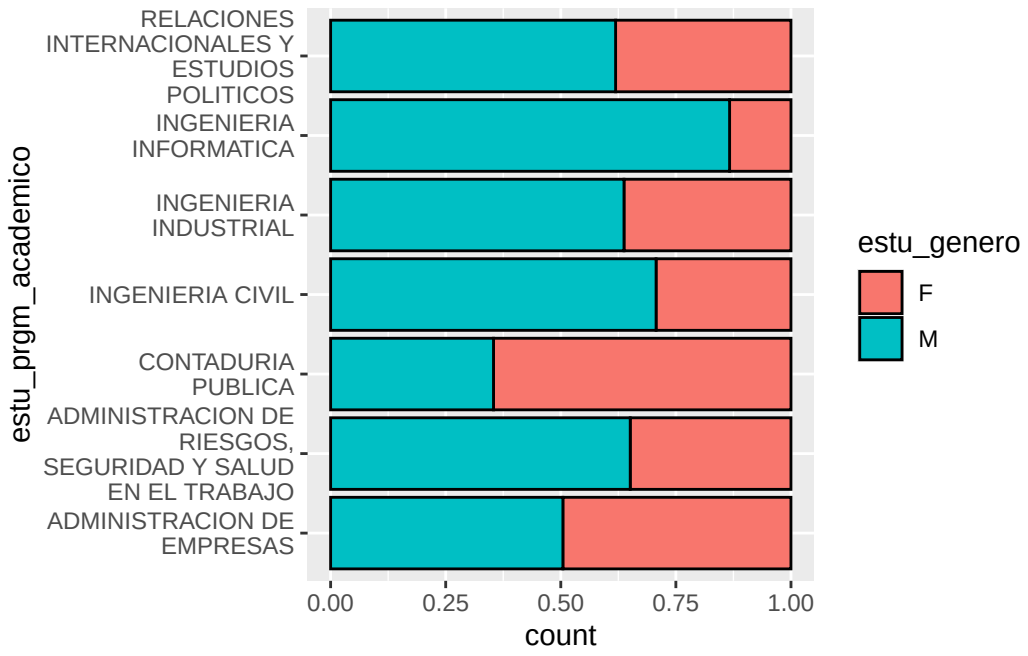
En el gráfico generado por el código del Listado 8.1 es difícil comparar la proporción de inscritos por género para cada programa académico debido a que se realiza un conteo absoluto. Para aislar el tamaño total de cada grupo y sus partes de tal manera que se pueda revisar la composición por género se puede aplicar una normalización en el rango $[0, 1]$ de tal manera que cada barra tenga una longitud de 1. Matemáticamente, este proceso divide el conteo de cada género entre el total de inscritos de su respectivo programa académico.

Para aplicar una normalización en el rango $[0, 1]$ en `ggplot2` es necesario modificar la posición dentro de la función `geom_bar()` debido a que esta geometría utiliza por defecto el parámetro `position` con el argumento `"stack"`, como `position = "stack"`. Si se

cambia el argumento `"stack"` por `"fill"` se pueden mantener las barras apiladas y que tengan además una longitud constante. Además es posible fijar el atributo estético `color` por fuera de `aes()` pero dentro de `geom_bar()` con el argumento `"black"` para resaltar la composición interna de cada grupo por género, tal como se señala a continuación:

Listado 8.2 Gráfico de barras apiladas y normalizadas en el rango $[0, 1]$

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(y = estu_prgm_academico)
) +
  geom_bar(
    mapping = aes(fill = estu_genero),
    color = "black",
    position = "fill"
  ) +
  scale_y_discrete(labels = label_wrap_gen(width = 18))
```



8.2.3. Ordenamiento de etiquetas

El gráfico generado en el Listado 8.2 presenta las etiquetas del eje asociado a la coordenada `y`, ordenadas de forma alfabética. Al igual que en la Sección 6.2.3.1, es fundamental

ordenarlas de manera lógica para facilitar la comparación de los programas académicos según la composición de sus inscritos por género. Por lo tanto, se utilizará la proporción de inscritos del género femenino (`F`) como criterio para reordenar dichas etiquetas.

Para aplicar este ordenamiento utilizando `ggplot2` y comprender su lógica, es necesario reducir el análisis a un conjunto más pequeño de datos. A modo de ejemplo, se pueden extraer los primeros 10 valores de la columna `estu_genero` utilizando el operador `$` (ver Sección 4.4) y una extracción por posición (ver Sección 7.4), tal como se señala a continuación:

```
estu_genero_muestra <- saber_pro_genericas_faedis_2021_2024$estu_genero[1:10]
estu_genero_muestra
```

```
 [1] M M M M F F F F F F
Levels: F M
```

Luego, utilizando el operador relacional `==` (ver Sección 5.5), es posible identificar cuáles valores en `estu_genero_muestra` corresponden al género femenino (`F`):

```
estu_genero_muestra == "F"
```

```
 [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE
[10]  TRUE
```

En ese sentido, al evaluar si cada elemento en `estu_genero_muestra` es igual a `"F"`, R devuelve un vector de tipo lógico. De este modo, cada posición donde se cumpla la condición se transformará en `TRUE`, mientras que, en caso contrario, se le asignará un `FALSE`.

Por otra parte, en R los vectores lógicos experimentan una coerción automática hacia vectores enteros en contextos donde se requiere un valor numérico, mapeando `TRUE` a `1` y `FALSE` a `0`. Es posible aprovechar este comportamiento calculando la media aritmética de la condición `estu_genero_muestra == "F"` mediante la función `mean()`, lo cual nos devuelve, por definición matemática, la proporción exacta de valores iguales a `"F"`:

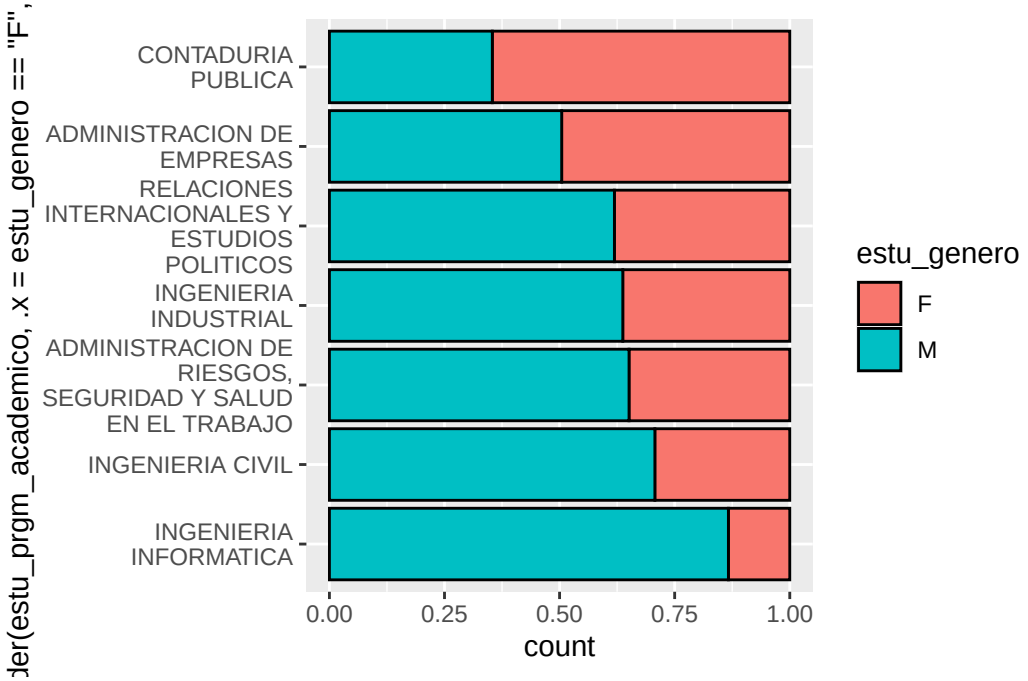
```
mean(estu_genero_muestra == "F")
```

```
[1] 0.6
```

Listado 8.3 Ordenamiento de etiquetas de un eje asociado a una coordenada utilizando la función `fct_reorder()`

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    y = fct_reorder(
      estu_prgm_academico,
      .x = estu_genero == "F",
      .fun = mean
    )
  )
) +
  geom_bar(
    mapping = aes(fill = estu_genero),
    color = "black",
    position = "fill"
  ) +
  scale_y_discrete(labels = label_wrap_gen(width = 18))
```

El proceso descrito anteriormente puede aplicarse a la columna `estu_prgm_academico` en el conjunto de datos `saber_pro_genericas_faedis_2021_2024` haciendo uso de la función `fct_reorder()`. Esta función permite ordenar un vector de caracteres con base en otra variable y haciendo uso de una función que corresponda usualmente a una estadística descriptiva, tal como se señala a continuación:



En el Listado 8.3 se utiliza la función `fct_reorder()` para reordenar `estu_prgm_academico` con base en la variable auxiliar `estu_genero == "F"` asociada al parámetro `.x`, y haciendo uso de la función `mean` asociada al parámetro `.fun`. Internamente, `ggplot2` junto con `fct_reorder()` genera un vector lógico con base en la condición `estu_genero == "F"` y calcula la media aritmética para cada programa académico, lo que en este caso es equivalente a la proporción de inscritos del género femenino para cada grupo. Con base en esta estadística descriptiva, R reordena las etiquetas del eje asociadas a la coordenada `y`.

8.2.4. Modificación de etiquetas

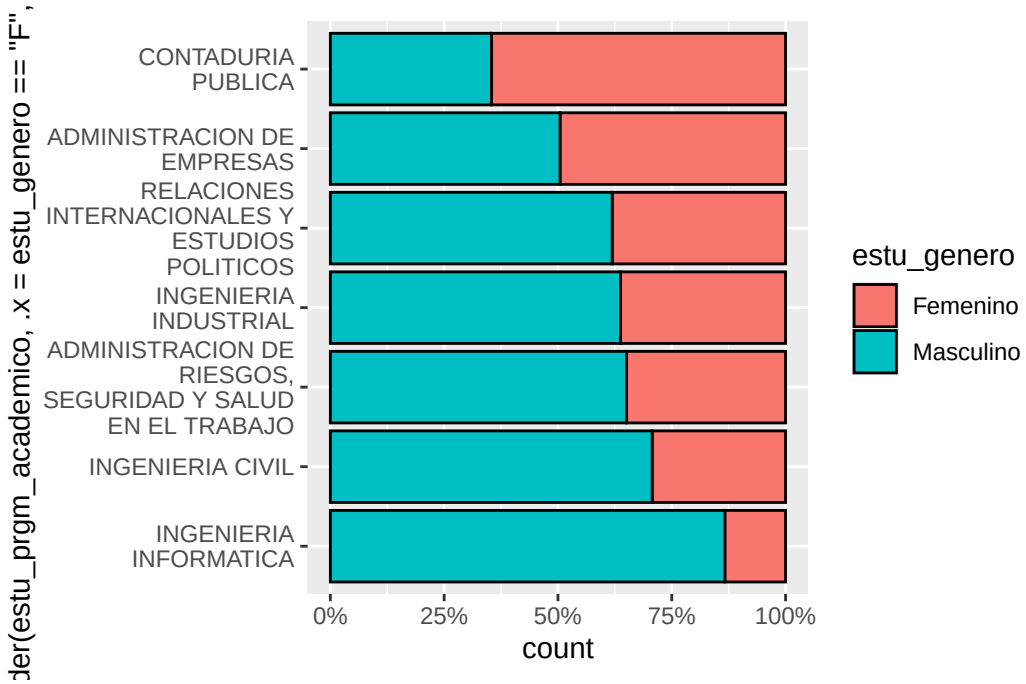
En el gráfico generado por el Listado 8.3 es posible generar etiquetas que especifiquen de manera más clara la información que se busca indicar. Este aspecto se puede aplicar de 2 maneras:

1. Modificando las escalas de manera directa a través de las funciones `scale_x_continuous()` y `scale_fill_discrete()`
2. Agregando etiquetas adicionales o modificando ciertos títulos con la función `labs()`

Respecto al primer aspecto utilizando el paquete `scales()` es posible utilizar funciones auxiliares como `label_percent()` para presentar los cortes en el eje asociado con la coordenada `x` en términos porcentuales. También se pueden modificar los valores `"F"` y `"M"` asociados con la leyenda para señalar las etiquetas del género por `"Femenino"` y `"Masculino"`, tal como se señala a continuación:

Listado 8.4 Modificación de las etiquetas en los cortes en el eje asociado con la coordenada `x` y las categorías dentro de la leyenda

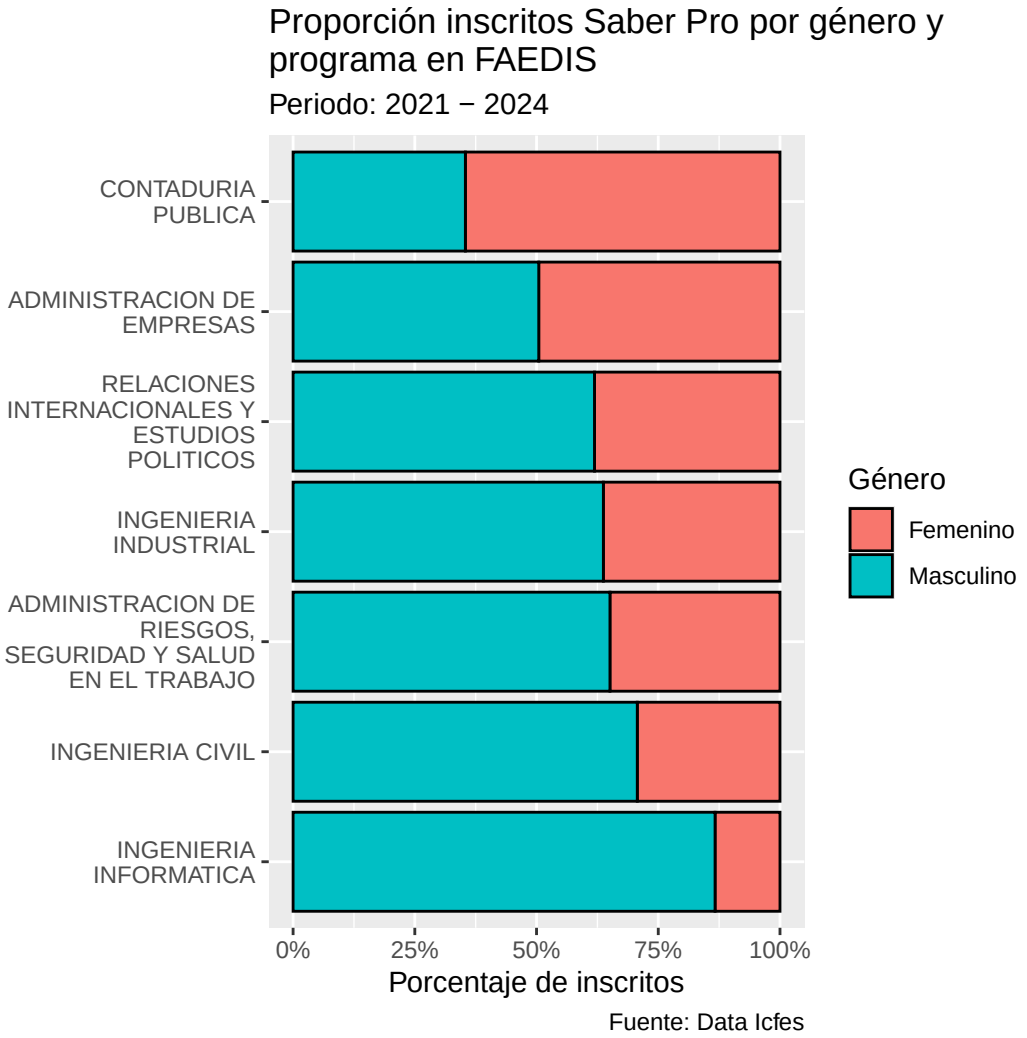
```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    y = fct_reorder(
      estu_prgm_academico,
      .x = estu_genero == "F",
      .fun = mean
    )
  )
) +
  geom_bar(
    mapping = aes(fill = estu_genero),
    color = "black",
    position = "fill"
  ) +
  scale_y_discrete(labels = label_wrap_gen(width = 18)) +
  scale_x_continuous(
    labels = label_percent()
  ) +
  scale_fill_discrete(
    labels = c("Femenino", "Masculino")
  )
```



En el paquete `scales` existe un conjunto de funciones auxiliares que se utilizan con el parámetro `labels` dentro de `ggplot2` para controlar el formato de las etiquetas en los ejes y las leyendas. En el Listado 8.4 dentro de la función `scale_x_continuous()` se incluyó la función `label_percent()` como argumento del parámetro `labels` para expresar los cortes como porcentajes.

También para que exista claridad respecto a las categorías de la leyenda asociada con el género, a través del parámetro `labels` dentro de la función `scale_fill_discrete()`, se utilizó como argumento un vector de caracteres que describiera mejor los colores asociados con el gráfico. El uso de la función `scale_fill_discrete()` se utiliza debido a que se requiere modificar un aspecto relacionado con el atributo estético `fill` y las categorías asociadas con el género toman valores discretos (`discrete` en inglés).

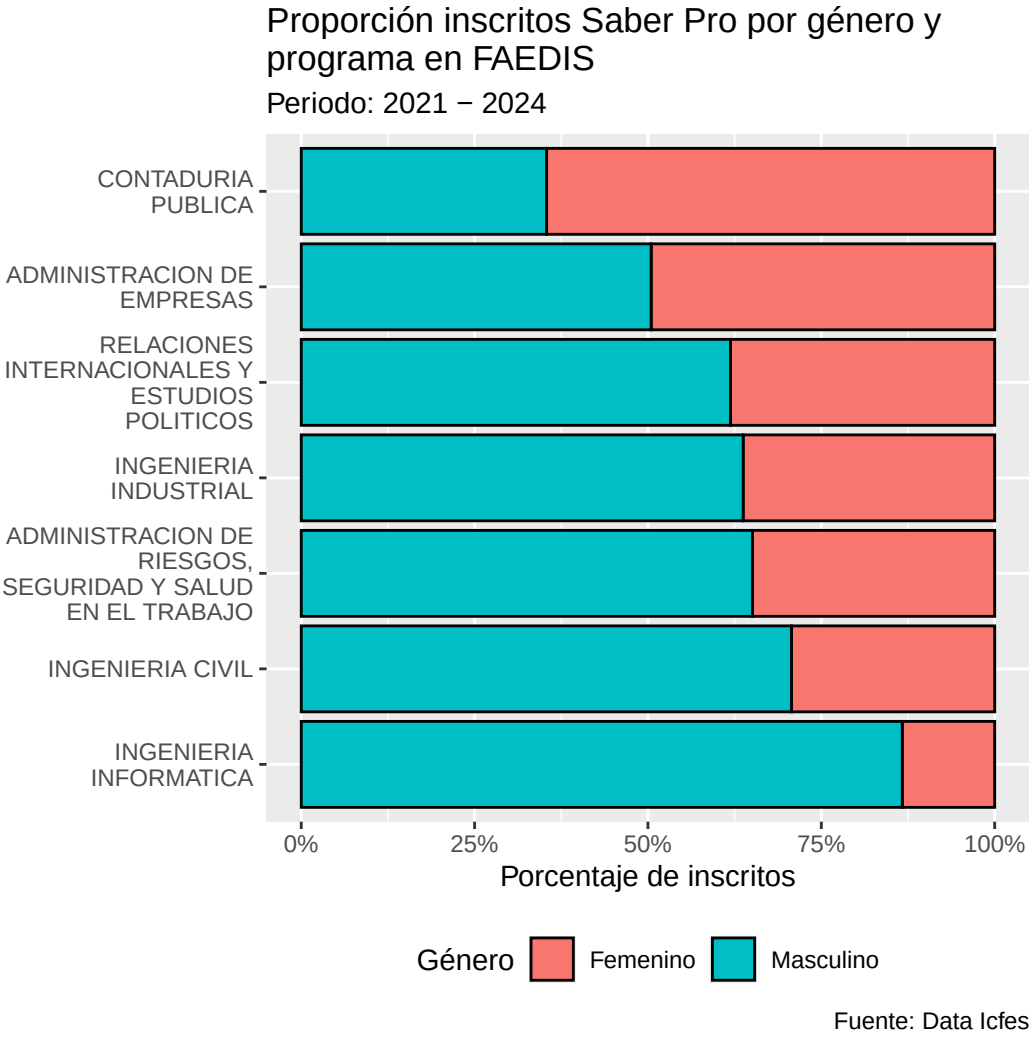
En relación al segundo aspecto se pueden eliminar o modificar diferentes etiquetas en el gráfico para incluir información de contexto mediante la función `labs()` de manera similar a como se procedió en el Listado 6.9 y el Listado 7.6. Sin embargo, en este caso, adicionalmente se puede hacer uso de la función `str_wrap()` perteneciente al paquete `stringr`, que se incluye dentro del `tidyverse` (ver Sección 5.3.1), para controlar la longitud del título del gráfico, tal como se señala a continuación:



Una pregunta natural respecto al Listado 8.5 es por qué no utilizar la función `label_wrap_gen()` para controlar la longitud del título, tal como se aplicó en la función `scale_y_discrete()` para las etiquetas del eje asociadas con la coordenada `y`. La función `label_wrap_gen()` fue construida como una función auxiliar dentro del paquete `ggplot2` para modificar de manera directa un conjunto de etiquetas en las escalas. Por el contrario, la función `str_wrap()` del paquete `stringr` se diseñó para dividir un texto determinado en párrafos con un formato adecuado, donde el primer parámetro de la función, denominado `string`, debe corresponder a un vector de caracteres (ver `?str_wrap`).

8.3. Tema

En la Sección 7.5 se hizo uso de la función `theme()` utilizando el parámetro `legend.position` junto con el argumento `"bottom"`, como `legend.position = "bottom"`. De nuevo, es posible utilizar este mismo aspecto para expandir el área del gráfico, tal como se señala a continuación:



De esa manera con el código señalado en el Listado 8.6, es posible recrear finalmente la Figura 8.1 para ilustrar la proporción de inscritos por género y programa académico utilizando las variables `estu_prgm_academico` y `estu_genero`.

Listado 8.5 Modificación de las etiquetas en los cortes en el eje asociado con la coordenada `x` y las categorías dentro de la leyenda

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    y = fct_reorder(
      estu_prgm_academico,
      .x = estu_genero == "F",
      .fun = mean
    )
  )
) +
  geom_bar(
    mapping = aes(fill = estu_genero),
    color = "black",
    position = "fill"
  ) +
  scale_y_discrete(labels = label_wrap_gen(width = 18)) +
  scale_x_continuous(
    labels = label_percent()
  ) +
  scale_fill_discrete(
    labels = c("Femenino", "Masculino")
  ) +
  labs(
    x = "Porcentaje de inscritos",
    y = NULL,
    fill = "Género",
    title = str_wrap(
      "Proporción inscritos Saber Pro por género y programa en FAEDIS",
      width = 50
    ),
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  )
```

Listado 8.6 Uso de la función `theme()` para modificar la posición de una leyenda

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    y = fct_reorder(
      estu_prgm_academico,
      .x = estu_genero == "F",
      .fun = mean
    )
  )
) +
  geom_bar(
    mapping = aes(fill = estu_genero),
    color = "black",
    position = "fill"
  ) +
  scale_y_discrete(labels = label_wrap_gen(width = 18)) +
  scale_x_continuous(
    labels = label_percent()
  ) +
  scale_fill_discrete(
    labels = c("Femenino", "Masculino")
  ) +
  labs(
    x = "Porcentaje de inscritos",
    y = NULL,
    fill = "Género",
    title = str_wrap(
      "Proporción inscritos Saber Pro por género y programa en FAEDIS",
      width = 50
    ),
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  ) +
  theme(
    legend.position = "bottom"
  )
```

9. Relaciones x-y

Este capítulo se basa en (Wickham, 2016) y (Wilke, 2019) así como en la documentación oficial de `ggplot2` donde tiene como propósito utilizar el modelo mental de la gramática de gráficos en capas para visualizar asociaciones entre variables numéricas y la evolución de una variable continua a lo largo del tiempo utilizando el paquete `ggplot2` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de la Facultad de Estudios a Distancia (FAEDIS) de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. ¿Cómo visualizar asociaciones entre pares de variables continuas para identificar posibles patrones?
2. ¿Cómo visualizar diferentes subconjuntos de datos en paneles individuales?
3. ¿Cómo visualizar la evolución de una variable continua por grupos a lo largo del tiempo?
4. ¿Cómo modificar la distribución de las etiquetas dentro de una leyenda en un gráfico?

> **Nota 18**
>
> Para desarrollar el contenido de este capítulo es necesario que hayas completado la descarga y ubicación del archivo `2021-2024_saber-pro-genericas_faedis.rds` dentro de la carpeta `data`, respecto a lo descrito en la Nota 11. Adicionalmente, es necesario que descargues el archivo señalado a continuación y lo incluyas también dentro de la carpeta `data`:
>
> **2021-2024__saber-pro-genericas__faedis__programas.rds**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `009_visualizar-relaciones-x-y-en-r` y dentro de ella genera el archivo `001_visualizar-relaciones-x-y-datos-faedis.R`, siguiendo las pautas de la Sección 4.4.

9.1. Gráfico de puntos

En esta sección realizarás un gráfico de puntos señalando la asociación del puntaje del módulo de lectura crítica con el puntaje del módulo de razonamiento cuantitativo aplicando facetas para los programas de pregrado de la Facultad de Estudios a Distancia de la Universidad Militar en el periodo 2021-2024, tal como se indica en la Figura 9.1.

9.1.1. Datos, mapeo y capas

Para generar la Figura 9.1 es necesario en primera instancia cargar los paquetes que se requieran utilizar y posteriormente importar los datos para explorarlos en R como se señala a continuación, teniendo en cuenta lo indicado en la Nota 18:

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
saber_pro_genericas_faedis_2021_2024 <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis.rds"
)
```

Una vez se carguen los paquetes que se utilizarán y se importen los datos, se pueden mapear las variables `mod_lectura_critica_punt` y `mod_razona_cuantitat_punt` a los ejes asociados con las coordenadas `x` y `y`. También se puede utilizar la función `geom_point()` para generar un gráfico de puntos de tal manera que se pueda visualizar la asociación entre 2 variables continuas, como `mod_lectura_critica_punt` y `mod_razona_cuantitat_punt`:

Listado 9.1 Gráfico de puntos

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    x = mod_lectura_critica_punt,
    y = mod_razona_cuantitat_punt
  )
) +
  geom_point()
```

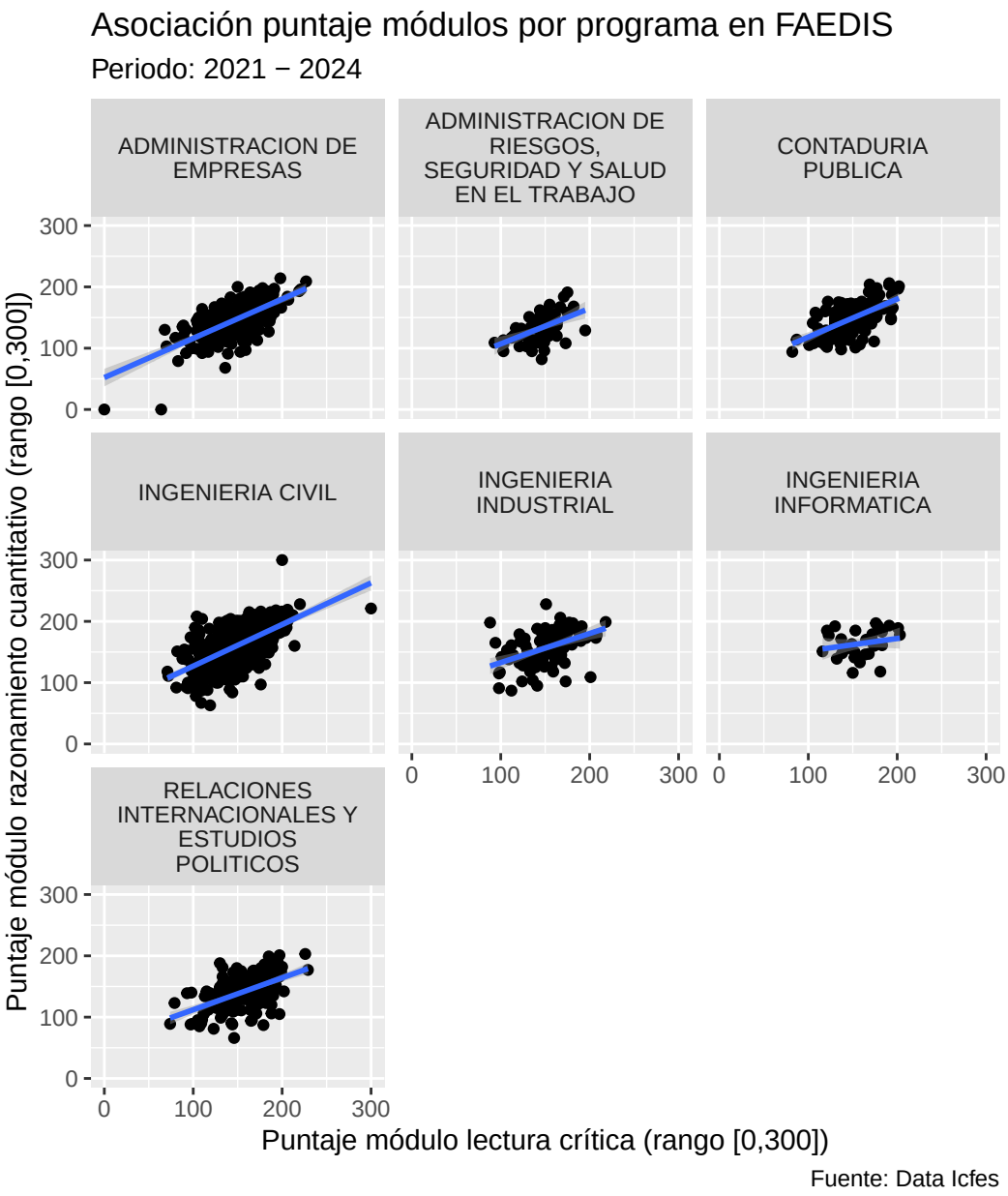
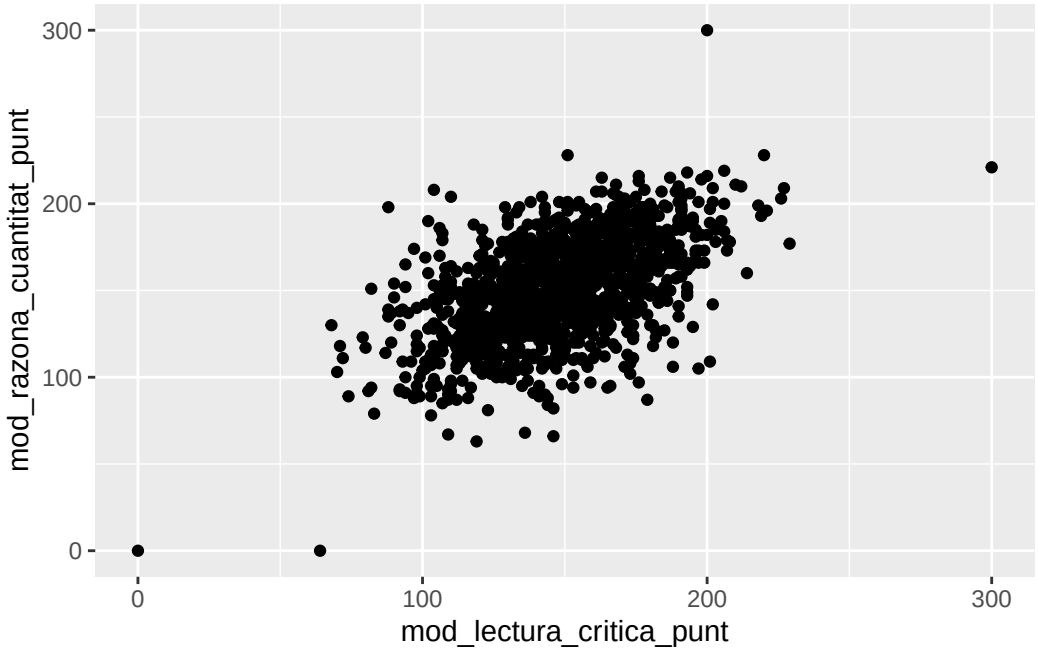


Figura 9.1.: Resultado final de un gráfico de puntos aplicando facetas y utilizando `ggplot2`

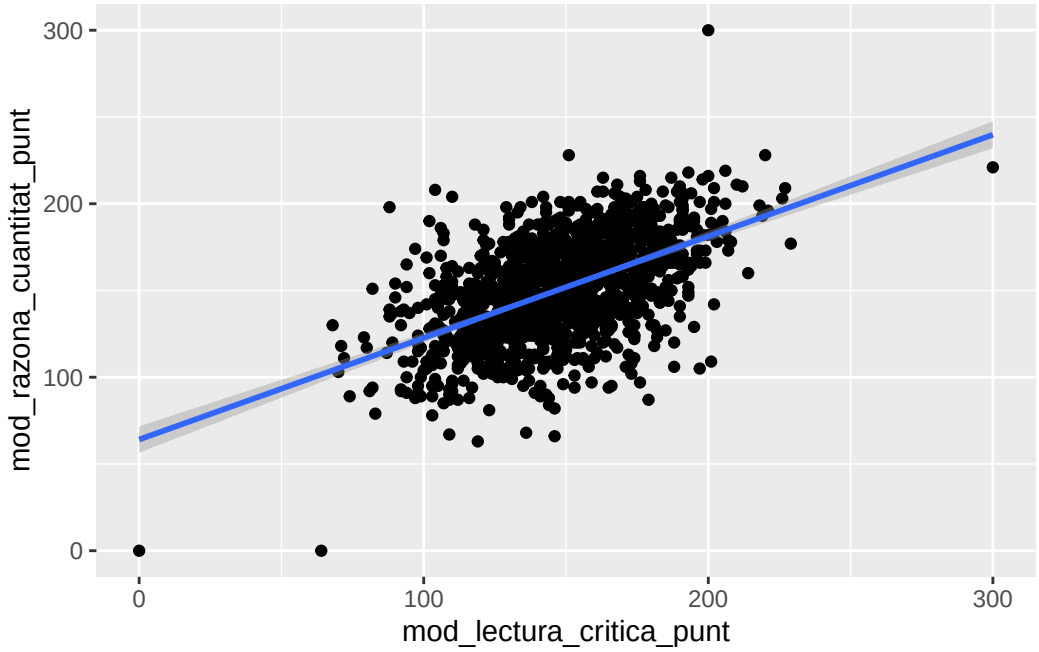


En un gráfico de puntos es posible superponer una recta para identificar una tendencia general en relación con la asociación de 2 variables continuas utilizando la función `geom_smooth()` y el parámetro `method` con el argumento `"lm"`, como `method = "lm"`:

Listado 9.2 Gráfico de puntos superponiendo una recta mediante el uso de la geometría `geom_smooth()` y el método `lm`

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    x = mod_lectura_critica_punt,
    y = mod_razona_cuantitat_punt
  )
) +
  geom_point() +
  geom_smooth(method = "lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Al ejecutar el código del Listado 9.2 se genera un mensaje indicando que la geometría `geom_smooth()` utiliza la fórmula `y ~ x` (ver `geom_smooth() using formula = 'y ~ x'`). Internamente `ggplot2`, mediante la función `lm()`, ajusta un modelo de regresión lineal. A partir de un modelo teórico poblacional (ver Ecuación 9.1), R estima un modelo lineal (ver Ecuación 9.2) mediante el método de mínimos cuadrados ordinarios:

$$y = \beta_0 + \beta_1 x + \epsilon \tag{9.1}$$

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x \tag{9.2}$$

Donde en la Ecuación 9.2 $\hat{\beta}_0$ corresponde al corte estimado con el eje asociado a la coordenada `y` y $\hat{\beta}_1$ se refiere a la pendiente de la línea estimada. En este caso particular, `y` corresponde a la variable `mod_razona_cuantitat_punt` y `x` corresponde a la variable `mod_lectura_critica_punt`.

Nota 18

En el libro no abarcaremos con detalle el tema de regresión lineal y modelos lineales. Sin embargo, para el lector interesado en el tema, una introducción amigable y clara se puede consultar en la lista de reproducción Linear Regression and Linear Models del canal StatQuest with Josh Starmer.
Por otro lado, aquel lector con conocimientos previos de estadística y probabilidad a nivel de pregrado, que tenga interés en profundizar de manera más detallada en los

Tabla 9.1.: Parámetros estimados utilizando la función `lm()` en una regresión lineal simple

Parámetro	Estimación
β_0	64.10
β_1	0.59

modelos lineales utilizando R, puede consultar (Faraway, 2025). Sin embargo, esta lectura no es un requisito para comprender los temas abordados en el Capítulo 9.

El paquete `ggplot2` no almacena o guarda los parámetros estimados del modelo. En ese sentido, `geom_smooth(method = "lm")` calcula la recta internamente solo de forma temporal para poder dibujarla. Si deseas conocer la estimación de los parámetros puedes utilizar para este caso el siguiente código:

Listado 9.3 Uso de la función `lm()` para obtener la estimación de los parámetros en una regresión lineal simple

```
lm(
  formula = mod_razona_cuantitat_punt ~ mod_lectura_critica_punt,
  data = saber_pro_genericas_faedis_2021_2024
)
```

```
Call:
lm(formula = mod_razona_cuantitat_punt ~ mod_lectura_critica_punt,
    data = saber_pro_genericas_faedis_2021_2024)

Coefficients:
             (Intercept)  mod_lectura_critica_punt
                 64.0985                   0.5856
```

En la Tabla 9.1 se señalan las estimaciones de los respectivos parámetros redondeados a 2 decimales y relacionados con la recta del gráfico generado por el Listado 9.2 mediante la función `geom_smooth()` y el parámetro `method` con el argumento `"lm"`.

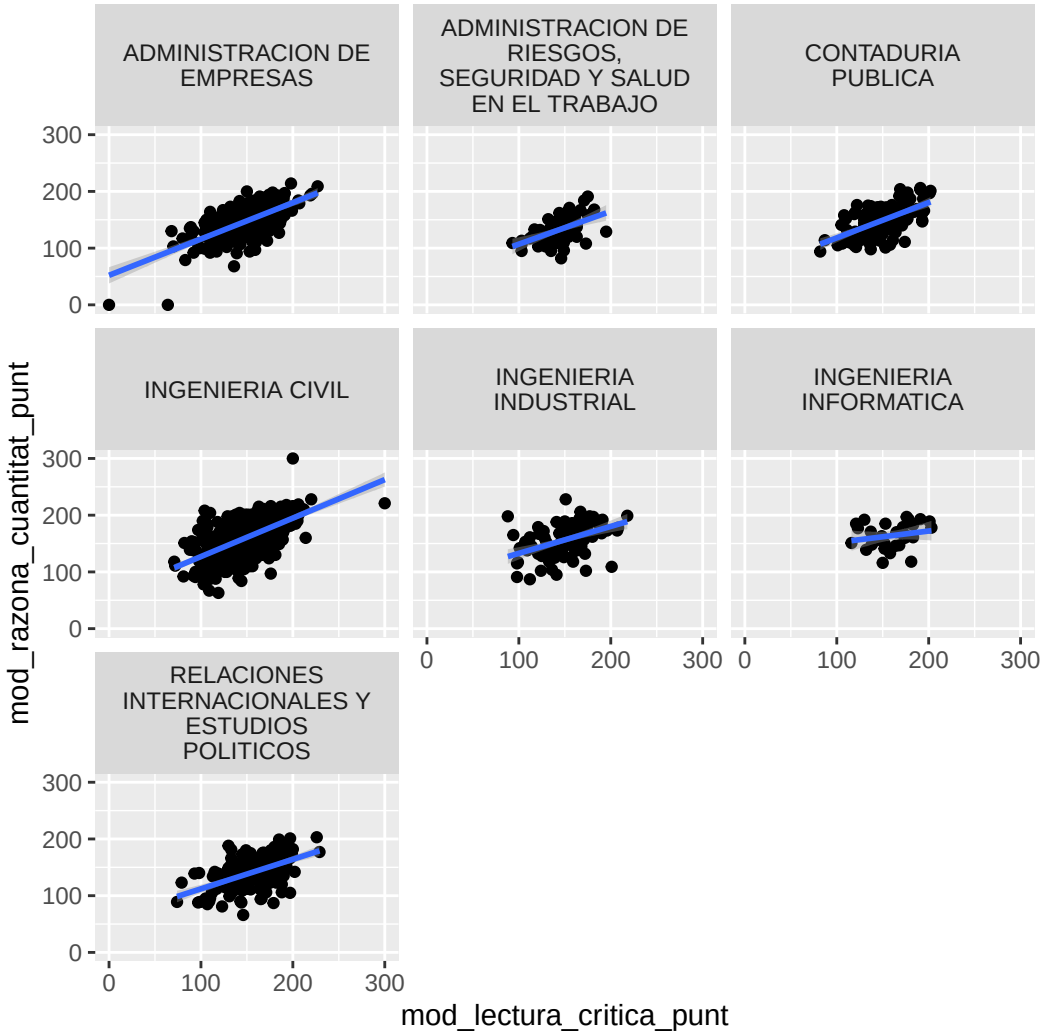
9.1.2. Facetas

En el gráfico generado por el Listado 9.2 la asociación entre los resultados de los módulos de razonamiento cuantitativo y lectura crítica se encuentran consolidados para cada uno de los programas académicos señalando una tendencia general. Para examinar si existe algún

tipo de heterogeneidad entre los programas académicos es posible utilizar el componente de facetas correspondiente a la gramática de gráficos en capas. Este componente permite visualizar el mismo gráfico generado por el Listado 9.2 pero para cada programa académico en paneles individuales utilizando la función `facet_wrap()`. Para utilizar esta función se requiere definir la variable que permitirá generar los grupos para generar cada gráfico utilizando el parámetro `facets` y como argumento la respectiva variable dentro de la función `vars()`. Además como en el Listado 6.8 y el Listado 8.1 es necesario limitar el ancho de los nombres de los programas donde este aspecto también se puede modificar mediante el parámetro `labeller` utilizando la función `label_wrap_gen()`, tal como se señala a continuación:

Listado 9.4 Uso de la función `facet_wrap()` junto con el parámetro `facets` y la función `vars()` como argumento para visualizar diferentes subconjuntos de datos

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    x = mod_lectura_critica_punt,
    y = mod_razona_cuantitat_punt
  )
) +
  geom_point() +
  geom_smooth(method = "lm") +
  facet_wrap(
    facets = vars(estu_prgm_academico),
    labeller = label_wrap_gen(width = 18)
  )
```



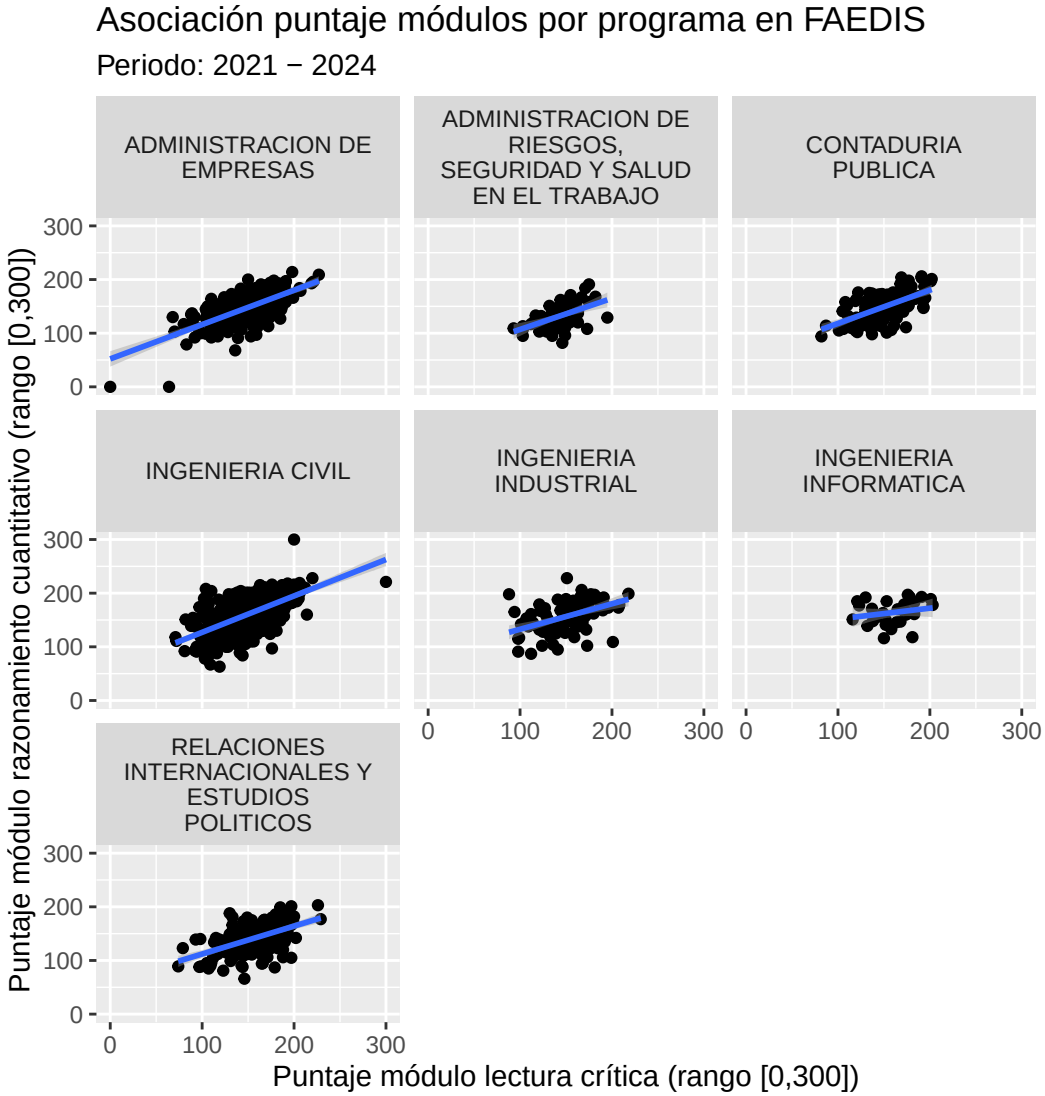
En el gráfico generado por el Listado 9.4 se puede identificar que la pendiente de la recta generada por la función `geom_smooth()` junto con el parámetro `method` y el argumento `"lm"` no es la misma cuando se examina la tendencia general para cada uno de los programas académicos. En la Tabla 9.2 se señalan las estimaciones de los respectivos parámetros y redondeados a 2 decimales para cada uno de los programas académicos relacionados con las rectas del gráfico generado por el Listado 9.4 mediante la función `geom_smooth()` y el parámetro `method` con el argumento `"lm"`.

9.1.3. Escalas

En el gráfico generado por el Listado 9.4 se pueden modificar diferentes etiquetas en el gráfico para incluir información de contexto de tal manera que sea más comprensible utilizando la función `labs()`, como se señala a continuación:

Tabla 9.2.: Parámetros estimados utilizando la función `lm()` en una regresión lineal simple por programa académico

Programa académico	β_0	β_1
ADMINISTRACION DE EMPRESAS	52.08	0.64
CONTADURIA PUBLICA	56.09	0.62
INGENIERIA CIVIL	59.20	0.68
INGENIERIA INDUSTRIAL	85.30	0.47
RELACIONES INTERNACIONALES Y ESTUDIOS POLITICOS	60.05	0.52
INGENIERIA INFORMATICA	131.58	0.20
ADMINISTRACION DE RIESGOS, SEGURIDAD Y SALUD EN EL TRABAJO	49.06	0.58



Fuente: Data Icfes

Listado 9.5 Uso de la función `labs()` para agregar y modificar etiquetas en un gráfico

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024,
  mapping = aes(
    x = mod_lectura_critica_punt,
    y = mod_razona_cuantitat_punt
  )
) +
  geom_point() +
  geom_smooth(method = "lm") +
  facet_wrap(
    facets = vars(estu_prgm_academico),
    labeller = label_wrap_gen(width = 18)
  ) +
  labs(
    x = "Puntaje módulo lectura crítica (rango [0,300])",
    y = "Puntaje módulo razonamiento cuantitativo (rango [0,300])",
    title = "Asociación puntaje módulos por programa en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  )
```

De esa manera con el código señalado en el Listado 9.5, es posible recrear finalmente la Figura 9.1 para ilustrar la asociación de las variables `mod_lectura_critica_punt` y `mod_razona_cuantitat_punt` correspondientes a los módulos de lectura crítica y razonamiento cuantitativo para cada uno de los programas académicos.

9.2. Gráfico de líneas

En esta sección realizarás un gráfico de líneas señalando la evolución a lo largo del tiempo del puntaje global promedio para los programas de pregrado de la Facultad de Estudios a Distancia de la Universidad Militar en el periodo 2021-2024, tal como se indica en la Figura 9.2.

9.2.1. Datos, mapeo y capas

Para generar la Figura 9.2 es necesario en primera instancia cargar los paquetes que se requieran utilizar y posteriormente importar los datos para explorarlos en R como se señala a continuación, teniendo en cuenta lo indicado en la Nota 18:

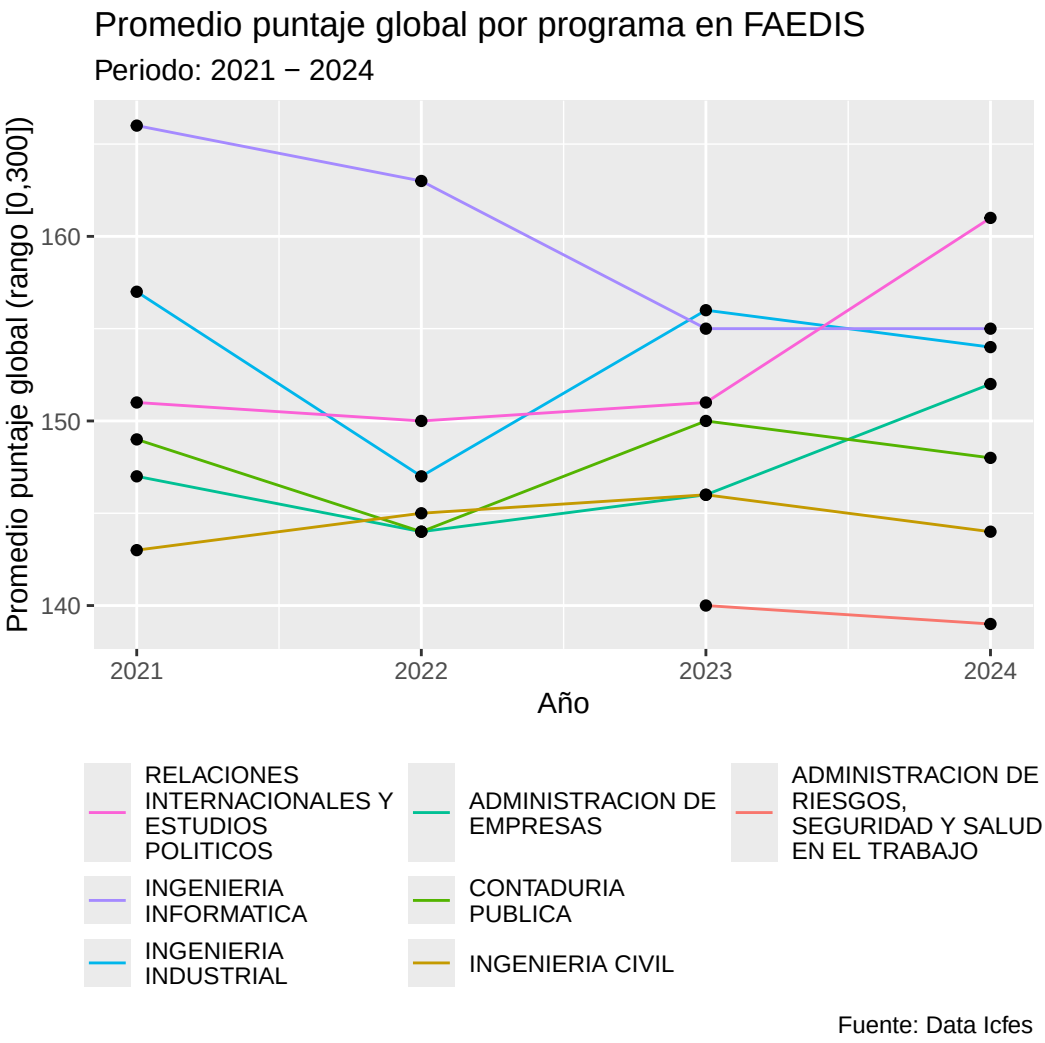


Figura 9.2.: Resultado final de un gráfico de líneas utilizando `ggplot2`

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
saber_pro_genericas_faedis_2021_2024_programas <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis_programas.rds"
)
```

Una vez importes el conjunto de datos, podrás darle un "vistazo" utilizando la función `glimpse()` para identificar el número y los tipos de variables que lo componen, así como la cantidad de observaciones disponibles.

```
# Inspeccionar datos ----
glimpse(saber_pro_genericas_faedis_2021_2024_programas)
```

```
Rows: 26
Columns: 9
$ ano                        <int> 2021, 2021, 2021, 2021, ~
$ estu_snies_prgmacademico   <int> 6527, 10963, 11004, 1142~
$ estu_prgm_academico        <chr> "ADMINISTRACION DE EMPRE~
$ mod_competen_ciudada_punt <dbl> 142, 158, 144, 155, 159,~
$ mod_comuni_escrita_punt    <dbl> 147, 149, 120, 145, 146,~
$ mod_ingles_punt            <dbl> 157, 151, 155, 150, 170,~
$ mod_lectura_critica_punt   <dbl> 142, 157, 141, 147, 150,~
$ mod_razona_cuantitat_punt <dbl> 145, 138, 154, 149, 161,~
$ punt_global                <dbl> 147, 151, 143, 149, 157,~
```

9.2.2. Descripción de variables

Esta base de datos es construida a partir de `saber_pro_genericas_faedis_2021_2024` donde aprenderás acerca de cómo construirla en el Capítulo 10 utilizando verbos de una sola tabla con el paquete `dplyr` perteneciente al `tidyverse`. A continuación, se describen cada una de las variables en `saber_pro_genericas_faedis_2021_2024_programas`:

1. **ano**: año correspondiente a los resultados del examen Saber Pro
2. **estu_snies_prgmacademico**: código SNIES del programa académico relacionado con los resultados del examen Saber Pro
3. **estu_prgm_academico**: nombre del programa académico relacionado con los resultados del examen Saber Pro
4. **mod_competen_ciudada_punt**: puntaje promedio redondeado al entero más próximo del módulo de competencias ciudadanas obtenido por los estudiantes del programa académico (el rango posible es $[0, 300]$)

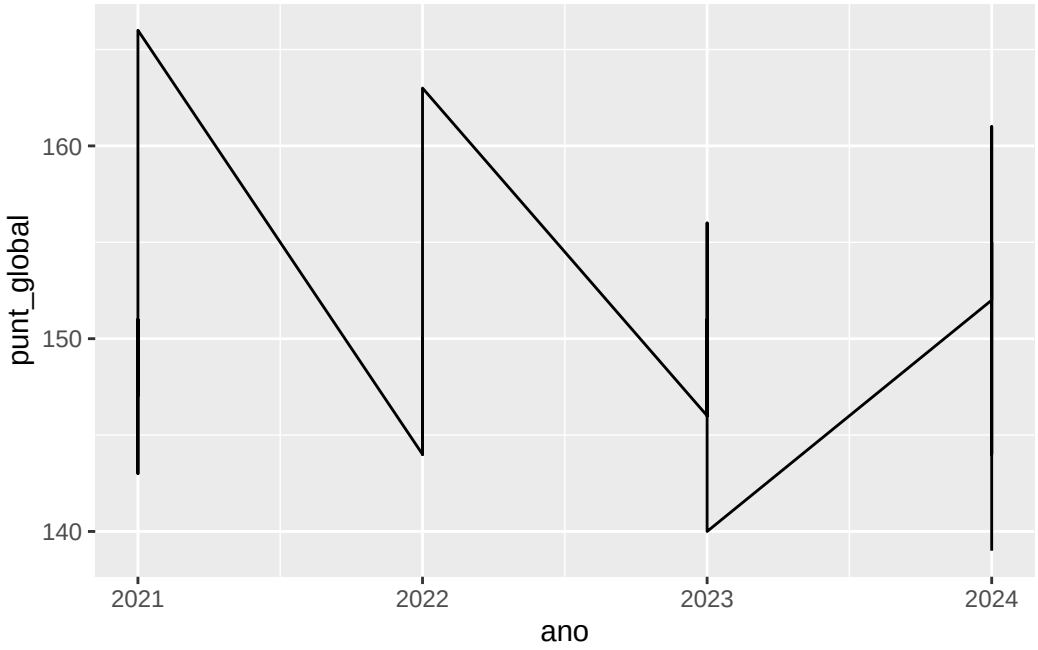
5. **mod_comuni_escrita_punt**: puntaje promedio redondeado al entero más próximo del módulo de comunicación escrita obtenido por los estudiantes del programa académico (el rango posible es $[0, 300]$)
6. **mod_ingles_punt**: puntaje promedio redondeado al entero más próximo del módulo de inglés obtenido por los estudiantes del programa académico (el rango posible es $[0, 300]$)
7. **mod_lectura_critica_punt**: puntaje promedio redondeado al entero más próximo del módulo de lectura crítica obtenido por los estudiantes del programa académico (el rango posible es $[0, 300]$)
8. **mod_razona_cuantitat_punt**: puntaje promedio redondeado al entero más próximo del módulo de razonamiento cuantitativo obtenido por los estudiantes del programa académico (el rango posible es $[0, 300]$)
9. **punt_global**: puntaje total promedio redondeado al entero más próximo obtenido por los estudiantes del programa académico en el grupo de competencias genéricas (este grupo comprende los módulos de competencias ciudadanas, comunicación escrita, inglés, lectura crítica y razonamiento cuantitativo)

9.2.3. Mapeo y capas

Para ilustrar el comportamiento del puntaje global promedio obtenido por cada uno de los programas académicos a través de las variables `ano` y `punt_global` mediante un gráfico de líneas es posible utilizar la función `geom_line()`, tal como se señala a continuación:

Listado 9.6 Uso de la función `geom_line()` para generar un gráfico de líneas

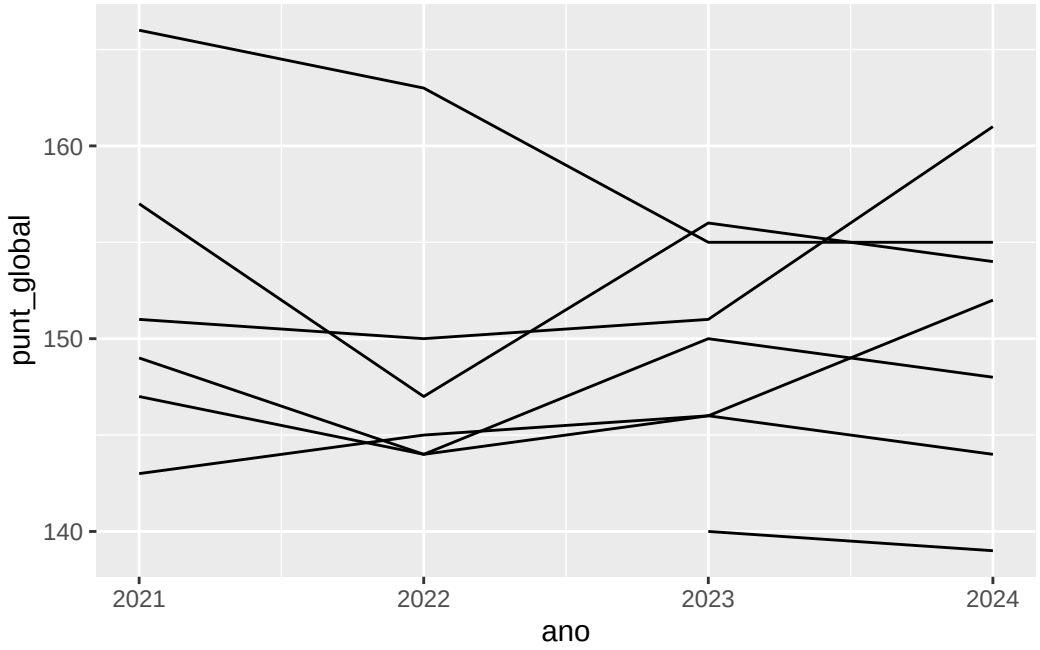
```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(
    x = ano,
    y = punt_global
  )
) +
  geom_line()
```



En el gráfico generado por el Listado 9.7 la función `geom_line()` conecta los puntos siguiendo el orden del eje asociado con la coordenada `x`. Debido a que se tienen diferentes programas académicos para cada año respecto al puntaje global, el gráfico generado no es claro debido a que conecta los puntos para cada año sin agrupar por cada programa académico. Para corregir este aspecto es posible utilizar el atributo estético `group` como parámetro y el argumento `estu_prgm_academico` dentro de la función `aes()` en conjunto con la función `geom_line()`, tal como se señala a continuación:

Listado 9.7 Uso del atributo estético `group` dentro de la función `aes()` en conjunto con la función `geom_line()` para generar un gráfico de líneas por cada categoría dentro de un grupo

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(
    x = ano,
    y = punt_global
  )
) +
  geom_line(
    aes(group = estu_prgm_academico)
  )
```



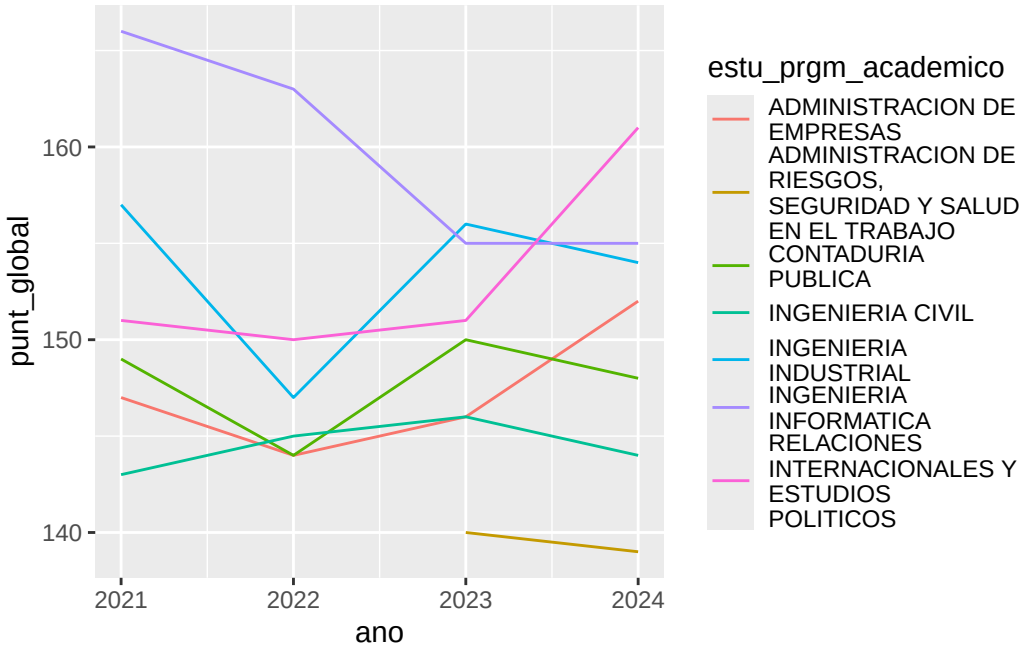
Aunque en el gráfico generado por el Listado 9.7 se agrupa el comportamiento de la variable `punt_global` a lo largo del tiempo no es claro a qué programa académico corresponde cada línea. En la Sección 9.2.4 corregiremos este aspecto utilizando el atributo estético `color` dentro de `aes()` en conjunto con la función `geom_line()`.

9.2.4. Capas y escalas

Para asociar cada línea con un programa académico es posible utilizar el atributo estético `color` como parámetro y la variable `estu_prgm_academico` como argumento dentro de la función `aes()` en conjunto con la función `geom_line()`. Al aplicar este aspecto `ggplot2` genera una leyenda que permite identificar el programa académico que se asocia con cada línea. Debido a que también necesitamos controlar la longitud de las etiquetas asociadas con el nombre de cada programa académico es necesario utilizar la función `label_wrap_gen()` como argumento junto con el parámetro `labels` dentro de la función `scale_color_discrete()` teniendo en cuenta que queremos modificar la escala asociada con el atributo estético `color` y `estu_prgm_academico` es una variable discreta, tal como se señala a continuación:

Listado 9.8 Uso del atributo estético `color` dentro de la función `aes()` en conjunto con la función `geom_line()` y modificación de las etiquetas dentro de una leyenda asociadas con este atributo estético

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(
    x = ano,
    y = punt_global
  )
) +
  geom_line(
    mapping = aes(
      group = estu_prgm_academico,
      color = estu_prgm_academico
    )
  ) +
  scale_color_discrete(
    labels = label_wrap_gen(width = 18)
  )
```

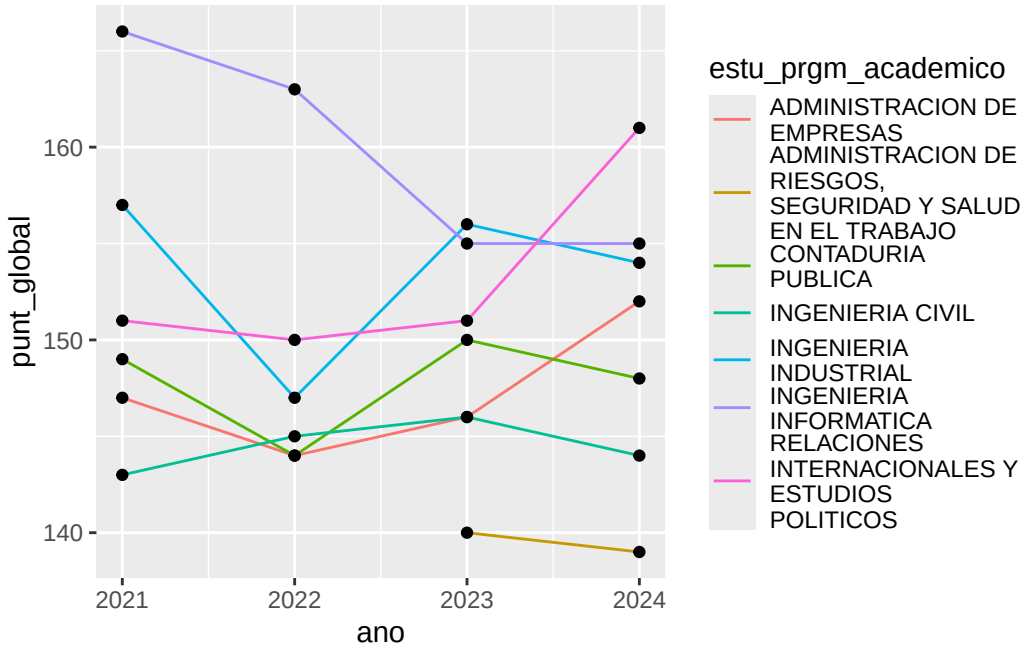


En el gráfico generado por el Listado 9.8 podemos agregar una capa adicional utilizando la geometría `geom_point()` para indicar de manera más clara las coordenadas de cada

dato, tal como se señala a continuación:

Listado 9.9 Uso de la función `geom_point()` para resaltar las coordenadas de cada dato dentro de un gráfico de líneas

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(
    x = ano,
    y = punt_global
  )
) +
  geom_line(
    mapping = aes(
      group = estu_prgm_academico,
      color = estu_prgm_academico
    )
  ) +
  geom_point() +
  scale_color_discrete(
    labels = label_wrap_gen(width = 18)
  )
```



En el código del Listado 9.9 existe un aspecto que no se ha señalado de manera explícita

con el objetivo de mantener de la forma más simple posible la explicación acerca del funcionamiento de la gramática de gráficos en capas implementada en `ggplot2`. Los parámetros `data` y `mapping` a través de la función `aes()` respecto a los atributos estéticos así como sus respectivos argumentos que se definen en la función `ggplot()` son heredados por parte de las geometrías que se definan posteriormente a través de las funciones de la forma `geom_*()`. En ese sentido, en el Listado 9.9 los datos asociados con el parámetro `data` son utilizados tanto por la función `geom_line()` como por la función `geom_point()`. De la misma forma ocurre con los atributos estéticos `x` y `y` definidos dentro de la función `aes()` donde la función `geom_line()` los utiliza para generar el trazo de las líneas y la función `geom_point()` para definir las coordenadas de los puntos que se generan en el gráfico.

Por otro lado, aquellos parámetros con sus respectivos argumentos que son definidos en las geometrías a través de las funciones de la forma `geom_*()` solo se utilizan por parte de la geometría donde se definan y no son heredados en otras geometrías que se hayan definido anterior o posteriormente. Por ejemplo, en el gráfico generado por el Listado 9.9 el color de los puntos asociados con la función `geom_point()` adquieren por defecto el color `"black"` pero ese mismo comportamiento no se mantiene para las líneas generadas por la función `geom_line()` debido a que el atributo estético `color` dentro de la función `aes()` tiene asociado la variable `estu_prgm_academico`.

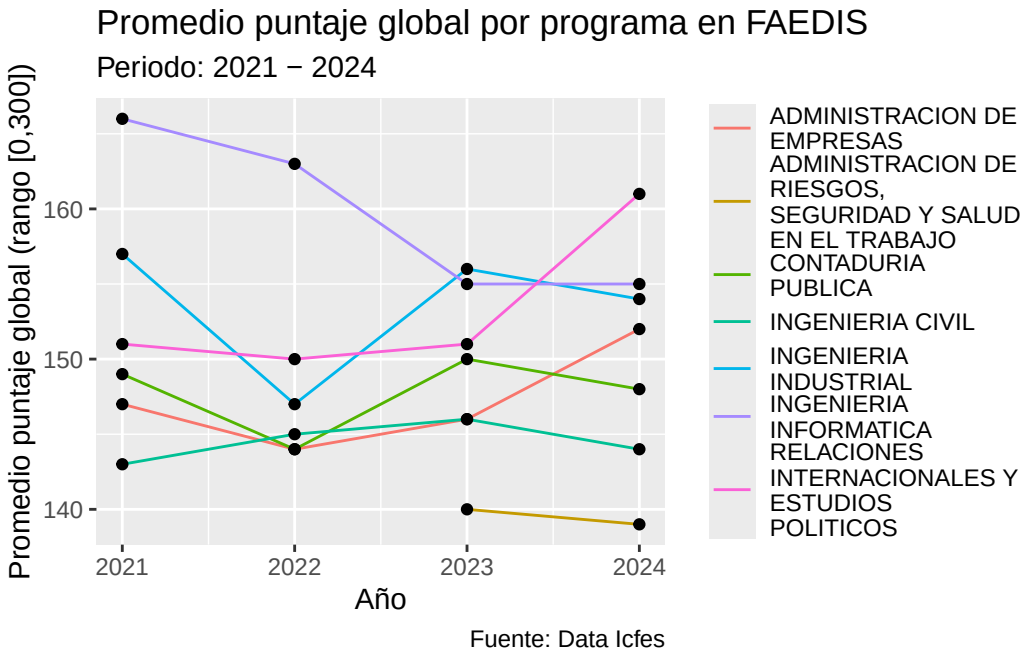
> **!** Importante 7
>
> Los parámetros `data` y `mapping` a través de la función `aes()` respecto a los atributos estéticos así como sus respectivos argumentos que se definen en la función `ggplot()` son heredados por parte de las geometrías que se definan posteriormente a través de las funciones de la forma `geom_*()`.
> Sin embargo, los parámetros que se definan dentro de una geometría particular a través de una función de la forma `geom_*()` no son heredados por otras geometrías que se definan anterior o posteriormente a través de otras funciones de la forma `geom_*()`.
> En ese sentido, `ggplot2` aplica un principio que se conoce en el desarrollo de software como **D**on't **R**epeat **Y**ourself (DRY) de tal manera que no sea necesario repetir en cada geometría parámetros o argumentos que se busquen definir de manera global para que sean utilizados en todas las geometrías. Sin embargo, mantiene un nivel de flexibilidad apropiado donde permite que se definan parámetros con sus respectivos argumentos para que sean utilizados dentro de una geometría en particular.

9.2.5. Escalas y tema

En el gráfico generado por el Listado 9.9 se pueden modificar diferentes etiquetas en el gráfico para incluir información de contexto de tal manera que sea más comprensible utilizando la función `labs()`, como se señala a continuación:

Listado 9.10 Uso de la función `labs()` para agregar y modificar etiquetas en un gráfico

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(x = ano, y = punt_global)
) +
  geom_line(
    mapping = aes(
      group = estu_prgm_academico,
      color = estu_prgm_academico
    )
  ) +
  geom_point() +
  scale_color_discrete(
    labels = label_wrap_gen(width = 18)
  ) +
  labs(
    x = "Año",
    y = "Promedio puntaje global (rango [0,300])",
    color = NULL,
    title = "Promedio puntaje global por programa en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  )
```

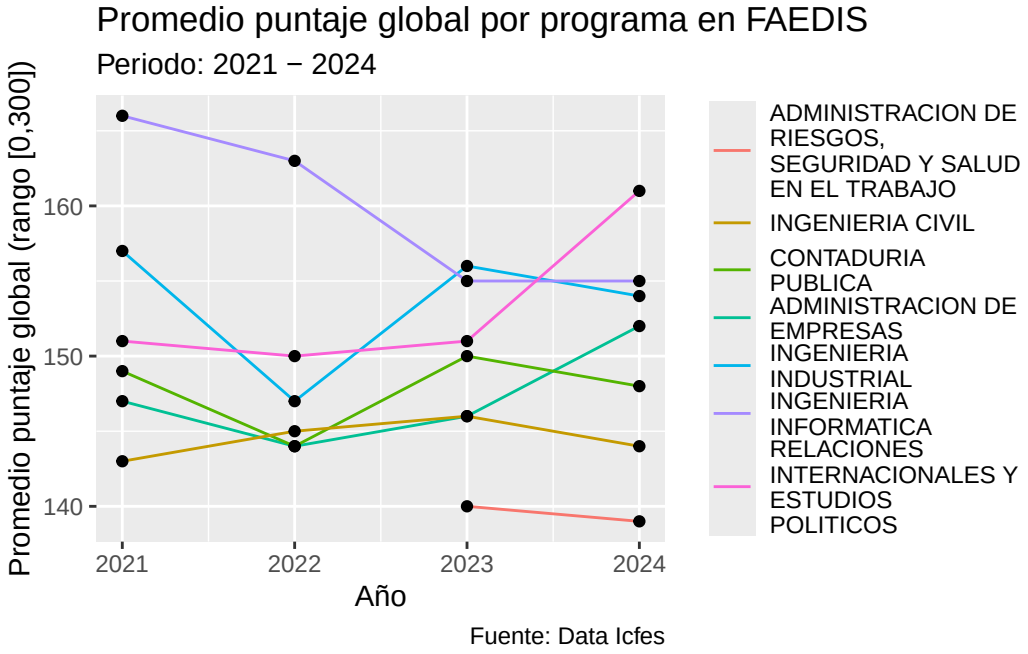


En el gráfico generado por el Listado 9.10 aunque los colores de las líneas se encuentran asociados con un programa académico y esta asociación se señala en una leyenda es posible mejorar la presentación del gráfico. Los nombres de los programas dentro de la leyenda se encuentran ordenados en orden alfabético. Una forma de generar un ordenamiento para que exista una asociación más clara con las etiquetas de la leyenda es utilizar el último valor del año del que se tiene información.

Al igual que en el Listado 8.3 es posible utilizar la función `fct_reorder()` pero en este caso afectando el atributo estético `color` en `geom_line()` con la función `last`, que proviene del paquete `dplyr` incluido en el `tidyverse`, donde permite identificar el último valor, tal como se señala a continuación:

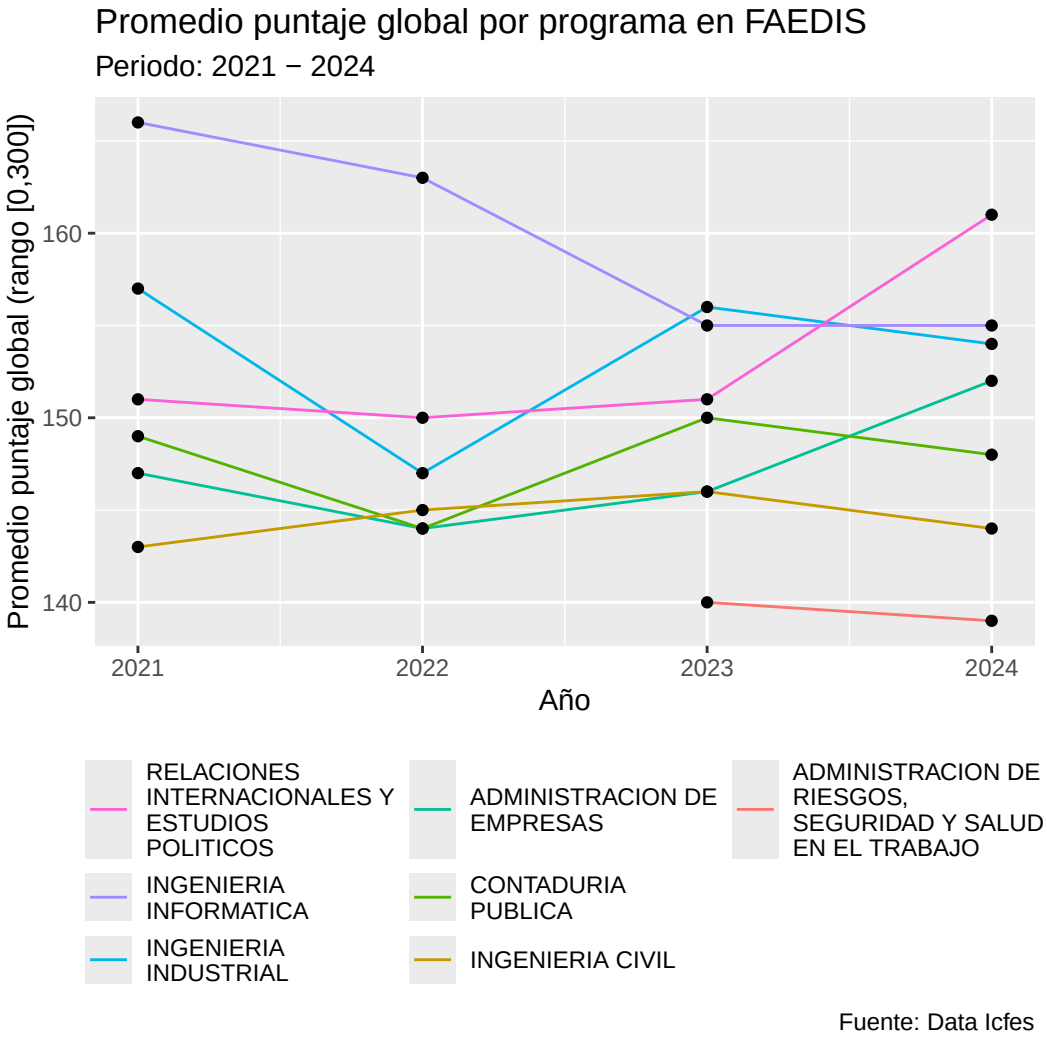
Listado 9.11 Ordenamiento de etiquetas dentro de una leyenda asociadas al atributo estético `color` utilizando la función `fct_reorder()`

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(x = ano, y = punt_global)
) +
  geom_line(
    mapping = aes(
      group = estu_prgm_academico,
      color = fct_reorder(
        estu_prgm_academico,
        .x = punt_global,
        .fun = last
      )
    )
  ) +
  geom_point() +
  scale_color_discrete(
    labels = label_wrap_gen(width = 18)
  ) +
  labs(
    x = "Año",
    y = "Promedio puntaje global (rango [0,300])",
    color = NULL,
    title = "Promedio puntaje global por programa en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  )
```



En el gráfico generado por el Listado 9.11 las etiquetas en la leyenda correspondientes a los programas académicos ya no se encuentran ordenadas alfabéticamente sino por el último valor de la variable `punt_global` en relación al año más reciente del que se tiene información. Sin embargo, en este caso el ordenamiento generado en la leyenda se aplica en forma descendente pero en el gráfico la variable `punt_global` se ordena de manera natural en forma ascendente.

Para mejorar este aspecto así como extender el ancho del gráfico, es posible utilizar tanto la función `theme()` con el parámetro `legend.position` y el argumento `"bottom"` así como el parámetro `guide` y la función `guide_legend()` como argumento dentro de `scale_color_discrete()` para modificar la leyenda del gráfico, tal como se señala a continuación:



En el gráfico generado por el código del Listado 9.12 se utiliza nuevamente la función `theme()` para modificar la posición de la leyenda donde este aspecto no es nuevo debido a que se introdujo en la Sección 7.5 y en la Sección 8.3. En el caso de la función `guide_legend()` el parámetro `nrow` permite definir el número de filas que se utiliza para ordenar en forma de grilla las categorías de la leyenda donde se utiliza como argumento el valor de `3` teniendo en cuenta la longitud de los nombres de los programas académicos. Por otro lado, el parámetro `reverse` junto con el argumento `TRUE` permite reversar el ordenamiento que por defecto se aplica a las etiquetas dentro de la leyenda de tal manera que los nombres de los programas académicos estén ordenados de arriba hacia abajo y de izquierda a derecha con respecto al ordenamiento ascendente de la variable `punt_global` respecto al último año que se tiene información dentro del gráfico.

De esa manera con el código señalado en el Listado 9.12, es posible recrear finalmente la Figura 9.2 para ilustrar el comportamiento del puntaje global promedio por programa

académico a lo largo del tiempo.

Listado 9.12 Uso de las funciones `guide_legend()` y `theme()` para modificar la leyenda dentro de un gráfico

```
ggplot(
  data = saber_pro_genericas_faedis_2021_2024_programas,
  mapping = aes(x = ano, y = punt_global)
) +
  geom_line(
    aes(
      group = estu_prgm_academico,
      color = fct_reorder(
        estu_prgm_academico,
        .x = punt_global,
        .fun = last
      )
    )
  ) +
  geom_point() +
  scale_color_discrete(
    labels = label_wrap_gen(width = 18),
    guide = guide_legend(nrow = 3, reverse = TRUE)
  ) +
  labs(
    x = "Año",
    y = "Promedio puntaje global (rango [0,300])",
    color = NULL,
    title = "Promedio puntaje global por programa en FAEDIS",
    subtitle = "Periodo: 2021 - 2024",
    caption = "Fuente: Data Icfes"
  ) +
  theme(
    legend.position = "bottom"
  )
```

Parte III.

Manipulación de datos

El objetivo de esta parte, es presentar la **gramática de manipulación de datos** mediante un conjunto consistente de verbos cuando se requiere transformar una sola tabla o se busca conectar dos o más tablas de datos de manera secuencial en R utilizando el paquete `dplyr`, a través de los siguientes dos capítulos:

- Capítulo 10: introduce la forma de manejar datos cuando se tiene una sola tabla mediante un conjunto consistente de verbos.
- Capítulo 11: introduce la forma de conectar o unir tablas de datos de manera secuencial a través de un marco flexible.

Una vez se entienda el funcionamiento del paquete `dplyr` a través de la **gramática de manipulación de datos**, el lector interesado en profundizar y explorar cómo conectarse a una base de datos y extraer datos mediante consultas SQL puede consultar los paquetes `dbplyr` y `DBI`.

El paquete `dbplyr` es el *backend* de bases de datos para `dplyr`, el cual permite trabajar con tablas de bases de datos remotas traduciendo automáticamente el código de `dplyr` a SQL, mientras que el paquete `DBI` facilita la conexión de R con **Sistemas de Gestión de Bases de Datos** (en inglés Database Management Systems (DBMS)).

10. Verbos de una sola tabla

Este capítulo se basa en (Wickham, 2023, Capítulo 3) así como en la documentación oficial de `dplyr` donde tiene como propósito introducir una **gramática de manipulación de datos** a través de un conjunto consistente de verbos para resolver las tareas más comunes en el manejo de datos cuando se tiene una sola tabla utilizando el paquete `dplyr` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes al periodo 2021-2024 para Colombia.

Exploraremos los siguientes aspectos:

1. ¿En qué consiste la gramática de manipulación de datos utilizada en `dplyr`?
2. ¿Qué y cuáles son los verbos de una sola tabla para filas?
3. ¿Qué y cuáles son los verbos de una sola tabla para columnas?
4. ¿Qué y cuáles son los verbos de una sola tabla para colapsar, agrupar o desagrupar filas?
5. ¿Qué es y para qué sirve el operador **pipe** (`|>`)?

> **Nota 19**
>
> Para desarrollar el contenido de este capítulo es necesario que descargues el archivo señalado a continuación y lo incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **2021_2024-saber_pro-genericas.csv.xz**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `010_verbos-una-sola-tabla-en-r` y dentro de ella genera el archivo `001_manipular-datos-una-sola-tabla.R`, siguiendo las pautas de la Sección 4.4.

10.1. Gramática de manipulación de datos en `dplyr`

El paquete `dplyr`, incluido en el `tidyverse`, permite manipular datos a partir de un conjunto de datos en 2 niveles:

1. Dentro de una sola tabla de datos
2. Entre diferentes tablas de datos

El primer nivel se abarcará en el Capítulo 10 y el segundo nivel se desarrollará en el Capítulo 11. Al igual que en el caso de `ggplot2` (ver Sección 6.1) el paquete `dplyr` funciona mejor si el conjunto de datos es ordenado (tidy data en inglés, (Wickham, 2014)) donde este concepto se tratará en más detalle en el Capítulo 16.

A medida que explores diferentes formas de manipular un conjunto de datos entenderás en más detalle la gramática de manipulación de datos propuesta en el paquete `dplyr`. Por ahora se describirán en la Sección 10.1.1 de manera general y gráfica los verbos más comunes que se utilizan para manipular datos en una sola tabla y que se utilizan en `dplyr`.

En el paquete `dplyr`, para mantener la consistencia de las funciones que lo componen, siempre se cumple que cada uno de los verbos obedece a los siguientes aspectos (Wickham, 2023, Capítulo 3):

1. El primer argumento es una tibble (o en su defecto un objeto de R similar).
2. Los argumentos subsiguientes especifican las columnas, filas o expresiones que operarán sobre las filas o columnas.
3. El resultado[1] de una función en `dplyr` es siempre una nueva tibble (o en su defecto un objeto de R similar).

10.1.1. Verbos

Los verbos para manipular datos en una sola tabla se encuentran organizados en 3 categorías teniendo en cuenta el elemento que se desea manipular:

a. Verbos para filas
b. Verbos para columnas
c. Verbos para colapsar, agrupar o desagrupar filas

Debido a que la manipulación de datos requiere realizar un conjunto de operaciones de manera secuencial, se hace necesario contar con un operador que "encadene" de forma armónica cada una de las transformaciones deseadas. Para este propósito se utiliza el operador **pipe** (`|>`), el cual se abordará con mayor detalle en la Sección 10.1.1.3.

10.1.1.1. Verbos para filas

Esta categoría incluye las funciones `filter()`, `slice()` y `arrange()`. La primera función, correspondiente a `filter()`, se utiliza para seleccionar filas con base en una expresión que retorne un vector lógico donde usualmente se hace uso de operadores relacionales

[1] En la documentación de ayuda de R, al buscar por ejemplo información acerca de la función `filter()` (ver Listado 5.10) con el comando `?filter` en la consola, la descripción del resultado se encuentra bajo la sección titulada **Value**. En este libro se usará el término "resultado" de la función y no "valor", con el fin de utilizar una expresión más intuitiva para el lector.

(ver Sección 5.5). En la Figura 10.1 se ilustra gráficamente la manipulación que se puede realizar con la función `filter()`.

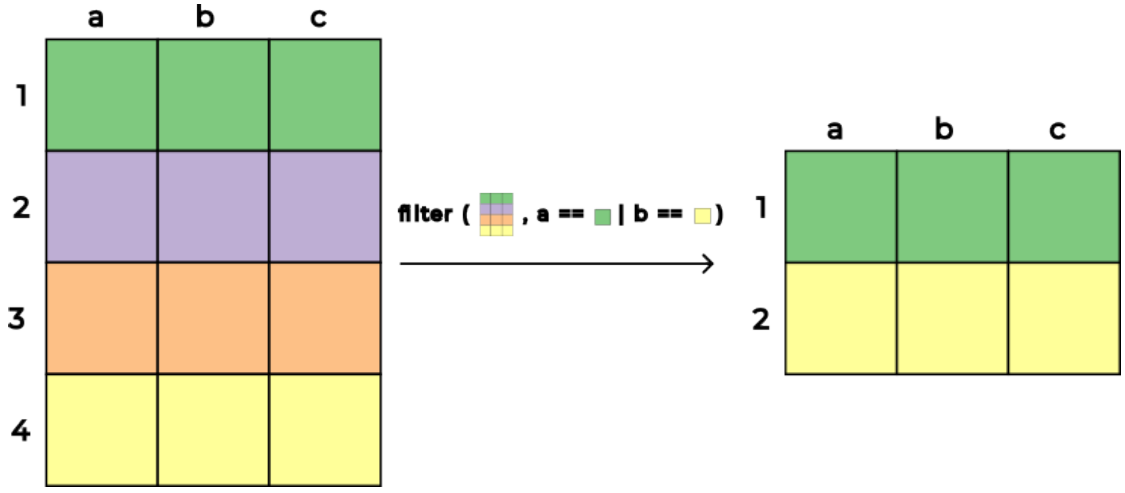


Figura 10.1.: Representación gráfica de la función `filter()`

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos utilizados en el Capítulo 6 respecto a `2021-2024_saber-pro-genericas_faedis.rds` para identificar las filas correspondientes al año 2022 y de estudiantes pertenecientes al género masculino:

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
saber_pro_genericas_faedis_2021_2024 <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis.rds"
)

# Filtrar ---
filter(
  saber_pro_genericas_faedis_2021_2024,
  ano == 2022 & estu_genero == "M"
)
```

```
# A tibble: 194 x 21
     ano estu_consecutivo estu_genero estu_semestrecursa
   <int> <chr>            <fct>       <ord>
 1  2022 EK202220120492   M           09
 2  2022 EK202220159443   M           08
 3  2022 EK202220121639   M           09
 4  2022 EK202220120505   M           09
```

```
 5  2022 EK202220120480   M           08
 6  2022 EK202220128207   M           08
 7  2022 EK202220121516   M           09
 8  2022 EK202220120491   M           08
 9  2022 EK202220120507   M           10
10  2022 EK202220120496   M           08
# i 184 more rows
# i 17 more variables: fami_estratovivienda <ord>,
#   fami_educacionmadre <ord>, fami_educacionpadre <ord>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_mcpio_presentacion <chr>, ...
```

La segunda función, correspondiente a `slice()`, se utiliza para seleccionar filas con base en su ubicación utilizando un vector de números enteros. Los números enteros deben ser todos estrictamente positivos o todos estrictamente negativos[2] donde los valores positivos se interpretan como la posición de filas que se desean conservar mientras que los valores negativos las posiciones de las filas que se desean descartar. En la Figura 10.2 se ilustra gráficamente la manipulación que se puede realizar con la función `slice()`.

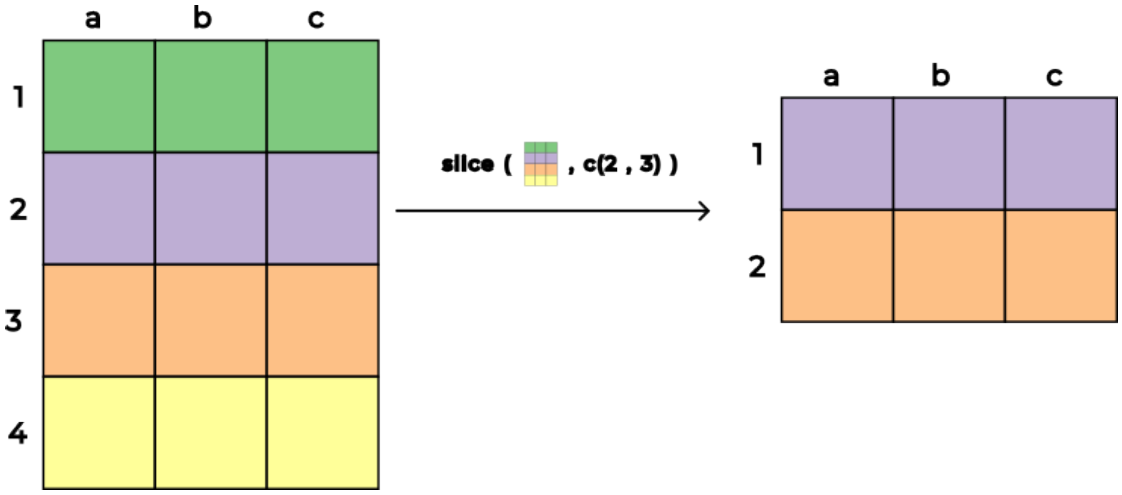


Figura 10.2.: Representación gráfica de la función `slice()`

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para seleccionar las filas correspondientes a las posiciones `1`, `3` y `5`:

[2]A diferencia de otros lenguajes de programación como Python, los índices de las posiciones comienzan en cero (`0`). En R al igual que en otros lenguajes de programación como Julia se utiliza una indexación basada en `1`. Esto significa que la primera fila o elemento siempre corresponde a la posición `1`, y el uso del `0` no se toma en cuenta para seleccionar elementos.

```
# Seleccionar filas por posición ----
slice(
  saber_pro_genericas_faedis_2021_2024,
  c(1, 3, 5)
)
```

```
# A tibble: 3 x 21
    ano estu_consecutivo estu_genero estu_semestrecursa
  <int> <chr>            <fct>       <ord>
1  2021 EK202120262469   M           09
2  2021 EK202120116868   M           09
3  2021 EK202120116814   F           09
# i 17 more variables: fami_estratovivienda <ord>,
#   fami_educacionmadre <ord>, fami_educacionpadre <ord>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_mcpio_presentacion <chr>,
#   estu_snies_prgmacademico <int>, ...
```

La función `slice()` tiene asociado un conjunto de funciones auxiliares de la forma `slice_*` que pueden ser útiles para seleccionar determinadas filas:

a. `slice_head()` y `slice_tail()` permiten seleccionar las primeras o últimas filas.
b. `slice_sample()` permite seleccionar filas de manera aleatoria.
c. `slice_min()` y `slice_max()` permiten seleccionar aquellas filas que contienen los valores más pequeños o más grandes de una determinada variable.

La tercera función, correspondiente a `arrange()`, se utiliza para cambiar el orden de las filas con base en un conjunto de columnas. Por defecto, este verbo ordena los datos de forma ascendente donde en caso de necesitar un ordenamiento descendente, se debe aplicar la función auxiliar `desc()` sobre la variable que se desee organizar bajo ese criterio. En la Figura 10.3 se ilustra gráficamente la manipulación que realiza `arrange()`.

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para ordenar la variable `ano` en forma descendente y luego al interior de cada año la variable `estu_semestrecursa` en forma ascendente:

```
arrange(
  saber_pro_genericas_faedis_2021_2024,
  desc(ano),
  estu_semestrecursa
)
```

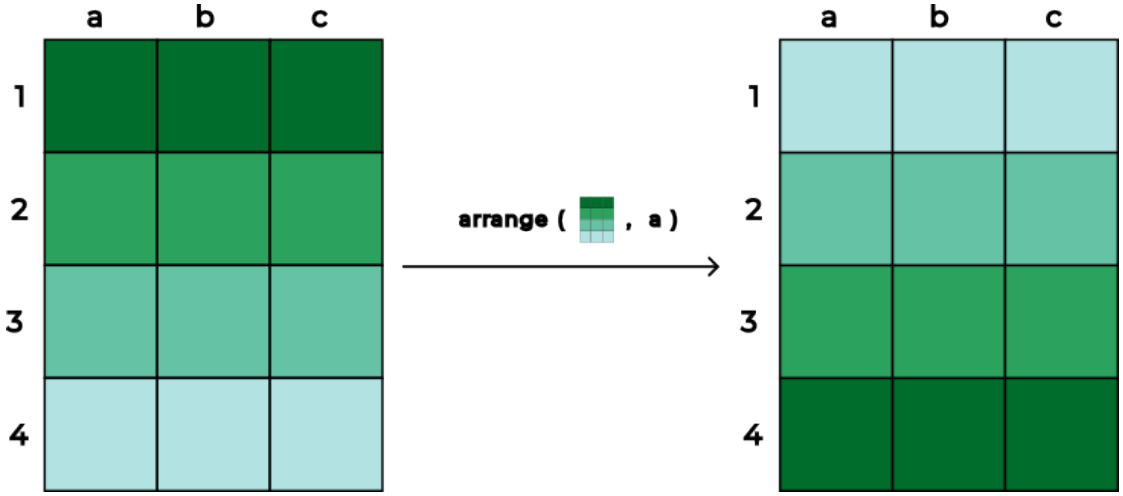


Figura 10.3.: Representación gráfica de la función `arrange()`

```
# A tibble: 1,239 x 21
     ano estu_consecutivo estu_genero estu_semestrecursa
   <int> <chr>            <fct>       <ord>
 1  2024 EK202430030817   F           06
 2  2024 EK202430030783   F           06
 3  2024 EK202410086302   F           07
 4  2024 EK202410000136   F           07
 5  2024 EK202410000145   M           07
 6  2024 EK202410129804   F           07
 7  2024 EK202410067203   M           07
 8  2024 EK202410067226   M           07
 9  2024 EK202410129912   M           07
10  2024 EK202410129981   M           07
# i 1,229 more rows
# i 17 more variables: fami_estratovivienda <ord>,
#   fami_educacionmadre <ord>, fami_educacionpadre <ord>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_mcpio_presentacion <chr>, ...
```

10.1.1.2. Verbos para columnas

Esta categoría incluye las funciones `select()`, `rename()`, `mutate()` y `relocate()`. La primera función, correspondiente a `select()`, se utiliza para determinar qué columnas se incluyen en una tibble. La selección de las columnas se realiza con base en el nombre

de las variables o a su tipo donde se pueden utilizar operadores o funciones auxiliares. En la Figura 10.4 se ilustra gráficamente la manipulación que se puede realizar con la función `select()`.

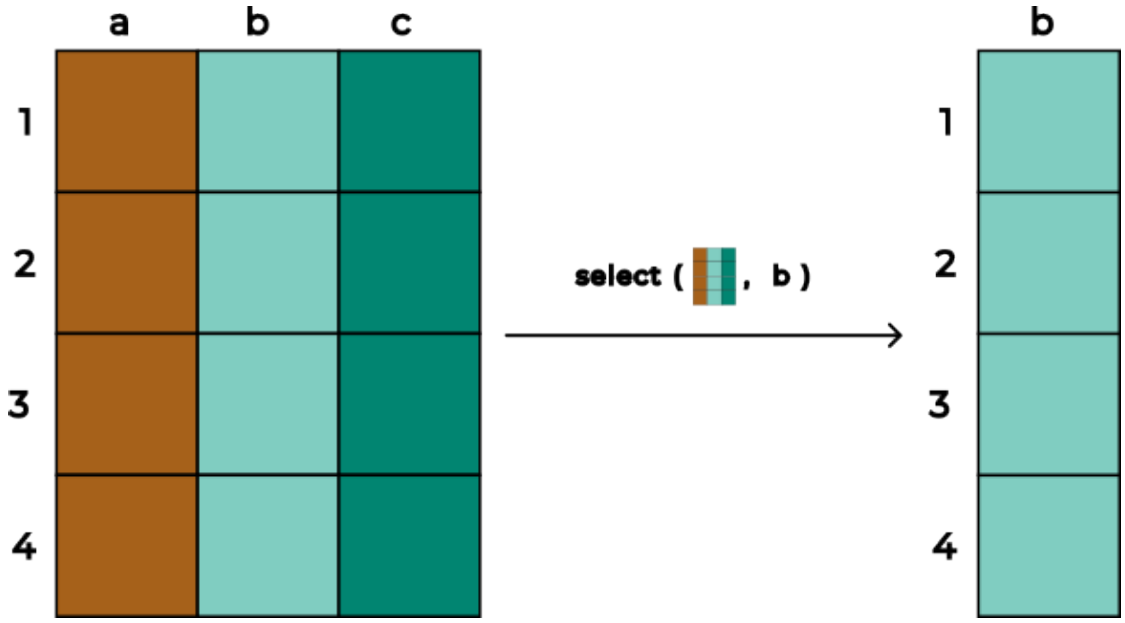


Figura 10.4.: Representación gráfica de la función `select()`

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para seleccionar las columnas asociadas con las variables `ano`, `estu_consecutivo` y `punt_global`:

```
select(
  saber_pro_genericas_faedis_2021_2024,
  ano,
  estu_consecutivo,
  punt_global
)
```

```
# A tibble: 1,239 x 3
     ano estu_consecutivo punt_global
   <int> <chr>                  <dbl>
 1  2021 EK202120262469           139
 2  2021 EK202120116785           170
 3  2021 EK202120116868           128
 4  2021 EK202120116831           155
 5  2021 EK202120116814           166
 6  2021 EK202120117053           142
```

```
 7   2021 EK202120262258           151
 8   2021 EK202120262692           145
 9   2021 EK202120117033           132
10   2021 EK202120117014           146
# i 1,229 more rows
```

En la Sección 10.2 se utilizarán operadores o funciones auxiliares para ilustrar la forma como se pueden seleccionar columnas de manera más precisa. A medida que explores más formas de seleccionar columnas de un conjunto de datos podrás consultar en más detalle la documentación de ayuda de la función `select()` (ver `?select`) respecto a las diferentes funcionalidades de selección.

La segunda función, correspondiente a `rename()`, se utiliza para cambiar el nombre de las columnas usando el esquema `nuevo_nombre = antiguo_nombre`. En la Figura 10.5 se ilustra gráficamente la manipulación que se puede realizar con la función `rename()`.

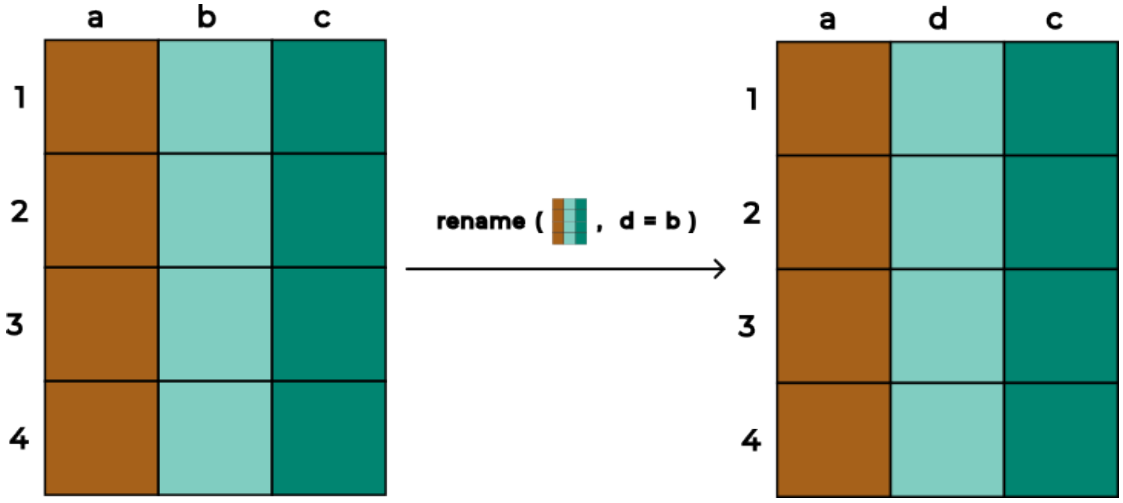


Figura 10.5.: Representación gráfica de la función `rename()`

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para renombrar la columna `estu_consecutivo` por un nombre más descriptivo como `id_saber_pro_estudiante`:

```
rename(
  saber_pro_genericas_faedis_2021_2024,
  id_saber_pro_estudiante = estu_consecutivo
)
```

```
# A tibble: 1,239 x 21
     ano id_saber_pro_estudiante estu_genero
```

```
   <int> <chr>                 <fct>
 1  2021 EK202120262469       M
 2  2021 EK202120116785       M
 3  2021 EK202120116868       M
 4  2021 EK202120116831       M
 5  2021 EK202120116814       F
 6  2021 EK202120117053       F
 7  2021 EK202120262258       F
 8  2021 EK202120262692       F
 9  2021 EK202120117033       F
10  2021 EK202120117014       F
# i 1,229 more rows
# i 18 more variables: estu_semestrecursa <ord>,
#   fami_estratovivienda <ord>, fami_educacionmadre <ord>,
#   fami_educacionpadre <ord>, estu_exterior <chr>,
#   estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>, ...
```

La tercera función, correspondiente a `mutate()`, se utiliza para crear nuevas columnas usualmente a partir de variables existentes dentro de una tibble. También se puede utilizar para modificar columnas si el nombre es el mismo a una columna existente. La función `mutate()` por defecto ubica las nuevas columnas al final de la tibble y mantiene todas las variables presentes en la tibble original. Sin embargo estos aspectos se pueden modificar con los parámetros `.keep`, `.before` y `.after` (ver `?mutate`). En la Figura 10.6 se ilustra gráficamente la manipulación que se puede realizar con la función `mutate()`.

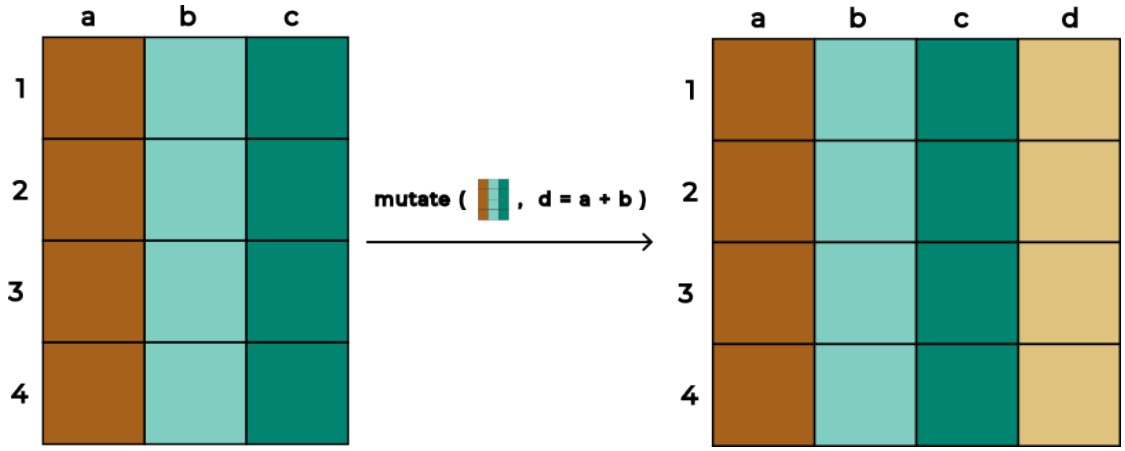


Figura 10.6.: Representación gráfica de la función `mutate()`

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para generar una nueva columna denominada

`punt_global_norm` donde corresponde a la variable `punt_global` normalizada en el intervalo $[0, 1]$[3] y conservando sólo las columnas utilizadas para generar ésta nueva variable a través del parámetro `.keep` con el argumento `"used"`, como `.keep = "used"`:

```
mutate(
  saber_pro_genericas_faedis_2021_2024,
  punt_global_norm = (punt_global - min(punt_global)) /
    (max(punt_global) - min(punt_global)),
  .keep = "used"
)
```

```
# A tibble: 1,239 x 2
   punt_global punt_global_norm
         <dbl>            <dbl>
 1         139            0.610
 2         170            0.746
 3         128            0.561
 4         155            0.680
 5         166            0.728
 6         142            0.623
 7         151            0.662
 8         145            0.636
 9         132            0.579
10         146            0.640
# i 1,229 more rows
```

La cuarta función, correspondiente a `relocate()`, se utiliza para modificar la posición de las columnas utilizando la misma sintaxis de la función `select`. Por defecto esta función reasigna las columnas en las primeras posiciones de la tibble pero a través de los parámetros `.before` y `.after` se puede controlar de manera más precisa la nueva asignación de posiciones indicando antes o después de que variable se desean asignar las nuevas posiciones de determinadas columnas. En la Figura 10.7 se ilustra gráficamente la manipulación que se puede realizar con la función `relocate()`.

Para utilizar un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para mover la variable `punt_global` después de la columna asociada con `estu_consecutivo` utilizando el parámetro `.after` junto con `estu_consecutivo`, como `.after = estu_consecutivo`:

[3] Si x es una variable numérica entonces su normalización en el intervalo $[0, 1]$ se puede definir como $x_{norm} = \frac{x - \text{mín}(x)}{\text{máx}(x) - \text{mín}(x)}$ donde mín(x) es el valor mínimo de la variable x y máx(x) es el valor máximo de la variable x.

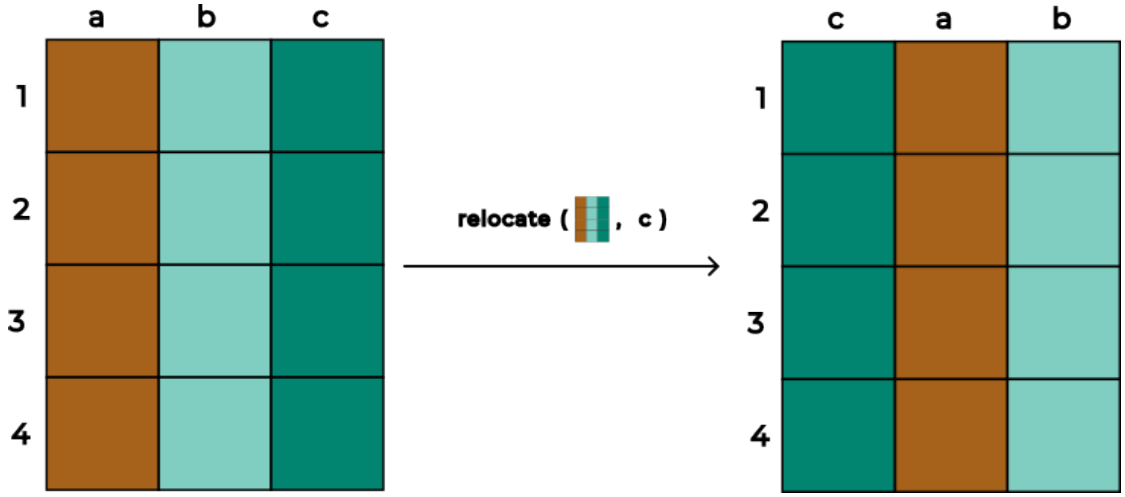


Figura 10.7.: Representación gráfica de la función `relocate()`

```
relocate(
  saber_pro_genericas_faedis_2021_2024,
  punt_global,
  .after = estu_consecutivo
)
```

```
# A tibble: 1,239 x 21
     ano estu_consecutivo punt_global estu_genero
   <int> <chr>                  <dbl> <fct>
 1  2021 EK202120262469           139 M
 2  2021 EK202120116785           170 M
 3  2021 EK202120116868           128 M
 4  2021 EK202120116831           155 M
 5  2021 EK202120116814           166 F
 6  2021 EK202120117053           142 F
 7  2021 EK202120262258           151 F
 8  2021 EK202120262692           145 F
 9  2021 EK202120117033           132 F
10  2021 EK202120117014           146 F
# i 1,229 more rows
# i 17 more variables: estu_semestrecursa <ord>,
#   fami_estratovivienda <ord>, fami_educacionmadre <ord>,
#   fami_educacionpadre <ord>, estu_exterior <chr>,
#   estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>, ...
```

Tabla 10.1.: Uso e interpretación del operador pipe (`|>`)

Uso	Interpretación
$a \mid> f(x)$	$f(x = a)$
$b \mid> f(x, y)$	$f(x = b, y)$
$c \mid> f(x) \mid> g(y)$	$g(y = f(x = c))$
$d \mid> f(x, y = _)$	$f(x, y = d)$

10.1.1.3. Operador pipe (`|>`)

Cuando se manipulan datos usualmente se requieren combinar múltiples verbos para contestar una determinada pregunta. El operador **pipe** (`|>`) permite realizar este proceso donde fue introducido en la versión `4.1.0` de R y motivado por un operador similar del paquete `magrittr` perteneciente al `tidyverse` (ver `` ?`|>` ``). Debido a que en el paquete `dplyr` se utilizan funciones para manipular datos en la Tabla 10.1 se ilustra el uso y la interpretación del operador `|>`.

Para utilizar un ejemplo concreto utilizando el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` y una aplicación del operador `|>` se podría por ejemplo estar interesado en identificar los casos atípicos en los cuales no sólo el puntaje del módulo de razonamiento cuantitativo o de lectura crítica es 0 (ver Listado 9.1) sino si se presenta en al menos alguno de los otros módulos utilizando las funciones `select()`, `filter()` y `arrange()` más `glimpse()` así como el operador lógico vectorizado `|` (ver Tabla 5.3), tal como se señala a continuación:

```
Rows: 85
Columns: 9
$ ano                      <int> 2021, 2021, 2022, 2022, ~
$ estu_consecutivo         <chr> "EK202120264262", "EK202~
$ estu_snies_prgmacademico <int> 6527, 6527, 6527, 6527, ~
$ estu_prgm_academico      <chr> "ADMINISTRACION DE EMPRE~
$ mod_competen_ciudada_punt <dbl> 77, 0, 0, 136, 122, 83, ~
$ mod_comuni_escrita_punt  <dbl> 0, 128, 0, 0, 0, 0, 0, 1~
$ mod_ingles_punt          <dbl> 187, 0, 0, 164, 144, 146~
$ mod_lectura_critica_punt <dbl> 92, 64, 0, 143, 132, 123~
$ mod_razona_cuantitat_punt <dbl> 130, 0, 0, 107, 155, 146~
```

> **Tip 2**
>
> En Positron, puedes insertar el operador `|>` utilizando la combinación de teclas `Shift-Command-M` (mac), `Ctrl-Shift-M` (linux), `Ctrl-Shift-M` (windows) de tal manera que no tengas que digitarlo de manera manual.

Listado 10.1 Ejemplo en el uso del operador pipe (|>) para manipular datos

```
saber_pro_genericas_faedis_2021_2024 |>
  select(
    ano,
    estu_consecutivo,
    estu_snies_prgmacademico,
    estu_prgm_academico,
    mod_competen_ciudada_punt,
    mod_comuni_escrita_punt,
    mod_ingles_punt,
    mod_lectura_critica_punt,
    mod_razona_cuantitat_punt
  ) |>
  filter(
    mod_competen_ciudada_punt == 0 |
      mod_comuni_escrita_punt == 0 |
      mod_ingles_punt == 0 |
      mod_lectura_critica_punt == 0 |
      mod_razona_cuantitat_punt == 0
  ) |>
  arrange(estu_prgm_academico) |>
  glimpse()
```

El Listado 10.1 ilustra las ventajas de hacer uso del operador |> para que el código sea más legible donde permite:

a. Estructurar el proceso de manipulación de datos de manera secuencial de izquierda a derecha y leer el código de arriba hacia abajo
b. Evita el uso de funciones anidadas
c. Minimiza la necesidad de crear variables intermedias o definiciones de funciones
d. Facilita agregar o eliminar pasos en cualquier parte de la secuencia en caso de que se requiera

Respecto al punto a. con relación al código del Listado 10.1 es posible identificar inicialmente que se parte de la variable saber_pro_genericas_faedis_2021_2024, luego se seleccionan determinadas columnas, posteriormente se filtran determinadas filas utilizando ciertas condiciones lógicas para finalmente ordenar las observaciones de interés con base en los nombres de los programas académicos para darles un "vistazo".

En caso de que no se use el operador |> existen 2 alternativas que se señalan en los puntos b. y c. donde el primero se ilustra en el Listado 10.2 utilizando el mismo código pero con funciones anidadas, tal como se señala a continuación:

Listado 10.2 Uso de funciones anidadas cuando no se usa el operador |>

```
glimpse(
  arrange(
    filter(
      select(
        saber_pro_genericas_faedis_2021_2024,
        ano,
        estu_consecutivo,
        estu_snies_prgmacademico,
        estu_prgm_academico,
        mod_competen_ciudada_punt,
        mod_comuni_escrita_punt,
        mod_ingles_punt,
        mod_lectura_critica_punt,
        mod_razona_cuantitat_punt
      ),
      mod_competen_ciudada_punt == 0 |
        mod_comuni_escrita_punt == 0 |
        mod_ingles_punt == 0 |
        mod_lectura_critica_punt == 0 |
        mod_razona_cuantitat_punt == 0
    ),
    estu_prgm_academico
  )
)
```

En el Listado 10.2 para entender el código, se debe leerlo de adentro hacia afuera donde es más difícil de comprender si se busca identificar la secuencia de pasos utilizada. También es posible crear variables intermedias donde este enfoque se ilustra en el Listado 10.3 utilizando el mismo código, tal como se señala a continuación:

En el Listado 10.3 se crean variables intermedias adicionales para un análisis sencillo. Sin embargo, esta práctica puede saturar el entorno de trabajo, aumentar el riesgo de cometer errores por confusión de nombres y dificulta el mantenimiento del código si se desea agregar un nuevo paso en la secuencia[4].

Esto no significa que se deba optar por el extremo opuesto y utilizar el operador |> de manera indiscriminada para generar secuencias interminables de pasos. En la práctica en el análisis de datos, tarde o temprano será necesario e incluso recomendable crear

[4]Aunque nombrar variables en este contexto parece un aspecto menor, en las ciencias de la computación es célebre la frase de Philip Lewis Karlton durante su tiempo en Netscape, donde señaló que *"There are only two hard things in Computer Science: cache invalidation and naming things."* (Karlton, 2017)

variables intermedias. La clave no es eliminar por completo la creación de variables, sino aprender a gestionar un conjunto mínimo y adecuado de variables con un propósito claro, en lugar de lidiar con una enorme cantidad de objetos.

En la Tabla 10.1 se señala en la última fila el uso del guion bajo (_). Este elemento funciona como un marcador de posición (*placeholder* en inglés) que sirve para controlar de manera explícita en qué lugar se insertará el objeto debido a que, por defecto, este se introduce como el primer argumento de una función al utilizar el operador `|>`. En algunos casos se hace necesario modificar este comportamiento por defecto donde por ejemplo el Listado 9.3 se puede reescribir utilizando el operador `|>`, tal como se señala a continuación:

```
Call:
lm(formula = mod_razona_cuantitat_punt ~ mod_lectura_critica_punt,
    data = saber_pro_genericas_faedis_2021_2024)

Coefficients:
             (Intercept)  mod_lectura_critica_punt
                 64.0985                    0.5856
```

10.1.1.4. Verbos para colapsar, agrupar y desagrupar filas

Esta categoría incluye las funciones `group_by()`, `summarise()` y `ungroup()`. La primera función, correspondiente a `group_by()`, se utiliza para convertir una tibble en una tibble agrupada de tal manera que los verbos posteriores que se utilicen se apliquen por grupo. En la Figura 10.8 se ilustra gráficamente la manipulación que se puede realizar con la función `group_by()`.

Para ilustrar el comportamiento de esta función con un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` para agruparlos por año y el género a través de las variables `ano` y `estu_genero`:

```
saber_pro_genericas_faedis_2021_2024 |>
  group_by(
    ano,
    estu_genero
  )
```

```
# A tibble: 1,239 x 21
# Groups:   ano, estu_genero [8]
     ano estu_consecutivo estu_genero estu_semestrecursa
   <int> <chr>            <fct>       <ord>
```

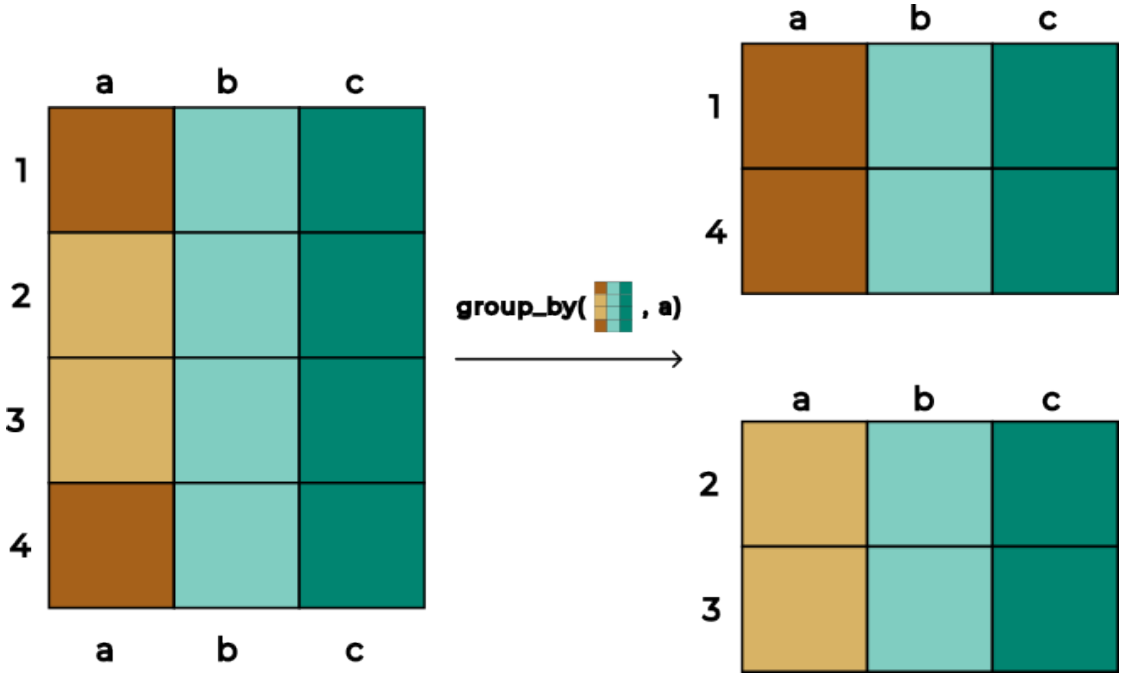


Figura 10.8.: Representación gráfica de la función `group_by()`

```
 1  2021 EK202120262469   M           09
 2  2021 EK202120116785   M           09
 3  2021 EK202120116868   M           09
 4  2021 EK202120116831   M           09
 5  2021 EK202120116814   F           09
 6  2021 EK202120117053   F           09
 7  2021 EK202120262258   F           08
 8  2021 EK202120262692   F           09
 9  2021 EK202120117033   F           08
10  2021 EK202120117014   F           08
# i 1,229 more rows
# i 17 more variables: fami_estratovivienda <ord>,
#   fami_educacionmadre <ord>, fami_educacionpadre <ord>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_mcpio_presentacion <chr>, ...
```

En la parte superior de la tibble además de indicar el número de filas así como las columnas que la componen (ver Listado 4.3) se incluye además un segundo comentario, `# Groups:  ano, estu_genero [8]`, que indica las variables que se utilizaron para realizar la agrupación así como el número de grupos formados. En este caso son 8 teniendo

en cuenta que se tienen 4 años (2021, 2022, 2023 y 2024) y 2 categorías de género (F, M) para un total de $4 \times 2 = 8$ grupos.

La segunda función, correspondiente a `summarise()`, se utiliza para crear una tibble donde si existen variables agrupadas genera una fila por cada combinación de variables agrupadas y una columna por cada estadística descriptiva que se especifique. En caso que no existan variables agrupadas genera una sola fila con la estadística descriptiva que se especifique y una columna por cada estadística descriptiva que se defina. En la Figura 10.9 se ilustra gráficamente la manipulación que se puede realizar con la función `summarise()`.

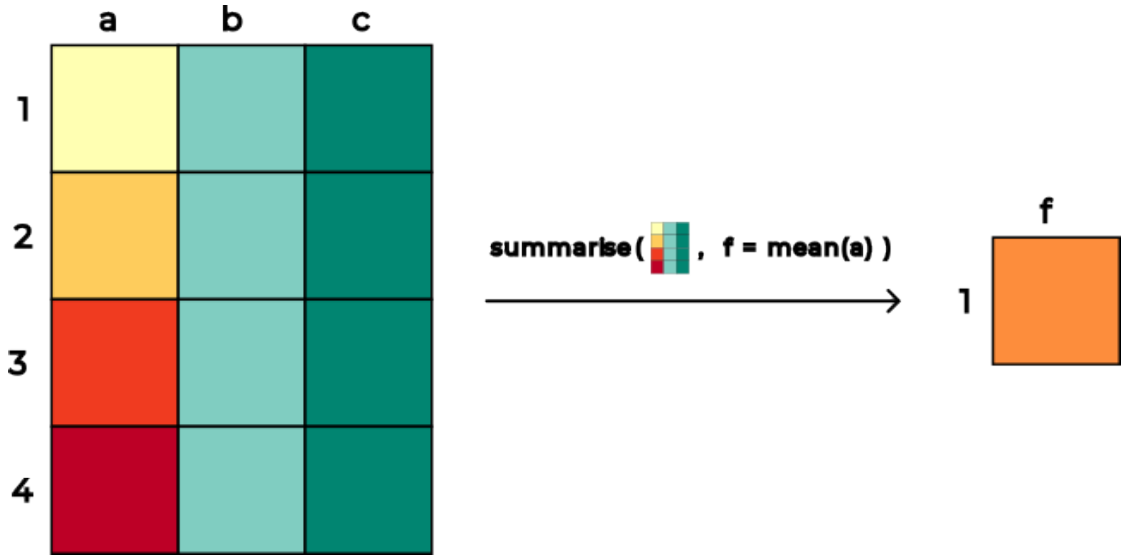


Figura 10.9.: Representación gráfica de la función `summarise()`

Para ilustrar el comportamiento de esta función con un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` donde con las variables agrupadas `ano` y `estu_genero` es posible calcular el puntaje global promedio redondeado a 0 decimales por año y género:

```
saber_pro_genericas_faedis_2021_2024 |>
  group_by(
    ano,
    estu_genero
  ) |>
  summarise(
    punt_global = mean(punt_global) |>
      round(digits = 0)
  )
```

```
`summarise()` has regrouped the output.
```

```
i Summaries were computed grouped by ano and estu_genero.
i Output is grouped by ano.
i Use `summarise(.groups = "drop_last")` to silence this
  message.
i Use `summarise(.by = c(ano, estu_genero))` for
  per-operation grouping (`?dplyr::dplyr_by`) instead.

# A tibble: 8 x 3
# Groups:   ano [4]
    ano estu_genero punt_global
  <int> <fct>             <dbl>
1  2021 F                   147
2  2021 M                   148
3  2022 F                   144
4  2022 M                   147
5  2023 F                   148
6  2023 M                   147
7  2024 F                   150
8  2024 M                   148
```

Por defecto cuando existen variables agrupadas la función `summarise()` desagrupa por la última variable el resultado final, en este caso por `estu_genero`. Debido a lo anterior el paquete `dplyr` genera un mensaje indicando este comportamiento así como diferentes alternativas para no utilizar la función `group_by()` mediante el uso del parámetro `.by` y el argumento `c(ano, estu_genero)`, como `.by = c(ano, estu_genero)`. Aunque existe la anterior alternativa (ver Importante 8) a lo largo del libro se utilizará de manera explícita la función `group_by()` cuando se busque hacer uso de la función `summarise()` para calcular estadísticas descriptivas por grupos.

! Importante 8

En el paquete `dplyr` existen 2 formas de agrupar:

1. Utilizando `group_by()` donde la agrupación persiste al aplicar diferentes verbos.
2. Utilizando los parámetros `.by` o `by` dentro de un determinado verbo donde la agrupación sólo afecta el verbo donde se utilice el parámetro.

El lector interesado puede consultar la documentación oficial de `dplyr` para conocer más detalles de este aspecto en caso de que sea de su interés. Sin embargo, a lo largo del libro utilizaremos la función `group_by()` de manera explícita y no los parámetros `.by` o `by` dentro de los verbos.
Una vez estés familiarizado con el paquete `dplyr` podrás examinar el comportamiento de utilizar una u otra opción. Por ahora el enfoque es conocer los diferentes verbos

disponibles en el contexto de la gramática de manipulación de datos.

La tercera función, correspondiente a `ungroup()`, se utiliza para desagrupar una tibble agrupada. En caso que no se especifiquen de manera explícita las variables a utilizar para realizar la desagrupación esta función genera una tibble totalmente desagrupada. En la Figura 10.10 se ilustra gráficamente la manipulación que se puede realizar con la función `ungroup()`.

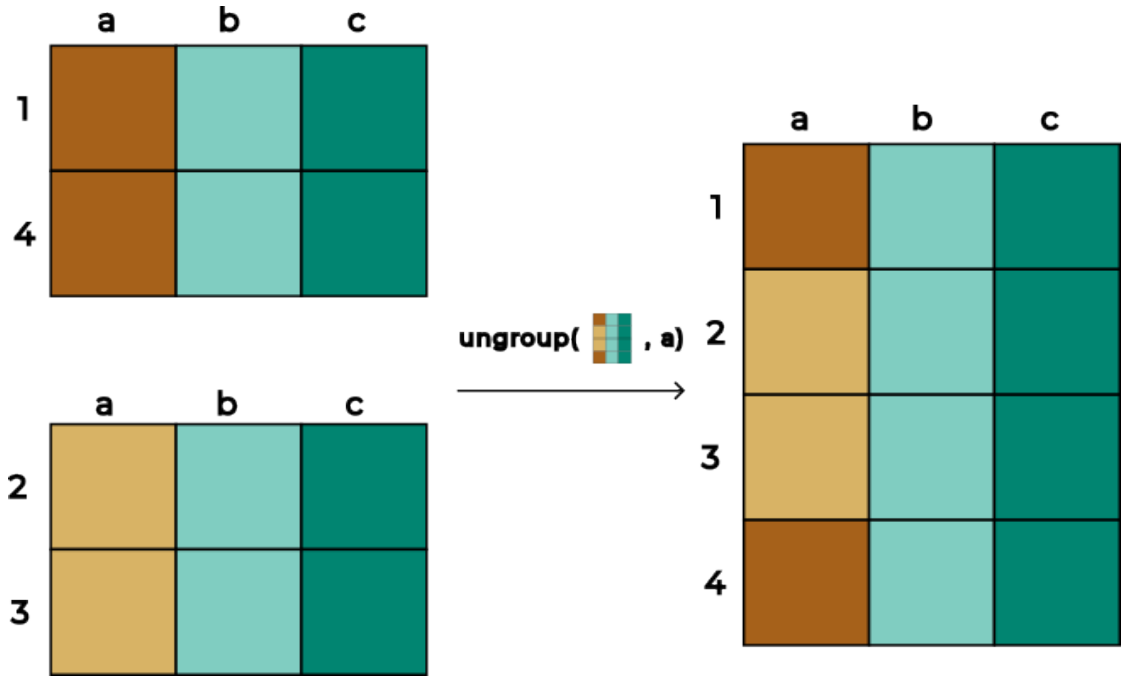


Figura 10.10.: Representación gráfica de la función `ungroup()`

Para ilustrar el comportamiento de esta función con un ejemplo concreto puedes utilizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` donde es posible desagrupar completamente una tibble, que puede ser utilizada como insumo para crear un gráfico similar a la Figura 9.2 pero por género:

```
saber_pro_genericas_faedis_2021_2024 |>
  group_by(
    ano,
    estu_genero
  ) |>
  summarise(
    punt_global = mean(punt_global) |>
      round(digits = 0)
  ) |>
```

```
  ungroup()
```

```
# A tibble: 8 x 3
    ano estu_genero punt_global
  <int> <fct>             <dbl>
1  2021 F                   147
2  2021 M                   148
3  2022 F                   144
4  2022 M                   147
5  2023 F                   148
6  2023 M                   147
7  2024 F                   150
8  2024 M                   148
```

10.2. Caso de estudio: obtención de datos de interés del Saber-Pro para FAEDIS

El propósito de la Sección 10.2 es aplicar la gramática de manipulación de datos para ilustrar su uso partiendo de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado en Colombia para el periodo 2021-2024. El objetivo es obtener el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` descrito en la Sección 6.2.1 así como el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds` descrito en la Sección 9.2.2.

10.2.1. Datos

Para desarrollar el caso de estudio es necesario en primera instancia importar los datos para explorarlos en R como se señala a continuación, teniendo en cuenta lo indicado en la Nota 19:

> **Nota 20**
>
> En el Capítulo 13 se abarcará con más detalle la importación de datos para archivos con extensión `.csv` mediante la función `read_csv()`, así como la razón detrás del uso del parámetro `col_types` y su respectivo argumento con la función `cols()`, teniendo en cuenta lo indicado en el Listado 10.5. Por ahora, el enfoque del Capítulo 10 será comprender la gramática de manipulación de datos mediante el paquete `dplyr`.
> Adicionalmente, es necesario señalar que este archivo cuenta con una extensión combinada `.csv.xz`. El sufijo `.xz` indica simplemente que el archivo de texto original (`.csv`) fue comprimido mediante un algoritmo de alta eficiencia para reducir

> drásticamente su tamaño en el disco. No obstante, no es necesario profundizar en este detalle técnico para el desarrollo del Capítulo 10 ni del Capítulo 13, dado que la función `read_csv()` detecta dicha extensión de forma automática y descomprime los datos de manera transparente al momento de realizar el proceso de importación.

Una vez importes el conjunto de datos, podrás darle un "vistazo" utilizando la función `glimpse()` para identificar el número y los tipos de variables que lo componen, así como la cantidad de observaciones disponibles.

```
# Inspeccionar datos ----
glimpse(saber_pro_genericas_2021_2024)
```

```
Rows: 1,015,295
Columns: 82
$ estu_actividadrefuerzoareas   <chr> NA, NA, NA, "Si", NA~
$ estu_actividadrefuerzogeneric <chr> NA, NA, NA, "Si", NA~
$ estu_agregado                 <chr> "S", "S", "S", "S", ~
$ estu_cod_depto_presentacion   <chr> "11", "11", "11", "1~
$ estu_cod_mcpio_presentacion   <chr> "11001", "11001", "1~
$ estu_consecutivo              <chr> "EK202120193457", "E~
$ estu_cursodocentesies         <chr> NA, NA, NA, "Entre 2~
$ estu_cursoiesapoyoexterno     <chr> NA, NA, NA, "Entre 2~
$ estu_depto_presentacion       <chr> "BOGOTÁ", "BOGOTÁ", ~
$ estu_estudiante               <chr> "ESTUDIANTE", "ESTUD~
$ estu_etnia                    <chr> NA, NA, NA, "Afrodes~
$ estu_exterior                 <chr> "NO", "NO", "NO", "N~
$ estu_fechanacimiento          <chr> "1992-10-05", "1994-~
$ estu_genero                   <chr> "M", "M", "M", "F", ~
$ estu_horassemanatrabaja       <chr> "Menos de 10 horas",~
$ estu_inse_individual          <dbl> 56.97800, 75.18547, ~
$ estu_inst_codmunicipio        <chr> "11001", "11001", "6~
$ estu_inst_departamento        <chr> "BOGOTÁ", "BOGOTÁ", ~
$ estu_inst_municipio           <chr> "BOGOTÁ D.C.", "BOGO~
$ estu_mcpio_presentacion       <chr> "BOGOTÁ D.C.", "BOGO~
$ estu_metodo_prgm              <chr> "PRESENCIAL", "PRESE~
$ estu_nacionalidad             <chr> "COLOMBIA", "COLOMBI~
$ estu_nivel_prgm_academico     <chr> "UNIVERSITARIO", "UN~
$ estu_nse_individual           <dbl> 3, 4, 4, 1, 1, 1, 2,~
$ estu_nucleo_pregrado          <chr> "INGENIERÍA DE SISTE~
$ estu_pagomatriculabeca        <chr> "No", "No", "No", "S~
$ estu_pagomatriculacredito     <chr> "No", "Si", "No", "N~
$ estu_pagomatriculapadres      <chr> "No", "Si", "Si", "N~
$ estu_pagomatriculapropio      <chr> "Si", "No", "No", "N~
```

```
$ estu_pais_reside              <chr> "COLOMBIA", "COLOMBI~
$ estu_prgm_academico           <chr> "INGENIERÍA DE SISTE~
$ estu_prgm_codmunicipio        <chr> "11001", "11001", "6~
$ estu_prgm_departamento        <chr> "BOGOTÁ", "BOGOTÁ", ~
$ estu_prgm_municipio           <chr> "BOGOTÁ D.C.", "BOGO~
$ estu_privado_libertad         <chr> "N", "N", "N", "N", ~
$ estu_semestrecursa            <chr> "10", "08", "10", "0~
$ estu_simulacrotipoicfes       <chr> NA, NA, NA, "Si", NA~
$ estu_snies_prgmacademico      <dbl> 106661, 1536, 2524, ~
$ estu_tieneetnia               <chr> "No", "No", "No", "S~
$ estu_tipodocumento            <chr> "CC", "CC", "CC", "C~
$ estu_tituloobtenidobachiller  <chr> "Bachiller técnico",~
$ fami_cuantoscompartebaño      <chr> "NINGUNA", "NINGUNA"~
$ fami_educacionmadre           <chr> "Educación profesion~
$ fami_educacionpadre           <chr> "Educación profesion~
$ fami_estratovivienda          <chr> "Estrato 3", "Estrat~
$ fami_ocupacionmadre           <chr> "Hogar", "Empleado d~
$ fami_ocupacionpadre           <chr> "Empleado de nivel t~
$ fami_tieneautomovil           <chr> "Si", "Si", "Si", NA~
$ fami_tienecomputador          <chr> "Si", "Si", "Si", NA~
$ fami_tieneconsolavideojuegos  <chr> "No", "Si", "Si", NA~
$ fami_tienehornomicroogas      <chr> "No", "Si", "Si", NA~
$ fami_tieneinternet            <chr> "Si", "Si", "Si", "S~
$ fami_tienelavadora            <chr> "Si", "Si", "Si", NA~
$ fami_tienemotocicleta         <chr> "No", "Si", "No", NA~
$ fami_tieneserviciotv          <chr> "No", "Si", "Si", "S~
$ inst_caracter_academico       <chr> "UNIVERSIDAD", "UNIV~
$ inst_cod_institucion          <dbl> 1101, 1813, 1723, 12~
$ inst_nombre_institucion       <chr> "UNIVERSIDAD NACIONA~
$ inst_origen                   <chr> "OFICIAL NACIONAL", ~
$ mod_competen_ciudada_desem    <dbl> 4, 3, 3, 2, 1, 2, 3,~
$ mod_competen_ciudada_pnal     <dbl> 100, 94, 75, 32, 25,~
$ mod_competen_ciudada_pnbc     <dbl> 100, 97, 66, 40, 30,~
$ mod_competen_ciudada_punt     <dbl> 220, 193, 168, 127, ~
$ mod_comuni_escrita_desem      <dbl> 4, 2, 2, 2, 2, 2, NA~
$ mod_comuni_escrita_pnal       <dbl> 96, 28, 51, 26, 45, ~
$ mod_comuni_escrita_pnbc       <dbl> 97, 31, 50, 25, 52, ~
$ mod_comuni_escrita_punt       <dbl> 186, 123, 139, 122, ~
$ mod_ingles_desem              <chr> "B2", "B2", "B2", "A~
$ mod_ingles_pnal               <dbl> 93, 99, 90, 48, 41, ~
$ mod_ingles_pnbc               <dbl> 91, 99, 89, 57, 55, ~
$ mod_ingles_punt               <dbl> 202, 218, 197, 149, ~
$ mod_lectura_critica_desem     <dbl> 4, 3, 3, 2, 3, 1, 3,~
$ mod_lectura_critica_pnal      <dbl> 98, 84, 81, 29, 67, ~
```

```
$ mod_lectura_critica_pnbc      <dbl> 98, 91, 75, 35, 81, ~
$ mod_lectura_critica_punt      <dbl> 205, 178, 174, 127, ~
$ mod_razona_cuantitat_punt     <dbl> 300, 181, 193, 137, ~
$ mod_razona_cuantitativo_pnal  <dbl> 100, 87, 93, 43, 73,~
$ mod_razona_cuantitativo_pnbc  <dbl> 100, 94, 71, 61, 79,~
$ percentil_global              <dbl> 100, 90, 87, 33, 50,~
$ ano                           <dbl> 2021, 2021, 2021, 20~
$ periodo                       <dbl> 2, 2, 2, 2, 2, 2, 2,~
$ punt_global                   <dbl> 223, 179, 174, 132, ~
```

El conjunto de datos `2021_2024-saber_pro-genericas.csv.xz` corresponden a los microdatos anonimizados de los resultados del examen Saber Pro del grupo de competencias genéricas para Colombia donde se consolidó la información de tal manera que todas las 82 variables que se incluyen estuvieran disponibles para todo el periodo 2021-2024. Al conjunto de datos se le realizó una mínima limpieza respecto a los datos originales disponibles en el portal DataIcfes con relación a las variables `ano`, `periodo`, `estu_fechanacimiento`, `estu_cod_depto_presentacion`, `estu_inst_departamento`, `estu_cod_mcpio_presentacion`, `estu_inst_codmunicipio` y `estu_prgm_codmunicipio`.

En el conjunto de datos `2021_2024-saber_pro-genericas.csv.xz` una variable clave a tener en cuenta y que se utilizará más adelante es **estu_agregado** donde indica si la observación es (`S` para Sí) o no (`N` para No) incluida en el informe nacional de resultados del Saber-Pro. Las demás variables no se describirán teniendo en cuenta que no son relevantes para el desarrollo del estudio de caso (ver Nota 19) o cuya descripción ya se señaló en la Sección 6.2.1.1.

> **Nota 21**
>
> Debido a que el objetivo es utilizar la gramática de manipulación de datos para reconstruir los conjuntos de datos `2021-2024_saber-pro-genericas_faedis.rds` y `2021-2024_saber-pro-genericas_faedis_programas.rds` en la Sección 10.2.1 no se describen cada una de las 82 variables. Sin embargo, el lector interesado puede descargar el diccionario de datos, publicado originalmente en el portal DataIcfes, donde se describen cada una de estas variables e información adicional:
>
> **diccionario-datos-examen-saber-pro.pdf**
>
> En la sección **2. Periodo 2021 - 2024** del anterior documento, a partir de la página 17, el lector podrá encontrar esta información respecto al periodo 2021-2024. La única variable que no encontrará es `ano` donde en los datos originales se agrupan en la variable `periodo` tanto el año como el periodo de presentación. En la limpieza de datos se separó esta información en las variables `ano` y `periodo`.

10.2.2. Filtrar (`filter()`)

El primer paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` comprende filtrar las filas relacionadas con las observaciones que se tienen en cuenta en el informe nacional de resultados del Saber-Pro a través de la variable `estu_agregado` así como de los programas de FAEDIS a partir de los códigos del SNIES.

Inicialmente respecto al segundo aspecto, se podría adoptar una estrategia similar a la descrita en el Listado 10.1, generando un conjunto de condiciones lógicas mediante el operador lógico vectorizado `|` para obtener las observaciones correspondientes a los programas académicos de FAEDIS, verificando si la variable `estu_snies_prgmacademico` es igual a cada uno de los códigos SNIES de los programas de FAEDIS. Sin embargo, R cuenta con el operador binario `%in%`, el cual ofrece una interfaz más intuitiva al devolver un vector lógico que indica si existe o no una coincidencia para su operando izquierdo dentro del conjunto de elementos ubicado a la derecha, tal como se señala a continuación:

```
# A tibble: 1,239 x 82
   estu_actividadrefuerzoareas estu_actividadrefuerzogeneric
   <chr>                       <chr>
 1 <NA>                        <NA>
 2 No                          Si
 3 <NA>                        <NA>
 4 <NA>                        <NA>
 5 <NA>                        <NA>
 6 <NA>                        <NA>
 7 <NA>                        <NA>
 8 <NA>                        <NA>
 9 <NA>                        <NA>
10 <NA>                        <NA>
# i 1,229 more rows
# i 80 more variables: estu_agregado <chr>,
#   estu_cod_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_consecutivo <chr>, estu_cursodocentesies <chr>,
#   estu_cursoiesapoyoexterno <chr>,
#   estu_depto_presentacion <chr>, ...
```

De esa manera, mediante el código del Listado 10.6 es posible obtener inicialmente las 1239 filas que se indicaron en la Sección 6.2.1.

10.2.3. Seleccionar (`select()`)

El segundo paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` comprende seleccionar las columnas relacionadas con las 20 variables indicadas en la Sección 6.2.1. Inicialmente se podría seleccionar nombre por nombre incluyendo cada una de las variables en la función `select()` pero se pueden utilizar operadores o funciones auxiliares para optimizar este proceso. A continuación se señalan algunos de ellos que se utilizarán más adelante en el estudio de caso:

- `:`: permite seleccionar un rango consecutivo de variables
- `&` y `|`: son operadores lógicos vectorizados (ver Tabla 5.3) que se pueden utilizar para seleccionar la unión o intersección de un conjunto de variables
- `starts_with()`: permite seleccionar variables que comienzan con un determinado prefijo
- `ends_with()`: permite seleccionar variables que terminan con un determinado sufijo

Utilizando estos operadores o funciones auxiliares es posible seleccionar las 20 variables correspondientes al conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds`, tal como se señala a continuación:

```
# A tibble: 1,239 x 21
     ano estu_consecutivo estu_genero estu_semestrecursa
   <dbl> <chr>            <chr>       <chr>
 1  2021 EK202120262469   M           09
 2  2021 EK202120116785   M           09
 3  2021 EK202120116868   M           09
 4  2021 EK202120116831   M           09
 5  2021 EK202120116814   F           09
 6  2021 EK202120117053   F           09
 7  2021 EK202120262258   F           08
 8  2021 EK202120262692   F           09
 9  2021 EK202120117033   F           08
10  2021 EK202120117014   F           08
# i 1,229 more rows
# i 17 more variables: fami_educacionmadre <chr>,
#   fami_educacionpadre <chr>, fami_estratovivienda <chr>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_mcpio_presentacion <chr>, ...
```

En el Listado 10.7 se utiliza `:` como `fami_educacionmadre:fami_estratovivienda` para seleccionar las variables `fami_educacionmadre`, `fami_educacionpadre` y `fami_estratovivienda` teniendo en cuenta que son variables que se encuentran

de manera consecutiva. En el caso de `ends_with()` esta función se utiliza como `ends_with("_presentacion")` para seleccionar los nombres y los códigos de los departamentos y municipios de presentación del Saber-Pro. Finalmente el operador lógico vectorizado `|` junto con las funciones `ends_with()` y `starts_with()` se utilizan como `ends_with("_punt") | starts_with("punt_")` para extraer el puntaje de los módulos correspondientes a las competencias genéricas así como el puntaje global.

10.2.4. Reubicar (`relocate()`)

El tercer paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` comprende reubicar algunas de las columnas relacionadas con las 20 variables indicadas en la Sección 6.2.1 utilizando la función `relocate`, tal como se señala a continuación:

```
# A tibble: 1,239 x 21
     ano estu_consecutivo estu_genero estu_semestrecursa
   <dbl> <chr>            <chr>       <chr>
 1  2021 EK202120262469   M           09
 2  2021 EK202120116785   M           09
 3  2021 EK202120116868   M           09
 4  2021 EK202120116831   M           09
 5  2021 EK202120116814   F           09
 6  2021 EK202120117053   F           09
 7  2021 EK202120262258   F           08
 8  2021 EK202120262692   F           09
 9  2021 EK202120117033   F           08
10  2021 EK202120117014   F           08
# i 1,229 more rows
# i 17 more variables: fami_estratovivienda <chr>,
#   fami_educacionmadre <chr>, fami_educacionpadre <chr>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_mcpio_presentacion <chr>, ...
```

En el Listado 10.8 la reubicación de columnas tiene el objetivo de agrupar las variables por categorías de tal manera que al analizar el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` sea más fácil de organizar. En este caso la reubicación tiene como propósito la siguiente agrupación por categorías:

- **Periodo**: `ano`
- **Id**: `estu_consecutivo`
- **Características de quien presenta la prueba**: `estu_genero` y `estu_semestrecursa`

- **Entorno familiar**: `fami_estratovivienda`, `fami_educacionmadre`, y `fami_educacionpadre`
- **Lugar de presentación de la prueba**: `estu_exterior`, `estu_cod_depto_presentacion`, `estu_depto_presentacion`, `estu_cod_mcpio_presentacion` y `estu_mcpio_presentacion`
- **Programa académico**: `estu_snies_prgmacademico` y `estu_prgm_academico`
- **Resultados de la prueba**:
  - **Ciudadanas**: `mod_competen_ciudada_punt`
  - **Comunicación escrita**: `mod_comuni_escrita_punt`
  - **Inglés**: `mod_ingles_punt` y `mod_ingles_desem`
  - **Lectura crítica**: `mod_lectura_critica_punt`
  - **Razonamiento cuantitativo**: `mod_razona_cuantitat_punt`
  - **Puntaje global**: `punt_global`

10.2.5. Modificar (`mutate()`)

El cuarto paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds` comprende modificar algunas de las columnas relacionadas con las 20 variables indicadas en la Sección 6.2.1 utilizando la función `mutate()`. Aunque la función `mutate()` también se puede utilizar para generar nuevas variables el propósito en este caso es asociar el tipo adecuado de ciertas variables dentro de R. Algunas variables no son continuas sino discretas y en otros casos existen variables categóricas que son nominales u ordinales pero que están codificadas como caracteres.

R implementa diferentes tipos de datos que se pueden utilizar para acomodar los aspectos mencionados anteriormente. Aunque no abarcaremos en la Sección 10.2.5 todos los tipos de datos que R implementa, en la Tabla 10.2 se indican algunos de los tipos más comunes y que se utilizarán más adelante. El lector interesado en conocer en más detalle los tipos de datos más comunes y su funcionamiento puede consultar (Wickham, 2019, Capítulo 3).

Tabla 10.2.: Algunos tipos de datos en R

Clase	Tipo	Ejemplo	Etiqueta en `tibble`
Vector atómico	`logical`	`TRUE`	`<lgl>`
	`integer`	`1L`	`<int>`
	`double`	`152.35`	`<dbl>`
	`character`	`"Ingeniería Civil"`	`<chr>`
Lista	`list`	`list(TRUE, 1L, 152.35, "Ingeniería Civil")`	`<list>`

Clase	Tipo	Ejemplo	Etiqueta en `tibble`
Objeto integrado	`factor`	`factor(x = c("Femenino", "Masculino"))`	`<fct>`
	`ordered`	`factor(x = c("10", "11", "12 o más"), ordered = TRUE)`	`<ord>`
	`Date`	`ymd("2021-06-01")`	`<date>`

En el Listado 10.9 se utilizan las funciones `as.integer()` y `factor()` junto con la función `mutate()` para transformar variables que son inicialmente continuas y caracteres a enteros y factores. La implementación en R de variables nominales y ordinales o en general de variables categóricas se realiza a través del concepto de factor (ver `?factor`). Para definir las categorías que abarca un factor se utiliza el parámetro `levels` junto con un vector como argumento donde se puede especificar si existe o no un ordenamiento a través del parámetro `ordered` donde el argumento puede tomar el valor de `FALSE` (si es una variable nominal) o `TRUE` (si es una variable ordinal), tal como se señala a continuación:

```
Rows: 1,239
Columns: 21
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_genero                  <fct> M, M, M, M, F, F, F, F~
$ estu_semestrecursa           <ord> 09, 09, 09, 09, 09, 09~
$ fami_estratovivienda         <ord> Estrato 3, Estrato 4, ~
$ fami_educacionmadre          <ord> Técnica o tecnológica ~
$ fami_educacionpadre          <ord> Secundaria (Bachillera~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion <chr> "11", "11", "11", "11"~
$ estu_depto_presentacion      <chr> "BOGOTÁ", "BOGOTÁ", "B~
$ estu_cod_mcpio_presentacion <chr> "11001", "11001", "110~
$ estu_mcpio_presentacion      <chr> "BOGOTÁ D.C.", "BOGOTÁ~
$ estu_snies_prgmacademico     <int> 6527, 11428, 11428, 11~
$ estu_prgm_academico          <chr> "ADMINISTRACION DE EMP~
$ mod_competen_ciudada_punt    <dbl> 162, 190, 154, 179, 17~
$ mod_comuni_escrita_punt      <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt              <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem             <ord> A2, A2, A2, A2, A2, A2~
$ mod_lectura_critica_punt     <dbl> 114, 168, 139, 148, 18~
```

```
$ mod_razona_cuantitat_punt   <dbl> 135, 164, 113, 149, 16~
$ punt_global                 <dbl> 139, 170, 128, 155, 16~
```

En el Listado 10.9 las variables `ano` y `estu_snies_prgmacademico` se transforman a enteros teniendo en cuenta la naturaleza de las mismas. Por otro lado, en primera instancia la variable `estu_genero` se codifica como un factor no ordenado teniendo en cuenta que se refiere al género del estudiante mientras que las variables `estu_semestrecursa`, `fami_estratovivienda`, `fami_educacionmadre`, `fami_educacionpadre` y `mod_ingles_desem` se codifican como factores ordenados. Cabe la pena señalar, que algunas categorías de las variables `fami_estratovivienda`, `fami_educacionmadre` y `fami_educacionpadre` como `"Sin Estrato"`, `"No sabe"` y `"No Aplica"` podrían codificarse como valores faltantes (en inglés **Not Available** o **Missing values**), donde R codifica este tipo de casos con la lógica constante `NA` (ver `?NA`), pero en `2021-2024_saber-pro-genericas_faedis.rds` se mantienen estas etiquetas para entender la razón por la cual se presentan estos casos de datos faltantes.

> **Nota 22**
>
> En este libro no abarcaremos a profundidad el manejo de datos faltantes, codificados en R como `NA`, así como las herramientas disponibles para llevar un análisis adecuado en presencia de este fenómeno, que es bastante común al realizar un análisis de datos.
> Si deseas profundizar en este tema más adelante, cuando estés familiarizado con R y Positron, puedes consultar (Buuren, 2018) y la documentación oficial de los paquetes `mice` y `naniar`.

10.2.6. Colapsar, agrupar y desagrupar (`summarise()`, `group_by()` y `ungroup()`)

A partir de `2021-2024_saber-pro-genericas_faedis.rds` es posible obtener el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds` utilizando las funciones `summarise()`, `group_by()` y `ungroup()`. Adicionalmente, en la Sección 10.2.6 conocerás el uso de una nueva función denominada `across()` del paquete `dplyr`, la cual permite aplicar la misma transformación a múltiples columnas, de tal manera que no tengas que repetir un código similar para cada variable dentro de un grupo de variables de interés.

El primer paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds` comprende agrupar las filas a partir de las variables `ano`, `estu_snies_prgmacademico` y `estu_prgm_academico`, de tal manera que se pueda obtener el puntaje promedio redondeado al entero más próximo de cada módulo de competencias genéricas así como del puntaje total promedio también redondeado al entero más próximo, tal como se señala a continuación:

```
# A tibble: 1,239 x 21
# Groups:   ano, estu_snies_prgmacademico,
#   estu_prgm_academico [26]
     ano estu_consecutivo estu_genero estu_semestrecursa
   <int> <chr>            <fct>       <ord>
 1  2021 EK202120262469   M           09
 2  2021 EK202120116785   M           09
 3  2021 EK202120116868   M           09
 4  2021 EK202120116831   M           09
 5  2021 EK202120116814   F           09
 6  2021 EK202120117053   F           09
 7  2021 EK202120262258   F           08
 8  2021 EK202120262692   F           09
 9  2021 EK202120117033   F           08
10  2021 EK202120117014   F           08
# i 1,229 more rows
# i 17 more variables: fami_estratovivienda <ord>,
#   fami_educacionmadre <ord>, fami_educacionpadre <ord>,
#   estu_exterior <chr>, estu_cod_depto_presentacion <chr>,
#   estu_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>,
#   estu_mcpio_presentacion <chr>, ...
```

El segundo paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds` comprende calcular el puntaje promedio de cada módulo y del puntaje global de las competencias genéricas para luego redondear estos valores al entero más cercano. En primera instancia, se podría utilizar un enfoque similar al descrito en la Sección 10.1.1.4 definiendo el puntaje promedio de manera individual para cada variable de interés. Sin embargo, debido a que se requiere calcular el puntaje promedio aplicando una misma transformación, es posible utilizar la función `across()` que permite a través del parámetro `.cols` seleccionar un conjunto de variables y aplicar una misma función a través del parámetro `.fns` utilizando como argumento el valor `mean`, tal como se señala a continuación:

```
# A tibble: 26 x 9
# Groups:   ano, estu_snies_prgmacademico [26]
     ano estu_snies_prgmacademico estu_prgm_academico
   <int>                    <int> <chr>
 1  2021                     6527 ADMINISTRACION DE EMPRESAS
 2  2021                    10963 RELACIONES INTERNACIONALE~
 3  2021                    11004 INGENIERIA CIVIL
 4  2021                    11428 CONTADURIA PUBLICA
 5  2021                    53703 INGENIERIA INDUSTRIAL
```

```
 6  2021                   105142 INGENIERIA INFORMATICA
 7  2022                     6527 ADMINISTRACION DE EMPRESAS
 8  2022                    10963 RELACIONES INTERNACIONALE~
 9  2022                    11004 INGENIERIA CIVIL
10  2022                    11428 CONTADURIA PUBLICA
# i 16 more rows
# i 6 more variables: mod_competen_ciudada_punt <dbl>,
#   mod_comuni_escrita_punt <dbl>, mod_ingles_punt <dbl>,
#   mod_lectura_critica_punt <dbl>,
#   mod_razona_cuantitat_punt <dbl>, punt_global <dbl>
```

El tercer paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds` comprende desagrupar las filas que se agruparon inicialmente por las variables de interés, tal como se señala a continuación:

```
# A tibble: 26 x 9
     ano estu_snies_prgmacademico estu_prgm_academico
   <int>                    <int> <chr>
 1  2021                     6527 ADMINISTRACION DE EMPRESAS
 2  2021                    10963 RELACIONES INTERNACIONALE~
 3  2021                    11004 INGENIERIA CIVIL
 4  2021                    11428 CONTADURIA PUBLICA
 5  2021                    53703 INGENIERIA INDUSTRIAL
 6  2021                   105142 INGENIERIA INFORMATICA
 7  2022                     6527 ADMINISTRACION DE EMPRESAS
 8  2022                    10963 RELACIONES INTERNACIONALE~
 9  2022                    11004 INGENIERIA CIVIL
10  2022                    11428 CONTADURIA PUBLICA
# i 16 more rows
# i 6 more variables: mod_competen_ciudada_punt <dbl>,
#   mod_comuni_escrita_punt <dbl>, mod_ingles_punt <dbl>,
#   mod_lectura_critica_punt <dbl>,
#   mod_razona_cuantitat_punt <dbl>, punt_global <dbl>
```

El cuarto paso para reconstruir el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds` comprende redondear los valores de las columnas que contienen los puntajes promedios tanto de los diferentes módulos como del puntaje global de las competencias específicas. En este caso, es posible utilizar nuevamente la función `across()` con el parámetro `.cols` para seleccionar las respectivas columnas y el parámetro `.fns` pero utilizando como argumento el valor `round` en conjunto con la función `mutate()`, tal como se señala a continuación:

```
Rows: 26
Columns: 9
$ ano                        <int> 2021, 2021, 2021, 2021, ~
$ estu_snies_prgmacademico   <int> 6527, 10963, 11004, 1142~
$ estu_prgm_academico        <chr> "ADMINISTRACION DE EMPRE~
$ mod_competen_ciudada_punt <dbl> 142, 158, 144, 155, 159,~
$ mod_comuni_escrita_punt    <dbl> 147, 149, 120, 145, 146,~
$ mod_ingles_punt            <dbl> 157, 151, 155, 150, 170,~
$ mod_lectura_critica_punt   <dbl> 142, 157, 141, 147, 150,~
$ mod_razona_cuantitat_punt <dbl> 145, 138, 154, 149, 161,~
$ punt_global                <dbl> 147, 151, 143, 149, 157,~
```

En el Listado 10.11 y el Listado 10.13 se separa el cálculo del puntaje promedio de cada módulo y del puntaje global de las competencias genéricas así como el proceso de redondear estos valores al entero más cercano. Sin embargo, es posible consolidar este aspecto en un solo paso utilizando una herramienta denominada **funciones anónimas** donde este aspecto se abarcará en detalle más adelante en el Capítulo 19.

Listado 10.3 Creación de variables intermedias cuando no se usa el operador `|>`

```
# Seleccionar columnas ----
datos_seleccionados <- select(
  saber_pro_genericas_faedis_2021_2024,
  ano,
  estu_consecutivo,
  estu_snies_prgmacademico,
  estu_prgm_academico,
  mod_competen_ciudada_punt,
  mod_comuni_escrita_punt,
  mod_ingles_punt,
  mod_lectura_critica_punt,
  mod_razona_cuantitat_punt
)

# Filtrar filas con ceros ----
datos_filtrados <- filter(
  datos_seleccionados,
  mod_competen_ciudada_punt == 0 |
    mod_comuni_escrita_punt == 0 |
    mod_ingles_punt == 0 |
    mod_lectura_critica_punt == 0 |
    mod_razona_cuantitat_punt == 0
)

# Ordenar por programa académico ---
datos_ordenados <- arrange(
  datos_filtrados,
  estu_prgm_academico
)

# "Vistazo" de datos ----
glimpse(datos_ordenados)
```

Listado 10.4 Uso del marcador de posición `_` y el operador `|>` en la función `lm()`

```
saber_pro_genericas_faedis_2021_2024 |>
  lm(
    formula = mod_razona_cuantitat_punt ~ mod_lectura_critica_punt,
    data = _
  )
```

Listado 10.5 Importación de datos para desarrollar el caso de estudio

```
# Importar datos ----
saber_pro_genericas_2021_2024 <- read_csv(
  file = "data/2021_2024-saber_pro-genericas.csv.xz",
  col_types = cols(
    estu_fechanacimiento = col_character()
  )
)
```

Listado 10.6 Filtrar filas para obtener las observaciones del conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds`

```
# Filtrar ----
## Vector auxiliar ----
snies_faedis <- c(
  6527,
  108241,
  11428,
  11004,
  53703,
  105142,
  10963
)

saber_pro_genericas_2021_2024_faedis <- saber_pro_genericas_2021_2024 |>
  filter(estu_agregado == "S") |>
  filter(estu_snies_prgmacademico %in% snies_faedis)

saber_pro_genericas_2021_2024_faedis
```

Listado 10.7 Seleccionar columnas para obtener las variables del conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds`

```
# Seleccionar ----
saber_pro_genericas_2021_2024_faedis <- saber_pro_genericas_2021_2024_faedis |>
  select(
    ano,
    estu_consecutivo,
    estu_genero,
    estu_semestrecursa,
    fami_educacionmadre:fami_estratovivienda,
    estu_exterior,
    ends_with("_presentacion"),
    estu_snies_prgmacademico,
    estu_prgm_academico,
    ends_with("_punt") | starts_with("punt_"),
    mod_ingles_desem
  )

saber_pro_genericas_2021_2024_faedis
```

Listado 10.8 Reubicar columnas para obtener el mismo ordenamiento de las variables del conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds`

```
# Reubicar ----
saber_pro_genericas_2021_2024_faedis <- saber_pro_genericas_2021_2024_faedis |>
  relocate(
    fami_estratovivienda,
    .before = fami_educacionmadre
  ) |>
  relocate(
    estu_depto_presentacion,
    .after = estu_cod_depto_presentacion
  ) |>
  relocate(
    estu_mcpio_presentacion,
    .after = estu_cod_mcpio_presentacion
  ) |>
  relocate(
    mod_ingles_desem,
    .after = mod_ingles_punt
  )

saber_pro_genericas_2021_2024_faedis
```

Listado 10.9 Transformar columnas para asociar el tipo de datos adecuado a cada variable del conjunto de datos `2021-2024_saber-pro-genericas_faedis.rds`

```
# Transformar ----
saber_pro_genericas_2021_2024_faedis <- saber_pro_genericas_2021_2024_faedis |>
  mutate(
    ano = as.integer(ano),
    estu_snies_prgmacademico = as.integer(estu_snies_prgmacademico)
  ) |>
  mutate(
    estu_genero = factor(
      x = estu_genero,
      ordered = FALSE
    ),
    estu_semestrecursa = factor(
      x = estu_semestrecursa,
      levels = c("06", "07", "08", "09", "10", "11", "12 o más"),
      ordered = TRUE
    ),
    fami_estratovivienda = factor(
      x = fami_estratovivienda,
      levels = c(
        "Sin Estrato",
        "Estrato 1",
        "Estrato 2",
        "Estrato 3",
        "Estrato 4",
        "Estrato 5",
        "Estrato 6"
      ),
      ordered = TRUE
    ),
    fami_educacionmadre = factor(
      x = fami_educacionmadre,
      levels = c(
        "No sabe",
        "No Aplica",
        "Ninguno",
        "Primaria incompleta",
        "Primaria completa",
        "Secundaria (Bachillerato) incompleta",
        "Secundaria (Bachillerato) completa",
        "Técnica o tecnológica incompleta",
        "Técnica o tecnológica completa",
        "Educación profesional incompleta",
        "Educación profesional completa",
        "Postgrado"
      ),
      ordered = TRUE
    ),
    fami_educacionpadre = factor(
      x = fami_educacionpadre,
      levels = c(
        "No sabe",
        "No Aplica",
        "Ninguno",
```

Listado 10.10 Agrupar filas a partir de un grupo de variables para generar el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds`

```
# Agrupar ----
saber_pro_genericas_faedis_2021_2024_programas <- saber_pro_genericas_faedis_2021_2
  group_by(
    ano,
    estu_snies_prgmacademico,
    estu_prgm_academico
  )

saber_pro_genericas_faedis_2021_2024_programas
```

Listado 10.11 Colapsar filas a partir de un grupo de variables utilizando el valor `mean` para generar el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds`

```
saber_pro_genericas_faedis_2021_2024_programas <- saber_pro_genericas_faedis_2021_2
  summarise(
    across(
      .cols = ends_with("punt") | starts_with("punt"),
      .fns = mean
    )
  )

saber_pro_genericas_faedis_2021_2024_programas
```

Listado 10.12 Desagrupar filas para generar el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds`

```
saber_pro_genericas_faedis_2021_2024_programas <- saber_pro_genericas_faedis_2021_2
  ungroup()

saber_pro_genericas_faedis_2021_2024_programas
```

Listado 10.13 Transformar columnas utilizando la función `across()` y el valor `round` para generar el conjunto de datos `2021-2024_saber-pro-genericas_faedis_programas.rds`

```
saber_pro_genericas_faedis_2021_2024_programas <- saber_pro_genericas_faedis_2021_2024_program
  mutate(
    across(
      .cols = ends_with("punt") | starts_with("punt"),
      .fns = round
    )
  )

saber_pro_genericas_faedis_2021_2024_programas |>
  glimpse()
```

11. Verbos de dos tablas

Este capítulo se basa en (Wickham, 2023, Capítulo 19) así como en la documentación oficial de `dplyr` donde tiene como propósito introducir a través de la **gramática de manipulación de datos** un marco flexible para conectar o unir tablas de datos utilizando el paquete `dplyr` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes al periodo 2021-2024 para Colombia.

Exploraremos los siguientes aspectos:

1. ¿Cómo entender la relación entre diferentes tablas de datos a través de un Diagrama Entidad-Relación (Entity-Relationship Diagram (ERD) en inglés)?
2. ¿Cómo interpretar las relaciones en un Diagrama Entidad-Relación utilizando la notación **Crow's Foot** ("pata de cuervo" en español)?

3. ¿Cuáles son los tipos de verbos de dos tablas?
4. ¿Cómo combinar variables de múltiples tablas utilizando **Mutating Joins** ("uniones de transformación" en español)?

> **Nota 23**
>
> Para desarrollar el contenido de este capítulo es necesario que descargues los archivos señalados a continuación y los incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **resultados-saber-pro.rds**
>
> **divipola-departamentos.rds**
>
> **divipola-municipios.rds**
>
> **programas-pregrado-faedis.rds**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `011_verbos-dos-tablas-en-r` y dentro de ella genera el archivo `001_manipular-datos-dos-tablas.R`, siguiendo las pautas de la Sección 4.4.

11.1. Diagrama Entidad-Relación

Al realizar un análisis de datos es habitual tener que combinar dos o más tablas, una necesidad que surge principalmente del diseño propio de las bases de datos relacionales. Aunque no se profundizará en cómo crearlas o conectarse a ellas (ver Nota 24), es conveniente conocer una representación gráfica que permita visualizar un conjunto de tablas de datos y las posibles relaciones que existen entre ellas para entender cómo se puede consolidar un determinado conjunto de información que se requiere para llevar a cabo un determinado análisis.

> **Nota 24**
>
> En el tidyverse existe el paquete `dbplyr` que junto al paquete DBI permite interactuar con bases de datos en R usando la sintaxis de `dplyr`. Así como, consultar y transformar tablas dentro de ellas sin necesidad de escribir código SQL directamente, ya que `dbplyr` traduce la sintaxis de `dplyr` a consultas en lenguaje SQL.
> En ese sentido, cuando estés familiarizado con los verbos de dos tablas puedes explorar este paquete si requieres conectarte más adelante a una base de datos relacional para realizar un análisis utilizando lo aprendido acerca de `dplyr`.

Una representación gráfica que cumple con este propósito es un Diagrama de Entidad-Relación que ilustra cómo una base de datos relacional compuesta por un conjunto de tablas se relacionan entre sí. Estos diagramas se componen de 3 elementos:

1. **Entidades**: representan cualquier objeto, concepto o evento.
2. **Atributos**: son las características, rasgos o propiedades de cada entidad.
3. **Relaciones**: es la asociación o vínculo lógico que existe entre dos o más entidades.

En la Figura 11.1 se indica un posible Diagrama de Entidad-Relación utilizando como insumo el conjunto de datos señalado en la Nota 11 y que se utilizó en la parte de visualización de datos para analizar los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de FAEDIS de la UMNG así como información adicional proveniente de los archivos señalados en la Nota 23.

11.2. Entidades

En la Figura 11.1 se señalan 4 entidades para representar una base de datos relacional que tenga como objetivo almacenar información relacionada con los resultados del examen Saber Pro de competencias genéricas en programas de pregrado de FAEDIS:

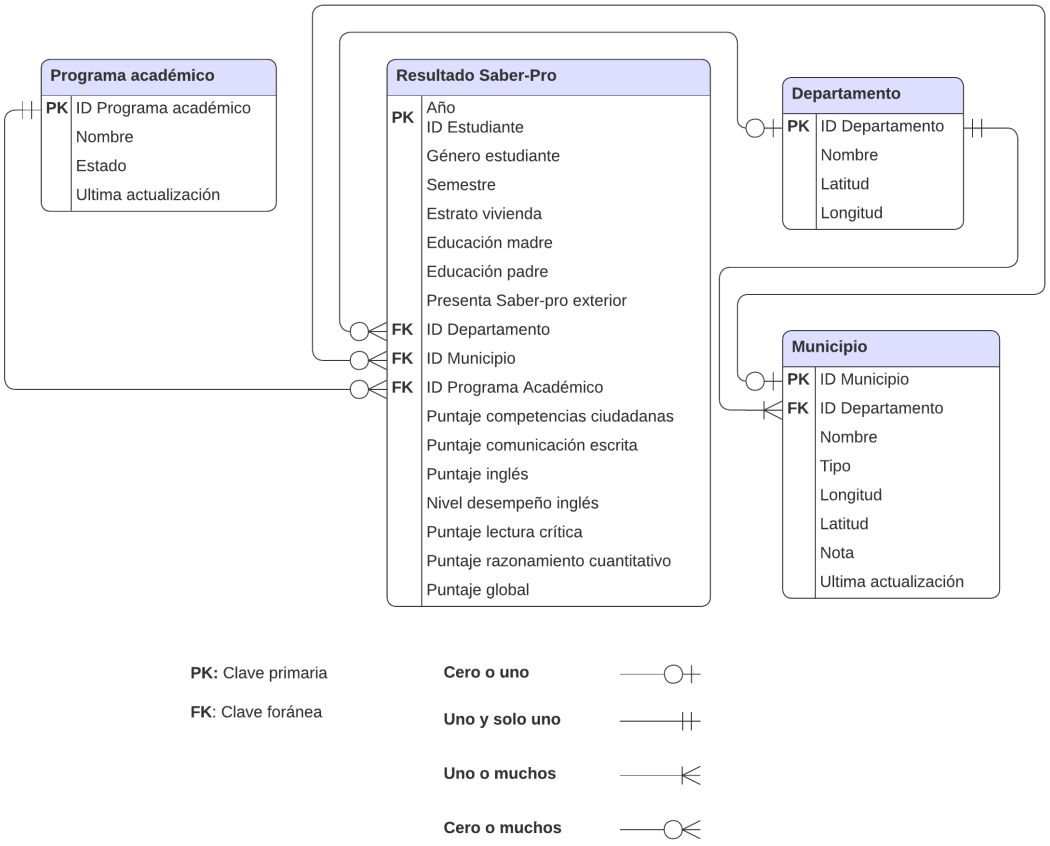


Figura 11.1.: Diagrama de Entidad-Relación correspondiente a los resultados del examen Saber Pro de competencias genéricas en programas de pregrado de FAEDIS

a. **Resultado Saber-Pro**: corresponde a un evento donde modela la acción de un estudiante presentando las pruebas Saber-Pro en un periodo y lugar específico junto con sus características socioeconómicas y los resultados obtenidos.

b. **Programa académico**: representa los programas académicos de pregrado de FAEDIS a los cuales puede potencialmente pertenecer un estudiante.

c. **Departamento**: representa la división político-administrativa a nivel departamental para Colombia según la codificación DIVIPOLA donde permite enriquecer los resultados obtenidos en la prueba Saber-Pro con información geográfica a nivel departamental.

d. **Municipio**: representa la división político-administrativa a nivel municipal para Colombia según la codificación DIVIPOLA donde permite enriquecer los resultados obtenidos en la prueba Saber-Pro con información geográfica a nivel municipal.

> **i** Nota 25
>
> A un elemento específico y concreto de una entidad se le denomina **instancia**. Por ejemplo, respecto a la Figura 11.1 *Amazonas* es una instancia de la entidad **Departamento**.

11.3. Atributos

Cada entidad en la Figura 11.1 se puede describir a través de características, rasgos o propiedades que se recopilan en una variable y se almacenan en una columna dentro de una tabla. Por ejemplo, la entidad **Departamento** se describe a través de las variables `ID Departamento` que identifica de manera única a cada instancia, `Nombre` que corresponde a su denominación oficial y `Latitud` junto con `Longitud` que brindan información sobre su ubicación geográfica.

11.3.1. Clave primaria

Dentro de los atributos se define una clave primaria (Primary Key (**PK**) en inglés) que tiene como propósito identificar de forma única cada registro dentro de una tabla. Es decir, no pueden existir dos filas con el mismo valor de la clave primaria. Adicionalmente una clave primaria debe cumplir con un conjunto de reglas fundamentales:

a. **Unicidad**: cada valor debe ser único en cada tabla
b. **No nulidad**: no puede contener valores ausentes o vacíos (`NA` o `NULL` en R). Es decir, todo registro debe estar asociado a una entidad.
c. **Inmutabilidad**: Su valor no debe cambiar con el tiempo.

Por otro lado, una clave primaria puede estar formada por una sola columna dentro de una tabla pero también se puede formar uniendo dos o más columnas cuando una sola no basta para garantizar unicidad. En el caso de la Figura 11.1 respecto a la entidad **Resultado Saber-Pro** se utilizan las variables `Año` y `ID estudiante` para identificar de manera única el evento relacionado con la acción de un estudiante presentando las pruebas Saber-Pro en un periodo determinado. Aunque también se puede utilizar una columna como en el caso de la entidad **Departamento** en la Figura 11.1 donde se hace uso de la variable `ID Departamento` para identificar de manera única esta entidad utilizando la codificación DIVIPOLA.

11.3.2. Clave foránea

Un atributo presente en una tabla cuyos valores hacen referencia directa a la clave primaria de otra tabla se conoce como clave foránea (Foreign Key (**FK**) en inglés). Este atributo es el puente que permite establecer una relación entre dos entidades así como vincular datos sin necesidad de duplicar toda la información. Por ejemplo, en lugar de repetir en la tabla relacionada con la entidad **Resultado Saber-Pro** el nombre del departamento así como su latitud y longitud, solo es necesario incluir la variable `ID Departamento` sin necesidad de repetir los demás atributos de la entidad **Departamento**.

A diferencia de una clave primaria (**PK**), los valores de una clave foránea (**FK**) se pueden repetir en una tabla. Por ejemplo, diferentes instancias de la entidad **Resultado Saber-Pro** pueden tener una misma `ID Departamento` porque muchos estudiantes pueden presentar el Saber-Pro en un mismo departamento. Adicionalmente, una clave foránea (**FK**) puede contener valores vacíos o ausentes. Por ejemplo, si un estudiante presenta el Saber-Pro en el exterior se esperaría que no tenga asociado un `ID Departamento` porque el lugar donde lo presentó no se encuentra dentro del territorio colombiano (ver Advertencia 1).

> ⚠ Advertencia 1
>
> En la Sección 11.7 utilizando `resultados-saber-pro.rds` respecto a los archivos señalados en Nota 23 se ajustará este aspecto debido a que la fuente donde se obtuvieron los datos (Data Icfes), utiliza por defecto a Bogotá como departamento de presentación del examen cuando se presenta la prueba Saber Pro en el exterior. También ocurre un comportamiento similar en este contexto para el caso de `ID Municipio`. Con los ajustes respectivos que aplicarás en la Sección 11.7, es posible generar de manera consistente las tablas de las entidades `Departamento` y `Municipio` para que reflejen lo señalado en la Figura 11.1.

11.4. Relaciones

Las relaciones definen cómo se vinculan lógicamente 2 entidades a través de las claves primarias (**PK**) y foráneas (**FK**) donde se utiliza el concepto de **cardinalidad**. La cardinalidad define las restricciones numéricas que gobiernan el vínculo lógico entre 2 entidades. En términos sencillos, la cardinalidad responde a la pregunta

> ¿Con cuántas instancias de una entidad `A` se puede vincular una instancia de la entidad `B`?

Para definir este aspecto, la cardinalidad se descompone en dos límites: mínimo y máximo. Por un lado, la cardinalidad mínima indica si la relación debería ser obligatoria u opcional:

- `0` (opcional): una instancia de una entidad puede existir sin estar asociada a ninguna instancia de otra entidad
- `1` (obligatoria): una instancia de una entidad debe estar asociada obligatoriamente a al menos una instancia de otra entidad

Por ejemplo, respecto al contexto de FAEDIS en la UMNG con base en la Figura 11.1 un estudiante que presenta la prueba en un determinado periodo debe estar asociado a un programa académico. De lo contrario, si el estudiante realiza por su lado la prueba sin inscribirse a través de la universidad no se tienen en cuenta los resultados obtenidos para analizar el desempeño de FAEDIS. Es decir, la relación de la entidad **Resultados Saber-Pro** hacia la entidad **Programa académico** respecto a la cardinalidad mínima implica que una instancia de la primera está asociada obligatoriamente con al menos una instancia de la segunda entidad.

En el otro sentido, un programa académico de FAEDIS no necesariamente está asociado con un estudiante que presenta la prueba del Saber-Pro en un determinado periodo dado que pueden existir programas académicos que se encuentran inactivos y no se tienen en cuenta en los resultados obtenidos para analizar el desempeño de FAEDIS. Es decir, la relación de la entidad **Programa académico** hacia la entidad **Resultados Saber-Pro** respecto a la cardinalidad mínima implica que una instancia de la primera no necesariamente está asociada con al menos una instancia de la segunda entidad.

Por otro lado, la cardinalidad máxima indica el número "tope" de instancias con las que se debería vincular:

- `1` (uno): como máximo una instancia de una entidad está asociada con una sola instancia de otra entidad
- `N` (muchos): una instancia de una entidad se puede vincular con varias instancias de otra entidad

Por ejemplo, respecto al contexto de FAEDIS en la UMNG con base en la Figura 11.1 un estudiante que presenta la prueba en un determinado periodo debe estar asociado a lo más con un programa académico. En el caso en que un estudiante realice dos o más programas académicos y presente la prueba dentro de los registros de DATA ICFES respecto a `resultados-saber-pro.rds`, se consideran como dos o más instancias diferentes dentro de **Resultados Saber-Pro**. Es decir, la relación de la entidad **Resultados Saber-Pro** hacia la entidad **Programa académico** respecto a la cardinalidad máxima implica que una instancia de la primera está asociada a lo más con una instancia de la segunda entidad.

En el otro sentido, un programa académico de FAEDIS puede estar asociado con muchos estudiantes que presentan la prueba del Saber-Pro en un determinado periodo porque muchos estudiantes pertenecientes a un mismo programa académico presentan la prueba en un mismo año. Es decir, la relación de la entidad **Programa académico** hacia la entidad **Resultados Saber-Pro** respecto a la cardinalidad máxima implica que una instancia de la primera está asociada con muchas instancias de la segunda entidad.

11.4.1. Notación Crow's Foot

Respecto a los elementos señalados en la Sección 11.4 existen diferentes tipos de notación para representar de manera gráfica la cardinalidad de las relaciones y los vínculos que existen entre entidades a través de las claves primarias y foráneas. En la Figura 11.1 se utiliza la notación **Crow's foot** para representar los vínculos entre las entidades así como su cardinalidad tanto mínima como máxima donde en la Tabla 11.1 se señalan los símbolos utilizados por esta notación y en la Tabla 11.2 la representación gráfica de la cardinalidad.

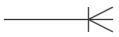
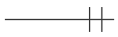
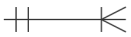
A modo de ejemplo, la cardinalidad de la relación de la entidad **Departamento** hacia la entidad **Municipio** así como la cardinalidad de la relación de la entidad **Municipio** hacia la entidad **Departamento** se puede representar de manera compacta a través de la unión de  y de  como  , tal como se señala en la Figura 11.1. Es decir, en el contexto de la codificación DIVIPOLA un departamento está vinculado con al menos un municipio y a lo más con muchos municipios pero un municipio está vinculado con uno y solo un departamento.

Tabla 11.1.: Símbolos en la notación Crow's Foot

Símbolo	Representación gráfica
0 (cero)	

Símbolo	Representación gráfica
`1` (uno)	
`N` (muchos)	

Tabla 11.2.: Cardinalidad de una relación y representación gráfica usando la notación Crow's Foot

Cardinalidad	Representación gráfica
`0` o `1` (mínimo cero, máximo uno)	
`0` o `N` (mínimo cero, máximo muchos)	
`1` y solo `1` (mínimo uno, máximo uno)	
`1` o `N` (mínimo uno, máximo muchos)	

11.5. Verbos

En el paquete `dplyr`, para manipular, conectar o unir 2 tablas de datos correspondientes a 2 entidades utilizando claves primarias y foráneas, existen 3 tipos de verbos:

1. **Mutating joins** (uniones de transformación en español): permiten combinar variables de diferentes tablas correspondientes a 2 entidades. En la Sección 11.6 utilizarás este tipo de verbos junto con el esquema de la Figura 11.1 y los archivos señalados en la Nota 23 para reconstruir el archivo `2021-2024_saber-pro-genericas_faedis.rds` pero teniendo en cuenta la Advertencia 1.
2. **Filtering joins** (uniones de filtrado en español): buscan coincidencias entre registros de forma similar a las **Mutating joins**, pero afectan a las observaciones y no a las variables.

3. **Set operations** (operaciones de conjuntos): esperan que ambas tablas compartan exactamente las mismas variables y tratan sus observaciones como conjuntos matemáticos mediante funciones como `intersect()`, `union()` y `setdiff()`.

Debido a que las **Set operations** no se requieren para los análisis que desarrollaremos a lo largo del libro, no se profundizará en ellas en los siguientes apartados. No obstante, el lector interesado puede consultar la documentación oficial de `dplyr` para conocer más detalles sobre este tipo de operaciones.

11.5.1. Filtering joins

Existen dos tipos de **Filtering joins**:

I. **Semi joins**: mantiene todas las observaciones de la tabla de una entidad que tienen una coincidencia en la tabla de otra entidad.
II. **Anti joins**: descarta todas las observaciones de la tabla de una entidad que tienen una coincidencia en la tabla de otra entidad.

Para ilustrar un ejemplo concreto utilizarás los archivos `divipola-departamentos.rds` y `resultados-saber-pro.rds` que corresponden a las tablas de las entidades **Departamento** y **Resultado Saber-Pro** en la Figura 11.1, tal como se señala a continuación:

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
divipola_departamentos <- read_rds(
  file = "data/divipola-departamentos.rds"
)

resultado_saber_pro <- read_rds(
  file = "data/resultados-saber-pro.rds"
)
```

Una vez importes las tablas, podrás darle un "vistazo" a cada una de ellas, tal como se señala a continuación:

```
# Inspeccionar datos ----
divipola_departamentos |>
  glimpse()
```

```
Rows: 33
Columns: 4
```

```
$ codigo   <chr> "05", "08", "11", "13", "15", "17", "18",~
$ nombre   <chr> "ANTIOQUIA", "ATLÁNTICO", "BOGOTÁ, D.C.",~
$ latitud  <dbl> 6.7020321, 10.6770095, 4.3161077, 8.07979~
$ longitud <dbl> -75.50456, -74.96522, -74.18107, -74.2351~
```

```
resultado_saber_pro |>
  glimpse()
```

```
Rows: 1,239
Columns: 18
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_genero                  <fct> M, M, M, M, F, F, F, F~
$ estu_semestrecursa           <ord> 09, 09, 09, 09, 09, 09~
$ fami_estratovivienda         <ord> Estrato 3, Estrato 4, ~
$ fami_educacionmadre          <ord> Técnica o tecnológica ~
$ fami_educacionpadre          <ord> Secundaria (Bachillera~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion <chr> "11", "11", "11", "11"~
$ estu_cod_mcpio_presentacion <chr> "11001", "11001", "110~
$ estu_snies_prgmacademico     <int> 6527, 11428, 11428, 11~
$ mod_competen_ciudada_punt    <dbl> 162, 190, 154, 179, 17~
$ mod_comuni_escrita_punt      <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt              <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem             <ord> A2, A2, A2, A2, A2, A2~
$ mod_lectura_critica_punt     <dbl> 114, 168, 139, 148, 18~
$ mod_razona_cuantitat_punt    <dbl> 135, 164, 113, 149, 16~
$ punt_global                  <dbl> 139, 170, 128, 155, 16~
```

Para utilizar los verbos de 2 tablas de `dplyr`, una vez importadas las tablas, es necesario entender a qué se refieren cada una de las variables que las componen. En ese sentido, a continuación se describen cada una de las variables en `divipola-departamentos.rds`:

1. **codigo**: código único asociado a cada departamento de acuerdo con la codificación DIVIPOLA donde Colombia se compone de 32 departamentos y adicionalmente para fines estadísticos se codifica como departamento el Distrito Capital de Bogotá.
2. **nombre**: nombre oficial del departamento de acuerdo con la codificación DIVIPOLA.
3. **latitud**: coordenada geográfica en grados decimales que mide la distancia respecto a la línea ecuatorial, indicando la posición norte-sur del punto representativo de cada departamento.

4. **longitud**: coordenada geográfica en grados decimales que mide la distancia respecto al meridiano de Greenwich, indicando la posición este-oeste del punto representativo de cada departamento.

Respecto a las variables en `resultados-saber-pro.rds` estas corresponden a las señaladas en la Sección 6.2.1.1 pero donde se excluyen `estu_depto_presentacion`, `estu_mcpio_presentacion` y `estu_prgm_academico` debido a que de acuerdo a la Figura 11.1 son atributos que pertenecen a otras entidades.

Teniendo claridad sobre el significado de las variables de cada entidad, es posible identificar cuáles departamentos de `divipola_departamentos` registran al menos una presentación del examen en `resultado_saber_pro`. Para lograr esto, se vincula la clave primaria `codigo` de la primera tabla con la clave foránea `estu_cod_depto_presentacion` de la segunda mediante la función `semi_join()`, tal como se señala a continuación:

Listado 11.1 Uso de la función `semi_join(x, y)` para filtrar las observaciones de `x` que coinciden con `y`

```
semi_join(
  x = divipola_departamentos,
  y = resultado_saber_pro,
  by = join_by(codigo == estu_cod_depto_presentacion)
)
```

```
# A tibble: 24 x 4
   codigo nombre       latitud longitud
   <chr>  <chr>          <dbl>    <dbl>
 1 05     ANTIOQUIA      6.70     -75.5
 2 08     ATLÁNTICO     10.7      -75.0
 3 11     BOGOTÁ, D.C.   4.32     -74.2
 4 13     BOLÍVAR        8.08     -74.2
 5 15     BOYACÁ         5.89     -72.6
 6 17     CALDAS         5.28     -75.3
 7 18     CAQUETÁ        0.799    -74.0
 8 19     CAUCA          2.40     -76.8
 9 20     CESAR          9.54     -73.5
10 23     CÓRDOBA        8.36     -75.8
# i 14 more rows
```

En el Listado 11.1 se conservan únicamente las filas de `divipola_departamentos` (tabla asignada al parámetro `x`) que tienen al menos una coincidencia en `resultado_saber_pro` (tabla asignada al parámetro `y`). Es importante destacar que la función `semi_join()` es una **Filtering join**. En ese sentido, no añade nuevas columnas de la tabla `y` ni duplica

filas de `x` cuando existen múltiples coincidencias por departamento, sino que únicamente filtra las observaciones de `x`.

Para conectar ambas tablas, el parámetro `by` se utiliza para indicar las claves que establecen la relación entre ellas. En este caso, a través de la función auxiliar `join_by()` como argumento de `by`, se señala qué clave de la primera tabla se relaciona con la clave de la segunda mediante la expresión `codigo == estu_cod_depto_presentacion` utilizando el operador relacional de igualdad `==` (ver Tabla 5.2), dado que las claves tienen nombres diferentes en cada tabla pero representan el mismo atributo.

También es posible identificar de manera similar cuáles departamentos de `divipola_departamentos` no registran ninguna presentación del examen en `resultado_saber_pro`. Para lograr esto, se vincula nuevamente la clave primaria `codigo` de la primera tabla con la clave foránea `estu_cod_depto_presentacion` de la segunda, pero mediante la función `anti_join()`, tal como se señala a continuación:

Listado 11.2 Uso de la función `anti_join(x, y)` para filtrar las observaciones de `x` que no coinciden con `y`

```
anti_join(
  x = divipola_departamentos,
  y = resultado_saber_pro,
  by = join_by(codigo == estu_cod_depto_presentacion)
)
```

```
# A tibble: 9 x 4
  codigo nombre     latitud longitud
  <chr>  <chr>        <dbl>    <dbl>
1 27     CHOCÓ         5.40    -76.9
2 44     LA GUAJIRA   11.5     -72.4
3 70     SUCRE         9.06    -75.1
4 81     ARAUCA        6.57    -71.0
5 91     AMAZONAS     -1.55    -71.5
6 94     GUAINÍA       2.73    -68.8
7 95     GUAVIARE      1.92    -72.1
8 97     VAUPÉS        0.646   -70.6
9 99     VICHADA       4.71    -69.4
```

En el Listado 11.2 se conservan únicamente las filas de `divipola_departamentos` (tabla asignada al parámetro `x`) que no tienen ninguna coincidencia en `resultado_saber_pro` (tabla asignada al parámetro `y`). Al igual que en la función `semi_join()`, el parámetro `by` se utiliza para indicar las claves que establecen la relación entre ambas tablas.

11.6. Mutating joins

Las **Mutating joins** permiten combinar variables de 2 tablas correspondientes a 2 entidades vinculando sus filas mediante claves. En `dplyr` existen 4 tipos principales, las cuales difieren en su comportamiento cuando una observación no tiene una coincidencia:

1. **`inner_join()`**: conserva únicamente las observaciones que tienen una coincidencia en ambas tablas (`x` y `y`).
2. **`left_join()`**: conserva todas las observaciones de la tabla izquierda (`x`), incorporando las variables de la tabla derecha (`y`) donde exista una coincidencia.
3. **`right_join()`**: conserva todas las observaciones de la tabla derecha (`y`), incorporando las variables de la tabla izquierda (`x`) donde exista una coincidencia.
4. **`full_join()`**: conserva todas las observaciones de ambas tablas (`x` y `y`), manteniendo los registros independientemente de si tienen o no una coincidencia.

Para comprender el funcionamiento de cada una de ellas, se utilizará un ejemplo ilustrativo mínimo compuesto por 2 tablas, `x` y `y`, vinculadas lógicamente mediante la clave `key`. La tabla `x` incluye las variables `key` y `z`, mientras que la tabla `y` cuenta con las variables `key`, `a` y `b`, tal como se señala a continuación:

Listado 11.3 Ejemplo ilustrativo mínimo para entender las **Mutating joins**

```
# Tabla x ----
x <- tibble(
  key = c(1, 2),
  z = c(3, 4)
)

x
```

```
# A tibble: 2 x 2
    key     z
  <dbl> <dbl>
1     1     3
2     2     4
```

```
# A tibble: 2 x 3
    key     a b
  <dbl> <dbl> <chr>
1     3    10 s
2     1    10 s
```

Listado 11.4 Ejemplo ilustrativo mínimo para entender las **Mutating joins**

```
# Tabla y ----
y <- tibble(
  key = c(3, 1),
  a = 10,
  b = "s"
)

y
```

A partir de estas 2 tablas, a continuación se examinará cómo opera cada una de las **Mutating joins**, examinando su comportamiento tanto de forma gráfica como mediante el uso de las funciones correspondientes. Posteriormente, estos conceptos se aplicarán de manera práctica para reconstruir el archivo `2021-2024_saber-pro-genericas_faedis.rds`, teniendo en cuenta la Advertencia 1.

11.6.1. Inner joins

La función `inner_join()` incluye únicamente las observaciones cuyas claves coinciden simultáneamente en ambas tablas (`x` y `y`), combinando todas sus variables en una nueva tabla, tal como se señala a continuación:

Listado 11.5 Uso de la función `inner_join(x, y)` para combinar las observaciones que coinciden tanto en `x` como en `y`

```
inner_join(
  x = x,
  y = y,
  by = join_by(key)
)
```

```
# A tibble: 1 x 4
    key     z     a b
  <dbl> <dbl> <dbl> <chr>
1     1     3    10 s
```

En el Listado 11.5, a través de la función auxiliar `join_by()` en el argumento `by`, se especifica directamente la variable `key` debido a que ambas tablas comparten el mismo

nombre para dicho atributo, a diferencia del Listado 11.1 donde fue necesario emplear el operador relacional de igualdad `==` al diferir los nombres.

Al aplicar la función sobre el ejemplo ilustrativo, la tabla resultante conserva una única fila con la clave `key = 1`, la cual representa la única coincidencia mutua, integrando las columnas `key` y `z` de `x` junto con las columnas `a` y `b` de `y`. Por el contrario, las observaciones con `key = 2` en la tabla `x` y `key = 3` en la tabla `y` se descartan automáticamente al no tener una coincidencia en la otra tabla.

En la Figura 11.2 se ilustra gráficamente este comportamiento. A la derecha se ubican las tablas de entrada `x` (arriba) y `y` (abajo), donde un nodo resalta la coincidencia exclusiva en `key = 1`, generando a la izquierda la tabla resultante con las 4 variables consolidadas.

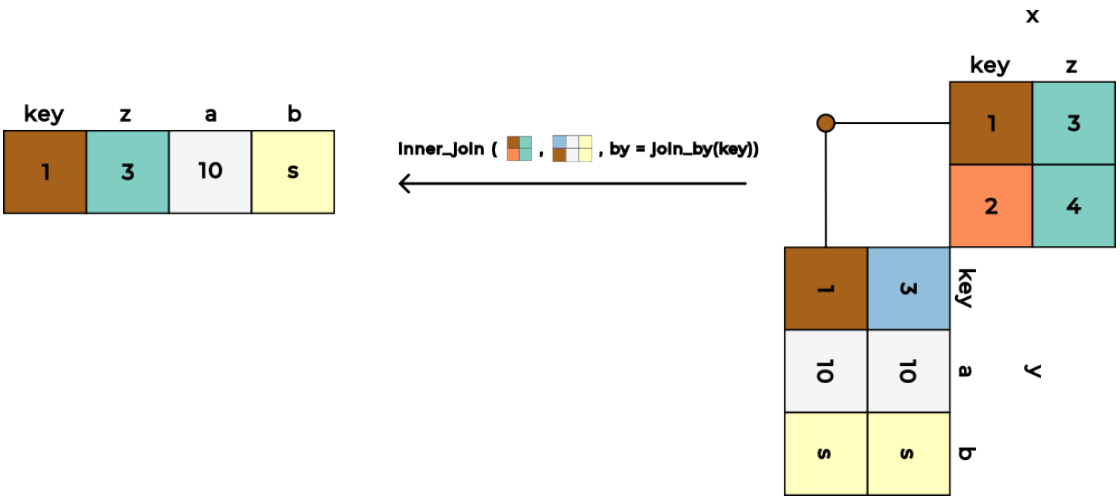


Figura 11.2.: Representación gráfica de la función `inner_join()`

11.6.2. Left joins

La función `left_join()` conserva todas las observaciones de la tabla izquierda (`x`), independientemente de si tienen o no una coincidencia en la tabla derecha (`y`), e incorpora las variables de `y` donde exista una coincidencia con la clave de `x`, tal como se señala a continuación:

```
# A tibble: 2 x 4
    key     z     a b
  <dbl> <dbl> <dbl> <chr>
1     1     3    10 s
2     2     4    NA <NA>
```

Listado 11.6 Uso de la función `left_join(x, y)` para incluir todas las observaciones en `x` e incorporar las variables de `y`

```
left_join(
  x = x,
  y = y,
  by = join_by(key)
)
```

En el Listado 11.6, al igual que en el Listado 11.5, a través de la función auxiliar `join_by()` en el argumento `by`, se especifica directamente la variable `key` debido a que ambas tablas comparten el mismo nombre para dicho atributo.

Al aplicar la función sobre el ejemplo ilustrativo, la tabla resultante conserva las 2 filas de la tabla `x` (`key = 1` y `key = 2`). Para la fila con `key = 1`, al existir una coincidencia en `y`, se agregan los valores correspondientes de las columnas `a` (`10`) y `b` (`"s"`). Por el contrario, para la fila con `key = 2`, como no existe una coincidencia en `y`, las variables `a` y `b` se rellenan automáticamente con valores faltantes (`NA`). Por su parte, la observación con `key = 3` de la tabla `y` se descarta al no estar presente en `x`.

En la Figura 11.3 se ilustra gráficamente este comportamiento. A la derecha se ubican las tablas de entrada `x` (arriba) y `y` (abajo), donde la fila con `key = 1` se enlaza con su coincidencia en `y`, mientras que la fila con `key = 2` se vincula con un registro de valores `NA`, generando a la izquierda la tabla resultante de 2 filas con las 4 variables consolidadas.

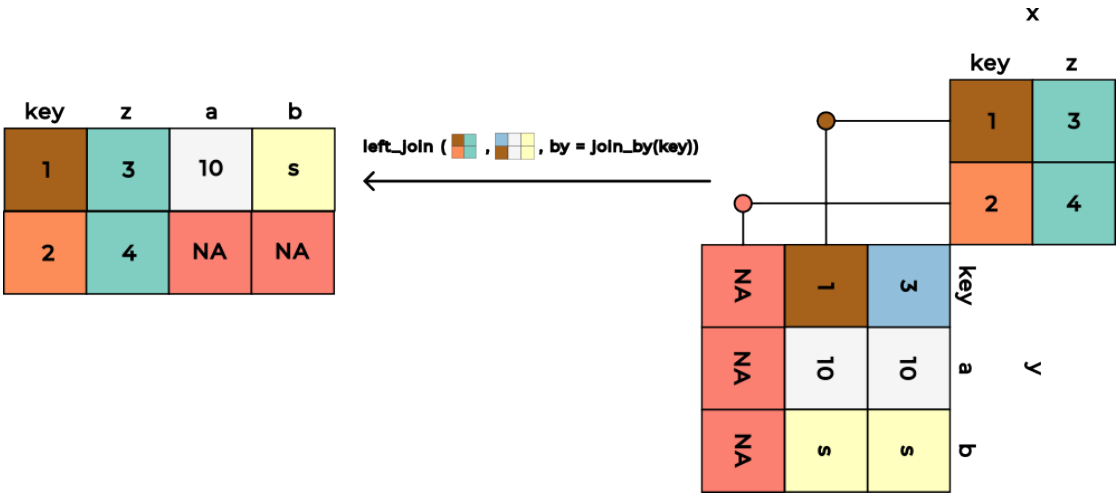


Figura 11.3.: Representación gráfica de la función `left_join()`

11.6.3. Right joins

La función `right_join()` conserva todas las observaciones de la tabla derecha (`y`), independientemente de si tienen o no coincidencia en la tabla izquierda (`x`), e incorpora las variables de `x` donde exista una coincidencia con la clave de `y`, tal como se señala a continuación:

Listado 11.7 Uso de la función `right_join(x, y)` para incluir todas las observaciones en `y` e incorporar las variables de `x`

```
right_join(
  x = x,
  y = y,
  by = join_by(key)
)
```

```
# A tibble: 2 x 4
    key     z     a b
  <dbl> <dbl> <dbl> <chr>
1     1     3    10 s
2     3    NA    10 s
```

En el Listado 11.7, al igual que en el Listado 11.5, a través de la función auxiliar `join_by()` en el argumento `by`, se especifica directamente la variable `key` debido a que ambas tablas comparten el mismo nombre para dicho atributo.

Al aplicar la función sobre el ejemplo ilustrativo, la tabla resultante conserva las 2 filas de la tabla `y` (`key = 3` y `key = 1`). Para la fila con `key = 3`, como no existe una coincidencia en `x`, la variable `z` se rellena automáticamente con un valor faltante (`NA`). Por el contrario, para la fila con `key = 1`, al existir una coincidencia en `x`, se agrega el valor correspondiente de la columna `z` (`3`). Por su parte, la observación con `key = 2` de la tabla `x` se descarta al no estar presente en `y`.

En la Figura 11.4 se ilustra gráficamente este comportamiento. A la derecha se ubican las tablas de entrada `x` (arriba) y `y` (abajo), donde la fila con `key = 3` se vincula con un registro de valor `NA`, mientras que la fila con `key = 1` se enlaza con su coincidencia en `x`, generando a la izquierda la tabla resultante de 2 filas con las 4 variables consolidadas.

11.6.4. Full joins

La función `full_join()` conserva todas las observaciones de ambas tablas (`x` y `y`), manteniendo los registros independientemente de si tienen o no una coincidencia en la otra tabla, tal como se señala a continuación:

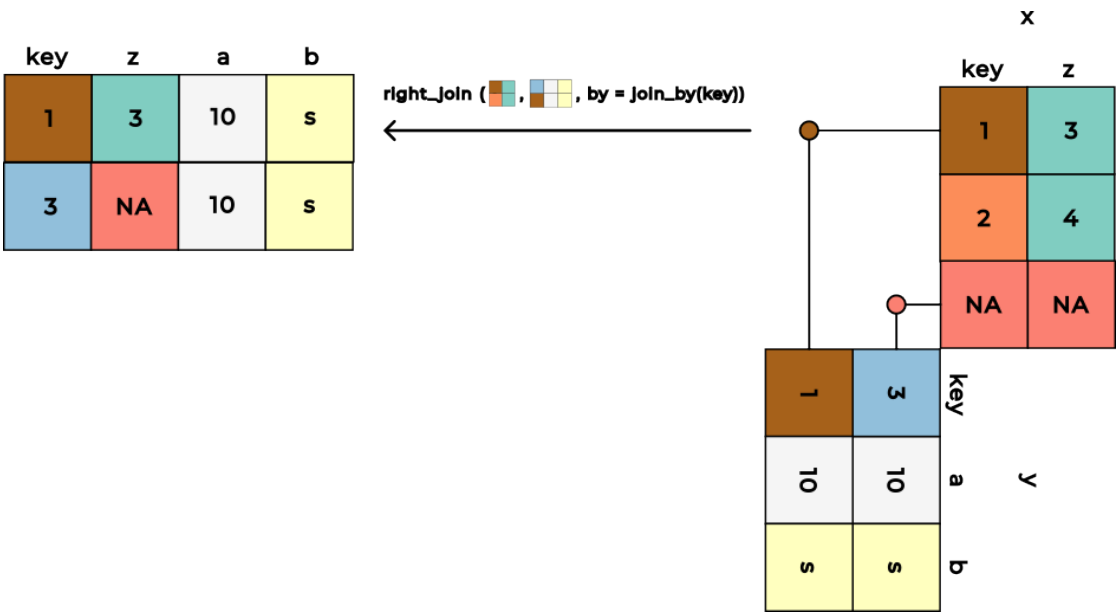


Figura 11.4.: Representación gráfica de la función `right_join()`

Listado 11.8 Uso de la función `full_join(x, y)` para incluir todas las observaciones en `x` y `y`

```
full_join(
  x = x,
  y = y,
  by = join_by(key)
)
```

```
# A tibble: 3 x 4
    key     z     a b
  <dbl> <dbl> <dbl> <chr>
1     1     3    10 s
2     2     4    NA <NA>
3     3    NA    10 s
```

En el Listado 11.8, al igual que en el Listado 11.5, a través de la función auxiliar `join_by()` en el argumento `by`, se especifica directamente la variable `key` debido a que ambas tablas comparten el mismo nombre para dicho atributo.

Al aplicar la función sobre el ejemplo ilustrativo, la tabla resultante conserva las 3 filas correspondientes a todas las claves presentes (`key = 1`, `key = 2` y `key = 3`). Para la fila con `key = 1`, al existir una coincidencia mutua, se integran las columnas `key` y `z` de `x`

junto con las columnas `a` (`10`) y `b` (`"s"`) de `y`. En el caso de la fila con `key = 2`, como no existe una coincidencia en `y`, las variables `a` y `b` se rellenan automáticamente con valores faltantes (`NA`). Por su parte, para la fila con `key = 3`, al no existir una coincidencia en `x`, la variable `z` se completa con un valor faltante (`NA`).

En la Figura 11.5 se ilustra gráficamente este comportamiento. A la derecha se ubican las tablas de entrada `x` (arriba) y `y` (abajo), donde la fila con `key = 1` se enlaza con su coincidencia mutua, mientras que las filas no coincidentes (`key = 2` en `x` y `key = 3` en `y`) se vinculan con sus respectivos registros de valores `NA`, generando a la izquierda la tabla resultante de 3 filas con las 4 variables consolidadas.

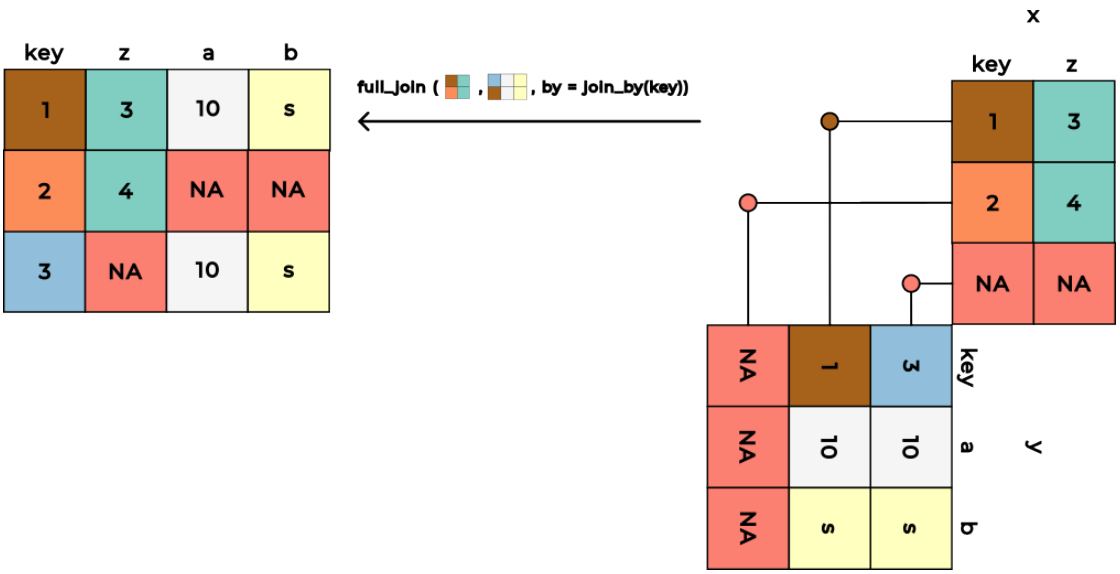


Figura 11.5.: Representación gráfica de la función `full_join()`

11.7. Caso de estudio: consolidación de resultados Saber-Pro para FAEDIS

El propósito de la Sección 11.7 es combinar las tablas señaladas en la Nota 23, teniendo en cuenta la Figura 11.1, mediante el uso de las funciones señaladas en la Sección 11.6. De esa manera es posible reconstruir el archivo `2021-2024_saber-pro-genericas_faedis.rds` e incorporar los aspectos señalados en la Advertencia 1.

Para alcanzar este objetivo en primera instancia se realizará una descripción de las variables respecto a las tablas asociadas con los archivos `divipola-municipios.rds` y `programas-pregrado-faedis.rds`, teniendo en cuenta que este mismo ejercicio se realizó previamente con las tablas restantes en la Sección 11.5.1. Posteriormente, se ajustarán los datos en el archivo `resultados-saber-pro.rds` para incorporar los aspectos señalados en la Advertencia 1 introduciendo la función `replace_when()` del paquete `dplyr`. Finalmente

se consolidarán los resultados del Saber-Pro para FAEDIS en un archivo denominado `2021-2024_saber-pro-genericas_faedis_ajustado.rds`.

11.7.1. Descripción de variables

Para utilizar los verbos de 2 tablas de `dplyr` es necesario entender a qué se refieren cada una de las variables en los archivos `divipola-municipios.rds` y `programas-pregrado-faedis.rds`, que corresponden a las tablas de las entidades **Municipio** y **Programa académico**. En primera instancia, importarás los datos, tal como se señala a continuación:

```
divipola_municipios <- read_rds(
  file = "data/divipola-municipios.rds"
)

programas_pregrado_faedis <- read_rds(
  file = "data/programas-pregrado-faedis.rds"
)
```

Una vez importes las tablas, podrás darle un "vistazo" a cada una de ellas, tal como se señala a continuación:

```
# Inspeccionar datos ----
divipola_municipios |>
  glimpse()
```

```
Rows: 1,122
Columns: 8
$ codigo_departamento  <chr> "05", "05", "05", "05", "05",~
$ codigo_municipio     <chr> "05001", "05002", "05004", "0~
$ nombre_municipio     <chr> "MEDELLÍN", "ABEJORRAL", "ABR~
$ tipo                 <chr> "Municipio", "Municipio", "Mu~
$ longitud             <dbl> -75.58177, -75.42874, -76.064~
$ latitud              <dbl> 6.246631, 5.789315, 6.632282,~
$ notas                <chr> NA, NA, NA, NA, NA, NA, NA, N~
$ ultima_actualizacion <date> 2025-12-30, 2025-12-30, 2025~
```

```
programas_pregrado_faedis |>
  glimpse()
```

```
Rows: 8
Columns: 4
$ codigo_snies_programa <int> 6527, 102005, 108241, 11428,~
$ nombre_programa       <chr> "ADMINISTRACION DE EMPRESAS"~
$ estado_programa       <chr> "Activo", "Inactivo", "Activ~
$ ultima_actualizacion  <date> 2026-08-25, 2026-08-25, 202~
```

Teniendo en cuenta la inspección de datos, a continuación se describen cada una de las variables en `divipola-municipios.rds`:

1. **codigo_departamento**: código único asociado al departamento al que pertenece cada municipio de acuerdo con la codificación DIVIPOLA.
2. **codigo_municipio**: código único asociado a cada municipio de acuerdo con la codificación DIVIPOLA.
3. **nombre_municipio**: nombre oficial del municipio de acuerdo con la codificación DIVIPOLA.
4. **tipo**: tipo de entidad territorial de acuerdo con la clasificación del DANE (`Municipio`, `Área no municipalizada` o `Isla`).
5. **longitud**: coordenada geográfica en grados decimales que mide la distancia respecto al meridiano de Greenwich, indicando la posición este-oeste del punto representativo de cada municipio.
6. **latitud**: coordenada geográfica en grados decimales que mide la distancia respecto a la línea ecuatorial, indicando la posición norte-sur del punto representativo de cada municipio.
7. **notas**: notas aclaratorias o precisiones sobre la delimitación o creación territorial de ciertos municipios o áreas geográficas de acuerdo con el DANE.
8. **ultima_actualizacion**: fecha de corte de la última consulta realizada en la base de datos DIVIPOLA del DANE.

De la misma forma, a continuación se describen cada una de las variables en `programas-pregrado-faedis.rds`:

1. **codigo_snies_programa**: código único asignado al programa académico de acuerdo con el SNIES.
2. **nombre_programa**: nombre oficial del programa académico de pregrado ofertado por FAEDIS.
3. **estado_programa**: estado del registro del programa académico ante el SNIES (`Activo` o `Inactivo`).
4. **ultima_actualizacion**: fecha de corte de la última consulta realizada en la base de datos del SNIES.

11.7.2. Ajuste de datos

Tal como se señaló en la Advertencia 1, la fuente original de datos (Data Icfes) asigna por defecto a Bogotá como lugar de presentación cuando el examen se realiza en el exterior. Esto genera una inconsistencia en las variables `estu_cod_depto_presentacion` y `estu_cod_mcpio_presentacion` para los registros donde los estudiantes presentaron la prueba por fuera de Colombia, tal como se señala a continuación:

```
resultado_saber_pro |>
  filter(estu_exterior == "SI") |>
  select(
    ano,
    estu_consecutivo,
    estu_exterior,
    estu_cod_depto_presentacion,
    estu_cod_mcpio_presentacion
  ) |>
  glimpse()
```

```
Rows: 64
Columns: 5
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202130321539", "EK2~
$ estu_exterior                <chr> "SI", "SI", "SI", "SI"~
$ estu_cod_depto_presentacion  <chr> "11", "11", "11", "11"~
$ estu_cod_mcpio_presentacion  <chr> "11001", "11001", "110~
```

Para corregir esta situación y asegurar la coherencia con el modelo relacional de la Figura 11.1, es necesario reemplazar los códigos de departamento y municipio por valores faltantes (`NA`) cuando `estu_exterior == "SI"`.

En el paquete `dplyr`, la función `replace_when()` permite actualizar de forma condicional los valores de un vector o columna existente. Para realizar este proceso, la función `replace_when()` utiliza los parámetros `x` y `...`, tal como se señala a continuación:

I. `x`: especifica el vector o variable de entrada cuyos valores se desean modificar.

II. `...`: permite suministrar una secuencia de fórmulas de 2 lados (en inglés **two-sided formulas**) bajo la estructura `condición ~ valor_de_reemplazo` mediante el uso del operador virgulilla (`~`):

- **Lado izquierdo** (en inglés **Left-Hand Side (LHS)**): define una condición lógica evaluada sobre cada observación para determinar cuáles cumplen el criterio (`TRUE`).

- **Lado derecho** (en inglés **Right-Hand Side (RHS)**): indica el valor con el cual se reemplazarán las observaciones que satisfacen la condición lógica del lado izquierdo.

A partir de estos parámetros, `replace_when()` evalúa las condiciones especificadas y actualiza únicamente los registros correspondientes, mientras que las observaciones que no cumplen ninguna condición conservan automáticamente su valor original desde `x` sin alterar el tipo de dato de la variable.

Para aplicar este ajuste sobre las variables de interés en `resultado_saber_pro`, se utiliza la función `replace_when()` en conjunto con `mutate()`, tal como se señala a continuación:

Listado 11.9 Uso de la función `replace_when()` junto con `mutate()` para actualizar parcialmente los valores de una variable dentro de una tibble

```
resultado_saber_pro_ajustado <- resultado_saber_pro |>
  mutate(
    estu_cod_depto_presentacion = replace_when(
      x = estu_cod_depto_presentacion,
      estu_exterior == "SI" ~ NA
    ),
    estu_cod_mcpio_presentacion = replace_when(
      x = estu_cod_mcpio_presentacion,
      estu_exterior == "SI" ~ NA
    )
  )
```

En el Listado 11.9, dentro de la función `mutate()`, se aplica `replace_when()` a las columnas `estu_cod_depto_presentacion` y `estu_cod_mcpio_presentacion`. En ambos casos, el parámetro `x` recibe la columna correspondiente como vector de entrada y en el argumento `...` se define la fórmula `estu_exterior == "SI" ~ NA`. En esta expresión, la condición lógica (**LHS**) identifica a los estudiantes que presentaron la prueba en el exterior y el valor de reemplazo (**RHS**) les asigna `NA`, mientras que las observaciones restantes conservan intactos sus códigos originales.

Para verificar que la transformación se aplicó de manera correcta, en primera instancia puedes inspeccionar los registros donde el examen se presentó en el exterior (`estu_exterior == "SI"`), comprobando que los códigos de departamento y municipio ahora contienen valores ausentes (`NA`), tal como se señala a continuación:

```
# Inspeccionar registros ajustados en el exterior ----
resultado_saber_pro_ajustado |>
  filter(estu_exterior == "SI") |>
```

```
  select(
    ano,
    estu_consecutivo,
    estu_exterior,
    estu_cod_depto_presentacion,
    estu_cod_mcpio_presentacion
  ) |>
  glimpse()
```

```
Rows: 64
Columns: 5
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202130321539", "EK2~
$ estu_exterior                <chr> "SI", "SI", "SI", "SI"~
$ estu_cod_depto_presentacion <chr> NA, NA, NA, NA, NA, NA~
$ estu_cod_mcpio_presentacion <chr> NA, NA, NA, NA, NA, NA~
```

De igual forma, puedes verificar que los registros correspondientes a los estudiantes que presentaron el examen dentro del territorio nacional (`estu_exterior == "NO"`) conservaron sus códigos originales de departamento y municipio sin ninguna alteración, tal como se señala a continuación:

```
# Inspeccionar registros en Colombia ----
resultado_saber_pro_ajustado |>
  filter(estu_exterior == "NO") |>
  select(
    ano,
    estu_consecutivo,
    estu_exterior,
    estu_cod_depto_presentacion,
    estu_cod_mcpio_presentacion
  ) |>
  glimpse()
```

```
Rows: 1,175
Columns: 5
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion <chr> "11", "11", "11", "11"~
$ estu_cod_mcpio_presentacion <chr> "11001", "11001", "110~
```

11.7.3. Consolidación de tablas

En la Sección 11.7.3 reconstruirás el archivo `2021-2024_saber-pro-genericas_faedis.rds` incorporando los aspectos señalados en la Advertencia 1.

Para realizar este proceso utilizarás inicialmente verbos para filas (ver Sección 10.1.1.1) y verbos para columnas (ver Sección 10.1.1.2), así como la función `left_join()` (ver Sección 11.6.2), para luego exportar los datos en un archivo denominado `2021-2024_saber-pro-genericas_faedis_ajustado.rds` en la carpeta `data`.

En la Figura 11.1, la entidad central que conecta cada una de las tablas es `Resultado Saber-Pro`, que corresponde al objeto `resultado_saber_pro_ajustado` (ver Sección 11.7.2). A partir de esta tabla es necesario conectar la información que se encuentra en los objetos `divipola_departamentos`, `divipola_municipios` y `programas_pregrado_faedis`, pero filtrando y seleccionando la información de interés, tal como se señala a continuación:

Listado 11.10 Selección y filtro de variables de interés

```
programas_pregrado_faedis <- programas_pregrado_faedis |>
  filter(estado_programa == "Activo") |>
  select(
    codigo_snies_programa,
    nombre_programa
  )

divipola_departamentos <- divipola_departamentos |>
  select(
    codigo,
    nombre
  )

divipola_municipios <- divipola_municipios |>
  select(
    codigo_municipio,
    nombre_municipio
  )
```

En el Listado 11.10 solo es necesario tener en cuenta los programas académicos de pregrado que se encuentran activos, así como seleccionar los atributos relacionados con las claves que identifican a cada instancia junto con sus nombres, con el fin de reconstruir el archivo `2021-2024_saber-pro-genericas_faedis.rds` ajustado.

Una vez aplicados estos filtros y selecciones, es posible integrar cada una de las tablas

de manera secuencial, vinculando en cada paso 2 tablas de datos correspondientes a 2 entidades mediante claves primarias y foráneas, tal como se señala a continuación:

Listado 11.11 Uso secuencial de la función `left_join()` para conectar o unir tablas de datos de diferentes entidades

```
resultado_saber_pro_ajustado <- resultado_saber_pro_ajustado |>
  left_join(
    y = programas_pregrado_faedis,
    by = join_by(
      estu_snies_prgmacademico == codigo_snies_programa
    )
  ) |>
  left_join(
    y = divipola_departamentos,
    by = join_by(
      estu_cod_depto_presentacion == codigo
    )
  ) |>
  left_join(
    y = divipola_municipios,
    by = join_by(
      estu_cod_mcpio_presentacion == codigo_municipio
    )
  )

resultado_saber_pro_ajustado |>
  glimpse()
```

```
Rows: 1,239
Columns: 21
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_genero                  <fct> M, M, M, M, F, F, F, F~
$ estu_semestrecursa           <ord> 09, 09, 09, 09, 09, 09~
$ fami_estratovivienda         <ord> Estrato 3, Estrato 4, ~
$ fami_educacionmadre          <ord> Técnica o tecnológica ~
$ fami_educacionpadre          <ord> Secundaria (Bachillera~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion  <chr> "11", "11", "11", "11"~
$ estu_cod_mcpio_presentacion  <chr> "11001", "11001", "110~
$ estu_snies_prgmacademico     <int> 6527, 11428, 11428, 11~
$ mod_competen_ciudada_punt    <dbl> 162, 190, 154, 179, 17~
```

```
$ mod_comuni_escrita_punt     <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt             <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem            <ord> A2, A2, A2, A2, A2, A2~
$ mod_lectura_critica_punt    <dbl> 114, 168, 139, 148, 18~
$ mod_razona_cuantitat_punt   <dbl> 135, 164, 113, 149, 16~
$ punt_global                 <dbl> 139, 170, 128, 155, 16~
$ nombre_programa             <chr> "ADMINISTRACION DE EMP~
$ nombre                      <chr> "BOGOTÁ, D.C.", "BOGOT~
$ nombre_municipio            <chr> "BOGOTÁ, D.C.", "BOGOT~
```

En el Listado 11.11, partiendo de la tabla `resultado_saber_pro_ajustado`, es posible unir de manera secuencial la información filtrada y seleccionada de las tablas `programas_pregrado_faedis`, `divipola_departamentos` y `divipola_municipios`, correspondientes a las entidades `Programa académico`, `Departamento` y `Municipio` (ver Figura 11.1).

A partir de la unión realizada en el Listado 11.11, es necesario renombrar y reorganizar las tres últimas columnas teniendo en cuenta que dicha información proviene de las tablas `programas_pregrado_faedis`, `divipola_departamentos` y `divipola_municipios`, tal como se señala a continuación:

Listado 11.12 Uso de la función `rename()` para renombrar columnas que provienen de otras tablas

```
resultado_saber_pro_ajustado <- resultado_saber_pro_ajustado |>
  rename(
    estu_prgm_academico = nombre_programa,
    estu_depto_presentacion = nombre,
    estu_mcpio_presentacion = nombre_municipio
  )
```

```
Rows: 1,239
Columns: 21
$ ano                         <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo            <chr> "EK202120262469", "EK2~
$ estu_genero                 <fct> M, M, M, M, F, F, F, F~
$ estu_semestrecursa          <ord> 09, 09, 09, 09, 09, 09~
$ fami_estratovivienda        <ord> Estrato 3, Estrato 4, ~
$ fami_educacionmadre         <ord> Técnica o tecnológica ~
$ fami_educacionpadre         <ord> Secundaria (Bachillera~
$ estu_exterior               <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion <chr> "11", "11", "11", "11"~
$ estu_depto_presentacion     <chr> "BOGOTÁ, D.C.", "BOGOT~
```

Listado 11.13 Uso de la función `relocate()` para reubicar columnas que provienen de otras tablas

```
resultado_saber_pro_ajustado <- resultado_saber_pro_ajustado |>
  relocate(
    estu_prgm_academico,
    .after = estu_snies_prgmacademico
  ) |>
  relocate(
    estu_depto_presentacion,
    .after = estu_cod_depto_presentacion
  ) |>
  relocate(
    estu_mcpio_presentacion,
    .after = estu_cod_mcpio_presentacion
  )

resultado_saber_pro_ajustado |>
  glimpse()
```

```
$ estu_cod_mcpio_presentacion <chr> "11001", "11001", "110~
$ estu_mcpio_presentacion     <chr> "BOGOTÁ, D.C.", "BOGOT~
$ estu_snies_prgmacademico    <int> 6527, 11428, 11428, 11~
$ estu_prgm_academico         <chr> "ADMINISTRACION DE EMP~
$ mod_competen_ciudada_punt   <dbl> 162, 190, 154, 179, 17~
$ mod_comuni_escrita_punt     <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt             <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem            <ord> A2, A2, A2, A2, A2, A2~
$ mod_lectura_critica_punt    <dbl> 114, 168, 139, 148, 18~
$ mod_razona_cuantitat_punt   <dbl> 135, 164, 113, 149, 16~
$ punt_global                 <dbl> 139, 170, 128, 155, 16~
```

Aplicando los cambios señalados en el Listado 11.12 y el Listado 11.13, se logra finalmente reconstruir el archivo `2021-2024_saber-pro-genericas_faedis.rds` ajustado. En ese sentido, es posible exportar el archivo y guardarlo en la carpeta `data` para un uso posterior, tal como se señala a continuación:

En el Listado 11.14 se utiliza la función `write_rds()` junto con el parámetro `file` asociado con el argumento `"data/2021-2024_saber-pro-genericas_faedis_ajustado.rds"` para definir tanto la ruta como el nombre del archivo en el que se guardará la información.

Listado 11.14 Uso de la función `write_rds()` para exportar y guardar el archivo de datos ajustado

```
resultado_saber_pro_ajustado |>
  write_rds(
    file = "data/2021-2024_saber-pro-genericas_faedis_ajustado.rds"
  )
```

> **Nota 26**
>
> En el Capítulo 12 se abarcarán con más detalle los archivos con extensión `.rds` mediante la función `read_rds()`, así como la función análoga `write_rds()` para exportar datos, teniendo en cuenta lo indicado en el Listado 11.14.

Parte IV.

Importación de datos

El objetivo de esta parte, es presentar cómo importar datos en R donde es el primer paso que se debe aplicar cuando se requiere realizar un análisis de datos sin importar el lenguaje de programación o herramienta que se utilice. En particular, se utilizará inicialmente el paquete `readr` y posteriormente otros paquetes, a través de los siguientes 4 capítulos:

- Capítulo 12: introduce la importación y exportación de datos en el formato nativo de R a través de la extensión `.rds` utilizando el paquete `readr`.
- Capítulo 13: introduce la importación y exportación de datos de archivos de texto plano rectangulares utilizando el paquete `readr`.
- Capítulo 14: introduce la importación de datos de hojas de cálculo de Excel (`.xls` y `.xlsx`) utilizando el paquete `readxl`.
- Capítulo 15: introduce la importación y posterior visualización de datos geoestadísticos a través del formato GeoPackage (`.gpkg`) utilizando los paquetes `sf` y `ggplot2`.

Una vez se entienda el funcionamiento del paquete `readr` así como de los otros paquetes utilizados, si estos no cubren tus necesidades o buscas una alternativa unificada para importar y exportar múltiples formatos con una sintaxis simple, puedes explorar el paquete `rio` (en inglés **R** **I**nput/**O**utput). El paquete `rio` simplifica el flujo de trabajo mediante 2 funciones principales: `import()` y `export()`. Estas funciones detectan automáticamente la extensión del archivo y seleccionan internamente el paquete adecuado en caso que exista.

Por otra parte, si requieres importar datos desde fuentes o formatos especializados, puedes también consultar los CRAN Task Views. Estas son guías temáticas curadas por la comunidad de R que agrupan y describen los paquetes más importantes según el área de trabajo. Entre las guías más útiles que contemplan la importación de datos, se destacan:

- Databases: paquetes para conectar R con bases de datos relacionales y no relacionales.
- Analysis of Spatial Data: herramientas para leer, escribir y manipular datos espaciales y geográficos.
- Web Technologies and Services: paquetes orientados a la descarga directa e ingestión de datos (en inglés **direct data download and ingestion**), interacción con APIs y extracción de datos desde la web.
- Official Statistics & Survey Statistics: herramientas especializadas para importar datos de oficinas y organismos oficiales de estadística, censos y encuestas.

12. Serialización de datos en R

Este capítulo se basa en (Wickham, 2023, Capítulo 7) así como en la documentación oficial de `readr` donde tiene como propósito introducir la importación y exportación de datos en el formato nativo de R a través de la extensión `.rds` utilizando el paquete `readr` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes al periodo 2021-2024 para Colombia.

Exploraremos los siguientes aspectos:

1. ¿Qué es el formato `.rds` y cuál es su propósito al almacenar e importar datos en R?
2. ¿Cúales son las ventajas y desventajas al utilizar archivos `.rds` frente a otros formatos en R?
3. ¿Cómo importar datos almacenados en formato `.rds` a R?
4. ¿Cómo exportar datos en formato `.rds` desde R?

> **Nota 27**
>
> Para desarrollar el contenido de este capítulo es necesario que descargues el archivo señalado a continuación y lo incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **2021-2024_saber-pro-genericas_faedis_ajustado.csv**
>
> Adicionalmente, es necesario que hayas exportado y guardado el archivo `2021-2024_saber-pro-genericas_faedis_ajustado.rds` en la carpeta `data`, tal como se señaló en el Listado 11.14. En caso que no lo hayas hecho es necesario que lo descargues y lo incluyas dentro de la carpeta `data`:
>
> **2021-2024_saber-pro-genericas_faedis_ajustado.rds**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `012_importar-datos-rds-en-r` y dentro de ella genera el archivo `001_importar-datos-formato-rds.R`, siguiendo las pautas de la Sección 4.4.

12.1. El formato `.rds`

El formato con extensión `.rds` es el formato nativo de R diseñado para guardar un único objeto en un archivo y restaurarlo cuando sea necesario. Al ser un formato propio de R, permite almacenar cualquier estructura de datos conservando sus componentes originales.

Este proceso se basa en el concepto de serialización, que es el proceso de traducir una estructura de datos o el estado de un objeto a un formato que pueda almacenarse o transmitirse y reconstruirse más adelante.

Dado que el archivo `.rds` almacena únicamente el contenido y la estructura de un solo objeto y no el nombre con el que fue creado, al momento de importarlo a R puedes asociarle cualquier nombre **sintácticamente válido** que desees (ver Importante 6).

12.2. Ventajas y desventajas de utilizar archivos `.rds`

Al trabajar con el formato `.rds` dentro de R, es conveniente considerar tanto sus aspectos favorables como sus limitaciones al momento de almacenar información en un flujo de trabajo orientado a proyectos.

Por un lado, entre sus principales ventajas se destaca la capacidad de conservar de forma íntegra los tipos de datos y los atributos de cada objeto, garantizando que no se pierda la estructura original de los datos en R. Asimismo, al tratarse de un archivo binario que incorpora compresión por defecto, ofrece una mayor eficiencia en el espacio de almacenamiento ocupado en disco y una mayor velocidad en los procesos de lectura y escritura.

Por otro lado, entre sus principales desventajas se resalta su falta de interoperabilidad, debido a que es un formato propio de R que no puede ser utilizado de manera directa por otras herramientas o lenguajes sin dependencias adicionales. Además, al ser un archivo binario, no es posible inspeccionar su contenido directamente desde un editor de texto convencional, requiriendo cargarlo en una sesión activa de R para examinar la información.

12.3. Importación de datos en formato `.rds`

Para importar un archivo en formato `.rds` a R, se utiliza la función `read_rds()` del paquete `readr`. Dado que `readr` forma parte del `tidyverse`, es necesario cargar previamente el paquete `tidyverse` mediante `library(tidyverse)` (ver Sección 4.3) en el archivo `001_importar-datos-formato-rds.R`. Para realizar la importación, la función `read_rds()` utiliza el parámetro `file`, en el cual se especifica como argumento la ruta y el nombre del archivo con extensión `.rds` que se desea leer.

Para importar el conjunto de datos ajustado de FAEDIS en formato `.rds` (ver Nota 27) y asignarlo a una variable denominada `resultado_saber_pro_ajustado_rds`, se utiliza la función `read_rds()`, tal como se señala a continuación:

Listado 12.1 Uso de la función `read_rds()` para importar un archivo de datos en formato `.rds`

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
resultado_saber_pro_ajustado_rds <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis_ajustado.rds"
)
```

En el Listado 12.1, primero se carga el `tidyverse` mediante `library(tidyverse)`. Posteriormente, se utiliza la función `read_rds()` con el parámetro `file` asociado al argumento `"data/2021-2024_saber-pro-genericas_faedis_ajustado.rds"` para leer el archivo y asignar explícitamente el objeto resultante a la variable `resultado_saber_pro_ajustado_rds` (ver Sección 12.1).

Con el propósito de contrastar la preservación de los tipos de datos frente a un formato `.csv`, es posible importar de forma análoga el archivo `2021-2024_saber-pro-genericas_faedis_ajustado.csv` utilizando la función `read_csv()` y asignarlo a la variable `resultado_saber_pro_ajustado_csv`, tal como se señala a continuación:

Listado 12.2 Uso de la función `read_csv()` para importar el archivo de datos en formato `.csv`

```
# Importar datos en formato .csv ----
resultado_saber_pro_ajustado_csv <- read_csv(
  file = "data/2021-2024_saber-pro-genericas_faedis_ajustado.csv"
)
```

```
Rows: 1239 Columns: 21
-- Column specification --------------------------------------
Delimiter: ","
chr (13): estu_consecutivo, estu_genero, estu_semestrecu...
dbl  (8): ano, estu_snies_prgmacademico, mod_competen_ci...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

En el Listado 12.2, al ejecutar la función `read_csv()`, R genera un mensaje informativo en la consola con la especificación de columnas (`Column specification`). Dado que los archivos de `.csv` no contienen metadatos sobre la estructura interna de los datos, `read_csv()` requiere identificar un delimitador (en este caso, `Delimiter: ","`) e inferir el tipo de dato de cada variable a partir de los primeros valores del archivo, clasificándolas en caracteres (`chr`) y números reales (`dbl`). En contraste, `read_rds()` no requiere delimitadores ni realiza inferencias, ya que lee directamente la representación binaria con los tipos originales almacenados en el archivo.

Al inspeccionar ambas tibbles con la función `glimpse()`, es posible examinar las diferencias en la estructura de los datos:

```
# Inspeccionar datos en rds ----
resultado_saber_pro_ajustado_rds |>
  glimpse()
```

```
Rows: 1,239
Columns: 21
$ ano                          <int> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_genero                  <fct> M, M, M, M, F, F, F, F~
$ estu_semestrecursa           <ord> 09, 09, 09, 09, 09, 09~
$ fami_estratovivienda         <ord> Estrato 3, Estrato 4, ~
$ fami_educacionmadre          <ord> Técnica o tecnológica ~
$ fami_educacionpadre          <ord> Secundaria (Bachillera~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion <chr> "11", "11", "11", "11"~
$ estu_depto_presentacion      <chr> "BOGOTÁ, D.C.", "BOGOT~
$ estu_cod_mcpio_presentacion <chr> "11001", "11001", "110~
$ estu_mcpio_presentacion      <chr> "BOGOTÁ, D.C.", "BOGOT~
$ estu_snies_prgmacademico     <int> 6527, 11428, 11428, 11~
$ estu_prgm_academico          <chr> "ADMINISTRACION DE EMP~
$ mod_competen_ciudada_punt    <dbl> 162, 190, 154, 179, 17~
$ mod_comuni_escrita_punt      <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt              <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem             <ord> A2, A2, A2, A2, A2, A2~
$ mod_lectura_critica_punt     <dbl> 114, 168, 139, 148, 18~
$ mod_razona_cuantitat_punt    <dbl> 135, 164, 113, 149, 16~
$ punt_global                  <dbl> 139, 170, 128, 155, 16~
```

```
# Inspeccionar datos en .csv ----
resultado_saber_pro_ajustado_csv |>
  glimpse()
```

```
Rows: 1,239
Columns: 21
$ ano                          <dbl> 2021, 2021, 2021, 2021~
$ estu_consecutivo             <chr> "EK202120262469", "EK2~
$ estu_genero                  <chr> "M", "M", "M", "M", "F~
$ estu_semestrecursa           <chr> "09", "09", "09", "09"~
$ fami_estratovivienda         <chr> "Estrato 3", "Estrato ~
$ fami_educacionmadre          <chr> "Técnica o tecnológica~
$ fami_educacionpadre          <chr> "Secundaria (Bachiller~
$ estu_exterior                <chr> "NO", "NO", "NO", "NO"~
$ estu_cod_depto_presentacion <chr> "11", "11", "11", "11"~
$ estu_depto_presentacion      <chr> "BOGOTÁ, D.C.", "BOGOT~
$ estu_cod_mcpio_presentacion <chr> "11001", "11001", "110~
$ estu_mcpio_presentacion      <chr> "BOGOTÁ, D.C.", "BOGOT~
$ estu_snies_prgmacademico     <dbl> 6527, 11428, 11428, 11~
$ estu_prgm_academico          <chr> "ADMINISTRACION DE EMP~
$ mod_competen_ciudada_punt    <dbl> 162, 190, 154, 179, 17~
$ mod_comuni_escrita_punt      <dbl> 133, 174, 85, 157, 154~
$ mod_ingles_punt              <dbl> 151, 155, 150, 144, 14~
$ mod_ingles_desem             <chr> "A2", "A2", "A2", "A2"~
$ mod_lectura_critica_punt     <dbl> 114, 168, 139, 148, 18~
$ mod_razona_cuantitat_punt    <dbl> 135, 164, 113, 149, 16~
$ punt_global                  <dbl> 139, 170, 128, 155, 16~
```

Al comparar ambos resultados, se observa que en `resultado_saber_pro_ajustado_rds` se preservan las variables categóricas como variables nominales (`<fct>`) y ordinales (`<ord>`) conservando sus niveles y orden, así como los números enteros (`<int>`). En contraste, al importar el archivo `.csv` en `resultado_saber_pro_ajustado_csv`, las columnas categóricas se convierten en caracteres (`<chr>`) y los enteros se interpretan como números reales (`<dbl>`), perdiendo los atributos originales definidos en R.

12.4. Exportación de datos en formato `.rds`

De forma análoga a la importación de datos con `read_rds()`, para exportar y guardar objetos de R en formato `.rds` se utiliza la función `write_rds()` del paquete `readr`. Esta función permite serializar cualquier estructura u objeto de datos directamente en un archivo binario mediante el parámetro `file`, en el cual se especifica la ruta y el nombre del archivo de destino.

El uso de esta función fue abordado en el Listado 11.14 del Capítulo 11, en el cual se empleó `write_rds()` para exportar y guardar el conjunto de datos ajustado tras la aplicación de los verbos de dos tablas.

13. Texto plano

Este capítulo se basa en (Wickham, 2023, Capítulo 7) así como en la documentación oficial de `readr` donde tiene como propósito introducir la importación y exportación de datos de archivos de texto plano rectangulares utilizando el paquete `readr` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes al periodo 2021 para Colombia.

Exploraremos los siguientes aspectos:

1. ¿Qué es un archivo `.csv` y cómo utilizar argumentos como `skip`, `comment`, `col_names` y `na` con `read_csv()`?
2. ¿Cómo importar otros formatos de archivos de texto plano rectangulares (`.tsv`, archivos delimitados por punto y coma (`;`) o delimitadores personalizados)?
3. ¿Cómo controlar los tipos de datos en las columnas mediante `col_types` y diagnosticar problemas de importación con `problems()`?
4. ¿Cómo exportar tablas de datos a archivos de texto plano desde R?

> **Nota 28**
>
> Para desarrollar el contenido de este capítulo es necesario que descargues el archivo señalado a continuación y lo incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **2021-saber_pro-genericas.txt.xz**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `013_importar-datos-texto-plano-en-r` y dentro de ella genera el archivo `001_importar-datos-formato-texto-plano.R`, siguiendo las pautas de la Sección 4.4.

13.1. Archivos de valores separados por comas (`.csv`)

Un archivo con formato `.csv` (en inglés comma-separated values) es un archivo de texto plano estructurado de forma rectangular donde cada línea del archivo representa una fila o registro y las columnas o campos se encuentran separados por comas (`,`). Generalmente, la primera línea del archivo contiene los encabezados o nombres de las variables.

Cuando uno de los campos de texto contiene una coma dentro de su propio contenido, dicho campo se delimita entre comillas dobles (") para evitar que la coma interna sea interpretada erróneamente como un separador de columnas.

Para importar archivos `.csv` a R se utiliza la función `read_csv()` del paquete `readr`. El primer parámetro de esta función es `file`, el cual usualmente recibe una ruta hacia un archivo almacenado en disco (p. ej., `"data/2021-2024_saber-pro-genericas_faedis_ajustado.csv"`). No obstante, para realizar pruebas o construir ejemplos ilustrativos, también es posible suministrar datos directamente desde una cadena de texto en memoria.

Para que `readr` distinga explícitamente que la cadena contiene datos literales en lugar de una ruta a un archivo en disco, se utiliza la función `I()` (**"as is"**) de R al asociar el argumento con `file`, tal como se ilustra a continuación:

Listado 13.1 Importación de una cadena literal en formato `.csv` utilizando `read_csv()` e `I()`

```
# Cargar paquetes ----
library(tidyverse)

# Definir datos en formato .csv ----
pregrado_faedis_csv <- "codigo_snies,nombre_programa
6527,Administración de Empresas
108241,\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428,Contaduría Pública
11004,Ingeniería Civil
53703,Ingeniería Industrial
105142,Ingeniería Informática
10963,Relaciones Internacionales y Estudios Políticos"

# Importar datos en formato .csv ----
pregrado_faedis_tbl <- read_csv(
  file = I(pregrado_faedis_csv)
)
```

```
Rows: 7 Columns: 2
-- Column specification --------------------------------------
Delimiter: ","
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

En el Listado 13.1, la cadena `pregrado_faedis_csv` define dos columnas (`codigo_snies` y `nombre_programa`). Observa los siguientes detalles:

1. **Uso de `I()`**: envolver la variable como `I(pregrado_faedis_csv)` le indica a `read_csv()` que interprete el contenido del objeto como los datos mismos y no como el nombre de un archivo que deba buscar en disco.
2. **Campos con comas internas**: el programa `"Administración de Riesgos, Seguridad y Salud en el Trabajo"` se encuentra entre comillas escapadas (`\"`) para que la coma no divida el texto en columnas adicionales.
3. **Especificación de columnas**: al ejecutar la función, `readr` imprime un mensaje informativo (`Column specification`) indicando que `nombre_programa` fue inferido como carácter (`<chr>`) y `codigo_snies` como número real (`<dbl>`).

13.1.1. Parámetros comunes al importar datos

Los archivos de texto plano pueden contener metadatos, comentarios o estructuras no convencionales. La función `read_csv()` proporciona diversos parámetros para controlar el proceso de importación:

1. **`skip`**: permite omitir un número determinado de líneas al inicio del archivo antes de comenzar la lectura de los datos. Resulta útil cuando el archivo incluye metadatos o encabezados explicativos:

Listado 13.2 Uso del parámetro `skip` para ignorar líneas iniciales de metadatos

```
# Archivo con metadatos en las dos primeras líneas ----
pregrado_skip_csv <- "Facultad de Estudios a Distancia - FAEDIS
Programas de pregrado vigentes
codigo_snies,nombre_programa
6527,Administración de Empresas
108241,\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428,Contaduría Pública
11004,Ingeniería Civil
53703,Ingeniería Industrial
105142,Ingeniería Informática
10963,Relaciones Internacionales y Estudios Políticos"

# Omitir las dos primeras líneas ----
read_csv(
  file = I(pregrado_skip_csv),
  skip = 2
)
```

```
Rows: 7 Columns: 2
```

```
-- Column specification --------------------------------------
Delimiter: ","
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this messag

# A tibble: 7 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

2. **`comment`**: permite especificar un carácter (comúnmente `#`) para que `read_csv()` ignore por completo todas las líneas que inicien con ese prefijo:

Listado 13.3 Uso del parámetro `comment` para omitir líneas comentadas

```
# Archivo con comentarios intercalados ----
pregrado_comment_csv <- "# Listado de programas FAEDIS
codigo_snies,nombre_programa
6527,Administración de Empresas
# 6527 en proceso de renovación registro calificado
108241,\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428,Contaduría Pública
11004,Ingeniería Civil
53703,Ingeniería Industrial
105142,Ingeniería Informática
10963,Relaciones Internacionales y Estudios Políticos"

# Ignorar líneas que inician con # ----
read_csv(
  file = I(pregrado_comment_csv),
  comment = "#"
)
```

```
Rows: 7 Columns: 2
-- Column specification --------------------------------------
Delimiter: ","
```

```
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# A tibble: 7 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

3. **col_names**: controla los nombres de las columnas. Si el archivo no incluye una fila de encabezado, se especifica `col_names = FALSE` (lo que asigna nombres automáticos `X1`, `X2`, etc.) o se proporciona un vector de caracteres con los nombres deseados:

Listado 13.4 Uso del parámetro `col_names` para archivos sin encabezados

```
# Archivo sin fila de encabezado ----
pregrado_no_header_csv <- "6527,Administración de Empresas
108241,\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428,Contaduría Pública
11004,Ingeniería Civil
53703,Ingeniería Industrial
105142,Ingeniería Informática
10963,Relaciones Internacionales y Estudios Políticos"

# Suministrar nombres explícitos de columnas ----
read_csv(
  file = I(pregrado_no_header_csv),
  col_names = c("codigo_snies", "nombre_programa")
)
```

```
Rows: 7 Columns: 2
-- Column specification --------------------------------------
Delimiter: ","
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this messag

# A tibble: 7 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

4. **na**: define qué valores de texto deben interpretarse como datos faltantes (`NA`), por defecto `na = c("", "NA")`. Si un archivo representa los valores ausentes con expresiones como `"Sin dato"` o `"N/D"`, pueden indicarse explícitamente mediante `na = c("", "NA", "Sin dato", "N/D")`.

Listado 13.5 Uso del parámetro **na** para archivos con datos faltantes y con expresiones personalizadas

```
# Archivo sin fila de encabezado ----
pregrado_no_header_csv <- "codigo_snies,nombre_programa
6527,
108241,\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428,Contaduría Pública
11004,Ingeniería Civil
NA,Ingeniería Industrial
105142,N/D
10963,Relaciones Internacionales y Estudios Políticos
102005,Sin dato"

# Suministrar nombres explícitos de columnas ----
read_csv(
  file = I(pregrado_no_header_csv),
  na = c("", "NA", "Sin dato", "N/D")
)
```

```
Rows: 8 Columns: 2
-- Column specification --------------------------------------
Delimiter: ","
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# A tibble: 8 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 <NA>
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5           NA Ingeniería Industrial
6       105142 <NA>
7        10963 Relaciones Internacionales y Estudios Políti~
8       102005 <NA>
```

13.2. Otros formatos de archivos de texto plano

Además de los archivos `.csv`, el paquete `readr` proporciona funciones dedicadas para importar otros formatos rectangulares de texto plano de uso común:

- **Valores separados por tabulaciones (`read_tsv()`)**: se utiliza cuando el delimitador entre columnas es una tabulación (`\t`):

Listado 13.6 Importación de datos separados por tabulaciones con `read_tsv()`

```
pregrado_faedis_tsv <- "codigo_snies\tnombre_programa
6527\tAdministración de Empresas
108241\t\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428\tContaduría Pública
11004\tIngeniería Civil
53703\tIngeniería Industrial
105142\tIngeniería Informática
10963\tRelaciones Internacionales y Estudios Políticos"

read_tsv(
  file = I(pregrado_faedis_tsv)
)
```

```
Rows: 7 Columns: 2
-- Column specification --------------------------------------
Delimiter: "\t"
chr (1): nombre_programa
dbl (1): codigo_snies
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this messag

# A tibble: 7 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

- **Archivos delimitados por punto y coma (`read_csv2()`)**: en países hispanohablantes y europeos donde la coma (`,`) se utiliza como separador decimal, los archivos `csv` suelen utilizar el punto y coma (`;`) como separador de columnas. La función `read_csv2()` gestiona automáticamente esta convención:

Listado 13.7 Importación de datos delimitados por punto y coma con `read_csv2()`

```
pregrado_faedis_csv2 <- "codigo_snies;nombre_programa
6527;Administración de Empresas
108241;\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428;Contaduría Pública
11004;Ingeniería Civil
53703;Ingeniería Industrial
105142;Ingeniería Informática
10963;Relaciones Internacionales y Estudios Políticos"

read_csv2(
  file = I(pregrado_faedis_csv2)
)
```

```
i Using "',' as decimal and "'.'" as grouping mark. Use `read_delim()` for mor

Rows: 7 Columns: 2
-- Column specification --------------------------------------
Delimiter: ";"
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this messag
```

```
# A tibble: 7 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

- **Delimitadores personalizados (`read_delim()`)**: permite leer archivos con cualquier delimitador arbitrario mediante el parámetro `delim` (p. ej., barra vertical (`|`), 2 puntos (`:`), o guiones (`-`)):

Listado 13.8 Importación con delimitadores personalizados utilizando `read_delim()`

```
pregrado_faedis_delim <- "codigo_snies|nombre_programa
6527|Administración de Empresas
108241|\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428|Contaduría Pública
11004|Ingeniería Civil
53703|Ingeniería Industrial
105142|Ingeniería Informática
10963|Relaciones Internacionales y Estudios Políticos"

read_delim(
  file = I(pregrado_faedis_delim),
  delim = "|"
)
```

```
Rows: 7 Columns: 2
-- Column specification --------------------------------------
Delimiter: "|"
chr (1): nombre_programa
dbl (1): codigo_snies

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# A tibble: 7 x 2
  codigo_snies nombre_programa
         <dbl> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
```

```
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

13.3. Control de tipos de columnas y resolución de problemas

Al importar un archivo de texto plano sin especificar explícitamente los tipos de columna con `col_types`, `readr` intenta inferirlos mediante una heurística, tal como se señala a continuación:

> De forma predeterminada, inspecciona 1000 valores, distribuidos uniformemente desde la primera hasta la última fila.

Esta es una heurística diseñada para ser siempre rápida sin importar qué tan grande sea el archivo y funciona bien en la mayoría de los casos de acuerdo a los autores de `readr`.

Para consultar la especificación que `readr` ha determinado se utiliza la función `spec()`, tal como se señala a continuación utilizando `pregrado_faedis_tbl` correspondiente al Listado 13.1:

Listado 13.9 Inspección de la especificación de tipos con `spec()`

```
spec(pregrado_faedis_tbl)
```

```
cols(
  codigo_snies = col_double(),
  nombre_programa = col_character()
)
```

13.3.1. Especificación explícita de tipos con `col_types`

Cuando los valores inspeccionados por dicha heurística no son representativos o cuando se desea asegurar un esquema de datos reproducible y robusto, es recomendable definir explícitamente los tipos de cada columna mediante el parámetro `col_types` y la función `cols()`:

- `col_integer()`: enteros (`<int>`).
- `col_double()`: números reales (`<dbl>`).
- `col_character()`: texto (`<chr>`).

- `col_logical()`: booleanos (`<lgl>`).
- `col_factor()`: variables categóricas o factores (`<fct>`).
- `col_date()`, `col_datetime()` o `col_time()`: fechas y horas.
- `col_skip()`: omite la columna durante la importación.

Listado 13.10 Especificación explícita de tipos de columna con `cols()`

```
read_csv(
  file = I(pregrado_faedis_csv),
  col_types = cols(
    codigo_snies = col_integer(),
    nombre_programa = col_character()
  )
)
```

```
# A tibble: 7 x 2
  codigo_snies nombre_programa
         <int> <chr>
1         6527 Administración de Empresas
2       108241 Administración de Riesgos, Seguridad y Salud~
3        11428 Contaduría Pública
4        11004 Ingeniería Civil
5        53703 Ingeniería Industrial
6       105142 Ingeniería Informática
7        10963 Relaciones Internacionales y Estudios Políti~
```

13.3.2. Diagnóstico de errores con `problems()`

Si el archivo contiene registros con formatos inesperados (por ejemplo, texto en una columna que fue declarada o inferida como numérica), `readr` no detiene la ejecución, sino que reemplaza el valor conflictivo con `NA` y emite un mensaje de advertencia (en inglés **warning**).

Por ejemplo, en la siguiente estructura de datos, la segunda fila del archivo contiene el texto `"Sin código"` en la columna `codigo_snies`, la cual se intenta forzar como un entero (`col_integer()`), tal como se señala a continuación:

```
Warning: One or more parsing issues, call `problems()` on your data
frame for details, e.g.:
  dat <- vroom(...)
  problems(dat)
```

Listado 13.11 Advertencia por incompatibilidad de tipos al importar datos con `read_csv()`

```
# Datos con un valor de texto en la columna numérica ----
pregrado_invalido_csv <- "codigo_snies,nombre_programa
Sin código,Administración de Empresas
108241,\"Administración de Riesgos, Seguridad y Salud en el Trabajo\"
11428,Contaduría Pública
11004,Ingeniería Civil
53703,Ingeniería Industrial
105142,Ingeniería Informática
10963,Relaciones Internacionales y Estudios Políticos"

# Importar forzando la columna a entero ----
pregrado_faedis_invalido <- read_csv(
  file = I(pregrado_invalido_csv),
  col_types = cols(
    codigo_snies = col_integer(),
    nombre_programa = col_character()
  )
)
```

Al ejecutar el Listado 13.11, `readr` emite una advertencia indicando que se encontraron problemas de análisis sintáctico (en inglés **parsing issues**). Esto ocurre porque en la segunda fila del archivo se incluye `"Sin código"`, el cual es una cadena de caracteres (`<chr>`) y no un número entero (`<int>`), por lo que no puede convertirse al tipo definido en `col_integer()`.

A partir de esta advertencia, se puede aplicar la función `problems()` sobre el objeto importado para diagnosticar e inspeccionar con exactitud en qué fila, columna y valor ocurrió la anomalía, tal como se señala a continuación:

Listado 13.12 Detección y diagnóstico de problemas de lectura con `problems()`

```
# Diagnosticar filas con problemas de conversión ----
problems(pregrado_faedis_invalido)
```

```
# A tibble: 1 x 5
    row   col expected   actual     file
  <int> <int> <chr>      <chr>      <chr>
1     2     1 an integer Sin código /tmp/Rtmplmouhm/vroom-c~
```

> **Advertencia 2**
>
> En el Listado 13.12, la columna `row` reportada por `problems()` indica el valor `2`. Es importante señalar que este número corresponde a la línea del archivo u objeto de origen y no a la fila de la tibble en R. La línea `1` contiene los encabezados y la línea `2` el registro problemático, que en la tibble pasa a ser la fila `1`.
> Esta distinción es esencial al depurar datos ya que si examinas la fila `2` de la tibble, encontrarás un registro válido. La columna `row` de `problems()` está diseñada específicamente para ubicar y corregir la anomalía directamente en la fuente original.

13.4. Exportación de archivos de texto plano

Para guardar tibbles en archivos de texto plano rectangulares, el paquete `readr` provee las funciones `write_csv()`, `write_tsv()`, `write_csv2()` y `write_delim()`.

Los parámetros más relevantes al exportar datos son:

- **`x`**: el objeto de R (usualmente una tibble o un objeto similar) que se desea guardar.
- **`file`**: la ruta y nombre del archivo de destino con su correspondiente extensión.
- **`na`**: la cadena de texto utilizada para escribir los valores faltantes (por defecto `"NA"`).
- **`col_names`**: valor lógico que indica si se escriben los encabezados de columna en la primera línea (por defecto `TRUE`).

Listado 13.13 Exportación de un objeto a un archivo `.csv` utilizando `write_csv()`

```
# Exportar datos a un archivo .csv ----
write_csv(
  x = pregrado_faedis_invalido,
  file = "data/pregrado_faedis_invalido.csv",
  na = "NA",
  colnames = TRUE
)
```

En comparación con el formato `.rds` (ver Capítulo 12), los archivos de texto plano (p. ej., `.csv`, `.tsv`) ofrecen la ventaja de ser universales e interoperables con cualquier herramienta de análisis, lenguaje de programación y gestores de bases de datos. No obstante, dado que no almacenan los metadatos propios de R, al volver a importarlos será necesario especificar nuevamente sus tipos de columna.

13.5. Caso de estudio: importación de datos del Saber-Pro para el año 2021 en Colombia

El propósito de esta sección es aplicar los conceptos desarrollados a lo largo del Capítulo 13 para importar y depurar los datos en el archivo `2021-saber_pro-genericas.txt.xz` (ver Nota 28).

Tal como se indicó en la Nota 20, el sufijo `.xz` en el nombre del archivo `2021-saber_pro-genericas.txt.xz` señala que se trata de un archivo comprimido mediante un algoritmo de alta eficiencia para reducir drásticamente su tamaño en el disco y facilitar su descarga. Al igual que con `read_csv()`, las funciones de importación de `readr` detectan automáticamente esta extensión y realizan la descompresión en memoria de manera transparente, sin necesidad de descomprimir previamente el archivo de forma manual.

Asimismo, es fundamental señalar que en el portal DataIcfes los microdatos originales suelen publicarse con la extensión de texto plano `.txt`, utilizando como delimitador de columnas el punto y coma (`;`). Por esta razón, la función apropiada del paquete `readr` para leer este archivo es `read_csv2()`, diseñada específicamente para satisfacer estas convenciones regionales.

13.5.1. Importación inicial e inspección de tipos

Para comenzar, procederás a importar el archivo `2021-saber_pro-genericas.txt.xz` utilizando la función `read_csv2()` sin suministrar tipos de columna explícitos. De esta manera, `readr` aplica su heurística predeterminada para inferir el tipo de cada variable. Posteriormente, se utiliza la función `spec()` para examinar la especificación resultante, tal como se señala a continuación:

Listado 13.14 Importación inicial con `read_csv2()` y consulta de especificación de tipos con `spec()`

```
# Importar datos ----
saber_pro_genericas_2021 <- read_csv2(
  file = "data/2021-saber_pro-genericas.txt.xz"
)
```

```
i Using "',' as decimal and "'.'" as grouping mark. Use `read_delim()` for more co

Rows: 253568 Columns: 95
-- Column specification --------------------------------------
Delimiter: ";"
chr (62): estu_consecutivo, estu_estudiante, estu_tipodo...
```

```
dbl (32): periodo, estu_cod_cole_mcpio_termino, estu_cod...
num  (1): estu_inse_individual

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Listado 13.15 Importación inicial con `read_csv2()` y consulta de especificación de tipos con `spec()`

```
# Consultar especificación de columnas ----
spec(saber_pro_genericas_2021)
```

```
cols(
  periodo = col_double(),
  estu_consecutivo = col_character(),
  estu_estudiante = col_character(),
  estu_tipodocumento = col_character(),
  estu_nacionalidad = col_character(),
  estu_actividadrefuerzoareas = col_character(),
  estu_actividadrefuerzogeneric = col_character(),
  estu_agregado = col_character(),
  estu_areareside = col_character(),
  estu_cod_cole_mcpio_termino = col_double(),
  estu_cod_depto_presentacion = col_double(),
  estu_cod_mcpio_presentacion = col_double(),
  estu_cod_reside_depto = col_double(),
  estu_cod_reside_mcpio = col_double(),
  estu_coddane_cole_termino = col_double(),
  estu_cole_termino = col_character(),
  estu_comocapacitoexamensb11 = col_character(),
  estu_cursodocentesies = col_character(),
  estu_cursoiesapoyoexterno = col_character(),
  estu_cursoiesexterna = col_character(),
  estu_depto_presentacion = col_character(),
  estu_depto_reside = col_character(),
  estu_estadocivil = col_character(),
  estu_etnia = col_character(),
  estu_exterior = col_character(),
  estu_fechanacimiento = col_character(),
  estu_genero = col_character(),
  estu_horassemanatrabaja = col_character(),
  estu_inse_individual = col_number(),
```

```
  estu_inst_codmunicipio = col_double(),
  estu_inst_departamento = col_character(),
  estu_inst_municipio = col_character(),
  estu_mcpio_presentacion = col_character(),
  estu_mcpio_reside = col_character(),
  estu_metodo_prgm = col_character(),
  estu_nivel_prgm_academico = col_character(),
  estu_nse_individual = col_double(),
  estu_nucleo_pregrado = col_character(),
  estu_otrocole_termino = col_character(),
  estu_pagomatriculabeca = col_character(),
  estu_pagomatriculacredito = col_character(),
  estu_pagomatriculapadres = col_character(),
  estu_pagomatriculapropio = col_character(),
  estu_pais_reside = col_character(),
  estu_prgm_academico = col_character(),
  estu_prgm_codmunicipio = col_double(),
  estu_prgm_departamento = col_character(),
  estu_prgm_municipio = col_character(),
  estu_privado_libertad = col_character(),
  estu_semestrecursa = col_character(),
  estu_simulacrotipoicfes = col_character(),
  estu_snies_prgmacademico = col_double(),
  estu_tieneetnia = col_character(),
  estu_tipodocumentosb11 = col_character(),
  estu_tituloobtenidobachiller = col_character(),
  estu_valormatriculauniversidad = col_character(),
  fami_cuantoscompartebaño = col_character(),
  fami_educacionmadre = col_character(),
  fami_educacionpadre = col_character(),
  fami_estratovivienda = col_character(),
  fami_ocupacionmadre = col_character(),
  fami_ocupacionpadre = col_character(),
  fami_tieneautomovil = col_character(),
  fami_tienecomputador = col_character(),
  fami_tieneconsolavideojuegos = col_character(),
  fami_tienehornomicroogas = col_character(),
  fami_tieneinternet = col_character(),
  fami_tienelavadora = col_character(),
  fami_tienemotocicleta = col_character(),
  fami_tieneserviciotv = col_character(),
  inst_caracter_academico = col_character(),
  inst_cod_institucion = col_double(),
  inst_nombre_institucion = col_character(),
```

```
  inst_origen = col_character(),
  mod_competen_ciudada_desem = col_double(),
  mod_competen_ciudada_pnal = col_double(),
  mod_competen_ciudada_pnbc = col_double(),
  mod_competen_ciudada_punt = col_double(),
  mod_comuni_escrita_desem = col_double(),
  mod_comuni_escrita_pnal = col_double(),
  mod_comuni_escrita_pnbc = col_double(),
  mod_comuni_escrita_punt = col_double(),
  mod_ingles_desem = col_character(),
  mod_ingles_pnal = col_double(),
  mod_ingles_pnbc = col_double(),
  mod_ingles_punt = col_double(),
  mod_lectura_critica_desem = col_double(),
  mod_lectura_critica_pnal = col_double(),
  mod_lectura_critica_pnbc = col_double(),
  mod_lectura_critica_punt = col_double(),
  mod_razona_cuantitat_punt = col_double(),
  mod_razona_cuantitativo_pnal = col_double(),
  mod_razona_cuantitativo_pnbc = col_double(),
  percentil_global = col_double(),
  punt_global = col_double()
)
```

Al inspeccionar el resultado de `spec(saber_pro_genericas_2021)`, puedes observar que entre las columnas identificadas, la variable `estu_fechanacimiento` fue interpretada como una columna de texto (`col_character()`):

```
  estu_fechanacimiento = col_character(),
```

13.5.2. Naturaleza de la variable y formato de fecha

De acuerdo con la documentación oficial de DataIcfes, la variable `estu_fechanacimiento` corresponde a la fecha de nacimiento de cada individuo que presenta el Saber-Pro. En la Nota 21 del Capítulo 10 se indicó que es posible descargar el diccionario de datos oficial publicado en el portal DataIcfes. En dicho documento y al examinar los primeros registros del archivo (p. ej., `"05/10/1992"`, `"02/05/1994"`, `"09/11/1996"`) puedes identificar que las fechas se encuentran registradas en el formato **día/mes/año** (`DD/MM/AAAA`).

Mantener una fecha como cadena de texto (`<chr>`) dificulta realizar operaciones cronológicas como calcular la edad de los individuos, filtrar por año de nacimiento o agrupar temporalmente los registros. Por ello, lo ideal es transformarla en un tipo fecha (`<date>`) directamente durante la lectura.

13.5.3. Intento inicial con `col_date()` y diagnóstico con `problems()`

Una primera alternativa consiste en definir explícitamente el tipo de la columna `estu_fechanacimiento` empleando `col_types = cols()` junto con la función `col_date()` con sus opciones predeterminadas, tal como se señala a continuación:

Listado 13.16 Intento de conversión a fecha con `col_date()` predeterminado

```
# Importar especificando col_date() ----
saber_pro_genericas_2021 <- read_csv2(
  file = "data/2021-saber_pro-genericas.txt.xz",
  col_types = cols(
    estu_fechanacimiento = col_date()
  )
)
```

```
i Using "','" as decimal and "'.'" as grouping mark. Use `read_delim()` for more co

Warning: One or more parsing issues, call `problems()` on your data
frame for details, e.g.:
  dat <- vroom(...)
  problems(dat)
```

Al ejecutar el Listado 13.16, `readr` emite una advertencia señalando problemas de lectura (`One or more parsing issues, ...`). Esto ocurre porque, de forma predeterminada, `col_date()` espera que las fechas cumplan con el estándar internacional ISO 8601 (ver Nota 29), el cual no coincide con la convención **día/mes/año** (`DD/MM/AAAA`) empleada en el archivo. Como consecuencia, `readr` no logra interpretar los valores y los sustituye por valores ausentes (`NA`).

> **i** Nota 29
>
> El estándar internacional ISO 8601 establece una norma técnica global para la representación de fechas y horas organizada jerárquicamente de mayor a menor unidad temporal:
>
> - Año con 4 dígitos
> - Mes con 2 dígitos
> - Día con 2 dígitos
>
> Es decir, en formato `AAAA-MM-DD` (en inglés `YYYY-MM-DD`), como por ejemplo `1992-10-05`, `1994-05-02` o `1996-11-09`.

> Por defecto, la función `col_date()` del paquete `readr` asume el estándar ISO 8601. Por esta razón, cuando los datos utilizan convenciones locales o regionales distintas, como el formato **día/mes/año** (`DD/MM/AAAA`), `readr` no puede interpretarlas automáticamente con la configuración predeterminada.

A partir de esta advertencia, se puede aplicar la función `problems()` para diagnosticar con exactitud las discrepancias encontradas durante la importación, tal como se señala a continuación:

Listado 13.17 Diagnóstico de problemas de conversión en fechas mediante `problems()`

```
# Diagnosticar problemas ----
problems(saber_pro_genericas_2021)
```

```
# A tibble: 253,568 x 5
     row   col expected         actual     file
   <int> <int> <chr>            <chr>      <chr>
 1     2    26 date in ISO8601 05/10/1992 ""
 2     3    26 date in ISO8601 02/05/1994 ""
 3     4    26 date in ISO8601 09/11/1996 ""
 4     5    26 date in ISO8601 09/04/1986 ""
 5     6    26 date in ISO8601 14/05/1993 ""
 6     7    26 date in ISO8601 21/06/1996 ""
 7     8    26 date in ISO8601 07/12/2000 ""
 8     9    26 date in ISO8601 27/02/1993 ""
 9    10    26 date in ISO8601 20/02/1986 ""
10    11    26 date in ISO8601 09/08/1996 ""
# i 253,558 more rows
```

Al examinar el Listado 13.17, la tibble de diagnósticos revela que en la columna 26 (`estu_fechanacimiento`), `readr` esperaba valores de tipo `"date in ISO8601"`, pero encontró cadenas de caracteres como `"05/10/1992"`, `"02/05/1994"` o `"09/11/1996"`, identificando anomalías para la totalidad de registros con fecha de nacimiento.

13.5.4. Solución con el parámetro `format` en `col_date()`

Para resolver este problema sin alterar el archivo original, la función `col_date()` incluye el parámetro `format`, el cual permite definir el patrón o máscara exacta que `readr` debe utilizar para interpretar las fechas.

En R, los formatos de fecha no se describen con texto libre (como `"día/mes/año"`), sino mediante especificadores de formato estandarizados. En estas especificaciones, el símbolo

de porcentaje (`%`) actúa como un prefijo o marcador especial que le indica a la función qué componente temporal (día, mes o año) debe extraer de esa posición:

- `%d`: representa el **día** del mes expresado en 2 dígitos (p. ej., `05`, `02` o `09`).
- `%m`: representa el **mes** expresado en 2 dígitos (p. ej., `10`, `05` u `11`).
- `%Y`: representa el **año** expresado con sus 4 dígitos (p. ej., `1992`, `1994` o `1996`).
- **Caracteres literales**: cualquier carácter que no esté precedido por `%`, como la barra inclinada (`/`), el guión (`-`) o el punto (`.`), se interpreta como un separador literal que debe coincidir exactamente con el texto del archivo.

De este modo, para leer registros como `"05/10/1992"`, `"02/05/1994"` o `"09/11/1996"` se construye la cadena de formato `"%d/%m/%Y"`, donde `%d` extrae el día, la primera `/` coincide con el separador, `%m` extrae el mes, la segunda `/` coincide con el segundo separador y `%Y` extrae el año de 4 dígitos, tal como se señala a continuación:

Listado 13.18 Importación exitosa especificando el formato de fecha con `col_date()` y el parámetro `format`

```
# Importar especificando formato de fecha ----
saber_pro_genericas_2021 <- read_csv2(
  file = "data/2021-saber_pro-genericas.txt.xz",
  col_types = cols(
    estu_fechanacimiento = col_date(format = "%d/%m/%Y")
  )
)
```

```
i Using "','" as decimal and "'.'" as grouping mark. Use `read_delim()` for more co
```

Para verificar que la importación se realizó de manera exitosa y sin incompatibilidades en los datos, puedes aplicar nuevamente la función `problems()` sobre el objeto importado, tal como se señala a continuación:

Listado 13.19 Verificación de importación exitosa mediante `problems()`

```
# Diagnosticar problemas ----
problems(saber_pro_genericas_2021)
```

```
# A tibble: 0 x 5
# i 5 variables: row <int>, col <int>, expected <chr>,
#   actual <chr>, file <chr>
```

Al consultar el Listado 13.19, se obtiene una tibble vacía con `0` filas y `5` columas (`A tibble: 0 × 5`), lo que confirma que todos los registros de fecha de nacimiento fueron procesados y convertidos correctamente al tipo `<date>`. Con esta configuración, R almacena la columna `estu_fechanacimiento` como un vector de fechas, garantizando una coherencia y facilitando análisis cronológicos posteriores sin que exista pérdida de información.

13.5.5. Especificaciones comunes de formato para fechas en `readr`

Además de los especificadores utilizados anteriormente (`%d`, `%m` y `%Y`), `readr` soporta un conjunto amplio de códigos de conversión estándar que se pueden consultar en la documentación de la función `col_date()` con `?readr::col_date` (ver Nota 10), para indicar la estructura de los componentes temporales de año, mes y día así como sus separadores:

- **Día**:
  - `%d`: día del mes expresado como un número decimal de 2 dígitos (`01` a `31`).
  - `%e`: día del mes en el que los valores del `1` al `9` pueden incluir un espacio inicial opcional en lugar de un cero.
- **Mes**:
  - `%m`: mes expresado como un número decimal de 2 dígitos (`01` a `12`).
  - `%b`: nombre abreviado del mes según la configuración local activa (p. ej., `"ene."`, `"feb."` en español o `"Jan"`, `"Feb"` en inglés)[1].
  - `%B`: nombre completo del mes (p. ej., `"enero"`, `"febrero"` en español o `"January"`, `"February"` en inglés).
- **Año**:
  - `%Y`: año completo con cuatro dígitos (p. ej., `2021`, `2022`).
  - `%y`: año con 2 dígitos (`00` a `99`, donde habitualmente `00` a `69` corresponden a **2000–2069** y `70` a `99` a **1970–1999**).
- **Caracteres literales**:
  - Los signos de puntuación como la barra inclinada (`/`), el guión (`-`) el punto (`.`) u otro tipo de separador adecuado que se utilice representan literalmente los caracteres separadores entre día, mes y año.

[1] En la configuración regional en español de `readr`, las abreviaturas de los meses incluyen el punto final (p. ej., `"ene."`o `"feb."`). Puedes consultar la lista completa de abreviaturas ejecutando en la consola `readr::locale("es")$date_names`.

14. Hojas de cálculo

Este capítulo se basa en (Wickham, 2023, Capítulo 20), así como en la documentación oficial de `readxl` donde tiene como propósito introducir la importación de datos de hojas de cálculo de Excel (`.xls` y `.xlsx`) utilizando el paquete `readxl` e información de proyecciones y retroproyecciones de población nacional por área, sexo y edad para el periodo 2018-2070 de Colombia con base en el Censo Nacional de Población y Vivienda (CNPV) 2018.

Exploraremos los siguientes aspectos:

1. ¿Cómo importar archivos u hojas de cálculo de Excel (`.xls` y `.xlsx`)?
2. ¿Cómo listar e importar hojas de trabajo individuales dentro de un archivo de Excel?
3. ¿Cómo importar solo una parte de una hoja de trabajo definiendo rangos de celdas específicos?

> **Nota 30**
>
> Para desarrollar el contenido de este capítulo es necesario que descargues el archivo señalado a continuación y lo incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `014_importar-datos-hojas-calculo-en-r` y dentro de ella genera el archivo `001_importar-datos-hojas-calculo.R`, siguiendo las pautas de la Sección 4.4.

14.1. Archivos con extensión `.xls` y `.xlsx`

Para importar archivos `.xls` o `.xlsx` a R existen 3 funciones en el paquete `readxl`:

1. `read_xls()`: se utiliza para importar archivos con extensión `.xls`. Hasta la versión de 2007, Microsoft Excel utilizaba como formato principal un formato binario propietario denominado **Excel Binary File Format**.
2. `read_xlsx()`: se utiliza para importar archivos con extensión `.xlsx`, formato que fue introducido a partir de Microsoft Excel 2007.
3. `read_excel()`: se utiliza para importar archivos tanto en formato `.xls` como `.xlsx`. Esta función infiere de forma automática el formato del archivo delegando internamente la lectura a `read_xls()` o `read_xlsx()` según corresponda.

> **Nota 31**
>
> En la práctica, `read_excel()` es la función que usualmente se utiliza porque identifica automáticamente el formato del archivo sin que tengas que preocuparte por su extensión. Las funciones específicas `read_xls()` y `read_xlsx()` existen para los casos en los que deseas asegurarte que tu código trabaje estrictamente con un formato determinado, alertándote con un error si por equivocación se suministra un tipo de archivo distinto.

El primer parámetro de estas funciones es `path`, el cual recibe una ruta hacia un archivo con extensión `.xls` o `.xlsx` almacenado en disco (p. ej., `"data/serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx"`).

14.2. Importación de datos en formato `.xlsx`

Para importar un archivo en formato `.xlsx` a R, se utiliza la función `read_excel()` del paquete `readxl`. Dado que `readxl` no forma parte del núcleo base de paquetes del `tidyverse`, es necesario cargar previamente el paquete mediante `library(readxl)` (ver Sección 4.3) en el archivo `001_importar-datos-hojas-calculo.R`. Para realizar la importación, la función `read_excel()` utiliza el parámetro `path`, en el cual se especifica como argumento la ruta y el nombre del archivo con extensión `.xlsx`.

Para importar inicialmente el conjunto de datos en formato `.xlsx` (ver Nota 30) y asignarlo a una variable denominada `poblacion_col_2018_2070` ejecutarás el siguiente código, tal como se señala a continuación:

```
# A tibble: 8 x 2
  PROYECCIONES DE POBLACIÓN Y ESTUDIOS DEMOGRÁFICOS ~1 ...2
  <chr>                                                <chr>
1 <NA>                                                 <NA>
2 DIRECCIÓN TÉCNICA DE CENSOS Y DEMOGRAFÍA (DCD)       <NA>
3 <NA>                                                 <NA>
4 <NA>                                                 <NA>
5 1.                                                   Intr~
```

Listado 14.1 Uso de la función `read_excel()` para importar un archivo de datos en formato `.xlsx`

```
# Cargar paquetes ----
library(readxl)

# Importar datos ----
path <- "data/serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx"

poblacion_col_2018_2070 <- read_excel(
  path = path
)

poblacion_col_2018_2070
```

```
6 <NA>                                                    Actu~
7 2                                                       Desa~
8 <NA>                                                    NACI~
# i abbreviated name:
#   1: `PROYECCIONES DE POBLACIÓN Y ESTUDIOS DEMOGRÁFICOS (PPED)`
```

En el Listado 14.1, primero se carga el paquete `readxl` mediante `library(readxl)`. Posteriormente, se define la ruta asociándole el nombre `path` y se utiliza la función `read_excel()` con el parámetro `path` asociado al argumento `path` para leer el archivo y asignar explícitamente el objeto resultante a la variable `poblacion_col_2018_2070`.

14.3. Estructura de archivos con extensión `.xlsx`

Un aspecto que diferencia a las hojas de cálculo de los archivos planos (ver Capítulo 13), es la noción de múltiples hojas de trabajo. El archivo `serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx` contiene múltiples hojas de cálculo y por ese motivo obtienes el resultado señalado en el Listado 14.1 dado que es necesario definir cuál es la hoja de trabajo que se desea importar.

Debido a que no se definió la hoja de trabajo que se deseaba importar, la función `read_excel()` importa por defecto la primera hoja de trabajo que se encuentre en el archivo. Para listar el nombre de las hojas de trabajo en un determinado archivo con extensión `.xlsx` el paquete `readxl` incluye la función `excel_sheets()`, la cual se puede utilizar tal como se señala a continuación:

```
[1] "Índice"                    "PPED"
```

Listado 14.2 Uso de la función `excel_sheets()` para listar los nombres de las hojas de trabajo en un archivo en formato `.xlsx`

```
# Listar hojas de trabajo ----
excel_sheets(
  path = path
)
```

```
[3] "PobNacionalxÁreaSexoEdad"
```

En el Listado 14.2 se utiliza la función `excel_sheets()` con el parámetro `path` asociado al argumento `path` para identificar los nombres de las 3 hojas de trabajo que componen el archivo `serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx`.

14.4. Importar datos de una hoja de trabajo

Una vez identificadas las hojas de trabajo correspondientes, es posible utilizar nuevamente la función `read_excel()` con el parámetro `sheet` asociado al argumento `"PobNacionalxÁreaSexoEdad"` para importar los datos de población por área, sexo y edad para el periodo 2018-2070 de Colombia, tal como se señala a continuación:

Listado 14.3 Uso del parámetro `sheet` en la función `read_excel()` para importar datos de una hoja de trabajo particular

```
# Importar datos hoja de trabajo ----
poblacion_col_2018_2070 <- read_excel(
  path = path,
  sheet = "PobNacionalxÁreaSexoEdad"
)

poblacion_col_2018_2070
```

```
# A tibble: 133 x 309
   PROYECCIONES DE POBLACIÓN~1 ...2  ...3  ...4  ...5  ...6
   <chr>                       <chr> <chr> <chr> <chr> <chr>
 1 <NA>                        <NA>  <NA>  <NA>  <NA>  <NA>
 2 NACIONAL. PROYECCIONES DE ~ <NA>  <NA>  <NA>  <NA>  <NA>
 3 <NA>                        <NA>  <NA>  <NA>  <NA>  <NA>
```

```
 4 PROYECCIONES DE POBLACIÓN ~ <NA>  <NA>  <NA>  <NA>  <NA>
 5 TERRITORIO                  AÑO   ÁREA~ TOTAL <NA>  <NA>
 6 <NA>                        <NA>  <NA>  Total Homb~ Muje~
 7 Total Nacional              2018  Cabe~ 3641~ 1745~ 1896~
 8 Total Nacional              2018  Cent~ 1184~ 6163~ 5680~
 9 Total Nacional              2018  Total 4825~ 2361~ 2464~
10 Total Nacional              2019  Cabe~ 3721~ 1784~ 1937~
# i 123 more rows
# i abbreviated name:
#   1: `PROYECCIONES DE POBLACIÓN Y ESTUDIOS DEMOGRÁFICOS (PPED)`
# i 303 more variables: ...7 <chr>, ...8 <chr>, ...9 <chr>,
#   ...10 <chr>, ...11 <chr>, ...12 <chr>, ...13 <chr>,
#   ...14 <chr>, ...15 <chr>, ...16 <chr>, ...17 <chr>,
#   ...18 <chr>, ...19 <chr>, ...20 <chr>, ...21 <chr>, ...
```

Usualmente, los archivos con extensión `.xlsx` se utilizan para compartir datos con el público en general, por lo que contienen encabezados con imágenes, celdas combinadas, anotaciones, así como otros elementos que pueden ser útiles para inspeccionar la información de manera visual, pero que no son adecuados para manipular datos.

En el Listado 14.3 se importan todos los datos disponibles en la hoja de trabajo seleccionada por lo que se obtiene el resultado correspondiente. Sin embargo, una revisión visual de la hoja de trabajo asociada con el nombre `"PobNacionalxÁreaSexoEdad"` permite identificar que los datos de interés se encuentran a partir de la fila 9. Para importar los datos de interés es posible utilizar el parámetro `skip` asociado al argumento `8` de manera similar a como se hizo en el caso de archivos de texto plano rectangulares (ver Sección 13.1.1), tal como se señala a continuación:

Listado 14.4 Uso del parámetro `skip` en la función `read_excel()` para omitir un número mínimo de filas antes de leer el contenido

```
# Omitir filas al importar datos ----
poblacion_col_2018_2070 <- read_excel(
  path = path,
  sheet = "PobNacionalxÁreaSexoEdad",
  skip = 8
)

poblacion_col_2018_2070
```

```
# A tibble: 127 x 309
   ...1   ...2 ...3   Total Hombres Mujeres `Hombres 0 años`
   <chr> <dbl> <chr>  <dbl>   <dbl>   <dbl>            <dbl>
```

```
 1 Tota~  2018 Cabe~ 3.64e7  1.75e7  1.90e7           267725
 2 Tota~  2018 Cent~ 1.18e7  6.16e6  5.68e6           111886
 3 Tota~  2018 Total 4.83e7  2.36e7  2.46e7           379611
 4 Tota~  2019 Cabe~ 3.72e7  1.78e7  1.94e7           267420
 5 Tota~  2019 Cent~ 1.20e7  6.27e6  5.78e6           112272
 6 Tota~  2019 Total 4.93e7  2.41e7  2.51e7           379692
 7 Tota~  2020 Cabe~ 3.81e7  1.83e7  1.98e7           272089
 8 Tota~  2020 Cent~ 1.23e7  6.38e6  5.88e6           113511
 9 Tota~  2020 Total 5.04e7  2.47e7  2.57e7           385600
10 Tota~  2021 Cabe~ 3.87e7  1.86e7  2.01e7           269870
# i 117 more rows
# i 302 more variables: `Hombres 1 año` <dbl>,
#   `Hombres 2 años` <dbl>, `Hombres 3 años` <dbl>,
#   `Hombres 4 años` <dbl>, `Hombres 5 años` <dbl>,
#   `Hombres 6 años` <dbl>, `Hombres 7 años` <dbl>,
#   `Hombres 8 años` <dbl>, `Hombres 9 años` <dbl>,
#   `Hombres 10 años` <dbl>, `Hombres 11 años` <dbl>, ...
```

En el Listado 14.4 es posible finalmente importar los datos de interés con su respectiva información relacionada única y exclusivamente con la población por área, sexo y edad para el periodo 2018-2070 de Colombia. Sin embargo, debido a que el archivo contiene celdas combinadas relacionadas con las primeras 3 columnas no es posible extraer los nombres de estas. Debido a lo anterior la función `read_excel()` genera nombres genéricos como `...1`, `...2` y `...3`. Por ello, es posible renombrar estas columnas utilizando la función `rename()` (ver Sección 10.1.1.2), tal como se señala a continuación:

```
# A tibble: 127 x 309
   territorio     ano area_geografica  Total Hombres Mujeres
   <chr>        <dbl> <chr>            <dbl>   <dbl>   <dbl>
 1 Total Nacio~  2018 Cabecera        3.64e7  1.75e7  1.90e7
 2 Total Nacio~  2018 Centros Poblad~ 1.18e7  6.16e6  5.68e6
 3 Total Nacio~  2018 Total           4.83e7  2.36e7  2.46e7
 4 Total Nacio~  2019 Cabecera        3.72e7  1.78e7  1.94e7
 5 Total Nacio~  2019 Centros Poblad~ 1.20e7  6.27e6  5.78e6
 6 Total Nacio~  2019 Total           4.93e7  2.41e7  2.51e7
 7 Total Nacio~  2020 Cabecera        3.81e7  1.83e7  1.98e7
 8 Total Nacio~  2020 Centros Poblad~ 1.23e7  6.38e6  5.88e6
 9 Total Nacio~  2020 Total           5.04e7  2.47e7  2.57e7
10 Total Nacio~  2021 Cabecera        3.87e7  1.86e7  2.01e7
# i 117 more rows
# i 303 more variables: `Hombres 0 años` <dbl>,
#   `Hombres 1 año` <dbl>, `Hombres 2 años` <dbl>,
#   `Hombres 3 años` <dbl>, `Hombres 4 años` <dbl>,
```

Listado 14.5 Uso de la función `rename()` para cambiar el nombre de un conjunto de columnas

```
# Cargar paquetes ----
library(tidyverse)

# Renombrar columnas ----
poblacion_col_2018_2070 <- read_excel(
  path = path,
  sheet = "PobNacionalxÁreaSexoEdad",
  skip = 8
) |>
  rename(
    territorio = `...1`,
    ano = `...2`,
    area_geografica = `...3`
  )

poblacion_col_2018_2070
```

```
#   `Hombres 5 años` <dbl>, `Hombres 6 años` <dbl>,
#   `Hombres 7 años` <dbl>, `Hombres 8 años` <dbl>,
#   `Hombres 9 años` <dbl>, `Hombres 10 años` <dbl>, ...
```

> ⚠ Advertencia 3
>
> En el Listado 14.5 los datos relacionados con la población por área, sexo y edad para el periodo 2018-2070 de Colombia se encuentran organizados de forma rectangular. Sin embargo, en el Capítulo 16 explorarás el concepto de datos ordenados (**tidy data** en inglés) debido a que la estructura en la que se encuentran genera dificultades para manipularlos y visualizarlos.

14.5. Importar datos de parte de una hoja de trabajo

Tal como se mencionó en la Sección 14.4, los archivos con extensión `.xlsx` usualmente contienen elementos que no son parte de los datos que se desean importar a R o que no se requieren para realizar un análisis determinado. La función `read_excel()` incluye el parámetro `range`, que permite tener un mejor control sobre el rango de celdas que se desean importar de una determinada hoja de trabajo.

A modo de ejemplo, si se desea importar el conjunto de datos asociado con la hoja de trabajo `"PobNacionalxÁreaSexoEdad"` correspondiente al territorio, año, área geográfica y el total de población así como su división por sexo, es posible utilizar el parámetro `range` asociado al argumento `"A9:F128"` como alternativa al uso del parámetro `skip` (ver Listado 14.4) para tener un mayor control, tal como se señala a continuación:

Listado 14.6 Uso del parámetro `range` en la función `read_excel()` para controlar el rango de celdas que se desean importar

```
# Importar datos de un rango de celdas ----
poblacion_col_2018_2070 <- read_excel(
  path = path,
  sheet = "PobNacionalxÁreaSexoEdad",
  range = "A9:F128"
) |>
  rename(
    territorio = `...1`,
    ano = `...2`,
    area_geografica = `...3`
  )

poblacion_col_2018_2070
```

```
# A tibble: 119 x 6
   territorio     ano area_geografica  Total Hombres Mujeres
   <chr>        <dbl> <chr>            <dbl>   <dbl>   <dbl>
 1 Total Nacio~  2018 Cabecera        3.64e7  1.75e7  1.90e7
 2 Total Nacio~  2018 Centros Poblad~ 1.18e7  6.16e6  5.68e6
 3 Total Nacio~  2018 Total           4.83e7  2.36e7  2.46e7
 4 Total Nacio~  2019 Cabecera        3.72e7  1.78e7  1.94e7
 5 Total Nacio~  2019 Centros Poblad~ 1.20e7  6.27e6  5.78e6
 6 Total Nacio~  2019 Total           4.93e7  2.41e7  2.51e7
 7 Total Nacio~  2020 Cabecera        3.81e7  1.83e7  1.98e7
 8 Total Nacio~  2020 Centros Poblad~ 1.23e7  6.38e6  5.88e6
 9 Total Nacio~  2020 Total           5.04e7  2.47e7  2.57e7
10 Total Nacio~  2021 Cabecera        3.87e7  1.86e7  2.01e7
# i 109 more rows
```

En el Listado 14.6, el argumento `"A9:F128"` asociado con el parámetro `range` establece el rango de celdas que se desean importar de la hoja de trabajo `"PobNacionalxÁreaSexoEdad"` especificando la celda del extremo superior izquierdo `A9` junto con la celda del extremo inferior derecho `F128` e integrando ambos extremos con el operador `:`.

15. GeoPackage (.gpkg)

Este capítulo se basa en (Lovelace et al., 2025), (Popovic, 2025) y (Wilke, 2019) así como en la documentación oficial de `ggplot2` y `sf` donde tiene como propósito introducir la importación y posterior visualización de datos geoestadísticos a través del formato GeoPackage (`.gpkg`) utilizando el paquete `sf` y `ggplot2` con información del Geoportal del Departamento Administrativo Nacional de Estadística (DANE) y los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de la Facultad de Estudios a Distancia (FAEDIS) de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. ¿Qué es el formato GeoPackage (`.gpkg`)?

2. ¿Cómo importar datos geoestadísticos a través del formato GeoPackage (`.gpkg`)?
3. ¿Cómo visualizar datos geoestadísticos integrándolos con otras variables a través de un mapa coroplético (en inglés **choropleth map**)?

> **Nota 32**
>
> Para desarrollar el contenido de este capítulo es necesario que instales el paquete `sf` teniendo en cuenta la Sección 4.2, tal como se señala a continuación:
>
> ```
> install.packages("sf")
> ```
>
> También es necesario que descargues el archivo señalado a continuación y lo incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **nivel-departamento_version-mgn-2025_geopackage.gpkg**
>
> Adicionalmente, es necesario que hayas exportado y guardado el archivo `2021-2024_saber-pro-genericas_faedis_ajustado.rds` en la carpeta `data`, tal como se señaló en el Listado 11.14. En caso que no lo hayas hecho es necesario que lo descargues y lo incluyas dentro de la carpeta `data`:
>
> **2021-2024_saber-pro-genericas_faedis_ajustado.rds**

> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `015_importar-datos-gpkg-en-r` y dentro de ella genera el archivo `001_importar-datos-formato-gpkg.R`, siguiendo las pautas de la Sección 4.4.

15.1. Mapa coroplético

En este capítulo realizarás un mapa coroplético señalando el número de inscritos a la prueba Saber Pro por departamento para los programas de pregrado de la Facultad de Estudios a Distancia de la Universidad Militar en el periodo 2021-2024, tal como se indica en la Figura 15.1.

15.2. El formato `.gpkg`

Cuando trabajes con datos espaciales es necesario utilizar un formato para almacenarlos y compartirlos. Durante muchos años, el estándar más utilizado en los Sistemas de Información Geográfica (SIG) fue el formato **Shapefile** (`.shp`). Sin embargo, el `.shp` presentaba limitaciones para el análisis de datos moderno (ver Jachym (2017)).

Para superar estas restricciones fue creado el formato GeoPackage (`.gpkg`). A diferencia de otros formatos tradicionales, GeoPackage almacena toda la información espacial y sus tablas asociadas dentro de un único archivo. Por "debajo", un archivo `.gpkg` es en realidad una base de datos SQLite ligera y portable.

Una de las mayores fortalezas del formato `.gpkg` es su naturaleza abierta, no propietaria e independiente de plataforma. Esto significa que cualquier persona puede utilizarlo sin necesidad de adquirir licencias comerciales de software y puede abrirse con las mismas garantías en lenguajes de programación como R.

Esta apertura es promovida y regulada por el Open Geospatial Consortium (OGC). El OGC es un consorcio internacional sin fines de lucro fundado en 1994, integrado por cientos de organizaciones de gobiernos, empresas comerciales y entidades académicas alrededor del mundo. Su misión es definir consensos técnicos y estándares abiertos para que los datos de localización sean fáciles de encontrar, accesibles, interoperables y reutilizables. Dentro de este marco, el estándar oficial OGC GeoPackage fue diseñado para garantizar la interoperabilidad universal, permitiendo que la información geoespacial fluya sin fricciones entre diferentes herramientas y sistemas.

Para acceder a datos geoestadísticos de Colombia, existen 2 plataformas institucionales donde es posible consultar y descargar información geográfica oficial en formatos abiertos como GeoPackage (`.gpkg`):

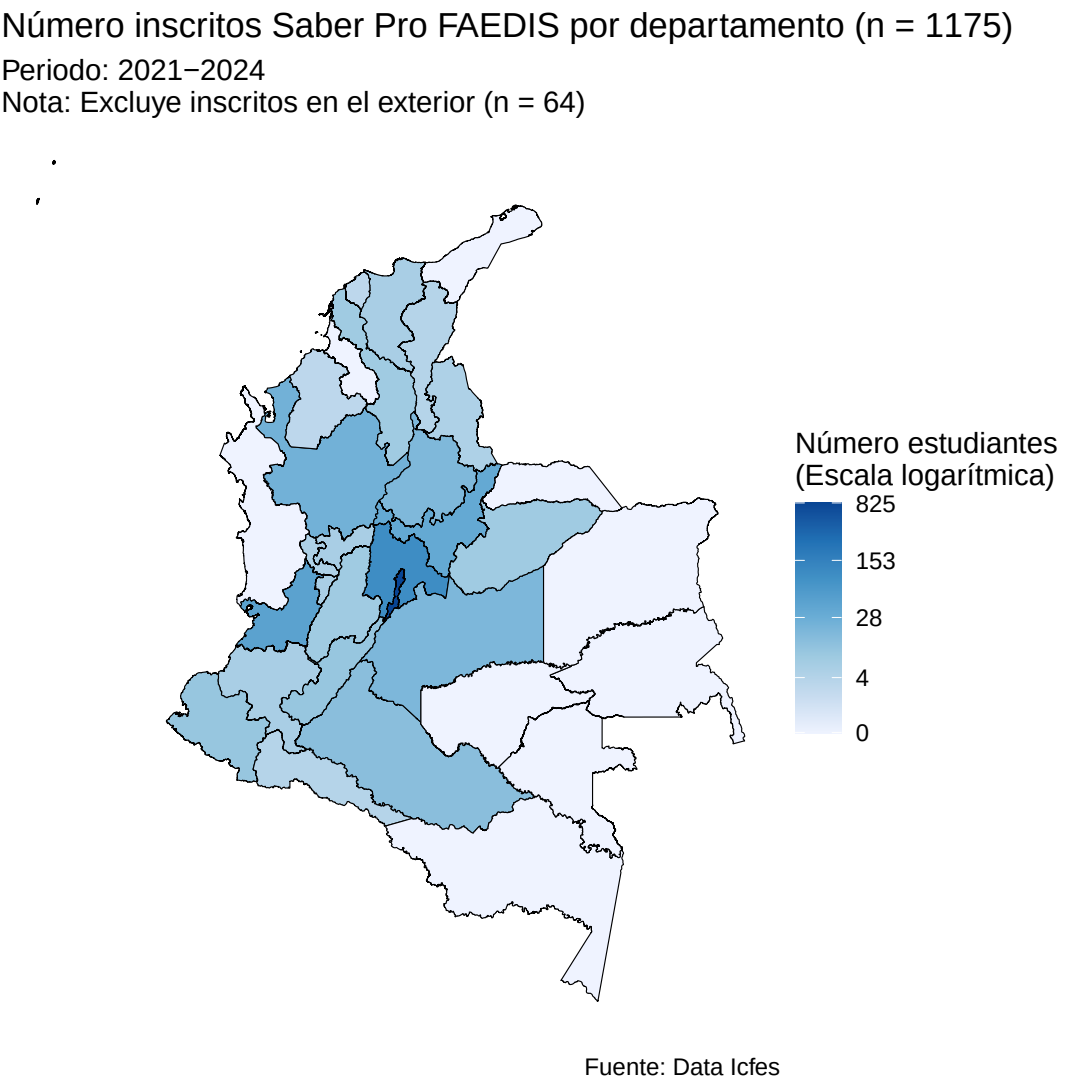


Figura 15.1.: Resultado final de un mapa coroplético elaborado con `sf` y `ggplot2` a partir de datos geoestadísticos en formato `.gpkg`

1. Geoportal del Departamento Administrativo Nacional de Estadística (DANE): a través del Marco Geoestadístico Nacional (MGN), el DANE ofrece un sistema diseñado para referenciar la información estadística con su localización geográfica en el país. Este marco estructura el territorio en áreas geoestadísticas como departamentos, municipios, cabeceras y centros poblados, abarcando los 1102 municipios, 18 áreas no municipalizadas y la isla de San Andrés. El archivo cartográfico departamental que utilizaremos a lo largo de este capítulo proviene de este sistema.
2. Portal de información geoespacial - Colombia en Mapas: es una plataforma administrada por el Instituto Geográfico Agustín Codazzi (IGAC) en conjunto con la Infraestructura Colombiana de Datos Espaciales (ICDE). Centraliza la información geográfica de Colombia y permite explorar y descargar capas temáticas de múltiples entidades públicas en formatos abiertos.

15.3. Importación de datos en formato `.gpkg`

Para importar un archivo en formato `.gpkg` a R, se utiliza la función `read_sf()` del paquete `sf`. Dado que `sf` no hace parte del `tidyverse`, es necesario cargar previamente el paquete mediante `library(sf)` (ver Sección 4.3) en el archivo `001_importar-datos-formato-gpkg.R`. Para realizar la importación, la función `read_sf()` utiliza el parámetro `dsn` (en inglés **Data Source Name**), en el cual se especifica como argumento la ruta y el nombre del archivo con extensión `.gpkg` que se desea leer.

Para importar el conjunto de datos de departamentos de Colombia en formato `.gpkg` (ver Nota 32) y asignarlo a una variable denominada `col_dpto_sf`, ejecutarás el siguiente código, tal como se señala a continuación:

Listado 15.1 Uso de la función `read_sf()` para importar un archivo de datos espaciales en formato `.gpkg`

```
# Cargar paquetes ----
library(sf)

# Importar datos ----
col_dpto_sf <- read_sf(
  dsn = "data/nivel-departamento_version-mgn-2025_geopackage.gpkg"
)

col_dpto_sf
```

```
Simple feature collection with 33 features and 8 fields
Geometry type: MULTIPOLYGON
```

```
Dimension:     XY
Bounding box:  xmin: -81.73562 ymin: -4.229406 xmax: -66.84722 ymax: 13.39473
Geodetic CRS:  WGS 84
# A tibble: 33 x 9
   dpto_ccdgo dpto_cnmbr   dpto_ano_c dpto_act_a  dpto_narea
   <chr>      <chr>             <int> <chr>            <dbl>
 1 05         ANTIOQUIA          1886 Constituci~     62789.
 2 08         ATLÁNTICO          1910 Ley 21 de ~      3324.
 3 11         BOGOTÁ, D.C.       1538 Constituci~      1621.
 4 13         BOLÍVAR            1886 Constituci~     26681.
 5 15         BOYACÁ             1886 Constituci~     23073.
 6 17         CALDAS             1905 11 de Abri~      7422.
 7 18         CAQUETÁ            1981 Ley 78 del~     92709.
 8 19         CAUCA              1857 15 de juni~     31261.
 9 20         CESAR              1967 Ley 25  21~     22498.
10 23         CÓRDOBA            1951 Ley 9 del ~     25083.
# i 23 more rows
# i 4 more variables: dpto_nano <int>, shape_Leng <dbl>,
#   shape_Area <dbl>, geom <MULTIPOLYGON [°]>
```

En el Listado 15.1, primero se carga el paquete `sf` mediante `library(sf)`. Posteriormente, se utiliza la función `read_sf()` con el parámetro `dsn` asociado al argumento `"data/nivel-departamento_version-mgn-2025_geopackage.gpkg"` para leer la capa cartográfica y asignar explícitamente el objeto resultante a la variable `col_dpto_sf`. Finalmente, al evaluar `col_dpto_sf` se imprime en la consola tanto el resumen de metadatos espaciales como las primeras observaciones. El objeto resultante `col_dpto_sf` se conoce como un objeto `sf` en R (ver Sección 15.3.1), el cual extiende el concepto de `tibble` (ver Sección 5.4) al incluir adicionalmente información geográfica.

15.3.1. Estructura de un objeto `sf`

El paquete `sf` recibe su nombre del estándar **Simple Features**, un estándar internacional abierto definido por el OGC. Este estándar describe una forma unificada de representar objetos geográficos del mundo real a través de geometrías y atributos. Al inspeccionar `col_dpto_sf` en la consola, notarás 2 componentes principales:

1. Un **encabezado de metadatos espaciales**

```
Simple feature collection with 33 features and 8 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -81.73562 ymin: -4.229406 xmax: -66.84722 ymax: 13.39473
Geodetic CRS:  WGS 84
# A tibble: 33 × 9
```

2. Una **tabla de datos**

El encabezado inicial resume las propiedades cartográficas fundamentales de la capa geográfica:

- `Simple feature collection with 33 features and 8 fields`: señala que el objeto contiene 33 observaciones espaciales correspondientes a los 32 departamentos de Colombia más Bogotá D.C y 8 variables adicionales.
- `Geometry type: MULTIPOLYGON`: indica que las geometrías son multipolígonos. Se utiliza este tipo porque varias entidades territoriales del país están conformadas por más de un fragmento de tierra discontinuo (p. ej., el archipiélago de San Andrés, Providencia y Santa Catalina).
- `Dimension: XY`: define que las coordenadas espaciales se representan en 2 dimensiones: longitud X y latitud Y.
- `Bounding box`: es la "caja" delimitadora que abarca los puntos geográficos extremos con occidente `xmin`, sur `ymin`, oriente `xmax` y norte `ymax`, dentro de los cuales se ubican todas las geometrías de la capa.
- `Geodetic CRS: WGS 84`: indica el Sistema de Referencia de Coordenadas (en inglés **Coordinate Reference System (CRS)**).

> **Nota 34**
>
> En caso que no entiendas a profundidad algunos elementos descritos anteriormente respecto al encabezado inicial del objeto `sf` relacionado con `col_dpto_sf`, puedes consultar una explicación detallada en (Lovelace et al., 2025, Capítulo 2).
> Sin embargo, esta lectura no es un requisito para comprender los temas abordados en el Capítulo 15 debido a que no se pretende desarrollar una explicación exhaustiva de sistemas de información geográfica en el libro. Más bien, se busca proporcionar al lector las bases prácticas necesarias para importar, transformar y visualizar datos espaciales combinándolos con otras variables de interés.

15.3.1.1. Una `tibble` con una columna geométrica

Inmediatamente después del encabezado de metadatos espaciales, el resultado muestra `# A tibble: 33 × 9`. Esto evidencia una de las mayores ventajas del paquete `sf`:

> Un objeto `sf` es en esencia una `tibble` (o un objeto similar) de R, extendido con capacidades geográficas.

Las 9 columnas de la `tibble` corresponden a las 8 variables no espaciales más una columna especial denominada **geometry list-column** donde en el caso de `col_dpto_sf` tiene asociada el nombre de `geom`. A diferencia de las columnas tradicionales que almacenan números o texto, la columna `geom`:

a. Es una **columna de lista** (en inglés **list-column**): cada celda contiene una lista (ver Nota 34) con todas las secuencias de coordenadas necesarias para dibujar los límites geográficos del departamento correspondiente.

b. Tiene una clase especial `<MULTIPOLYGON [°]>`: donde `MULTIPOLYGON` especifica el tipo de figura geométrica y el símbolo `[°]` indica que las unidades de medida son **grados decimales** (latitud y longitud en grados decimales), en concordancia con el Sistema de Referencia de Coordenadas `WGS 84`.

> **Nota 34**
>
> En el Capítulo 20 se abarcará con más detalle el concepto de **lista**, un tipo de dato señalado de manera superficial en la Tabla 10.2.

Gracias a esta arquitectura, no es necesario aprender un conjunto totalmente diferente de herramientas para manipular información geográfica. Puedes aplicar sobre un objeto `sf` las mismas funciones del `tidyverse` relacionadas con manipulación de datos (ver Sección 15.3.2). Además haciendo uso del paquete `ggplot2` es posible visualizar datos geoestadísticos integrándolos con otras variables de interés (ver Sección 15.4)

Finalmente, con respecto a la tabla de datos, a continuación se describen cada una de las variables en `nivel-departamento_version-mgn-2025_geopackage.gpkg`:

1. **dpto_ccdgo**: código del departamento según codificación DIVIPOLA
2. **dpto_cnmbr**: nombre del departamento de acuerdo a DIVIPOLA
3. **dpto_ano_c**: año en el que fue creado el departamento
4. **dpto_act_a**: acto administrativo con el que fue creado el departamento
5. **dpto_narea**: área del departamento en kilómetros cuadrados
6. **dpto_nano**: año de la versión de la información del departamento
7. **shape_Leng**: perímetro del departamento calculado en las unidades del sistema de referencia de coordenadas que en este caso corresponden a grados decimales
8. **shape_Area**: superficie o área del departamento calculada en las unidades del sistema de referencia de coordenadas que en este caso corresponden a grados decimales cuadrados
9. **geom**: columna de geometría espacial que almacena las coordenadas y formas geométricas mediante el uso de multipolígonos que delimitan el territorio de cada departamento

Donde para la realización de la Figura 15.1 utilizarás sólo las variables `dpto_ccdgo`, `dpto_cnmbr` y `geom`.

15.3.2. Importación de variables no espaciales y manipulación de datos

Para incorporar los resultados del examen Saber Pro y llevar a cabo la manipulación de datos, primero es necesario cargar el paquete `tidyverse` e importar el conjunto de datos `2021-2024_saber-pro-genericas_faedis_ajustado.rds` (ver Sección 12.1) mediante la función `read_rds()` del paquete `readr`, tal como se indica a continuación:

Listado 15.2 Cargar el paquete `tidyverse` e importar el conjunto de datos de variables no espaciales

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
resultado_saber_pro_ajustado <- read_rds(
  file = "data/2021-2024_saber-pro-genericas_faedis_ajustado.rds"
)
```

En el Listado 15.2, se carga el paquete `tidyverse` y se utiliza la función `read_rds()` asignando la ruta del archivo `.rds` al parámetro `file` para asociar el conjunto de datos con el nombre `resultado_saber_pro_ajustado`.

15.3.2.1. Selección y creación de variables de interés

Una vez importados los datos espaciales y no espaciales, el siguiente paso consiste en preparar las variables requeridas para construir el mapa coroplético de la Figura 15.1, tal como se señala a continuación:

Respecto al Listado 15.3 en `col_dpto_sf`, se conservan únicamente el código del departamento (`dpto_ccdgo`), el nombre del departamento (`dpto_cnmbr`) y la geometría (`geom`) utilizando la función `select()` de `dplyr` (ver Sección 10.1.1.2). Luego en `resultado_saber_pro_ajustado`, se calcula el número de estudiantes que presentaron la prueba en cada departamento utilizando la función `count()` de `dplyr`.

La función `count()` de `dplyr` permite contabilizar de forma rápida el número de observaciones asociadas a los valores de una o más variables. Al suministrarle como argumento `estu_cod_depto_presentacion`, `count()` agrupa las observaciones por departamento y crea automáticamente una nueva columna denominada `n`, la cual contiene la cantidad total de estudiantes que presentaron el Saber Pro en cada departamento. Cabe resaltar que la función `count()` no descarta los valores ausentes (`NA`), sino que también los contabiliza y los incluye por defecto en la última fila de la tabla resultante, lo cual corresponde a los estudiantes que presentaron el examen en el exterior.

Listado 15.3 Selección de variables en el objeto espacial y conteo de observaciones con `count()`

```
# Seleccionar ----
col_dpto_sf <- col_dpto_sf |>
  select(
    dpto_ccdgo,
    dpto_cnmbr,
    geom
  )

# Contar ----
resultado_saber_pro_ajustado <- resultado_saber_pro_ajustado |>
  count(
    estu_cod_depto_presentacion
  )
```

15.3.2.2. Creación de variables auxiliares

Para generar parte del título y subtítulo del mapa coroplético en la Figura 15.1 de forma dinámica y reproducible, es conveniente calcular 2 valores:

1. El número de estudiantes que presentaron el examen en el exterior donde el código del departamento no se encuentra disponible y se registra como `NA`.

2. El total de estudiantes inscritos

Para ello, se crean 2 variables auxiliares denominadas `n_na` y `n_total`, tal como se señala a continuación:

Listado 15.4 Creación de variables auxiliares para generar parte del título y subtítulo del mapa coroplético

```
# Variables auxiliares ----
n_na <- resultado_saber_pro_ajustado |>
  pull(n) |>
  last()

n_total <- resultado_saber_pro_ajustado |>
  pull(n) |>
  sum()
```

En el Listado 15.4, se utiliza la función `pull()`, la cual se introdujo al final de la Sección 4.4 con el propósito de extraer una columna de una tabla de datos y convertirla en un vector atómico de R. La lógica detrás del uso de `last()` y `sum()` junto con `pull()` comprende los siguientes aspectos:

- Al aplicar `resultado_saber_pro_ajustado |> pull(n)`, se extrae la columna con los conteos en forma de vector numérico.
- La función `last()` extrae el último elemento de dicho vector. Dado que `count()` ubica por defecto los valores faltantes (`NA`) en la última fila del conteo, `last()` recupera directamente el número de estudiantes inscritos en el exterior (`n_na`).
- Por su parte, la función `sum()` suma todos los elementos del vector numérico obtenido con `pull(n)`, permitiendo calcular el total de estudiantes inscritos (`n_total`).

15.3.2.3. Consolidación y limpieza de datos

El siguiente paso consiste en consolidar la información cartográfica con los conteos de estudiantes. Para ello, se utilizan **Mutating joins** (ver Sección 11.6) mediante la función `left_join()` (ver Sección 11.6.2), tal como se señala a continuación:

Listado 15.5 Consolidación mediante `left_join()` y tratamiento de valores ausentes con `replace_na()`

```
# Consolidar tablas ----
col_dpto_saber_pro_n <- col_dpto_sf |>
  left_join(
    y = resultado_saber_pro_ajustado,
    by = join_by(
      dpto_ccdgo == estu_cod_depto_presentacion
    )
  )

# Limpiar valores ausentes ----
col_dpto_saber_pro_n <- col_dpto_saber_pro_n |>
  replace_na(
    replace = list(
      n = 0
    )
  )
```

En el Listado 15.5 se llevan a cabo 2 tareas fundamentales:

1. **Unión de tablas con `left_join()`**: se conservan todas las entidades espaciales de la capa geográfica `col_dpto_sf` a la izquierda, incorporando a la derecha la columna `n` de `resultado_saber_pro_ajustado`. La correspondencia entre ambas tablas se define a través de la función `join_by()`, emparejando el código del departamento `dpto_ccdgo` con el código de presentación `estu_cod_depto_presentacion`.
2. **Tratamiento de valores ausentes con `replace_na()`**: en aquellos departamentos donde ningún estudiante presentó la prueba Saber Pro no existe una fila correspondiente en la tabla de conteos, por lo que `left_join()` les asigna inicialmente un valor ausente (`NA`) en la columna `n`. Para corregir este aspecto y reflejar que en esos departamentos hubo 0 estudiantes, se utiliza la función `replace_na()` del paquete `tidyr`.

El propósito de la función `replace_na()` es reemplazar los valores ausentes (`NA`) por un valor específico en una o más columnas. Para ello, se utiliza el parámetro `replace`, el cual recibe como argumento un objeto de tipo lista (ver Nota 34). Al especificar `replace = list(n = 0)`, se indica de forma explícita que cualquier valor ausente (`NA`) en la columna `n` debe sustituirse por `0`.

15.3.2.4. Inspección del objeto espacial consolidado

Una vez finalizado el proceso de selección, conteo, unión y limpieza, puedes darle un "vistazo" al objeto final `col_dpto_saber_pro_n` mediante la función `glimpse()` del paquete `dplyr` (ver Listado 4.5). De esta manera, podrás verificar la estructura de las variables y cerciorarte de que la tabla espacial esté lista para ser utilizada en la construcción del mapa coroplético de la Figura 15.1, tal como se señala a continuación:

Listado 15.6 Inspección del objeto espacial consolidado `col_dpto_saber_pro_n` con `glimpse()`

```
# Inspeccionar datos ----
glimpse(col_dpto_saber_pro_n)
```

```
Rows: 33
Columns: 4
$ dpto_ccdgo <chr> "05", "08", "11", "13", "15", "17", "18~
$ dpto_cnmbr <chr> "ANTIOQUIA", "ATLÁNTICO", "BOGOTÁ, D.C.~
$ geom       <MULTIPOLYGON [°]> MULTIPOLYGON (((-76.41355 ~
$ n          <int> 24, 3, 825, 8, 34, 6, 14, 6, 4, 3, 102,~
```

Como se observa en el Listado 15.6, `col_dpto_saber_pro_n` conserva las 33 observaciones correspondientes a las entidades territoriales departamentales de Colombia y contiene exactamente las 4 variables que se emplearán en la visualización:

1. **dpto_ccdgo**: código del departamento según codificación DIVIPOLA
2. **dpto_cnmbr**: nombre del departamento de acuerdo a DIVIPOLA
3. **geom**: columna de geometría espacial que almacena las coordenadas y formas geométricas mediante el uso de multipolígonos que delimitan el territorio de cada departamento
4. **n**: número total de estudiantes inscritos al examen Saber Pro, donde aquellos departamentos sin estudiantes inscritos ahora tienen el valor de 0.

15.4. Visualización de datos geoespaciales

Una vez consolidados los datos geoestadísticos con las variables de interés en el objeto `col_dpto_saber_pro_n`, el siguiente paso consiste en representar visualmente dicha información mediante un mapa coroplético (ver Figura 15.1). Para construir este gráfico utilizarás el paquete `ggplot2` siguiendo el modelo mental de la **gramática de gráficos en capas** (ver Sección 6.1).

En las siguientes secciones se dividirá la construcción del mapa en 3 fases secuenciales:

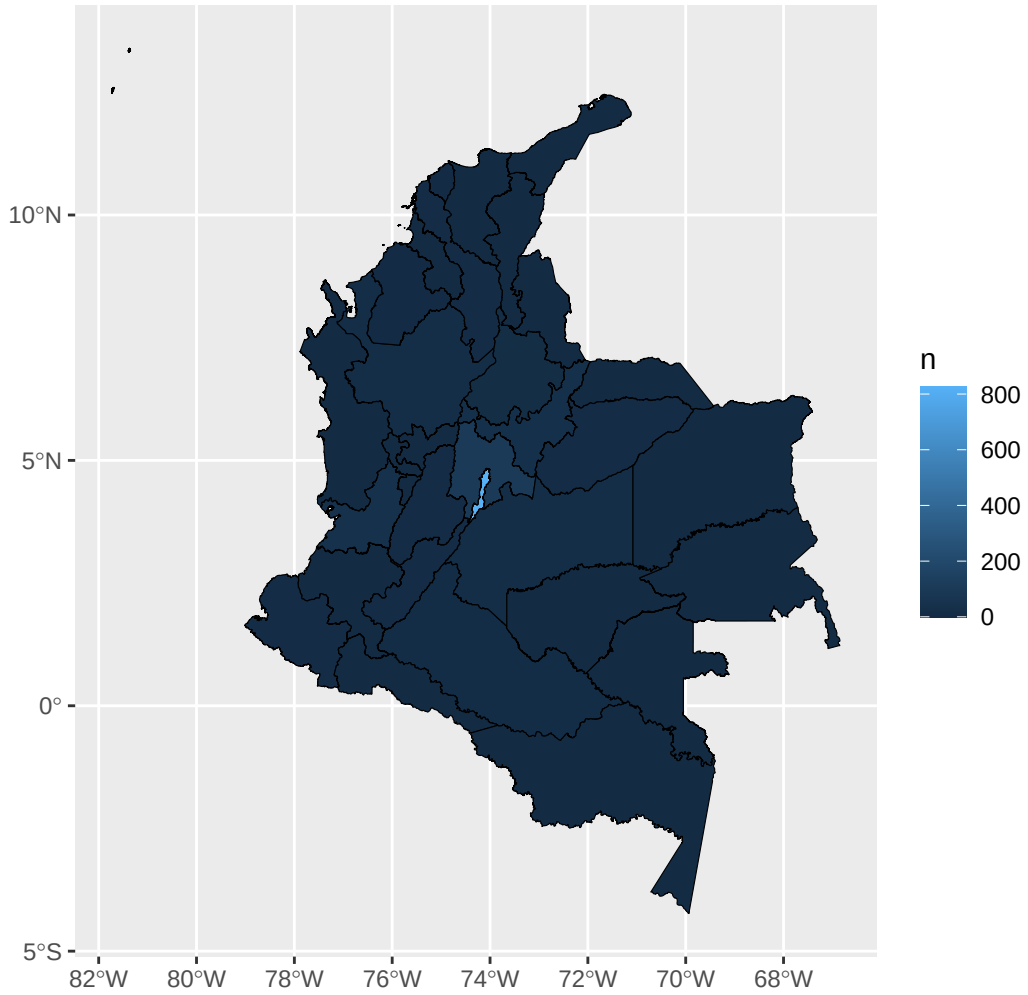
1. **Datos, mapeo y capas**: allí se establece el conjunto de datos espaciales, el mapeo de las variables a las geometrías y la representación gráfica de los departamentos en el plano mediante la geometría `geom_sf()`.
2. **Escalas**: en esta parte ajustarás la escala del relleno y se definirás etiquetas informativas para el título, subtítulo y la leyenda.
3. **Tema**: finalmente en esta sección simplificarás el aspecto visual removiendo ejes y cuadrículas innecesarias para obtener un mapa limpio y profesional.

15.4.1. Datos, mapeo y capas

Para iniciar la construcción del mapa coroplético, se define el objeto `col_dpto_saber_pro_n` como el insumo de datos y se añade la capa geométrica `geom_sf()`, tal como se señala a continuación:

Listado 15.7 Representación base del mapa coroplético utilizando `geom_sf()` con relleno según el número de estudiantes

```
# Visualización ----
col_dpto_saber_pro_n |>
  ggplot() +
  geom_sf(
    mapping = aes(
      geometry = geom,
      fill = n
    ),
    color = "black"
  )
```



En el Listado 15.7 intervienen varios aspectos clave de la gramática de gráficos aplicada a datos espaciales. En primer lugar, la función `geom_sf()` es la geometría especializada de `ggplot2` para visualizar objetos espaciales que cumplen con el estándar Simple Features (`sf`). A diferencia de geometrías convencionales como `geom_point()` o `geom_col()`, `geom_sf()` es capaz de interpretar automáticamente geometrías complejas compuestas por ejemplo por múltiples multipolígonos.

Asimismo, mientras que en gráficos tradicionales es necesario especificar de forma separada las posiciones horizontal y vertical mediante los atributos `x` e `y` (p. ej., `aes(x = ..., y = ...)`), en datos espaciales `geom_sf()` utiliza el atributo estético `geometry`. Al asociarle la columna `geom` mediante `geometry = geom`, se define el atributo estético único que encapsula toda la información geográfica del objeto, correspondiente a las secuencias de coordenadas que delimitan cada departamento.

Con respecto al componente de **Coordenadas** (ver Sección 6.1), en gráficos estándar `ggplot2` utiliza por defecto el sistema de coordenadas cartesianas. Sin embargo, los datos geoespaciales no se ubican sobre un plano cartesiano arbitrario, sino que se rigen por un Sistema de Referencia de Coordenadas (CRS) sobre la superficie curva de la Tierra. De forma automática, `geom_sf()` añade e integra la función de coordenadas `coord_sf()`, la cual asegura que las proporciones del mapa no se distorsionen y que los polígonos se proyecten respetando el CRS de los datos.

Finalmente, en relación con la apariencia visual de los departamentos, se utilizan de manera combinada los parámetros `fill` y `color`. Por un lado, `fill = n` se define dentro de `aes()` como un mapeo estético dependiente de los datos, asignando un color de relleno al interior de cada multipolígono en función del número de estudiantes inscritos. Por otro lado, `color = "black"` se define fuera de la función `aes()` como un parámetro que no depende de los datos, lo que permite trazar el perímetro de todos los departamentos con una línea negra constante para delimitar las fronteras territoriales.

15.4.2. Escalas

Al examinar el mapa que se obtiene en el Listado 15.7, surge un problema común en la visualización de datos en relación a la presencia de valores atípicos (en inglés **outliers**). En el conjunto de datos Bogotá D.C. cuenta con 825 estudiantes inscritos, mientras que la mayoría de los departamentos presentan valores considerablemente menores (p. ej., Guaviare con 0, Córdoba con 3, Cauca con 6, Antioquia con 24 o Boyacá con 34).

Con una escala lineal convencional, la amplia distancia entre 0 y 825 comprime todos los valores bajos e intermedios hacia el mismo extremo de la paleta. Como resultado, casi todos los departamentos se perciben con un tono idéntico y prácticamente indistinguible, impidiendo al lector apreciar por ejemplo las diferencias relativas entre departamentos con 0, 3, 6, 24 o 34 inscritos. Además, la paleta cromática por defecto requiere invertirse para que los valores cercanos a 0 se asocien a tonalidades claras y los valores cercanos al máximo se representen con tonos progresivamente más oscuros.

Para resolver esta limitación y lograr una discriminación visual adecuada en presencia de distribuciones altamente asimétricas, es necesario transformar la variable numérica mediante una **escala logarítmica en base** e ($\log_e$ o logaritmo natural).

15.4.2.1. La transformación logarítmica y el caso del cero

A diferencia de una escala lineal que mide incrementos aditivos constantes ($+10, +20, +30$), una escala logarítmica mide incrementos multiplicativos por órdenes de magnitud ($\times 10, \times 100, \times 1000$). Sin embargo, la función logarítmica presenta una restricción matemática fundamental ya que el logaritmo de 0 no está definido ($\log_e(0) \to -\infty$). Dado que en el conjunto de datos existen departamentos donde ningún estudiante presentó la prueba ($n = 0$), no es posible aplicar directamente la función logarítmica convencional $\log_e(x)$.

Para solucionar esta restricción se utiliza la transformación $\log_e(x + 1)$, conocida en R como `log1p`. Al sumar 1 a cada observación antes de calcular el logaritmo, se garantiza que:

$$\log_e(0 + 1) = \log_e(1) = 0$$

De este modo, el valor 0 permanece fijo en 0, mientras que los valores positivos se transforman de manera suave y continua. Para comprender intuitivamente el efecto de esta transformación, considera un vector de ejemplo con valores en distintos órdenes de magnitud: `c(0, 30, 100, 1000)`. En la Figura 15.2 se compara cómo se distribuyen estos mismos datos sobre una escala lineal frente a una escala logarítmica $\log_e(x + 1)$.

Como se evidencia en la Figura 15.2, en la escala lineal los valores 0 y 30 quedan completamente comprimidos e indistinguibles en el extremo izquierdo debido a la influencia del valor extremo 1000. En contraste, en la escala logarítmica $\log_e(x + 1)$ los valores se distribuyen de manera equilibrada y proporcional a sus órdenes de magnitud, haciendo visibles las diferencias entre magnitudes pequeñas sin perder la información de los valores altos.

Matemáticamente, la transformación directa que asigna cada valor original x a su posición en la escala logarítmica y se expresa como:

$$y = \log_e(x + 1)$$

Para recuperar el valor original a partir de su posición logarítmica, donde este aspecto es indispensable para ubicar e interpretar los cortes (en inglés **breaks**) de la leyenda en las unidades originales de los datos, se aplica la función exponencial inversa:

$$e^y = x + 1 \iff x = e^y - 1$$

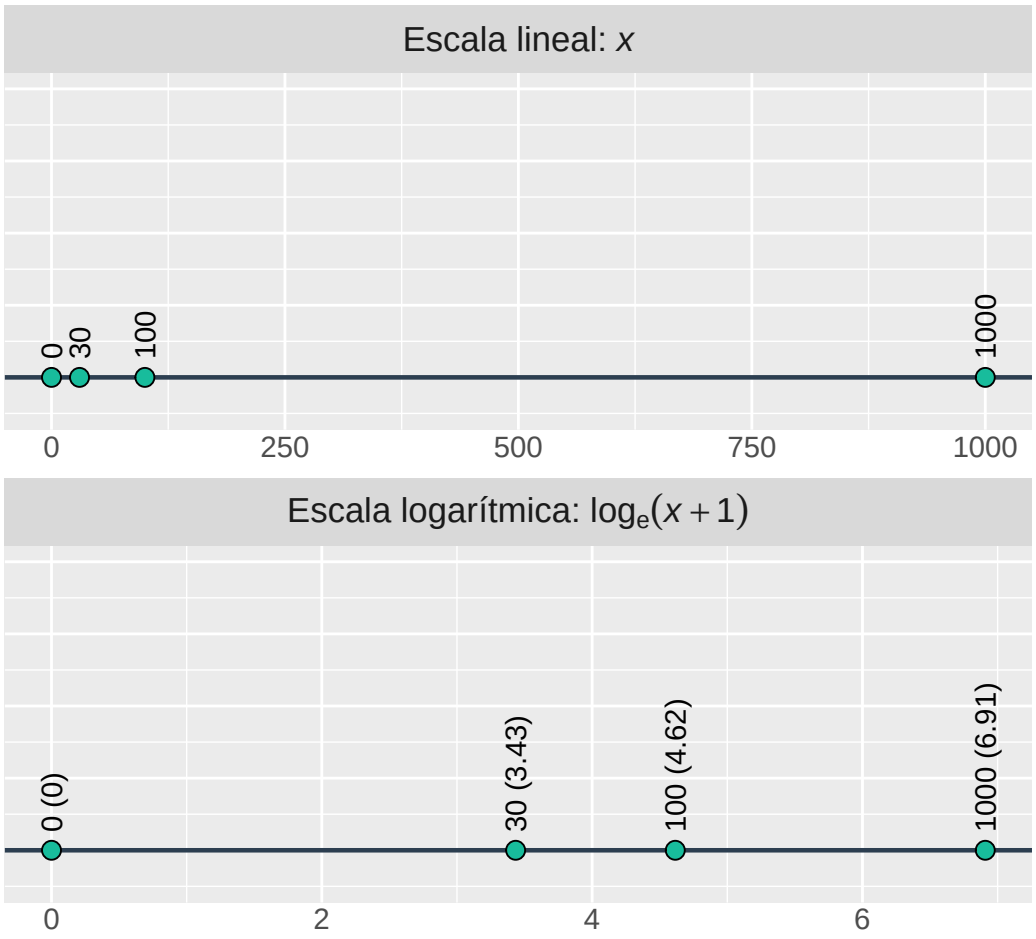


Figura 15.2.: Comparación gráfica entre la distribución de valores en una escala lineal (superior) y una escala logarítmica transformada con $\log_e(x+1)$ (inferior)

En R, esta relación inversa se implementa mediante la función `expm1(y)` ($e^y - 1$), la cual revierte con exactitud la transformación realizada por `log1p(x)`.

> **Nota 35**
>
> Para una introducción intuitiva y visual a los conceptos fundamentales de los logaritmos y su aplicación en el análisis de datos, puedes consultar el video explicativo sobre logaritmos del canal StatQuest with Josh Starmer.

15.4.2.2. Definición de variables auxiliares y puntos de corte

Para aplicar esta transformación a la escala de relleno y construir una leyenda legible con marcas distribuidas uniformemente a lo largo del espectro de color, primero se calculan los límites inferior y superior de la variable `n` a través de las funciones `min()` y `max()` combinadas con `pull()` y luego los cortes, tal como se señala a continuación:

Listado 15.8 Creación de variables auxiliares y cálculo de puntos de corte equidistantes en la escala logarítmica

```
# Variables auxiliares ----
## Mínimo y máximo ----
n_min <- col_dpto_saber_pro_n |>
  pull(n) |>
  min()

n_max <- col_dpto_saber_pro_n |>
  pull(n) |>
  max()

## Breaks ----
breaks <- seq.int(
  from = log1p(n_min),
  to = log1p(n_max),
  length.out = 5
) |>
  expm1() |>
  round()
```

En el Listado 15.8 se obtiene `n_min` (0) y `n_max` (825). Posteriormente, se construye el vector `breaks` con 5 valores. Para que los cortes queden repartidos de forma equidistante en el espacio perceptivo del color logarítmico, la función `seq.int()` genera una secuencia regular desde `log1p(n_min)` hasta `log1p(n_max)`. Luego, el resultado se devuelve a la

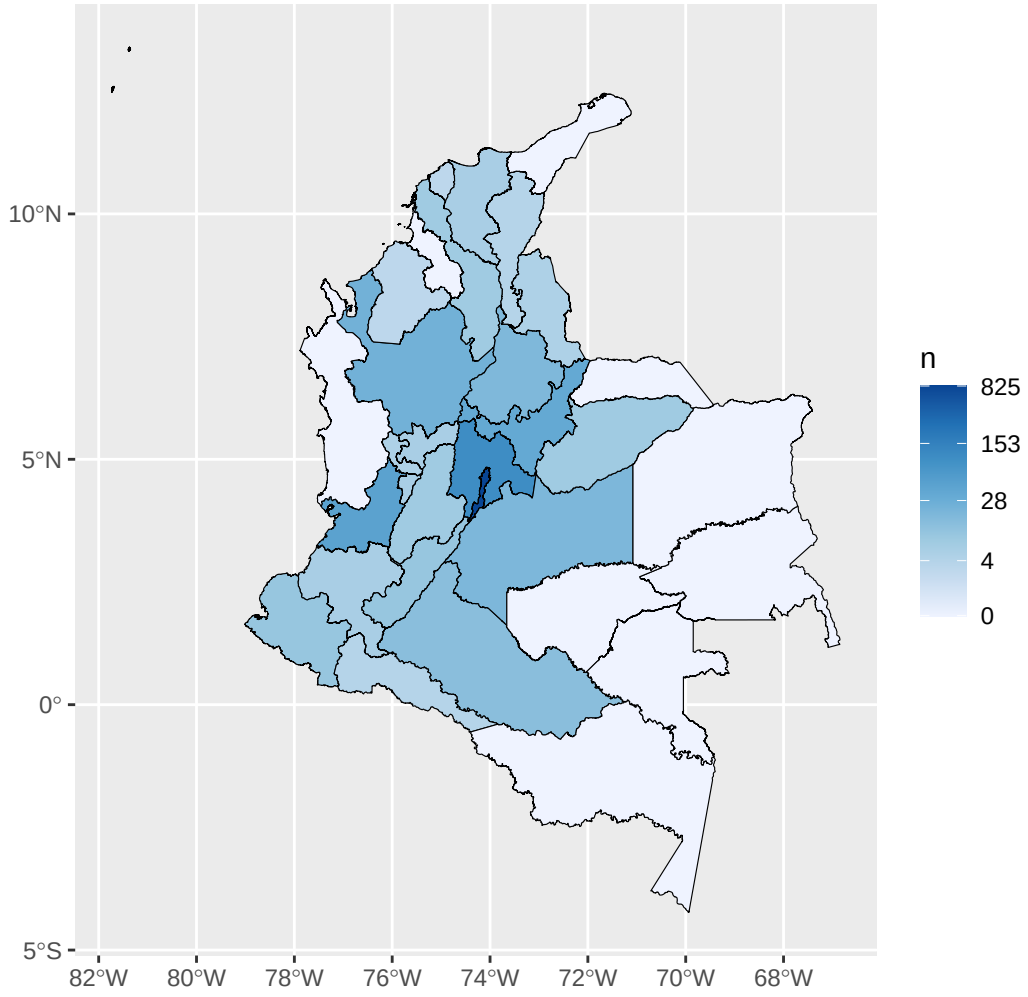
escala natural de conteos mediante la función inversa `expm1()` y se redondea a números enteros con `round()`, produciendo cortes interpretables por el usuario.

15.4.2.3. Aplicación de la escala de color

Con los cortes calculados, se incorpora la función `scale_fill_distiller()` al gráfico previamente definido, tal como se señala a continuación:

Listado 15.9 Incorporación de `scale_fill_distiller()` con transformación `log1p` y paleta secuencial

```
col_dpto_saber_pro_n |>
  ggplot() +
  geom_sf(
    mapping = aes(
      geometry = geom,
      fill = n
    ),
    color = "black"
  ) +
  scale_fill_distiller(
    palette = "Blues",
    direction = 1,
    transform = "log1p",
    breaks = breaks
  )
```



En el Listado 15.9 se configuran varios aspectos fundamentales de la escala de relleno del mapa. En primer lugar, la función `scale_fill_distiller()` se utiliza para interpolar paletas de colores provenientes del proyecto ColorBrewer sobre variables numéricas continuas. En el contexto del mapa coroplético, esta función permite generar un esquema de colores secuencial, el cual resulta idóneo para representar magnitudes numéricas ordenadas como el número de estudiantes inscritos. A través de una gradación cromática continua, el lector puede identificar de manera intuitiva el incremento o la concentración de individuos en cada territorio.

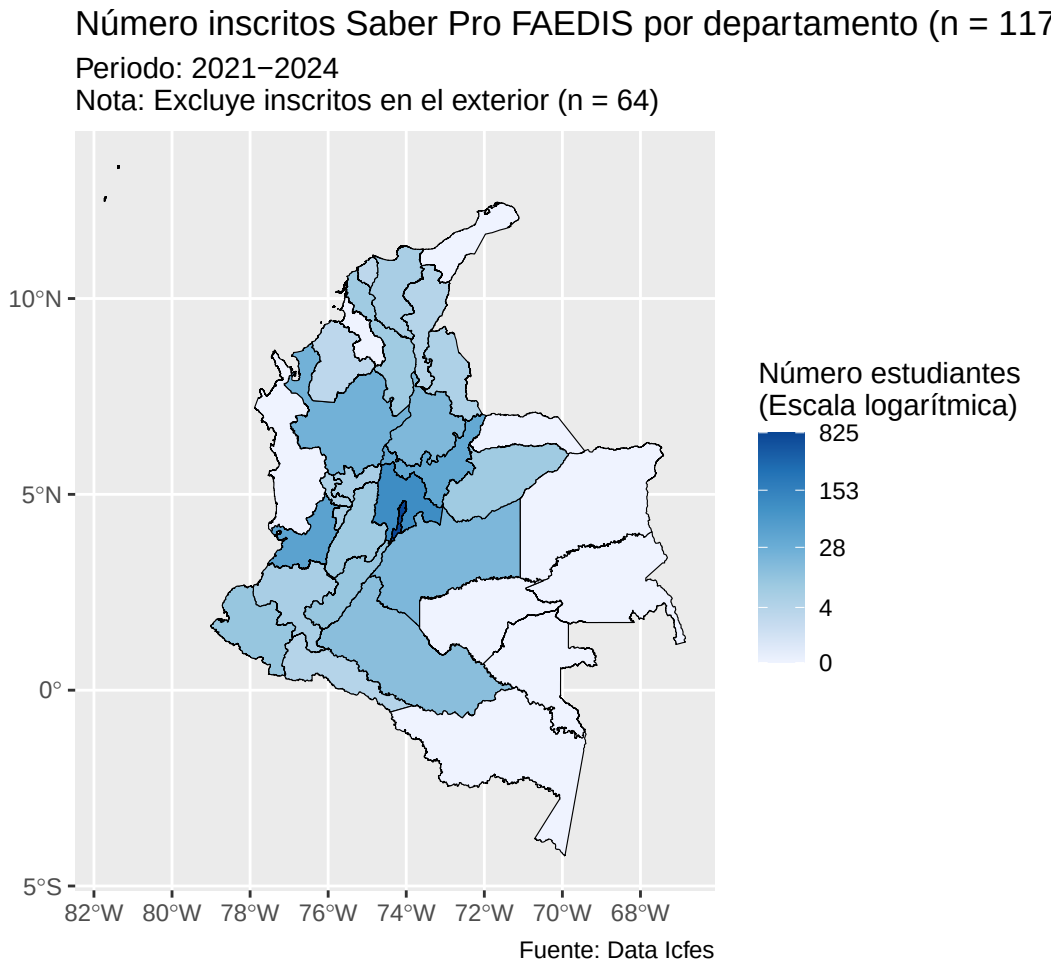
Para este fin, se utiliza el parámetro `palette = "Blues"`, el cual aplica una paleta compuesta por diversas tonalidades de azul. No obstante, por defecto las paletas de

ColorBrewer en `ggplot2` asignan los colores más oscuros a las magnitudes bajas y los colores claros a las altas, tal como se observaba de forma poco intuitiva en el mapa del Listado 15.7. Por esta razón, se especifica el parámetro `direction = 1`, el cual invierte el sentido de la paleta para que los departamentos con valores cercanos a cero se representen con tonalidades azules más claras y a medida que el número de estudiantes se incrementa, el color evolucione hacia azules progresivamente más oscuros.

Asimismo, para mitigar la compresión visual originada por el valor atípico de Bogotá D.C y distinguir los departamentos con pocos inscritos, se aplica la transformación logarítmica a través del parámetro `transform = "log1p"`. Este parámetro instruye internamente a `ggplot2` para calcular el valor $\log_e(x+1)$ antes de asignar el color a cada polígono. Complementariamente, mediante el parámetro `breaks = breaks` se suministra el vector de cortes previamente calculado, asegurando que en la barra continua de la leyenda se visualicen marcas numéricas exactas, legibles y expresadas en las unidades originales de los datos.

15.4.2.4. Aplicación de etiquetas informativas

Una vez ajustada la escala cromática, la capa `labs()` se encarga de contextualizar la visualización mediante títulos y descripciones informativas que facilitan la comprensión del mapa, tal como se señala a continuación:



En el Listado 15.10 se incorporan los elementos textuales que documentan el mapa utilizando la capa `labs()`. Para construir estas etiquetas se emplea la función `str_glue()` del paquete `stringr` perteneciente al `tidyverse`.

Construir cadenas de texto dinámicas suele requerir combinaciones engorrosas de texto y variables. La función `str_glue()` simplifica este proceso permitiendo intercalar texto literal con código de R evaluado dentro de llaves `{ }`. Por ejemplo, en la expresión `str_glue("... (n = {n_total - n_na})")`, R detecta las llaves, ejecuta internamente la operación matemática con las variables auxiliares `n_total` y `n_na`, e incrusta el resultado numérico directamente en el título sin necesidad de digitarlo manualmente.

Además, `str_glue()` admite saltos de línea de forma natural al presionar la tecla `Return`

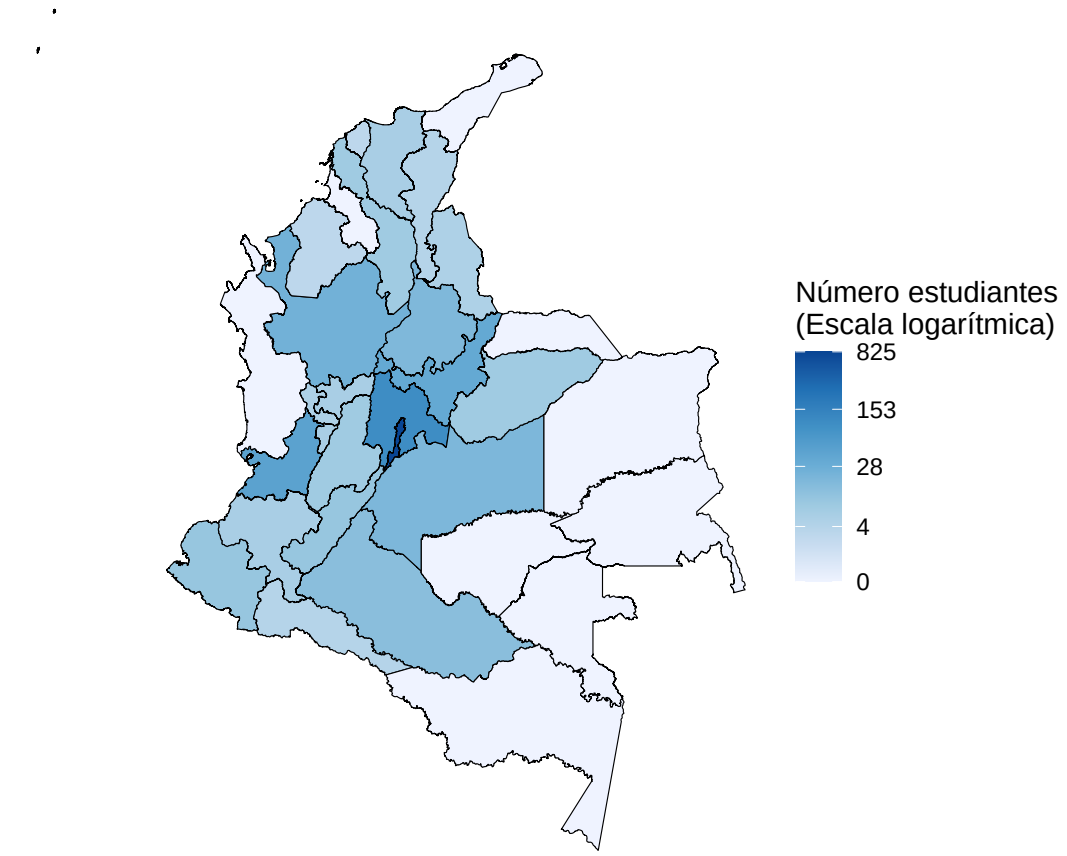
(mac), `Enter` (linux), `Enter` (windows) dentro de la cadena de texto en el editor, evitando tener que recurrir a símbolos especiales. De este modo, mediante los argumentos `fill`, `title`, `subtitle` y `caption`, el mapa queda documentado de forma clara, reproducible y profesional.

15.4.3. Tema

Como se introdujo en el Sección 6.1 y se desarrolló en la Sección 7.5 y la Sección 8.3, este componente se encarga de controlar los elementos visuales del gráfico que son ajenos a los datos. En dichas secciones se utilizó la función `theme()` para modificar aspectos puntuales del diseño, como la posición de la leyenda: `legend.position = "bottom"`.

Sin embargo, además de permitir la personalización de parámetros individuales mediante `theme()`, `ggplot2` cuenta con un conjunto de temas completos preconfigurados. Por defecto, `ggplot2` aplica `theme_grey()`, el cual añade un fondo gris con cuadrículas blancas y etiquetas con marcas en los ejes. Si bien estos elementos son esenciales en gráficos cartesianos convencionales, en un mapa coroplético las coordenadas geográficas y las líneas de cuadrícula, en este caso, no aportan información relevante y suelen generar "ruido" visual.

Para presentar el mapa de una forma limpia y libre de distracciones, es posible utilizar la función `theme_void()`. Esta función aplica un tema completamente vacío que remueve los fondos, las cuadrículas y las etiquetas y marcas de los ejes. De esa manera, se conservan únicamente las geometrías espaciales, la leyenda de color y los textos informativos definidos en `labs()`, tal como se señala a continuación:



Con el código señalado en el Listado 15.11 es posible recrear finalmente la Figura 15.1, obteniendo un mapa coroplético limpio donde la atención se enfoca exclusivamente en la distribución territorial y en la concentración del número de estudiantes inscritos a la prueba Saber Pro en cada departamento.

Listado 15.10 Incorporación de etiquetas descriptivas mediante `labs()` y cadenas dinámicas con `str_glue()`

```
col_dpto_saber_pro_n |>
  ggplot() +
  geom_sf(
    mapping = aes(
      geometry = geom,
      fill = n
    ),
    color = "black"
  ) +
  scale_fill_distiller(
    palette = "Blues",
    direction = 1,
    transform = "log1p",
    breaks = breaks
  ) +
  labs(
    fill = str_glue(
      "Número estudiantes
  (Escala logarítmica)"
    ),
    title = str_glue(
      "Número inscritos Saber Pro FAEDIS por departamento (n = {n_total - n_na})"
    ),
    subtitle = str_glue(
      "Periodo: 2021-2024
    Nota: Excluye inscritos en el exterior (n = {n_na})"
    ),
    caption = "Fuente: Data Icfes"
  )
```

Listado 15.11 Aplicación de la función `theme_void()` para remover elementos no espaciales y obtener una visualización limpia del mapa

```
col_dpto_saber_pro_n |>
  ggplot() +
  geom_sf(
    mapping = aes(
      geometry = geom,
      fill = n
    ),
    color = "black"
  ) +
  scale_fill_distiller(
    palette = "Blues",
    direction = 1,
    transform = "log1p",
    breaks = breaks
  ) +
  labs(
    fill = str_glue(
      "Número estudiantes
  (Escala logarítmica)"
    ),
    title = str_glue(
      "Número inscritos Saber Pro FAEDIS por departamento (n = {n_total - n_na})"
    ),
    subtitle = str_glue(
      "Periodo: 2021-2024
    Nota: Excluye inscritos en el exterior (n = {n_na})"
    ),
    caption = "Fuente: Data Icfes"
  ) +
  theme_void()
```

Parte V.

Organización y limpieza de datos

El objetivo de esta parte, es presentar los fundamentos y herramientas para estructurar conjuntos de datos en un formato ordenado (en inglés **tidy data**) en R utilizando el paquete `tidyr`, a través de los siguientes 3 capítulos:

- Capítulo 16: introduce el concepto de datos ordenados, los principios que vinculan la semántica con la estructura rectangular de las tablas y los problemas más comunes presentes en datos no ordenados junto con las estrategias para solucionarlos.
- Capítulo 17: introduce la reestructuración de tablas de datos alternando entre formato ancho y formato largo mediante el uso de las funciones `pivot_longer()` y `pivot_wider()`.
- Capítulo 18: introduce la separación de caracteres en columnas mediante la familia de funciones `separate_wider_*()` y la combinación de múltiples columnas a través de `unite()`, complementado con la preparación y visualización de pirámides poblacionales.

Una vez se comprenda el funcionamiento básico del paquete `tidyr`, el lector interesado en profundizar en la reestructuración y limpieza avanzada de datos puede consultar la documentación de referencia de `tidyr`, donde se abordan aspectos adicionales tales como:

- **Separación en filas**: la familia de funciones `separate_longer_*()` como `separate_longer_delim()` y `separate_longer_position()`, que permiten dividir cadenas de caracteres distribuyendo los valores en múltiples filas en lugar de columnas.
- **Aplanamiento de datos** (en inglés **rectangling**) y **anidamiento** (en inglés **nesting**): técnicas y funciones como `unnest_longer()`, `unnest_wider()`, `hoist()`, `nest()` y `unnest()` orientadas a transformar estructuras de datos complejas o profundamente anidadas en tablas rectangulares ordenadas (ver **Rectangling** y **Nesting**).
- **Valores ausentes y miscelánea**: funciones dedicadas al tratamiento explícito e implícito de datos faltantes como `complete()`, `fill()`, `drop_na()` y `replace_na()`, así como funciones auxiliares para cruces y combinaciones de variables como `expand()` y `crossing()`.

16. Datos ordenados

Este capítulo se basa en (Wickham, 2023, Capítulo 5) y (Wickham, 2014) así como en la documentación oficial de `tidyr` donde tiene como propósito introducir la forma de estructurar conjuntos de datos para facilitar su análisis utilizando el paquete `tidyr`.

Exploraremos los siguientes aspectos:

1. ¿Qué son datos ordenados (**tidy data** en inglés)?
2. ¿Cuáles son los problemas más comunes con datos no ordenados?

> **Nota 36**
>
> Para desarrollar el contenido de este capítulo es necesario que crees una nueva carpeta llamada `016_datos-ordenados-en-r` y dentro de ella generes el archivo `001_datos-ordenados.R`, siguiendo las pautas de la Sección 4.4.

16.1. Definición de datos ordenados

El concepto de datos ordenados, propuesto por Hadley Wickham (Wickham, 2014), proporciona un marco para vincular la **estructura** de un conjunto de datos (su disposición visual en filas y columnas) con su **semántica** (su significado real). Este estándar simplifica las etapas iniciales de limpieza, facilita la exploración y análisis de la información y permite que los paquetes pertenecientes al `tidyverse` o que comparten su filosofía interactúen sin fricciones.

16.1.1. Estructura de datos

Gran parte de los conjuntos de datos se organizan en tablas rectangulares compuestas por **filas** y **columnas**. En la práctica las columnas casi siempre se encuentran identificadas mediante encabezados mientras que las filas a veces cuentan con etiquetas que las distinguen (Wickham, 2014).

Para ilustrar este principio considera los datos de un experimento médico hipotético analizado en (Wickham, 2014), donde se evalúa el resultado de 2 tratamientos (**a** y **b**) en 3 individuos. En la Tabla 16.1 se presenta la estructura típica con la que frecuentemente se diseñan las tablas para reportes o de presentación visual.

Tabla 16.1.: Tabla de presentación con filas y columnas etiquetadas

individuo	**a**	**b**
John Smith	NA	2
Jane Doe	16	11
Mary Johnson	3	1

Esta misma información puede organizarse de múltiples formas. Por ejemplo, en la Tabla 16.2 se muestran los mismos datos del experimento, pero con las filas y las columnas transpuestas:

Tabla 16.2.: Los mismos datos del experimento pero organizados de forma transpuesta

tratamiento	John Smith	Jane Doe	Mary Johnson
a	NA	16	3
b	2	11	1

A pesar de que ambas tablas transmiten exactamente la misma información, su disposición física y espacial es totalmente distinta. Esto evidencia que el vocabulario habitual de "filas" y "columnas" no es suficiente para explicar por qué ambas tablas representan el mismo conjunto de datos ni cómo un computador debe interpretarlos (Wickham, 2014). Por esta razón, más allá de la apariencia, se requiere examinar la **semántica** de los datos.

16.1.2. Semántica de datos

Un conjunto de datos es una colección de **valores** (en inglés **values**), los cuales usualmente corresponden a números o a cadenas de texto (Wickham, 2014). En la semántica de datos, todo valor pertenece simultáneamente a 2 conceptos fundamentales una **variable** y una **observación**:

1. **Variable**: contiene todos los valores que miden un mismo atributo o característica como la estatura, la temperatura, la duración o el resultado de una prueba a través de distintas unidades de análisis.
2. **Observación**: contiene todos los valores medidos sobre una misma unidad de análisis como una persona, un día específico o una muestra médica a través de diferentes atributos.
3. **Valor**: corresponde a la medición individual de un atributo específico registrada en una celda.

Si reorganizamos los datos del experimento médico vinculando de forma unívoca su estructura con su semántica, obtenemos la disposición mostrada en la Tabla 16.3.

Tabla 16.3.: Los mismos datos del experimento con variables en columnas y observaciones en filas

individuo	tratamiento	resultado
John Smith	a	NA
Jane Doe	a	16
Mary Johnson	a	3
John Smith	b	2
Jane Doe	b	11
Mary Johnson	b	1

La Tabla 16.3 hace explícitos los 18 valores que componen el conjunto de datos correspondientes a 3 variables y 6 observaciones:

a. `individuo` : con 3 valores posibles (`John Smith`, `Jane Doe` y `Mary Johnson`).
b. `tratamiento` : con 2 valores posibles (`a` y `b`).
c. `resultado` : con los valores cuantitativos registrados y el valor ausente (`NA`, `16`, `3`, `2`, `11`, `1`).

El diseño experimental ofrece pautas esenciales sobre la estructura de las observaciones. En este caso particular, cada combinación entre persona y tratamiento fue medida. Asimismo, el diseño experimental determina cómo deben gestionarse los valores ausentes:

- **Valores ausentes estructurales**: representan mediciones que no pueden ocurrir por la propia naturaleza del fenómeno (por ejemplo, el número de partos en individuos masculinos). Estos registros pueden descartarse sin pérdida de rigor.
- **Valores ausentes explícitos**: representan observaciones que debieron haberse registrado pero por alguna circunstancia se omitieron o no se obtuvieron, como el tratamiento `a` en `John Smith` en la Tabla 16.3. En estos casos, es fundamental conservarlos como valores ausentes (`NA` en R).

16.1.3. Características de datos ordenados

Los datos ordenados constituyen un estándar para vincular la semántica de un conjunto de datos con su estructura. De acuerdo con (Wickham, 2014) y (Wickham, 2023), un conjunto de datos es ordenado si satisface las siguientes reglas fundamentales:

1. Cada variable forma una columna.
2. Cada observación forma una fila.
3. Cada valor es una celda.

4. Cada tipo de unidad observacional forma una tabla.

Cualquier disposición física de los datos que no cumpla con estas reglas se clasifica como **datos no ordenados** (en inglés **non-tidy data**).

16.2. Problemas más comunes con datos no ordenados

En la práctica es usual encontrar los conjuntos de datos en un formato no ordenado. Aunque los datos no ordenados pueden adoptar diversas formas, la gran mayoría de las dificultades estructurales se reducen a 5 problemas habituales identificados por en (Wickham, 2014):

1. Nombres de columnas que representan valores y no nombres de variables.
2. Múltiples variables combinadas en una sola columna.
3. Variables distribuidas simultáneamente en filas y en columnas.
4. Múltiples unidades observacionales mezcladas en una sola tabla.
5. Una misma unidad observacional fragmentada en múltiples tablas o archivos.

Para resolver estos problemas, el paquete `tidyr` proporciona un conjunto pequeño, consistente y potente de herramientas de reestructuración, encabezado por las funciones `pivot_longer()` y `pivot_wider()`. A continuación se examina cada uno de estos problemas a partir de los ejemplos destacados en la documentación de `tidyr` o señalados en (Wickham, 2014).

16.2.1. Nombres de columnas que son valores, no variables

Un caso muy común de datos no ordenados ocurre en tablas elaboradas para presentaciones visuales o resúmenes ejecutivos, donde los encabezados de las columnas no son nombres de variables sino valores específicos de una variable.

El paquete `tidyr` incluye el conjunto de datos `relig_income`, derivado de una encuesta sobre la distribución del ingreso en diversas confesiones religiosas en los Estados Unidos, tal como se señala a continuación:

```
# Cargar paquetes ----
library(tidyverse)

# Import datos ----
relig_income
```

```
# A tibble: 18 x 11
   religion  `<$10k` `$10-20k` `$20-30k` `$30-40k` `$40-50k`
   <chr>       <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1 Agnostic       27        34        60        81        76
 2 Atheist        12        27        37        52        35
 3 Buddhist       27        21        30        34        33
 4 Catholic      418       617       732       670       638
 5 Don't kn~      15        14        15        11        10
 6 Evangeli~     575       869      1064       982       881
 7 Hindu           1         9         7         9        11
 8 Historic~     228       244       236       238       197
 9 Jehovah'~      20        27        24        24        21
10 Jewish         19        19        25        25        30
11 Mainline~     289       495       619       655       651
12 Mormon         29        40        48        51        56
13 Muslim          6         7         9        10         9
14 Orthodox       13        17        23        32        32
15 Other Ch~       9         7        11        13        13
16 Other Fa~      20        33        40        46        49
17 Other Wo~       5         2         3         4         2
18 Unaffili~     217       299       374       365       341
# i 5 more variables: `$50-75k` <dbl>, `$75-100k` <dbl>,
#   `$100-150k` <dbl>, `>150k` <dbl>,
#   `Don't know/refused` <dbl>
```

En este conjunto de datos existen 3 variables:

- `religion`: confesión religiosa del encuestado.
- `ingreso` (`income`): rango o nivel socioeconómico de ingresos.
- `frecuencia` (`frequency`): cantidad de individuos ubicados en cada intersección.

Sin embargo, los encabezados de las columnas (`<$10k`, `$10-20k`, `$20-30k`, … , `Don't know/refused`) representan en realidad categorías o valores de la variable ingreso. Para ordenar esta tabla se utiliza la función `pivot_longer()`, la cual transforma múltiples columnas en un par de columnas `nombre-valor` (en inglés `name-value`) aumentando el número de filas y reduciendo el de columnas, tal como se señala en el Listado 16.1.

```
# A tibble: 180 x 3
   religion ingreso           frecuencia
   <chr>    <chr>                  <dbl>
 1 Agnostic <$10k                     27
 2 Agnostic $10-20k                   34
 3 Agnostic $20-30k                   60
 4 Agnostic $30-40k                   81
```

Listado 16.1 Reestructuración del conjunto de datos `relig_income` utilizando `pivot_longer()`

```
relig_income |>
  pivot_longer(
    cols = `<$10k`:`Don't know/refused`,
    names_to = "ingreso",
    values_to = "frecuencia"
  )
```

```
 5 Agnostic $40-50k                  76
 6 Agnostic $50-75k                 137
 7 Agnostic $75-100k                122
 8 Agnostic $100-150k               109
 9 Agnostic >150k                    84
10 Agnostic Don't know/refused       96
# i 170 more rows
```

Otro ejemplo, corresponde al conjunto de datos `billboard`, el cual registra la posición en el ranking musical **Billboard Top 100** durante el año 2000, tal como se señala a continuación:

```
billboard
```

```
# A tibble: 317 x 79
   artist   track date.entered   wk1   wk2   wk3   wk4   wk5
   <chr>    <chr> <date>       <dbl> <dbl> <dbl> <dbl> <dbl>
 1 2 Pac    Baby~ 2000-02-26      87    82    72    77    87
 2 2Ge+her  The ~ 2000-09-02      91    87    92    NA    NA
 3 3 Doors~ Kryp~ 2000-04-08      81    70    68    67    66
 4 3 Doors~ Loser 2000-10-21      76    76    72    69    67
 5 504 Boyz Wobb~ 2000-04-15      57    34    25    17    17
 6 98^0     Give~ 2000-08-19      51    39    34    26    26
 7 A*Teens  Danc~ 2000-07-08      97    97    96    95   100
 8 Aaliyah  I Do~ 2000-01-29      84    62    51    41    38
 9 Aaliyah  Try ~ 2000-03-18      59    53    38    28    21
10 Adams, ~ Open~ 2000-08-26      76    76    74    69    68
# i 307 more rows
# i 71 more variables: wk6 <dbl>, wk7 <dbl>, wk8 <dbl>,
#   wk9 <dbl>, wk10 <dbl>, wk11 <dbl>, wk12 <dbl>,
#   wk13 <dbl>, wk14 <dbl>, wk15 <dbl>, wk16 <dbl>,
#   wk17 <dbl>, wk18 <dbl>, wk19 <dbl>, wk20 <dbl>,
```

```
#   wk21 <dbl>, wk22 <dbl>, wk23 <dbl>, wk24 <dbl>,
#   wk25 <dbl>, wk26 <dbl>, wk27 <dbl>, wk28 <dbl>, ...
```

En este conjunto, el ranking semanal de cada canción se encuentra disperso en 76 columnas: `wk1` hasta `wk76`. Los encabezados (`wk1`, `wk2`, … , `wk76`), no son nombres de variables diferentes sino valores temporales de una única variable que corresponde a la semana en la lista. En ese sentido, es posible ordenar este conjunto de datos transformando las 76 columnas en nuevas variables (`semana` y `posicion`) y empleando el parámetro `values_drop_na = TRUE` para descartar las semanas en que la canción no figuraba en la lista, tal como se señala en el Listado 16.2:

Listado 16.2 Reestructuración del conjunto de datos `billboard` utilizando `pivot_longer()`

```
billboard |>
  pivot_longer(
    cols = wk1:wk76,
    names_to = "semana",
    values_to = "posicion",
    values_drop_na = TRUE
  )
```

```
# A tibble: 5,307 x 5
   artist  track                   date.entered semana posicion
   <chr>   <chr>                   <date>       <chr>     <dbl>
 1 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk1          87
 2 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk2          82
 3 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk3          72
 4 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk4          77
 5 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk5          87
 6 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk6          94
 7 2 Pac   Baby Don't Cry (Kee~ 2000-02-26   wk7          99
 8 2Ge+her The Hardest Part Of~ 2000-09-02   wk1          91
 9 2Ge+her The Hardest Part Of~ 2000-09-02   wk2          87
10 2Ge+her The Hardest Part Of~ 2000-09-02   wk3          92
# i 5,297 more rows
```

16.2.2. Múltiples variables en una sola columna

Otro problema surge cuando los encabezados de las columnas unen 2 o más variables distintas dentro de una sola etiqueta compuesta.

Un ejemplo clásico es el conjunto de datos de casos confirmados de tuberculosis (`tb`), recopilado por la Organización Mundial de la Salud (OMS). En este conjunto, los datos se encuentran discriminados por país, año y grupo demográfico, tal como se señala en el Listado 16.3:

Listado 16.3 Muestra representativa del conjunto de datos de tuberculosis (OMS)

```
tb_muestra <- tribble(
  ~iso2 , ~year , ~m_014 , ~m_1524 , ~f_014 , ~f_1524 ,
  "AD"  ,  2000 ,      0 ,       0 ,      1 ,       1 ,
  "AE"  ,  2000 ,      2 ,       4 ,      3 ,       4 ,
  "AF"  ,  2000 ,     52 ,     228 ,     93 ,     140
)

tb_muestra
```

```
# A tibble: 3 x 6
  iso2   year m_014 m_1524 f_014 f_1524
  <chr> <dbl> <dbl>  <dbl> <dbl>  <dbl>
1 AD     2000     0      0     1      1
2 AE     2000     2      4     3      4
3 AF     2000    52    228    93    140
```

> **Nota 37**
>
> En el Listado 16.3 se utiliza la función `tribble()` del paquete `tibble` para construir una tibble fila por fila. A diferencia de la función `tibble()` introducida en la Sección 5.4 (ver Listado 5.8), que organiza los datos columna por columna, `tribble()` resulta idónea para ingresar tablas pequeñas o ejemplos demostrativos en el código:
>
> 1. Los nombres de las columnas se especifican anteponiendo una virgulilla (`~`).
> 2. Los valores se ingresan secuencialmente a lo largo de las filas separados por comas.

Las columnas como `m_014` o `f_1524` agrupan 2 variables diferentes:

a. El sexo: `m` para masculino y `f` para femenino.
b. El rango de edad: `014` para 0 a 14 años o `1524` para 15 a 24 años.

Para ordenar estos datos:

1. Se aplica `pivot_longer()` sobre las columnas compuestas para agruparlas en una variable temporal denominada `demo`.

2. Se utiliza la función `separate_wider_delim()` de `tidyr` para dividir la cadena de texto de la columna `demo` en 2 variables independientes (`sexo` y `edad`) con base al delimitador `_`, tal como se señala en el Listado 16.4:

Listado 16.4 Reestructuración y separación de variables compuestas en `tb_muestra` con `pivot_longer()` y `separate_wider_delim()`

```
tb_muestra |>
  pivot_longer(
    cols = `m_014`:`f_1524`,
    names_to = "demo",
    values_to = "casos"
  ) |>
  separate_wider_delim(
    cols = demo,
    delim = "_",
    names = c("sexo", "edad")
  )
```

```
# A tibble: 12 x 5
   iso2   year sexo  edad  casos
   <chr> <dbl> <chr> <chr> <dbl>
 1 AD     2000 m     014       0
 2 AD     2000 m     1524      0
 3 AD     2000 f     014       1
 4 AD     2000 f     1524      1
 5 AE     2000 m     014       2
 6 AE     2000 m     1524      4
 7 AE     2000 f     014       3
 8 AE     2000 f     1524      4
 9 AF     2000 m     014      52
10 AF     2000 m     1524    228
11 AF     2000 f     014      93
12 AF     2000 f     1524    140
```

Ordenar este conjuntos de datos permite por ejemplo incorporar con facilidad variables adicionales como la población de cada país y calcular de manera directa tasas de incidencia por grupo etario y sexo.

16.2.3. Variables almacenadas tanto en filas como en columnas

La forma más compleja de datos no ordenados ocurre cuando ciertas variables se encuentran distribuidas a lo largo de las columnas, mientras que otras variables tienen sus nombres almacenados como valores dentro de filas.

Este patrón se evidencia en los registros meteorológicos diarios de la estación climatológica MX17004 en México provenientes de la Global Historical Climatology Network para el año 2010, tal como se señala en el Listado 16.5:

Listado 16.5 Muestra representativa del registro meteorológico diario

```
clima_muestra <- tribble(
  ~id       , ~ano , ~mes , ~element , ~`1` , ~`2` , ~`3` ,
  "MX17004" , 2010 ,    1 , "tmax"    , NA   , NA   , NA   ,
  "MX17004" , 2010 ,    1 , "tmin"    , NA   , NA   , NA   ,
  "MX17004" , 2010 ,    2 , "tmax"    , NA   , 27.3 , 24.1 ,
  "MX17004" , 2010 ,    2 , "tmin"    , NA   , 14.4 , 14.4 ,
  "MX17004" , 2010 ,    3 , "tmax"    , NA   , NA   , NA   ,
  "MX17004" , 2010 ,    3 , "tmin"    , NA   , NA   , NA
)

clima_muestra
```

```
# A tibble: 6 x 7
  id        ano   mes element `1`     `2`   `3`
  <chr>   <dbl> <dbl> <chr>   <lgl> <dbl> <dbl>
1 MX17004  2010     1 tmax    NA     NA    NA
2 MX17004  2010     1 tmin    NA     NA    NA
3 MX17004  2010     2 tmax    NA     27.3  24.1
4 MX17004  2010     2 tmin    NA     14.4  14.4
5 MX17004  2010     3 tmax    NA     NA    NA
6 MX17004  2010     3 tmin    NA     NA    NA
```

En esta tabla coexisten 2 problemas estructurales:

- Los días del mes se encuentran dispersos en columnas (`1`, `2`, `3`).
- La columna `element` no es una variable en sí misma ya que almacena los nombres de 2 variables meteorológicas medidas: la temperatura máxima (`tmax`) y la temperatura mínima (`tmin`).

Para ordenar este conjunto se combina el uso de `pivot_longer()` y `pivot_wider()`, tal como se señala en el Listado 16.6:

1. **Colapsar los días**: se aplica `pivot_longer()` sobre las columnas de días para recopilarlas en una sola variable `dia`.
2. **Pivote inverso**: se utiliza `pivot_wider()` para transformar los valores de `element` (`tmax` y `tmin`) en columnas independientes, asignando a cada una su respectiva medición tomada de la columna `valor`.

Listado 16.6 Reestructuración de `clima_muestra` combinando `pivot_longer()` y `pivot_wider()`

```
clima_muestra |>
  pivot_longer(
    cols = `1`:`3`,
    names_to = "dia",
    values_to = "valor",
    values_drop_na = TRUE
  ) |>
  mutate(dia = as.integer(dia)) |>
  pivot_wider(
    names_from = element,
    values_from = valor
  )
```

```
# A tibble: 2 x 6
  id        ano   mes   dia  tmax  tmin
  <chr>   <dbl> <dbl> <int> <dbl> <dbl>
1 MX17004  2010     2     2  27.3  14.4
2 MX17004  2010     2     3  24.1  14.4
```

En la tabla resultante del Listado 16.6, cada fila representa un día específico de observación y cada columna una variable medida, cumpliendo con el formato de datos ordenados.

16.2.4. Diferentes unidades observacionales en una sola tabla

En ocasiones, un conjunto de datos recopila información de diferentes unidades observacionales dentro de una misma tabla. Esta práctica genera duplicación redundante de registros y aumenta el riesgo de inconsistencias.

Un ejemplo representativo de esta situación se ilustró en la Capítulo 11 mediante el modelo relacional del examen Saber Pro (ver Figura 11.1). Si todos los datos estuviesen consolidados en una única tabla no ordenada, por cada evaluación de un estudiante se repetirían innecesariamente los atributos correspondientes a las entidades **Departamento**, **Municipio** y **Programa académico**.

Para que el conjunto de datos sea ordenado los datos se estructuran en tablas independientes conectadas por claves primarias (**PK**) y foráneas (**FK**):

1. Una tabla central de eventos (**Resultado Saber-Pro**), donde cada fila representa una evaluación individual identificada por el año y el consecutivo del estudiante.
2. Tablas complementarias para cada entidad u observación particular: **Departamento**, **Municipio** y **Programa académico**.

Como se exploró en la Capítulo 11, esta separación evita redundancias y permite recuperar la información integrada cuando sea necesario mediante **Mutating joins** como por ejemplo `left_join()`.

16.2.5. Una misma unidad observacional almacenada en múltiples tablas

El caso opuesto ocurre cuando los datos correspondientes a una misma unidad observacional se encuentran fragmentados en múltiples tablas o archivos independientes.

Un escenario común se presenta con los resultados del examen Saber-Pro. Es habitual que la información de cada periodo se publique y almacene en archivos separados por año, como por ejemplo:

- `2021-saber_pro-genericas.csv`
- `2022-saber_pro-genericas.csv`
- `2023-saber_pro-genericas.csv`
- `2024-saber_pro-genericas.csv`.

Aunque estos archivos comparten variables comunes y la misma unidad observacional, su fragmentación dificulta realizar análisis longitudinales o comparativos. Para ordenar y consolidar este tipo de datos, el flujo de trabajo estándar comprende:

1. Identificar las rutas de cada uno de los archivos.
2. Leer de forma iterativa cada archivo asegurando registrar el año o archivo de origen.
3. Unir todas las filas en una única tabla consolidada como `2021_2024-saber_pro-genericas.csv` identificando las variables en común para el conjunto de tablas que se buscan consolidar.

En la Capítulo 20 se abordará en detalle cómo automatizar este proceso utilizando las herramientas de programación funcional del paquete `purrr` de tal manera que se puedan consolidar múltiples archivos en una única tabla ordenada para su posterior análisis utilizando el `tidyverse`.

17. Pivoteo

Este capítulo se basa en (Wickham, 2023, Capítulo 5) así como en la documentación oficial de `tidyr` donde tiene como propósito introducir la forma de estructurar un conjuntos de datos en formato largo y ancho (en inglés **longer** y **wider**) mediante las funciones `pivot_longer()` y `pivot_wider()` utilizando el paquete `tidyr` e información de proyecciones y retroproyecciones de población nacional por área, sexo y edad para el periodo 2018-2070 de Colombia con base en el Censo Nacional de Población y Vivienda (CNPV) 2018.

Exploraremos los siguientes aspectos:

1. ¿Qué es el formato largo para estructurar datos?
2. ¿Qué es el formato ancho para estructurar datos?
3. ¿Cómo realizar limpieza de nombres utilizando el paquete `janitor`?
4. ¿Cómo estructurar datos para generar pirámides poblaciones?

> **Nota 38**
>
> Para desarrollar el contenido de este capítulo es necesario que hayas guardado el archivo `serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx` en la carpeta `data`, tal como se señaló en la Nota 30. En caso que no lo hayas hecho es necesario que lo descargues y lo incluyas dentro de la carpeta `data`:
>
> **serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx**
>
> También es necesario que instales el paquete `janitor` teniendo en cuenta la Sección 4.2, tal como se señala a continuación:
>
> ```
> install.packages("janitor")
> ```
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `017_pivotear-datos-en-r` y dentro de ella genera el archivo `001_pivotear.R`, siguiendo las pautas de la Sección 4.4.

17.1. Pirámides poblacionales

En el Capítulo 17 y en el Capítulo 18 ordenarás un conjunto de datos utilizando el paquete `tidyr` y utilizarás tanto la **gramática de manipulación de datos** (ver Sección 10.1) como la **gramática de gráficos en capas** (ver Sección 6.1) para realizar un gráfico de pirámides de población señalando la participación en la población por grupos etareos en Colombia para los años 2018, 2035, 2052 y 2070, tal como se indica en la Figura 17.1.

17.2. Formato largo

El formato largo (en inglés **longer format**) reorganiza una tabla aumentando el número de filas y reduciendo el de columnas, un proceso frecuentemente denominado "alargar" los datos. Esta transformación se requiere cuando los nombres de ciertas columnas no corresponden a nombres de variables sino a valores o niveles específicos de una variable.

En el paquete `tidyr`, la función `pivot_longer()` permite transformar este tipo de tablas en una estructura ordenada mediante 3 parámetros principales:

1. `cols`: especifica el conjunto de columnas cuyos nombres representan valores y que deben reestructurarse.
2. `names_to`: define el nombre de la nueva columna en la que se recopilarán los nombres de las columnas seleccionadas en `cols`.
3. `values_to`: define el nombre de la nueva columna que incluirá los valores numéricos o caracteres almacenados en las celdas de las columnas seleccionadas.

En la Figura 17.2 se ilustra conceptualmente la reestructuración realizada por la función `pivot_longer()`.

Tal como se observa en la Figura 17.2, la tabla inicial contiene 2 columnas (`m_014` y `f_014`) asociadas a categorías demográficas de sexo y edad. Al aplicar `pivot_longer()`, los encabezados de estas 2 columnas pasan a formar parte de una única columna de categorías (`name`), mientras que los valores numéricos correspondientes (`20`, `15`, `3` y `5`) se recopilan en la columna `value`. Este proceso preserva el atributo identificador (`id`) duplicando sus filas según sea necesario para "alargar" la tabla de 2 a 4 filas.

> **Nota 39**
>
> La aplicación en código de la función `pivot_longer()` y su sintaxis básica fueron introducidas en el Capítulo 16 a través de conjuntos de datos como `relig_income` y `billboard`. En la Sección 17.4 aplicarás esta función en un escenario práctico de demografía nacional.

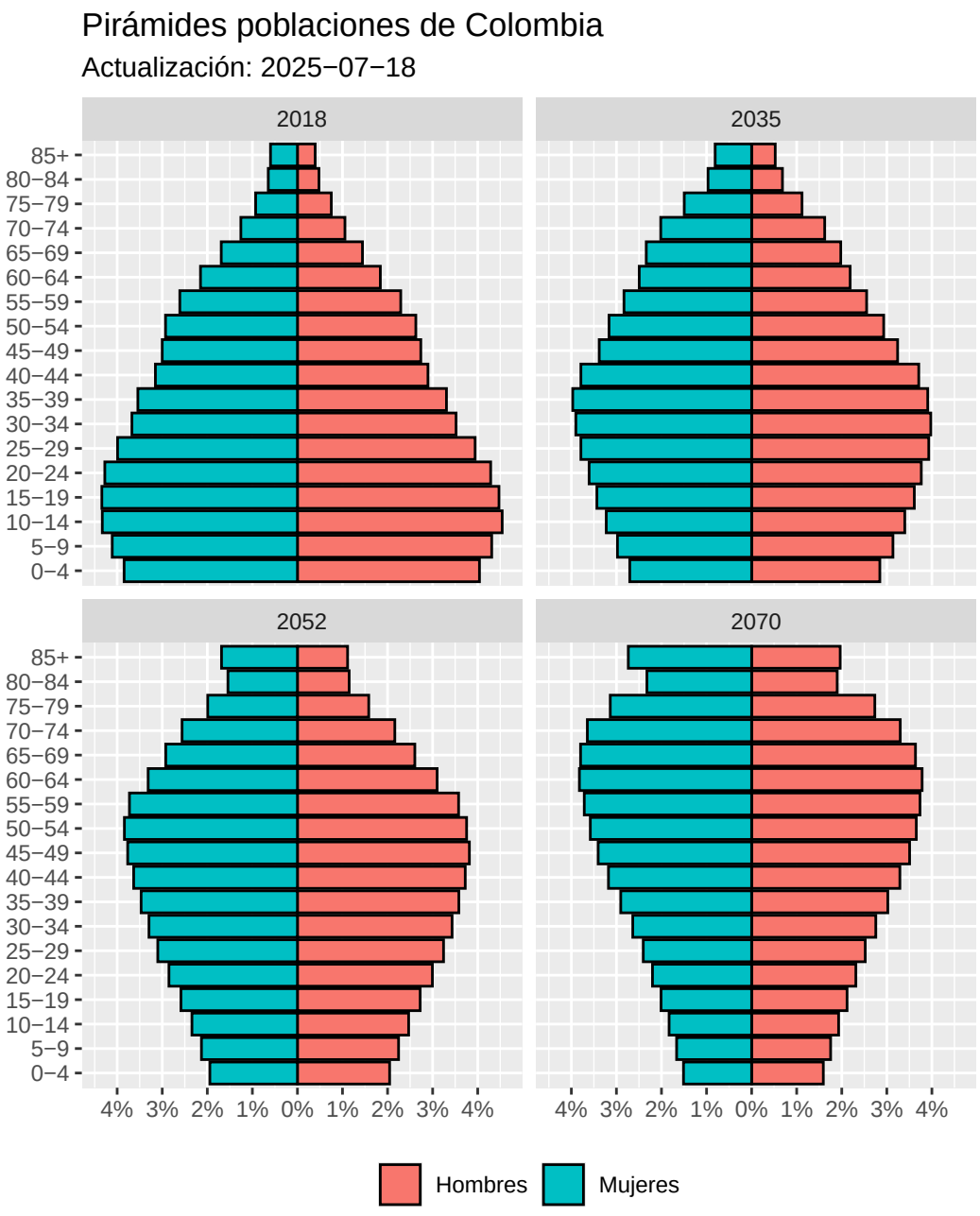


Figura 17.1.: Resultado final de un gráfico de pirámides poblaciones utilizando `readxl`, `janitor`, `tidyr`, `ggplot2` y `scales`

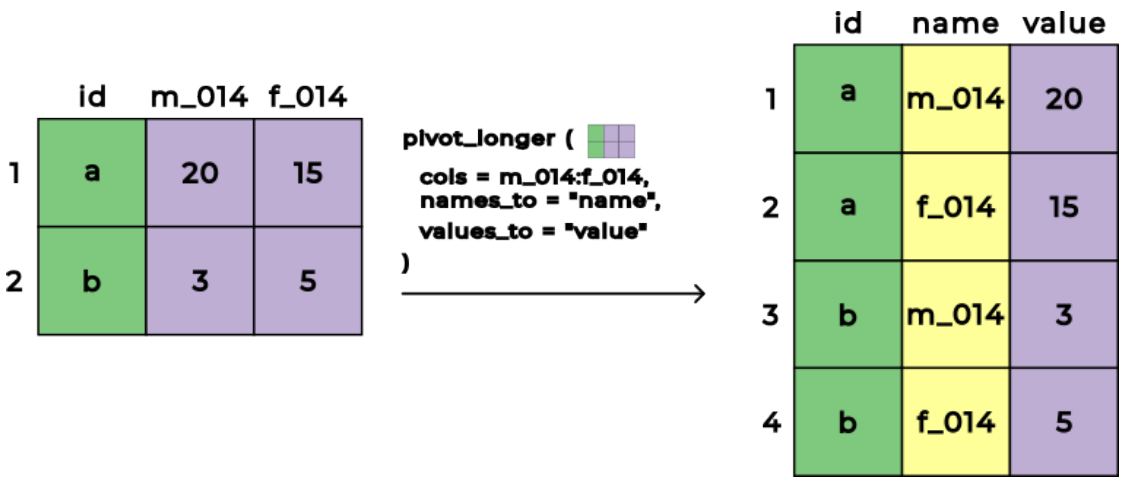


Figura 17.2.: Representación conceptual de la función `pivot_longer()`

17.3. Formato ancho

El formato ancho (en inglés **wider format**) corresponde a la operación inversa del formato largo, reorganizando una tabla al aumentar el número de columnas y disminuir el de filas, un proceso comúnmente llamado "ensanchar" los datos. Esta transformación resulta fundamental cuando una observación particular se encuentra fragmentada a lo largo de múltiples filas o cuando se desea preparar una matriz o tabla de presentación donde las categorías de una variable se exhiben como encabezados de columnas independientes.

En el paquete `tidyr`, la función `pivot_wider()` implementa esta reestructuración mediante 2 parámetros principales:

1. `names_from`: especifica la columna existente cuyos valores únicos se convertirán en los encabezados de las nuevas columnas.
2. `values_from`: define la columna existente desde la cual se extraerán los valores de las celdas que poblarán las nuevas columnas generadas.

En la Figura 17.3 se ilustra conceptualmente la reestructuración realizada por la función `pivot_wider()`.

Como se observa en la Figura 17.3, la tabla inicial contiene 4 observaciones distribuidas en las columnas `id`, `name` y `value`. Al ejecutar `pivot_wider()`, los valores diferenciados de la columna `name` (`m_014` y `f_014`) se elevan a nuevos encabezados de columna y las celdas correspondientes se rellenan con los registros numéricos de `value`. De este modo, la tabla se consolida pasando de 4 filas a 2 filas agrupando los atributos de cada identificador `id` en una sola fila.

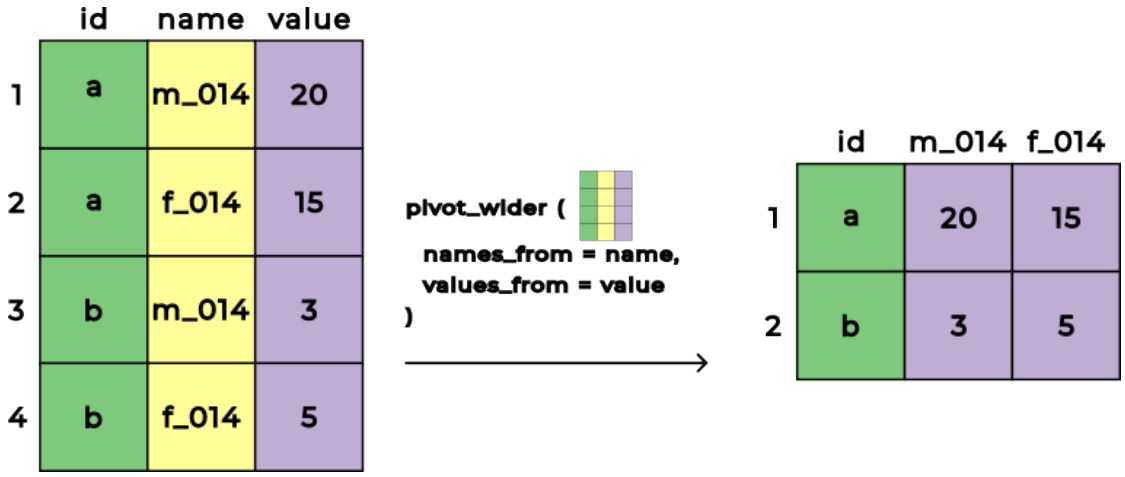


Figura 17.3.: Representación conceptual de la función `pivot_wider()`

> **Nota 40**
>
> El uso de `pivot_wider()` en conjunto con `pivot_longer()` se exploró en el Capítulo 16 mediante el análisis de registros meteorológicos diarios en `clima_muestra`. Aunque en la gran mayoría de los análisis dentro del `tidyverse` se prefiere el formato largo para facilitar la manipulación y visualización, el formato ancho resulta idóneo para reportes tabulares finales o matrices.

17.4. Caso de estudio: estructurar datos en formato largo para generar pirámides poblacionales de Colombia

El propósito de la Sección 17.4 es aplicar la limpieza de nombres y la transformación hacia formato largo mediante `pivot_longer()` partiendo de los datos oficiales de proyecciones y retroproyecciones de población nacional para Colombia elaborados por el Departamento Administrativo Nacional de Estadística (DANE) para el periodo 2018-2070.

El objetivo final de este caso de estudio es generar el archivo `poblacion_col_2018_2070_largo.rds`, el cual servirá como insumo fundamental para separar variables compuestas y preparar la visualización de pirámides poblacionales en el Capítulo 18.

17.4.1. Datos

Para desarrollar el caso de estudio, en primera instancia se cargan los paquetes `readxl`, `janitor` y `tidyverse` y lueog se importan los datos aplicando lo desarrollado en el Capí-

tulo 14 con respecto al archivo `serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx`, tal como se señala a continuación:

Listado 17.1 Carga de paquetes e importación de datos aplicando lo desarrollado en el Capítulo 14

```
# Cargar paquetes ----
library(readxl)
library(janitor)
library(tidyverse)

# Importar datos ----
path <- "data/serie-nacional-de-poblacion_area-sexo-edad_2018-2070_2025-07-18.xlsx"

poblacion_col_2018_2070 <- read_excel(
  path = path,
  sheet = "PobNacionalxÁreaSexoEdad",
  skip = 8
) |>
  rename(
    territorio = `...1`,
    ano = `...2`,
    area_geografica = `...3`
  )
```

Al inspeccionar el objeto importado `poblacion_col_2018_2070` es posible observar la estructura inicial de la tabla, tal como se señala a continuación:

```
# Inspeccionar datos importados ----
poblacion_col_2018_2070
```

```
# A tibble: 127 x 309
   territorio     ano area_geografica  Total Hombres Mujeres
   <chr>        <dbl> <chr>            <dbl>   <dbl>   <dbl>
 1 Total Nacio~  2018 Cabecera        3.64e7  1.75e7  1.90e7
 2 Total Nacio~  2018 Centros Poblad~ 1.18e7  6.16e6  5.68e6
 3 Total Nacio~  2018 Total           4.83e7  2.36e7  2.46e7
 4 Total Nacio~  2019 Cabecera        3.72e7  1.78e7  1.94e7
 5 Total Nacio~  2019 Centros Poblad~ 1.20e7  6.27e6  5.78e6
 6 Total Nacio~  2019 Total           4.93e7  2.41e7  2.51e7
 7 Total Nacio~  2020 Cabecera        3.81e7  1.83e7  1.98e7
 8 Total Nacio~  2020 Centros Poblad~ 1.23e7  6.38e6  5.88e6
 9 Total Nacio~  2020 Total           5.04e7  2.47e7  2.57e7
```

```
10 Total Nacio~  2021 Cabecera        3.87e7  1.86e7  2.01e7
# i 117 more rows
# i 303 more variables: `Hombres 0 años` <dbl>,
#   `Hombres 1 año` <dbl>, `Hombres 2 años` <dbl>,
#   `Hombres 3 años` <dbl>, `Hombres 4 años` <dbl>,
#   `Hombres 5 años` <dbl>, `Hombres 6 años` <dbl>,
#   `Hombres 7 años` <dbl>, `Hombres 8 años` <dbl>,
#   `Hombres 9 años` <dbl>, `Hombres 10 años` <dbl>, ...
```

La tabla contiene 127 filas y múltiples columnas donde se combinan variables geográficas, temporales y columnas demográficas cuyos nombres contienen espacios, mayúsculas y caracteres especiales como por ejemplo `` `Hombres 0 años` `` o `` `Mujeres 100 años y más` ``.

17.4.2. Limpieza de nombres

Trabajar con nombres de columnas que contienen espacios o caracteres especiales en R exige escribir los nombres entre comillas invertidas, como por ejemplo `` `Hombres 0 años` ``. Este aspecto dificulta la escritura de código y puede generar errores tipográficos recurrentes.

Para resolver este inconveniente el paquete `janitor` provee la función `clean_names()`, la cual transforma de forma automatizada los encabezados de cualquier tabla a un estilo consistente en minúsculas y separado por guiones bajos conocido en inglés con el nombre de **snake case**. De esa manera se eliminan tildes y caracteres especiales, tal como se señala a continuación:

Listado 17.2 Limpieza de nombres de variables mediante `clean_names()`

```
# Limpiar nombres ----
poblacion_col_2018_2070_limpiar_nombres <- poblacion_col_2018_2070 |>
  clean_names()
```

```
# Inspeccionar nombres limpios ----
poblacion_col_2018_2070_limpiar_nombres
```

```
# A tibble: 127 x 309
   territorio     ano area_geografica  total hombres mujeres
   <chr>        <dbl> <chr>            <dbl>   <dbl>   <dbl>
 1 Total Nacio~  2018 Cabecera        3.64e7  1.75e7  1.90e7
 2 Total Nacio~  2018 Centros Poblad~ 1.18e7  6.16e6  5.68e6
 3 Total Nacio~  2018 Total           4.83e7  2.36e7  2.46e7
```

```
 4 Total Nacio~  2019 Cabecera        3.72e7  1.78e7  1.94e7
 5 Total Nacio~  2019 Centros Poblad~ 1.20e7  6.27e6  5.78e6
 6 Total Nacio~  2019 Total           4.93e7  2.41e7  2.51e7
 7 Total Nacio~  2020 Cabecera        3.81e7  1.83e7  1.98e7
 8 Total Nacio~  2020 Centros Poblad~ 1.23e7  6.38e6  5.88e6
 9 Total Nacio~  2020 Total           5.04e7  2.47e7  2.57e7
10 Total Nacio~  2021 Cabecera        3.87e7  1.86e7  2.01e7
# i 117 more rows
# i 303 more variables: hombres_0_anos <dbl>,
#   hombres_1_ano <dbl>, hombres_2_anos <dbl>,
#   hombres_3_anos <dbl>, hombres_4_anos <dbl>,
#   hombres_5_anos <dbl>, hombres_6_anos <dbl>,
#   hombres_7_anos <dbl>, hombres_8_anos <dbl>,
#   hombres_9_anos <dbl>, hombres_10_anos <dbl>, ...
```

Al aplicar `clean_names()`, encabezados como `` `Hombres 0 años` `` y `` `Mujeres 100 años y más` `` se convierten de forma transparente en `hombres_0_anos` y `mujeres_100_anos_y_mas` facilitando su selección y manipulación en los pasos posteriores.

17.4.3. Filtrar categoría de interés

El conjunto de datos del DANE clasifica las proyecciones anuales según el área geográfica en 3 niveles: `"Cabecera"`, `"Centros Poblados y Rural Disperso"` y `"Total"`. Dado que el objetivo es analizar la estructura demográfica a nivel nacional, es necesario filtrar únicamente los registros correspondientes al total nacional.

Para lograr este aspecto, se utiliza la función `filter()` (ver Sección 10.1.1.1) sobre la columna `area_geografica` seleccionando el valor `"Total"`, tal como se señala a continuación:

Listado 17.3 Filtrar observaciones del total nacional mediante `filter()`

```
# Filtrar ----
poblacion_col_2018_2070_filtrar_total <- poblacion_col_2018_2070_limpiar_nombres |>
  filter(
    area_geografica == "Total"
  )
```

```
# Inspeccionar datos filtrados ----
poblacion_col_2018_2070_filtrar_total
```

```
# A tibble: 53 x 309
   territorio     ano area_geografica  total hombres mujeres
   <chr>        <dbl> <chr>            <dbl>   <dbl>   <dbl>
 1 Total Nacio~  2018 Total           4.83e7  2.36e7  2.46e7
 2 Total Nacio~  2019 Total           4.93e7  2.41e7  2.51e7
 3 Total Nacio~  2020 Total           5.04e7  2.47e7  2.57e7
 4 Total Nacio~  2021 Total           5.11e7  2.50e7  2.61e7
 5 Total Nacio~  2022 Total           5.16e7  2.53e7  2.63e7
 6 Total Nacio~  2023 Total           5.21e7  2.55e7  2.66e7
 7 Total Nacio~  2024 Total           5.26e7  2.58e7  2.68e7
 8 Total Nacio~  2025 Total           5.31e7  2.60e7  2.71e7
 9 Total Nacio~  2026 Total           5.34e7  2.62e7  2.72e7
10 Total Nacio~  2027 Total           5.37e7  2.63e7  2.74e7
# i 43 more rows
# i 303 more variables: hombres_0_anos <dbl>,
#   hombres_1_ano <dbl>, hombres_2_anos <dbl>,
#   hombres_3_anos <dbl>, hombres_4_anos <dbl>,
#   hombres_5_anos <dbl>, hombres_6_anos <dbl>,
#   hombres_7_anos <dbl>, hombres_8_anos <dbl>,
#   hombres_9_anos <dbl>, hombres_10_anos <dbl>, ...
```

Mediante esta operación se reducen las observaciones a 53 filas, correspondientes a los 53 años comprendidos en el horizonte de proyección 2018 a 2070.

17.4.4. Seleccionar variables de interés

La tabla resultante contiene agregados globales (`total`, `hombres` y `mujeres`) así como totales por edad simple junto con las columnas desagregadas por sexo y edad simple desde `hombres_0_anos` hasta `mujeres_100_anos_y_mas`.

Para construir la pirámide poblacional en los capítulos posteriores, se requiere únicamente la variable temporal (`ano`), el total nacional (`total`) para calcular porcentajes poblacionales y las columnas que desglosan la población por sexo y edad. Para seleccionarlas eficientemente se utiliza la función `select()` junto con el operador de rango consecutivo `:` (ver Sección 10.2.3), tal como se señala a continuación:

```
# Inspeccionar variables seleccionadas ----
poblacion_col_2018_2070_seleccionar
```

```
# A tibble: 53 x 204
     ano   total hombres_0_anos hombres_1_ano hombres_2_anos
   <dbl>   <dbl>          <dbl>         <dbl>          <dbl>
 1  2018  4.83e7         379611        384392         389812
```

Listado 17.4 Selección de variables requeridas para la pirámide poblacional mediante `select()`

```
# Seleccionar ----
poblacion_col_2018_2070_seleccionar <- poblacion_col_2018_2070_filtrar_total |>
  select(
    ano,
    total,
    hombres_0_anos:mujeres_100_anos_y_mas
  )
```

```
 2  2019  4.93e7         379692         386682         391193
 3  2020  5.04e7         385600         388121         394740
 4  2021  5.11e7         382677         389658         392098
 5  2022  5.16e7         379263         384581         391602
 6  2023  5.21e7         375915         380514         385906
 7  2024  5.26e7         372539         377208         381864
 8  2025  5.31e7         367902         373397         378139
 9  2026  5.34e7         361692         367778         373403
10  2027  5.37e7         355501         361449         367664
# i 43 more rows
# i 199 more variables: hombres_3_anos <dbl>,
#   hombres_4_anos <dbl>, hombres_5_anos <dbl>,
#   hombres_6_anos <dbl>, hombres_7_anos <dbl>,
#   hombres_8_anos <dbl>, hombres_9_anos <dbl>,
#   hombres_10_anos <dbl>, hombres_11_anos <dbl>,
#   hombres_12_anos <dbl>, hombres_13_anos <dbl>, ...
```

De esta forma, la tabla queda restringida a 204 columnas: `ano`, `total` y las 202 categorías demográficas individuales correspondientes a 101 para hombres y 101 para mujeres desde 0 hasta 100 años y más.

17.4.5. Ordenar datos en formato largo

Aunque la tabla seleccionada contiene toda la información demográfica necesaria, no se encuentra en formato ordenado. Este aspecto se debe a que los encabezados desde `hombres_0_anos` hasta `mujeres_100_anos_y_mas` son en realidad valores de variables demográficas y no nombres de variables.

Para estructurar los datos en formato largo, se aplica la función `pivot_longer()`:

- `cols = hombres_0_anos:mujeres_100_anos_y_mas`: especifica el rango de variables que se colapsarán en filas.

- `names_to = "genero_edad"`: define el nombre de la nueva columna que almacenará los nombres de las categorías.
- `values_to = "poblacion"`: define el nombre de la nueva columna que contendrá el conteo de población respectivo.

Este procedimiento se realiza tal como se señala a continuación:

Listado 17.5 Reestructuración de variables demográficas a formato largo mediante `pivot_longer()`

```
# Tidy ----
poblacion_col_2018_2070_largo <- poblacion_col_2018_2070_seleccionar |>
  pivot_longer(
    cols = hombres_0_anos:mujeres_100_anos_y_mas,
    names_to = "genero_edad",
    values_to = "poblacion"
  )
```

```
# Inspeccionar datos en formato largo ----
poblacion_col_2018_2070_largo
```

```
# A tibble: 10,706 x 4
     ano    total genero_edad    poblacion
   <dbl>    <dbl> <chr>              <dbl>
 1  2018 48258494 hombres_0_anos    379611
 2  2018 48258494 hombres_1_ano     384392
 3  2018 48258494 hombres_2_anos    389812
 4  2018 48258494 hombres_3_anos    395361
 5  2018 48258494 hombres_4_anos    400077
 6  2018 48258494 hombres_5_anos    405013
 7  2018 48258494 hombres_6_anos    410040
 8  2018 48258494 hombres_7_anos    415839
 9  2018 48258494 hombres_8_anos    421676
10  2018 48258494 hombres_9_anos    427467
# i 10,696 more rows
```

El resultado es una tibble en formato largo de 10706 filas (53 años×202 categorías demográficas) y 4 columnas: `ano`, `total`, `genero_edad` y `poblacion`. En este punto, los datos se encuentran en formato largo y listos para proceder con la separación de la columna `genero_edad` en sus 2 componentes individuales (`genero` y `edad`) en el Capítulo 18.

17.4.6. Exportar datos en formato largo

Una vez estructurado el conjunto de datos en formato largo mediante la función `pivot_longer()`, es posible exportar el archivo y guardarlo en la carpeta `data` para utilizarlo posteriormente en el Capítulo 18, tal como se señala a continuación:

Listado 17.6 Uso de la función `write_rds()` para exportar y guardar el archivo de datos en formato largo

```
poblacion_col_2018_2070_largo |>
  write_rds(
    file = "data/poblacion_col_2018_2070_largo.rds"
  )
```

En el Listado 17.6 se utiliza la función `write_rds()` junto con el parámetro `file` asociado con el argumento `"data/poblacion_col_2018_2070_largo.rds"` para definir tanto la ruta como el nombre del archivo en el que se guardará la información en formato largo.

18. Separación y unión de caracteres en columnas

Este capítulo se basa en (Wickham, 2023, Capítulo 5) así como en la documentación oficial de `tidyr` respecto a la familia de funciones `separate_wider_*()` y la función `unite()` donde tiene como propósito introducir la forma de separar caracteres en columnas o unir múltiples columnas en una sola mediante concatenación de caracteres, utilizando el paquete `tidyr` e información de proyecciones y retroproyecciones de población nacional por área, sexo y edad para el periodo 2018-2070 de Colombia con base en el Censo Nacional de Población y Vivienda (CNPV) 2018.

Exploraremos los siguientes aspectos:

1. ¿Cómo separar caracteres en columnas?
2. ¿Cómo unir múltiples columnas mediante concatenación de caracteres?
3. ¿Cuál es el proceso para separar caracteres en columnas, transformar y preparar datos para generar pirámides poblacionales?
4. ¿Cuál es el proceso para visualizar datos a través de pirámides poblacionales?

> **Nota 41**
>
> Para desarrollar el contenido de este capítulo es necesario que hayas exportado y guardado el archivo `poblacion_col_2018_2070_largo.rds` en la carpeta `data`, tal como se señaló en el Listado 17.6. En caso que no lo hayas hecho es necesario que lo descargues y lo incluyas dentro de la carpeta `data`:
>
> **poblacion_col_2018_2070_largo.rds**
>
> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `018_separar-unir-caracteres-cols-en-r` y dentro de ella genera el archivo `001_separar-unir-caracteres-cols.R`, siguiendo las pautas de la Sección 4.4.

18.1. Separar caracteres en columnas

En el Capítulo 16 se examinaron los problemas más comunes presentes en datos no ordenados, destacando aquel donde múltiples variables se encuentran combinadas dentro de una sola columna (ver Listado 16.3). Para solucionar este inconveniente y garantizar que cada variable conforme su propia columna, el paquete `tidyr` proporciona la familia de funciones `separate_wider_*()`, las cuales dividen una columna de caracteres en múltiples columnas rectangulares.

En la Figura 18.1 se ilustra gráficamente el funcionamiento conceptual de la separación de caracteres en columnas mediante `separate_wider_delim()`.

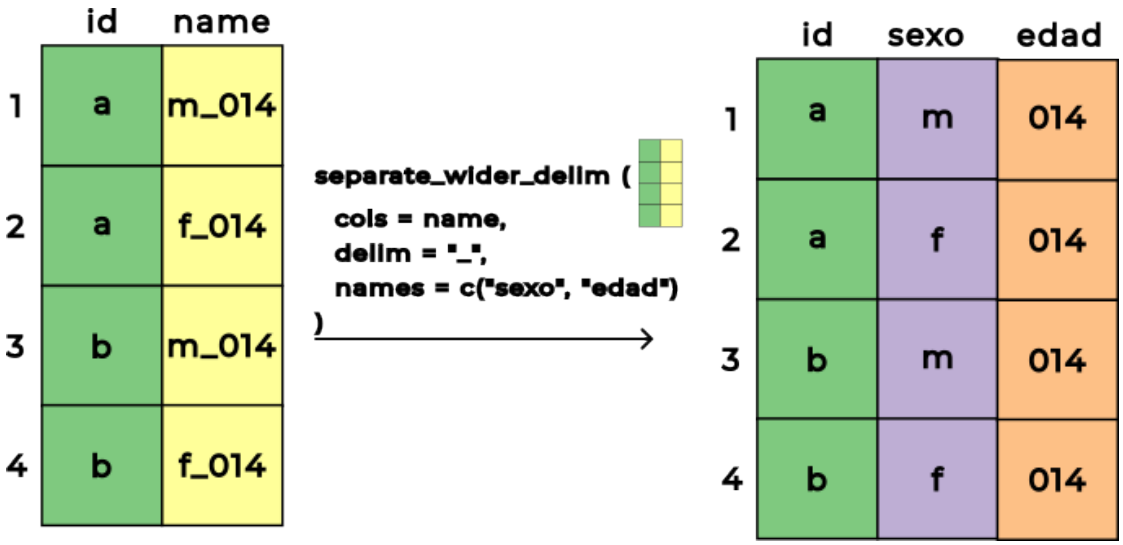


Figura 18.1.: Representación gráfica de la función `separate_wider_delim()`

De acuerdo con la documentación oficial de `tidyr`, la familia de funciones `separate_wider_*()` se compone de 3 alternativas especializadas según el criterio empleado para efectuar la división:

1. `separate_wider_delim()`: separa una columna en múltiples columnas basándose en un delimitador o carácter separador fijo, como por ejemplo un guion bajo `_`, una coma `,` o un guion `-`. Esta es la función que utilizaremos en el caso de estudio de este capítulo debido a que las categorías a separar se encuentran delimitadas por patrones textuales definidos.
2. `separate_wider_position()`: separa una columna en múltiples columnas basándose en anchos o posiciones numéricas fijas de los caracteres. Esta función resulta idónea cuando los datos provienen de archivos de ancho fijo donde cada variable ocupa un número invariable de caracteres y no existen delimitadores explícitos.
3. `separate_wider_regex()`: separa una columna en múltiples columnas basándose en patrones de expresiones regulares (en inglés **regular expressions** o **regex**).

Esta función ofrece la máxima flexibilidad cuando la estructura interna del texto es variable o compleja, permitiendo extraer patrones específicos.

> **Nota 42**
>
> Las expresiones regulares constituyen un lenguaje conciso y formal para describir patrones dentro de cadenas de caracteres. Si bien en este capítulo utilizaremos `separate_wider_delim()` debido a que los datos poseen un delimitador claro, el lector interesado en profundizar en el uso de expresiones regulares y su aplicación con funciones como `separate_wider_regex()` puede consultar (Wickham, 2023, Capítulo 15).

Al utilizar la función `separate_wider_delim()`, los parámetros principales a especificar son:

- `cols`: especifica la columna o variable que se desea dividir.
- `delim`: indica la cadena de texto o carácter que actúa como separador.
- `names`: suministra un vector de caracteres con los nombres de las nuevas columnas resultantes.
- `too_many`: define la acción a tomar si existen más divisiones de las esperadas por el argumento `names`, permitiendo descartar elementos sobrantes mediante `"drop"` o emitir un mensaje de advertencia o error.

18.2. Unir múltiples columnas mediante concatenación de caracteres

La función `unite()` del paquete `tidyr` realiza la operación inversa a las funciones de la familia `separate_wider_*()`. Mientras que las funciones de separación toman una única columna compuesta y la descomponen en varias variables independientes, `unite()` toma múltiples columnas y las combina o concatena en una sola columna de caracteres uniendo sus valores mediante un delimitador común.

En la Figura 18.2 se ilustra gráficamente el funcionamiento de la función `unite()`.

De acuerdo con la documentación oficial de `tidyr`, los parámetros principales de la función `unite()` son:

- `data`: la tibble u objeto similar sobre el cual se aplicará la transformación.
- `col`: el nombre de la nueva columna resultante suministrado como una cadena de texto o símbolo sin comillas.
- `...`: las columnas que se desean unir, las cuales pueden especificarse individualmente o utilizando rangos y funciones auxiliares de selección idénticas a las empleadas en `select()` (ver Sección 10.1.1.2).

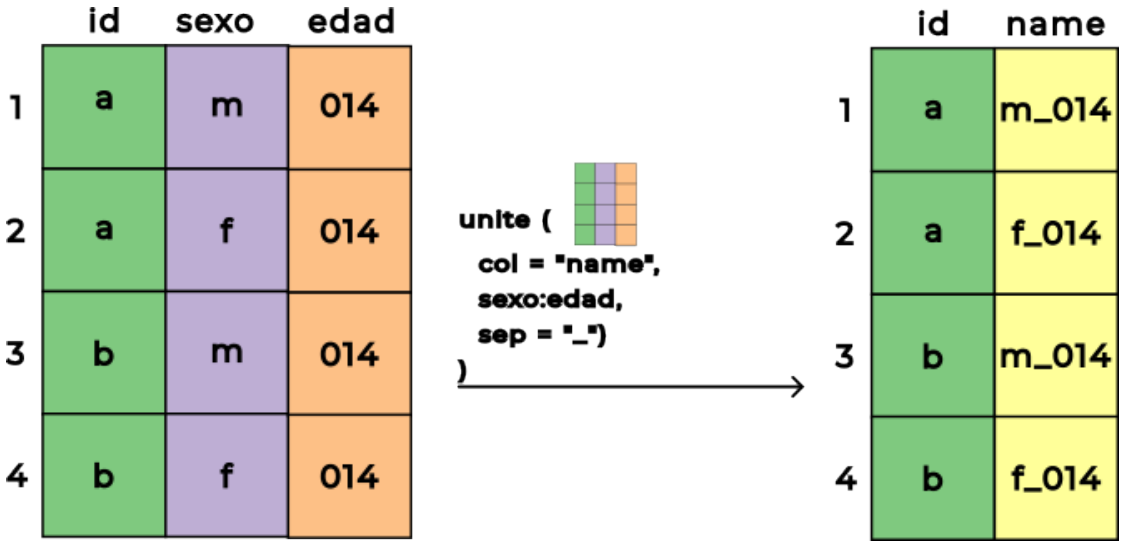


Figura 18.2.: Representación gráfica de la función `unite()`

- `sep`: el separador o delimitador que se insertará entre los valores concatenados de cada columna donde por defecto corresponde a un guion bajo `_`.
- `remove`: un valor lógico que indica si se deben eliminar de la tabla resultante las columnas originales que fueron unidas (`TRUE` por defecto o si se deben conservar `FALSE`).
- `na.rm`: un valor lógico que determina si los valores ausentes (`NA`) deben excluirse durante la concatenación (`FALSE` por defecto).

18.3. Caso de estudio: separar caracteres en columnas, transformar y preparar datos para generar pirámides poblacionales de Colombia

En el Capítulo 17 se inició la reestructuración de las proyecciones de población nacional para Colombia provenientes del CNPV 2018, transformando las variables demográficas en formato largo y almacenando el resultado en el archivo `poblacion_col_2018_2070_largo.rds`. El propósito de este caso de estudio es completar el procesamiento de dicha información a través de 3 fases:

1. Separar la columna compuesta `genero_edad` en sus variables constituyentes individuales (`genero` y `edad`).
2. Transformar las variables asociando sus tipos adecuados y calculando la participación porcentual de cada grupo.
3. Preparar los datos agrupándolos en intervalos etarios quinquenales para consolidar las cohortes necesarias para la construcción de pirámides poblacionales.

Para iniciar con el desarrollo del caso de estudio, es necesario en primera instancia cargar los paquetes requeridos e importar el archivo guardado previamente en la carpeta `data`, tal como se señala a continuación:

Listado 18.1 Importación del conjunto de datos en formato largo para desarrollar el caso de estudio

```
# Cargar paquetes ----
library(tidyverse)

# Importar datos ----
poblacion_col_2018_2070_largo <- read_rds(
  file = "data/poblacion_col_2018_2070_largo.rds"
)
```

18.3.1. Separar caracteres en columnas

Al examinar la columna `genero_edad`, se observa que cada celda contiene etiquetas compuestas como `"hombres_0_anos"` o `"mujeres_100_anos_y_mas"`. En estas cadenas, el primer componente define el género (`"hombres"` o `"mujeres"`) y el segundo la edad numérica (`0`, `1`, ..., `100`) mientras que el resto de palabras (`"anos"`, `"ano"` `"anos_y_mas"`) corresponden a sufijos redundantes.

Para separar esta información en 2 columnas independientes denominadas `genero` y `edad`, se utiliza la función `separate_wider_delim()`. Dado que algunas cadenas poseen múltiples guiones bajos posteriores al número de edad, se emplea el parámetro `too_many = "drop"` para conservar únicamente los 2 primeros elementos extraídos y descartar los fragmentos adicionales sobrantes, tal como se señala a continuación:

```
# A tibble: 10,706 x 5
     ano    total genero  edad  poblacion
   <dbl>    <dbl> <chr>   <chr>     <dbl>
 1  2018 48258494 hombres 0        379611
 2  2018 48258494 hombres 1        384392
 3  2018 48258494 hombres 2        389812
 4  2018 48258494 hombres 3        395361
 5  2018 48258494 hombres 4        400077
 6  2018 48258494 hombres 5        405013
 7  2018 48258494 hombres 6        410040
 8  2018 48258494 hombres 7        415839
 9  2018 48258494 hombres 8        421676
10  2018 48258494 hombres 9        427467
# i 10,696 more rows
```

Listado 18.2 Separación de la columna compuesta `genero_edad` mediante `separate_wider_delim()`

```
# Separar ----
poblacion_col_2018_2070_separar <- poblacion_col_2018_2070_largo |>
  separate_wider_delim(
    cols = genero_edad,
    delim = "_",
    names = c("genero", "edad"),
    too_many = "drop"
  )

poblacion_col_2018_2070_separar
```

En el Listado 18.2 la función `separate_wider_delim()` divide la variable `genero_edad` utilizando el delimitador `_`. Con `names = c("genero", "edad")` se asignan las 2 nuevas variables resultantes y con `too_many = "drop"` se asegura que expresiones como `"100_anos_y_mas"` asignen únicamente `"100"` a la columna `edad`, omitiendo los segmentos posteriores.

18.3.2. Transformar variables

Una vez separadas las variables, la columna `edad` queda almacenada como una cadena de caracteres (`<chr>`) y la columna `ano` puede requerir ser convertida a un número entero. Asimismo, para elaborar una pirámide poblacional se requiere analizar la participación de cada cohorte respecto a la población total del país en cada año.

Para llevar a cabo estas transformaciones, se utiliza la función `mutate()` convirtiendo `ano` y `edad` en enteros con `as.integer()` y calculando la variable `poblacion_pct` mediante el cociente entre la población de la cohorte (`poblacion`) y el total nacional (`total`), tal como se señala a continuación:

```
# A tibble: 10,706 x 6
     ano    total genero   edad poblacion poblacion_pct
   <int>    <dbl> <chr>   <int>     <dbl>         <dbl>
 1  2018 48258494 hombres     0    379611       0.00787
 2  2018 48258494 hombres     1    384392       0.00797
 3  2018 48258494 hombres     2    389812       0.00808
 4  2018 48258494 hombres     3    395361       0.00819
 5  2018 48258494 hombres     4    400077       0.00829
 6  2018 48258494 hombres     5    405013       0.00839
 7  2018 48258494 hombres     6    410040       0.00850
```

Listado 18.3 Transformación de tipos de datos y cálculo de la proporción poblacional mediante `mutate()`

```
# Transformar ----
poblacion_col_2018_2070_transformar <- poblacion_col_2018_2070_separar |>
  mutate(
    ano = as.integer(ano),
    edad = as.integer(edad),
    poblacion_pct = poblacion / total
  )

poblacion_col_2018_2070_transformar
```

```
 8  2018 48258494 hombres     7    415839       0.00862
 9  2018 48258494 hombres     8    421676       0.00874
10  2018 48258494 hombres     9    427467       0.00886
# i 10,696 more rows
```

En el Listado 18.3, el cálculo `poblacion / total` genera una medida estandarizada en el intervalo $[0, 1]$ que permite comparar de forma consistente la estructura demográfica entre distintos años, independientemente del crecimiento del tamaño absoluto de la población.

18.3.3. Preparar datos para visualizarlos

En el análisis demográfico y la construcción de pirámides poblacionales tradicionales, las edades individuales suelen consolidarse en grupos o intervalos de 5 años a través de cohortes quinquenales `0-4`, `5-9`, …, `80-84` y un grupo abierto final `85+`.

Para generar estos rangos se utiliza la función `cut()` de R junto con las funciones `seq.int()` y `str_c()`:

- `breaks = c(seq.int(0, 85, 5), "Inf")`: define los puntos de corte numéricos cada 5 años desde 0 hasta 85, incluyendo el infinito para agrupar todas las edades de 85 años en adelante.
- `labels`: asigna las etiquetas correspondientes concatenando las secuencias de inicio y fin (`seq(0, 80, 5)` y `seq(4, 85, 5)`) separadas por un guion mediante `str_c()` y añadiendo `"85+"` al final.
- `right = FALSE`: especifica que los intervalos son cerrados a la izquierda y abiertos a la derecha. Es decir $[0, 5)$, abarcando las edades de 0 a 4 años cumplidos.

Posteriormente, se utiliza `group_by()` junto con `summarize()` para colapsar las filas sumando la población absoluta (`sum(poblacion)`) y la proporción porcentual (`sum(poblacion_pct)`) para cada combinación de año, género y rango de edad, tal como se señala a continuación:

Listado 18.4 Discretización de edades en intervalos quinquenales y agregación demográfica mediante `cut()`, `group_by()` y `summarize()`

```
# Preparar datos ----
poblacion_col_2018_2070_preparar <- poblacion_col_2018_2070_transformar |>
  group_by(ano, genero) |>
  mutate(
    rango_edad = cut(
      edad,
      breaks = c(seq.int(0, 85, 5), "Inf"),
      labels = c(
        str_c(seq(0, 80, 5), seq(4, 85, 5), sep = "-"),
        "85+"
      ),
      right = FALSE
    )
  ) |>
  ungroup() |>
  group_by(ano, genero, rango_edad) |>
  summarize(
    poblacion = sum(poblacion),
    poblacion_pct = sum(poblacion_pct)
  ) |>
  ungroup()

poblacion_col_2018_2070_preparar
```

```
# A tibble: 1,908 x 5
     ano genero  rango_edad poblacion poblacion_pct
   <int> <chr>   <fct>          <dbl>         <dbl>
 1  2018 hombres 0-4          1949253        0.0404
 2  2018 hombres 5-9          2080035        0.0431
 3  2018 hombres 10-14        2192498        0.0454
 4  2018 hombres 15-19        2158466        0.0447
 5  2018 hombres 20-24        2069100        0.0429
 6  2018 hombres 25-29        1902635        0.0394
 7  2018 hombres 30-34        1698592        0.0352
 8  2018 hombres 35-39        1595799        0.0331
```

```
 9  2018 hombres 40-44        1397601        0.0290
10  2018 hombres 45-49        1320949        0.0274
# i 1,898 more rows
```

En el Listado 18.4, los datos quedan consolidados a nivel de cohorte quinquenal para cada año y género. Esta tabla constituye la estructura ordenada y preparada requerida como insumo para generar las pirámides poblacionales en la siguiente sección.

18.4. Caso de estudio: visualizar datos a través de pirámides poblacionales de Colombia

El ordenamiento, separación y transformación de datos no constituyen un fin en sí mismos. Disponer de datos ordenados debe responder a un propósito claro: facilitar la exploración, el análisis y la comunicación efectiva de los patrones en la información. En este caso de estudio, dicho propósito consiste en visualizar la estructura demográfica de Colombia mediante **pirámides poblacionales**. A través de estas representaciones gráficas es posible analizar la dinámica demográfica del país a lo largo del horizonte 2018-2070, extrayendo conclusiones sobre la evolución de las cohortes generacionales, el descenso en la natalidad y el progresivo proceso de envejecimiento poblacional.

18.4.1. Creación de variables auxiliares para selección de años de interés

El conjunto de datos preparado en el Sección 18.3.3 abarca anualmente las proyecciones demográficas desde 2018 hasta 2070, reuniendo un total de 53 periodos. Graficar las pirámides poblacionales correspondientes a los 53 periodos en una sola visualización saturaría la figura e impediría una comparación visual efectiva entre cohortes.

Por esta razón, resulta conveniente seleccionar un subconjunto representativo de años a lo largo del periodo de proyección. En lugar de definir dichos años de forma fija o manual, es una buena práctica calcularlos dinámicamente mediante variables auxiliares a partir del propio conjunto de datos:

1. Con la función `pull()` de `dplyr` se extrae el vector numérico correspondiente a la columna `ano`.
2. Con las funciones `min()` y `max()` se identifican los valores extremos del periodo proyectado: `ano_min` (2018) y `ano_max` (2070).
3. Con la función `seq.int()` se define una secuencia de 4 hitos temporales equidistantes especificando `from = ano_min`, `to = ano_max` y `length.out = 4`, convirtiendo el resultado en números enteros mediante `as.integer()` para obtener los años 2018, 2035, 2052 y 2070.

4. Con la función `filter()` y el operador de pertenencia `%in%` se seleccionan únicamente las observaciones del conjunto de datos preparado cuyos años coincidan con los del vector `filtrar_ano`, asignando el resultado a la tibble `poblacion_col_filtrar`.

Este procedimiento se ejecuta, tal como se señala a continuación:

Listado 18.5 Creación de variables auxiliares para la selección y filtrado de años de interés mediante `pull()`, `seq.int()` y `filter()`

```
# Filtrar ----
## Variables auxiliares ----
ano <- poblacion_col_2018_2070_preparar |>
  pull(ano)

ano_min <- min(ano)
ano_max <- max(ano)

filtrar_ano <- seq.int(
  from = ano_min,
  to = ano_max,
  length.out = 4
) |>
  as.integer()

poblacion_col_filtrar <- poblacion_col_2018_2070_preparar |>
  filter(
    ano %in% filtrar_ano
  )

poblacion_col_filtrar
```

```
# A tibble: 144 x 5
     ano genero  rango_edad poblacion poblacion_pct
   <int> <chr>   <fct>          <dbl>         <dbl>
 1  2018 hombres 0-4          1949253        0.0404
 2  2018 hombres 5-9          2080035        0.0431
 3  2018 hombres 10-14        2192498        0.0454
 4  2018 hombres 15-19        2158466        0.0447
 5  2018 hombres 20-24        2069100        0.0429
 6  2018 hombres 25-29        1902635        0.0394
 7  2018 hombres 30-34        1698592        0.0352
 8  2018 hombres 35-39        1595799        0.0331
 9  2018 hombres 40-44        1397601        0.0290
```

```
10  2018 hombres 45-49        1320949        0.0274
# i 134 more rows
```

En el Listado 18.5, la tabla resultante `poblacion_col_filtrar` conserva únicamente las cohortes quinquenales correspondientes a los 4 años seleccionados, proporcionando el conjunto de datos necesario para construir las pirámides poblacionales en las secciones siguientes.

18.4.2. Creación de pirámides poblacionales

Para construir las pirámides poblacionales utilizaremos el paquete `ggplot2` y los principios de la gramática de gráficos en capas, abordando sucesivamente los componentes de:

1. **Datos, mapeo y capas**: definición de las coordenadas estéticas y la geometría de columnas para representar la bilateralidad demográfica por género.
2. **Facetas**: división del gráfico en paneles individuales para comparar los 4 años seleccionados.
3. **Escalas**: ajuste de los valores absolutos en el eje horizontal, formato porcentual, etiquetas de categorías y textos descriptivos.
4. **Tema**: ajuste de la posición de la leyenda para optimizar el espacio horizontal de visualización.

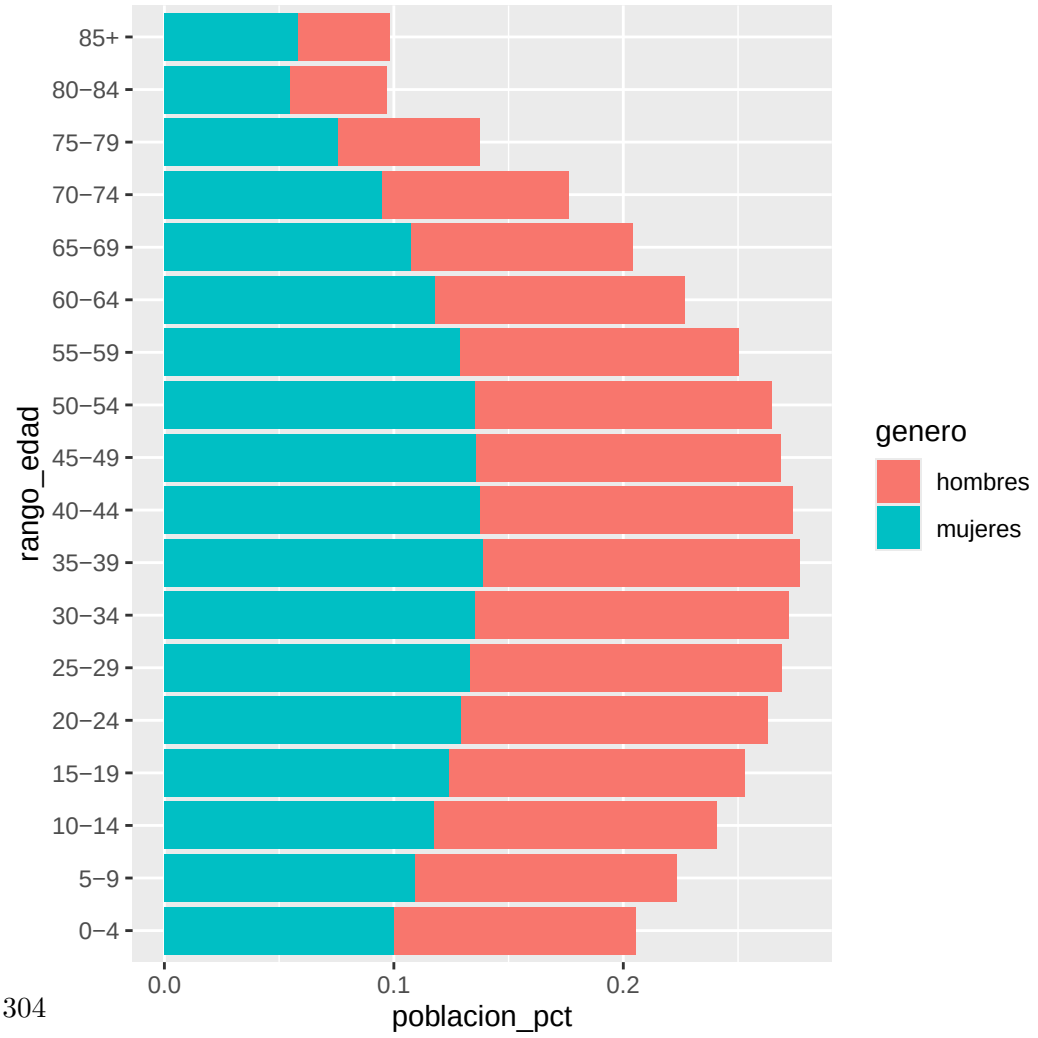
18.4.2.1. Datos, mapeo y capas

Una pirámide poblacional es en esencia una extensión de un gráfico de barras que representa cantidades. En el Capítulo 6 se introdujo la función `geom_bar()` para ilustrar la frecuencia o número de observaciones de una variable categórica mediante el parámetro `stat = "count"`, donde `ggplot2` efectúa internamente el conteo de filas a partir de datos individuales no agregados. En contraste, en `poblacion_col_filtrar` la información ya se encuentra resumida previamente, disponiendo de valores numéricos explícitos tanto para las cohortes de edad (`rango_edad`) como para su proporción demográfica (`poblacion_pct`). Por consiguiente, se utiliza la geometría `geom_col()`, la cual opera con `stat = "identity"` representando directamente los valores numéricos suministrados en la tabla.

Para comprender cómo se origina una pirámide poblacional a partir de `geom_col()`, considera un primer mapeo básico donde se asigna la variable continua `poblacion_pct` al eje horizontal (`x`), la variable categórica `rango_edad` al eje vertical (`y`) y el relleno de las barras según el género con `aes(fill = genero)`, tal como se señala a continuación:

Listado 18.6 Mapeo inicial de cantidades mediante `geom_col()` con barras apiladas en dirección positiva

```
# Cargar paquetes ----
library(scales)

# Visualización ----
poblacion_col_filtrar |>
  ggplot(
    aes(
      x = poblacion_pct,
      y = rango_edad
    )
  ) +
  geom_col(
    aes(fill = genero)
  )
```



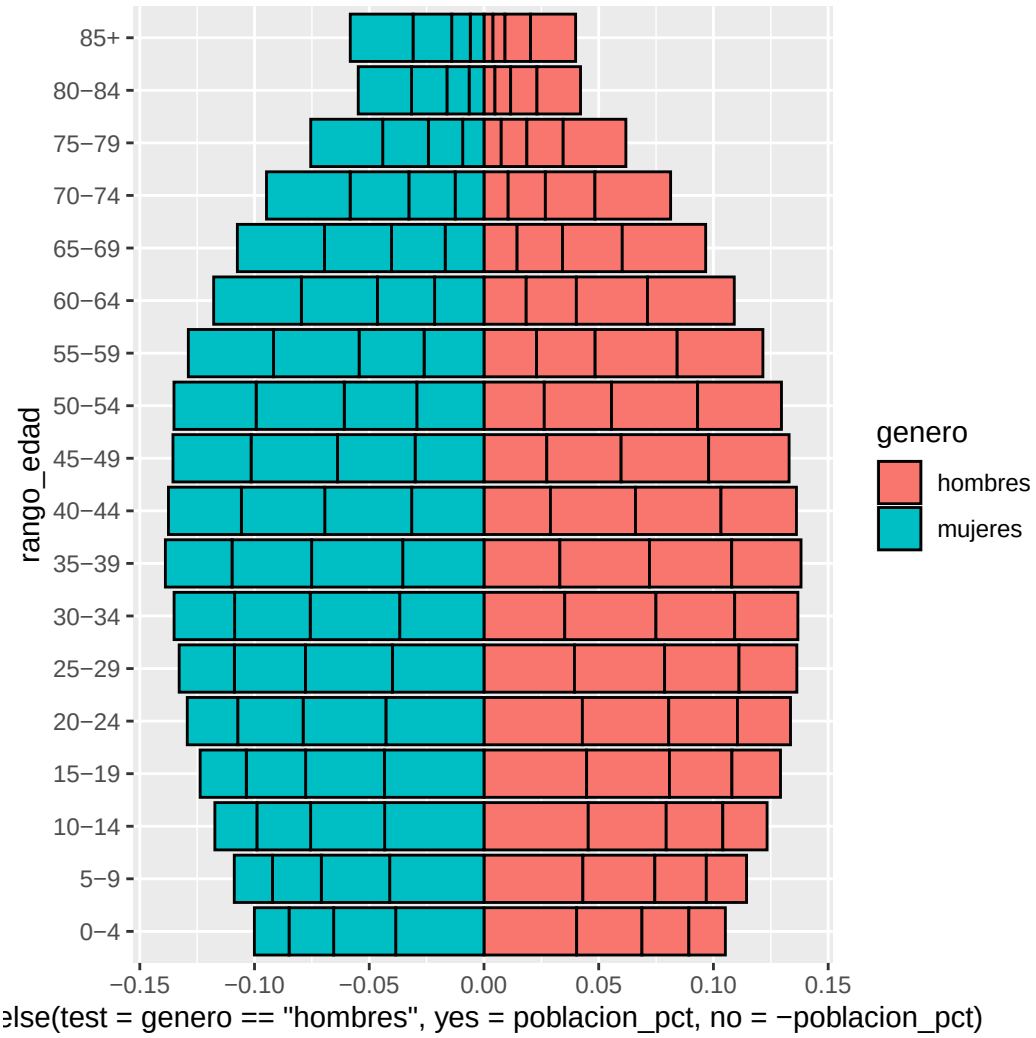
En el Listado 18.6, ambas categorías (`"hombres"` y `"mujeres"`) parten desde el valor 0 y se extienden en la misma dirección positiva sobre el eje horizontal, apilándose una a continuación de la otra debido al ajuste de posición por defecto (`position = "stack"`). Si bien este gráfico permite observar la suma porcentual conjunta de cada cohorte, no refleja la estructura comparativa tradicional de una pirámide demográfica, cuyo rasgo distintivo es la simetría bilateral. Es decir, un género debe situarse a la derecha del eje central y el otro hacia la izquierda.

Para lograr esta disposición bilateral a partir de `geom_col()`, es indispensable que las proporciones de uno de los dos grupos adopten valores negativos, de tal manera que sus barras se extiendan en sentido opuesto hacia la izquierda del origen 0.

Este comportamiento se consigue directamente dentro de `aes()` mediante la función `ifelse()`: al evaluar `test = genero == "hombres"`, se conserva el valor positivo original `poblacion_pct` (`yes`), mientras que para las mujeres se asigna el valor negativo opuesto `-poblacion_pct` (`no`). Asimismo, en `geom_col()` se añade el parámetro `color = "black"` fuera de `aes()` para delinear con precisión el contorno de cada barra, tal como se señala a continuación:

Listado 18.7 Representación base de la pirámide poblacional con simetría bilateral mediante `ifelse()` y `geom_col()`

```
poblacion_col_filtrar |>
  ggplot(
    aes(
      x = ifelse(
        test = genero == "hombres",
        yes = poblacion_pct,
        no = -poblacion_pct
      ),
      y = rango_edad
    )
  ) +
  geom_col(
    aes(fill = genero),
    color = "black"
  )
```



En el gráfico generado por el Listado 18.7 se observa con claridad cómo la función `ifelse()` transforma el gráfico de barras convencional en la característica estructura bilateral de una pirámide poblacional. No obstante, al contener los datos correspondientes a los 4 años seleccionados, las barras de los distintos periodos se superponen entre sí impidiendo distinguir la estructura de cada año.

18.4.2.2. Facetas

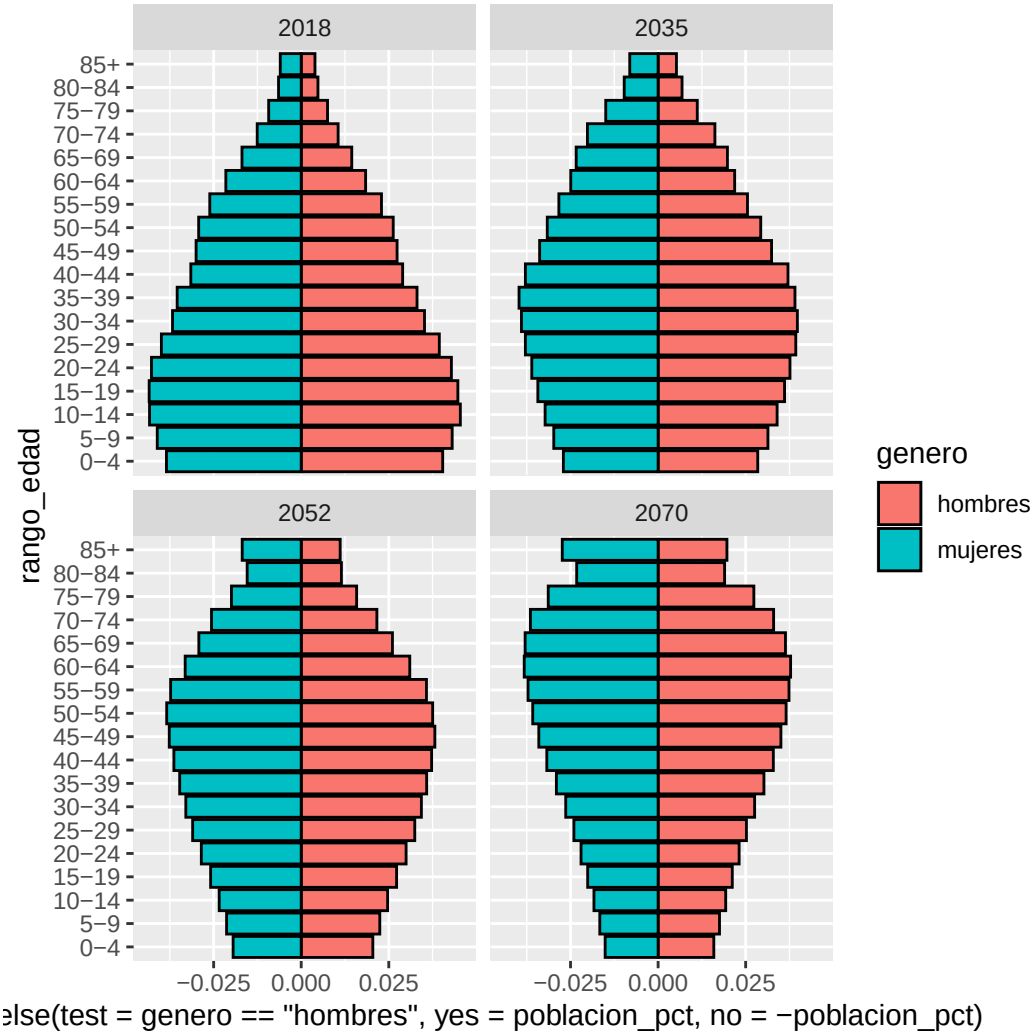
Para separar las barras superpuestas y comparar la evolución de la estructura demográfica entre los periodos seleccionados, se incorpora el componente de facetas utilizando la función `facet_wrap()` (ver Capítulo 9).

Mediante el argumento `facets = vars(ano)`, la función `facet_wrap()` divide el conjunto

de datos y genera un panel individual para cada uno de los años de interés (2018, 2035, 2052 y 2070), tal como se señala a continuación:

Listado 18.8 Incorporación de paneles individuales para cada año seleccionado mediante `facet_wrap()`

```
poblacion_col_filtrar |>
  ggplot(
    aes(
      x = ifelse(
        test = genero == "hombres",
        yes = poblacion_pct,
        no = -poblacion_pct
      ),
      y = rango_edad
    )
  ) +
  geom_col(
    aes(fill = genero),
    color = "black"
  ) +
  facet_wrap(
    facets = vars(ano)
  )
```



Al examinar el gráfico del Listado 18.8, los 4 paneles permiten apreciar con claridad cómo la distribución por edades se transforma a lo largo del tiempo. Sin embargo, se identifican aspectos estéticos y de formato que requieren ser modificados mediante el componente de escalas.

18.4.2.3. Escalas

En el gráfico obtenido en el Listado 18.8 se presentan aspectos a mejorar en relación a las escalas:

1. Debido a la transformación con `-poblacion_pct` para ubicar a las mujeres hacia la izquierda, el eje horizontal presenta valores numéricos negativos (p. ej., `-0.025`), lo cual resulta inadecuado dado que una magnitud poblacional es estrictamente

positiva. Asimismo, los valores se muestran en notación decimal en lugar de formato de porcentaje.
2. Los nombres de las categorías en la leyenda (`"hombres"` y `"mujeres"`) inician en letra minúscula.
3. Se requiere incorporar títulos, subtítulos y notas explicativas que contextualicen adecuadamente la visualización.

Para solucionar el primer aspecto, se define la variable auxiliar `poblacion_pct_breaks` mediante `seq.int()` con marcas que van desde `-0.04` hasta `0.04` en incrementos de `0.01` (1 %). Luego, en la función `scale_x_continuous()` se especifican `breaks = poblacion_pct_breaks` y se formatea el texto del eje mediante el parámetro `labels`.

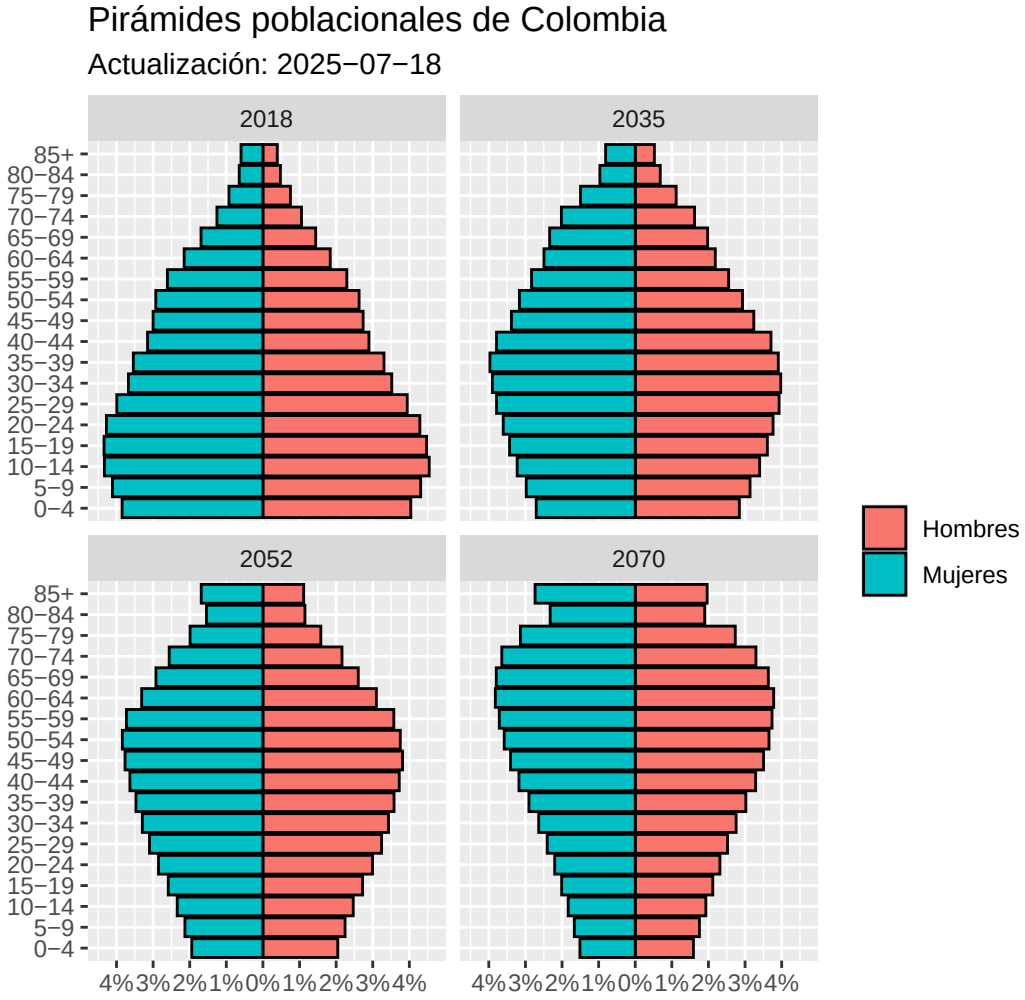
Para mostrar los valores siempre positivos y con símbolo de porcentaje, se define una **función anónima** con la sintaxis `\(x)`:

$$\backslash(x)\ \text{label_percent}(\text{accuracy} = 1)(\text{abs}(x))$$

En el Capítulo 19 se explorará en detalle qué son y cómo operan las funciones anónimas en R. Por el momento, basta señalar que `\(x)` define una función puntual que recibe cada valor `x` del eje, le aplica la función `abs()` para tomar su valor absoluto transformando valores como `-0.04` en `0.04` y finalmente le aplica la función `label_percent()` del paquete `scales` para formatearlo como un porcentaje entero (`4%`).

Para solucionar el segundo aspecto, se emplea la función `scale_fill_discrete()` especificando `labels = str_to_title`. La función `str_to_title()` del paquete `stringr` convierte automáticamente a mayúscula la primera letra de cada categoría, transformando `"hombres"` en `"Hombres"` y `"mujeres"` en `"Mujeres"` sin necesidad de alterar los datos originales.

Por último, se agregan las etiquetas informativas mediante la función `labs()`. Siguiendo el enfoque del Capítulo 15, se define la variable auxiliar `actualizacion_fecha <- ymd("2025-07-18")` con la función `ymd()` del paquete `lubridate` perteneciente al `tidyverse` y se incluye dinámicamente en el subtítulo utilizando la función `str_glue()` con: `subtitle = str_glue("Actualización: {actualizacion_fecha}")`. Además, se suprimen las etiquetas redundantes de los ejes y la leyenda asignando `NULL` a `x`, `y` y `fill`, tal como se señala a continuación:



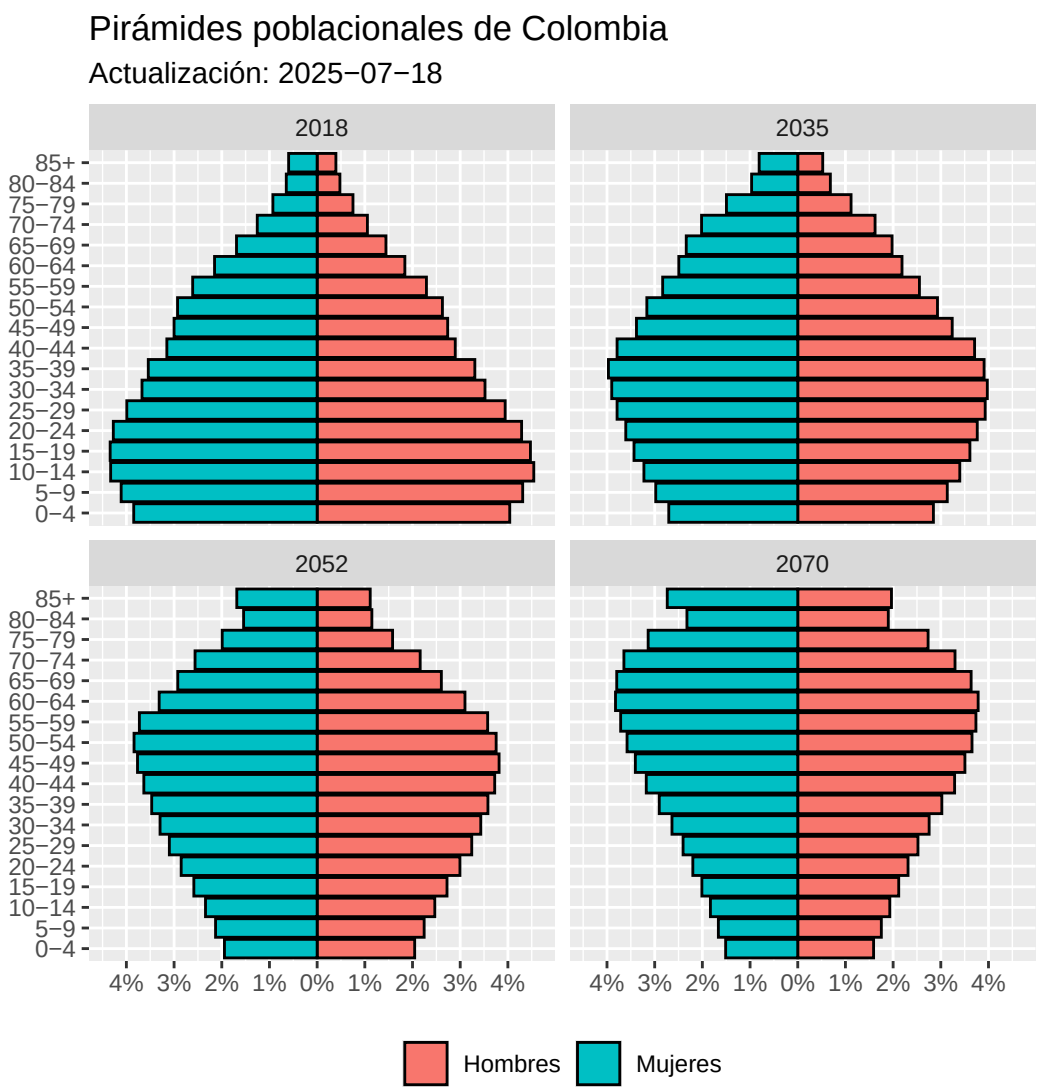
En el Listado 18.9 el gráfico cuenta con escalas legibles, valores en porcentaje positivos en ambos sentidos del eje horizontal, leyenda capitalizada y documentación completa de título, fecha de actualización y fuente oficial.

18.4.2.4. Tema

Por defecto, `ggplot2` sitúa la leyenda en la parte lateral derecha del gráfico. Al utilizar facetas con múltiples paneles distribuidos en columnas, esta posición lateral reduce el ancho disponible para las pirámides, comprimiendo las barras y dificultando la lectura de los intervalos quinquenales.

Para optimizar el uso del espacio horizontal y otorgar mayor amplitud a cada faceta, se utiliza la función `theme()` ajustando el parámetro `legend.position = "bottom"`. De

este modo, la leyenda se reubica en la parte inferior del gráfico para maximizar el área visual asignada a los paneles, tal como se señala a continuación:



Fuente: Dane − Proyecciones de Población y Estudios Demográficos (PPED)

El gráfico obtenido en el Listado 18.10 concluye la construcción de la visualización demográfica. Al comparar los 4 paneles temporales se evidencia con claridad la transición demográfica de Colombia proyectada por el DANE:

- En **2018**, la pirámide presenta una base aún moderadamente ancha en las edades jóvenes y productivas (15 a 29 años), con una cúspide reducida en edades avanzadas.
- Conforme se avanza hacia **2035** y **2052**, la base se contrae progresivamente como consecuencia del descenso en la tasa de natalidad mientras que los grupos de edades intermedias y adultas aumentan su peso relativo.

- Para **2070**, la pirámide adopta una estructura inversa, característica del envejecimiento poblacional, donde las cohortes de adultos mayores de 60 años concentran una proporción considerablemente mayor en comparación con la población infantil y juvenil.

Listado 18.9 Ajuste de escalas continuas, discretas y adición de textos informativos mediante `scale_x_continuous()`, `scale_fill_discrete()` y `labs()`

```
# Visualización ----
## Variables auxiliares ----
poblacion_pct_breaks <- seq.int(
  from = -0.04,
  to = 0.04,
  by = 0.01
)

actualizacion_fecha <- ymd("2025-07-18")

poblacion_col_filtrar |>
  ggplot(
    aes(
      x = ifelse(
        test = genero == "hombres",
        yes = poblacion_pct,
        no = -poblacion_pct
      ),
      y = rango_edad
    )
  ) +
  geom_col(
    aes(fill = genero),
    color = "black"
  ) +
  facet_wrap(
    facets = vars(ano)
  ) +
  scale_x_continuous(
    breaks = poblacion_pct_breaks,
    labels = \(x) label_percent(accuracy = 1)(abs(x))
  ) +
  scale_fill_discrete(
    labels = str_to_title
  ) +
  labs(
    x = NULL,
    y = NULL,
    fill = NULL,
    title = "Pirámides poblacionales de Colombia",
    subtitle = str_glue("Actualización: {actualizacion_fecha}"),
    caption = "Fuente: Dane - Proyecciones de Población y Estudios Demográficos (PPED)"
  )
```

Listado 18.10 Personalización del tema y reubicación de la leyenda mediante `theme()` para la pirámide poblacional final

```
poblacion_col_filtrar |>
  ggplot(
    aes(
      x = ifelse(
        test = genero == "hombres",
        yes = poblacion_pct,
        no = -poblacion_pct
      ),
      y = rango_edad
    )
  ) +
  geom_col(
    aes(fill = genero),
    color = "black"
  ) +
  facet_wrap(
    facets = vars(ano)
  ) +
  scale_x_continuous(
    breaks = poblacion_pct_breaks,
    labels = \(x) label_percent(accuracy = 1)(abs(x))
  ) +
  scale_fill_discrete(
    labels = str_to_title
  ) +
  labs(
    x = NULL,
    y = NULL,
    fill = NULL,
    title = "Pirámides poblacionales de Colombia",
    subtitle = str_glue("Actualización: {actualizacion_fecha}"),
    caption = "Fuente: Dane - Proyecciones de Población y Estudios Demográficos (PP
  ) +
  theme(
    legend.position = "bottom"
  )
```

Parte VI.

Programación funcional

El objetivo de esta parte, es presentar los fundamentos y aplicaciones de la **programación funcional** (en inglés **functional programming**) para automatizar tareas repetitivas y manipular conjuntos de datos en R utilizando el paquete `purrr`, a través de los siguientes 2 capítulos:

- Capítulo 19: introduce los conceptos y requisitos indispensables para comprender la iteración funcional en R, incluyendo la distinción entre vectores atómicos y listas, la definición y anatomía de funciones anónimas y el funcionamiento básico de `map()`, `reduce()` y `list_rbind()`.
- Capítulo 20: introduce la aplicación práctica de `map()`, `reduce()` y `list_rbind()` en combinación con el paquete `fs` para importar múltiples archivos desde un directorio, identificar y seleccionar variables comunes, verificar especificaciones de columnas y consolidar los datos en una sola estructura rectangular.

Una vez se comprenda el funcionamiento básico del paquete `purrr`, el lector interesado en profundizar puede consultar la documentación de referencia de `purrr` y la hoja de referencia de `purrr`, donde se abordan otras familias de funciones tales como:

- **Variantes de mapeo por tipo de salida** (en inglés **type-specific map variants**): funciones como `map_lgl()`, `map_int()`, `map_dbl()`, `map_chr()` y `map_vec()`, diseñadas para devolver vectores atómicos homogéneos en lugar de listas, así como `walk()` para ejecutar funciones orientadas exclusivamente a efectos secundarios sin modificar los datos de entrada.
- **Mapeo con múltiples argumentos e iteración paralela** (en inglés **multi-argument and parallel map**): la familia de funciones `map2()` para iterar sobre 2 vectores de manera simultánea, y `pmap()` (junto con su variante para efectos secundarios `pwalk()`) para aplicar una función a un número arbitrario de argumentos suministrados en una lista o en las columnas de una tabla de datos.
- **Mapeo indexado y condicional** (en inglés **indexed and conditional map**): funciones como `imap()` e `iwalk()` para iterar sobre elementos junto con sus nombres o índices de posición, así como `map_if()` y `map_at()` para aplicar transformaciones únicamente a los elementos que cumplen con un predicado lógico o se encuentran en posiciones determinadas.
- **Funciones de predicado, filtrado e inspección** (en inglés **predicate and filtering functions**): herramientas para examinar colecciones de datos como `some()`, `every()` y `none()` para evaluar si algún, todos o ningún elemento satisface una condición lógica, `keep()` y `discard()` para conservar o descartar elementos con base en un predicado y `detect()` o `detect_index()` para localizar el primer elemento o posición que cumple con un criterio.
- **Reducción acumulativa** (en inglés **accumulate**): la función `accumulate()`, la cual complementa a `reduce()` al conservar de manera secuencial los resultados intermedios de cada etapa de la iteración.

- **Operadores de funciones y control de errores** (en inglés **adverbs and function operators**): modificadores o "adverbios" como `possibly()`, `safely()` y `quietly()`, que envuelven funciones ordinarias para capturar errores, advertencias y mensajes sin interrumpir la ejecución de un flujo de procesamiento, junto con utilidades como `compose()` para encadenar funciones y `negate()` para invertir el resultado lógico de una función de predicado.
- **Manipulación, extracción y transformación de listas** (en inglés **list manipulation**): utilidades para navegar, extraer y reestructurar elementos en listas y jerarquías anidadas como `pluck()`, `chuck()`, `list_flatten()`, `list_c()`, `list_cbind()`, `list_transpose()` y la familia `modify()`.

19. Funciones anónimas y `purrr`

Este capítulo se basa en (Grolemund, 2014, Capítulo 1), (Wickham, 2023, Capítulo 26) y (Wickham, 2019, Capítulo 9) así como en la documentación oficial de `purrr` donde tiene como propósito introducir los requisitos necesarios para entender cómo utilizar el paquete `purrr` para manipular archivos en un directorio respecto a lo que se expondrá en el Capítulo 20.

Exploraremos los siguientes aspectos:

1. ¿Qué es el paquete `purrr`?
2. ¿Cómo se utiliza el paquete `purrr` con vectores atómicos y listas?
3. ¿Cuál es la estructura de las funciones y en particular de las funciones anónimas en R?
4. ¿Cómo funcionan las funciones `map()`, `reduce()` y `list_rbind()` del paquete `purrr`?

> **Nota 43**
>
> Para desarrollar el contenido de este capítulo es necesario que crees una nueva carpeta llamada `019_funciones-anonimas-purrr-en-r` y dentro de ella genera el archivo `001_funciones-anonimas-purrr.R`, siguiendo las pautas de la Sección 4.4.

19.1. `purrr`

El paquete `purrr` es el componente del `tidyverse` diseñado para facilitar la programación funcional en R (Wickham, 2023, Capítulo 26). En el análisis de datos es habitual enfrentarse a tareas repetitivas donde se debe aplicar la misma operación a múltiples elementos independientes, tales como importar decenas de archivos desde una carpeta o transformar varias columnas de manera simultánea.

Tradicionalmente, la repetición de tareas en programación se resuelve mediante bucles imperativos como `for` o `while`. Aunque los bucles son válidos, suelen requerir una considerable cantidad de código repetitivo, como inicializar previamente estructuras vacías para almacenar los resultados, gestionar manualmente variables de índice y controlar los límites de la iteración.

En contraste, la **programación funcional** implementada en `purrr` sustituye los bucles tradicionales por **funcionales** (en inglés **functionals**). Es decir, funciones de orden superior que reciben una función como argumento y la aplican de forma sistemática a cada uno de los elementos de una estructura de datos (Wickham, 2019, Capítulo 9). Esto permite expresar directamente **qué** transformación se desea realizar sobre cada elemento en lugar de especificar **cómo** administrar los pasos del ciclo, dando lugar a un código más conciso, legible, expresivo y fácil de depurar.

19.2. Vectores

Para comprender las herramientas de iteración que ofrece `purrr`, es indispensable profundizar en las estructuras de datos sobre las cuales opera: los **vectores**. En R, los vectores constituyen la estructura de datos más básica e importante y prácticamente cualquier objeto más complejo se fundamenta en ellos (Wickham, 2019, Capítulo 3).

Los vectores en R se dividen en 2 categorías:

1. **Vectores atómicos** (en inglés **atomic vectors**): colecciones unidimensionales y homogéneas donde la totalidad de sus elementos debe pertenecer obligatoriamente al mismo tipo de datos básico.
2. **Listas** (en inglés **lists**): colecciones unidimensionales pero heterogéneas donde cada elemento puede poseer un tipo de datos diferente y albergar objetos de cualquier longitud y complejidad.

En el Sección 10.2.5 se presentó la Tabla 10.2, en la cual se expusieron algunos de los tipos de datos más comunes utilizados en R, diferenciando entre vectores atómicos (`logical`, `integer`, `double`, `character`), listas (`list`) y objetos integrados como factores y fechas (`factor`, `ordered`, `Date`).

19.2.1. Vectores atómicos

Los vectores atómicos son estrictamente homogéneos donde R no permite almacenar elementos de distinta naturaleza en un mismo vector atómico. En R existen 6 tipos de vectores atómicos: `logical`, `integer`, `double` (cuyo sinónimo habitual en R es `numeric`), `character`, `complex` y `raw`.

> **Nota 44**
>
> En la Tabla 10.2 del Sección 10.2.5 únicamente se incluyeron los tipos `logical`, `integer`, `double` y `character`. Los tipos `complex` (como por ejemplo números complejos `1 + 2i`) y `raw` (relacionados con el almacenamiento de bytes binarios puros) no fueron mencionados allí debido a que corresponden a tipos de datos

> especializados, y donde no es necesario utilizar los tipos `complex` y `raw` para comprender la lógica del análisis de datos respecto a los temas expuestos en este libro.

Cuando se combinan elementos para conformar un vector atómico mediante la función básica `c()`, R exige homogeneidad. Si se intentan combinar valores de tipos diferentes, R aplica una regla de coerción implícita transformando automáticamente todos los elementos al tipo más flexible. Por ejemplo, convirtiendo enteros a caracteres.

19.2.2. Listas

A diferencia de los vectores atómicos, las **listas** son vectores heterogéneos y genéricos. Cada elemento de una lista no está restringido a un solo escalar de tipo primitivo. En ese sentido puede ser un vector, una función o una tabla de datos rectangular como una `tibble`.

Esta flexibilidad hace que la introducción de las listas sea estrictamente necesaria al realizar análisis de datos e iteración funcional. En muchas ocasiones se requiere manipular y conservar colecciones de objetos que no pueden incluirse dentro de un vector atómico sin perder su estructura.

Para ilustrar esta necesidad fundamental, supongamos que se dispone de 2 tablas de datos en `tibble`s independientes y se busca agruparlas en un solo contenedor para su posterior procesamiento, tal como se señala en el Listado 19.1:

Listado 19.1 Creación de una lista para almacenar tibbles independientes

```
# Cargar paquetes ----
library(tidyverse)

# Lista de tibbles ----
list(
  tibble(x = "a", y = "b"),
  tibble(y = "c", z = "d")
)
```

```
[[1]]
# A tibble: 1 x 2
  x     y
  <chr> <chr>
1 a     b
```

```
[[2]]
# A tibble: 1 x 2
  y     z
  <chr> <chr>
1 c     d
```

Al utilizar la función `list()`, tal como se observa en el Listado 19.1, R genera una lista con 2 componentes independientes: `[[1]]` y `[[2]]`. Cada uno de estos componentes conserva íntegramente su condición y estructura de `tibble` con sus respectivas dimensiones y columnas rectangulares.

En contraste, si se intentara utilizar la función `c()` para agrupar estas 2 tablas de datos, el resultado no conservará las tablas como entidades independientes, tal como se señala en el Listado 19.2:

Listado 19.2 Pérdida de la estructura tabular al utilizar la función `c()` con tibbles

```
# Intento de combinación con c() ----
c(
  tibble(x = "a", y = "b"),
  tibble(y = "c", z = "d")
)
```

```
$x
[1] "a"

$y
[1] "b"

$y
[1] "c"

$z
[1] "d"
```

Tal como se aprecia en el resultado del Listado 19.2, la función `c()` no crea una colección de 2 `tibbles`. En su lugar, R descompone internamente cada tabla en sus columnas y las concatena en una única lista "aplanada" de 4 vectores columna: `$x`, `$y`, `$y`, `$z`. Con ello se pierde por completo la identidad tabular rectangular de cada conjunto de datos original.

Por esta razón, las listas son el contenedor en R para almacenar tablas de datos múltiples y otros objetos. El paquete `purrr` utiliza tanto vectores atómicos como listas para iterar

(Wickham, 2023, Capítulo 26). En cualquier flujo de trabajo funcional para análisis de datos, se identifica el vector sobre el cual se desea operar, como por ejemplo un vector atómico de caracteres con las rutas de varios archivos o una lista con las tablas importadas, y luego se itera a través de cada uno de sus elementos mediante una función.

19.3. Funciones

Para llevar a cabo una iteración funcional con `purrr`, no basta con disponer de un vector sobre el cual iterar. También se requiere una **función** que determine la acción que se ejecutará sobre cada elemento (Wickham, 2019, Capítulo 9). Las funciones permiten en R encapsular un conjunto de instrucciones reutilizables para automatizar cálculos repetitivos, evitar la duplicación de código y disminuir la probabilidad de cometer errores (Grolemund, 2014, Capítulo 1).

Para comprender cómo se construye una función en R, considera un ejemplo clásico de probabilidad en relación a la simulación del lanzamiento de una moneda donde existen 2 posibles resultados excluyentes, cara o cruz, tal como se ilustra en la Figura 19.1.

Figura 19.1.: Dos caras de una moneda

En R, es posible simular este experimento mediante la función `sample()`, la cual extrae una muestra aleatoria a partir de los elementos provistos en el parámetro `x`, tal como se señala a continuación:

```
[1] "cruz"
```

Listado 19.3 Simulación de un lanzamiento de moneda mediante la función `sample()`

```
# Simulación de lanzamiento ----
moneda <- c("cara", "cruz")

sample(x = moneda, size = 1)
```

En el Listado 19.3, la función `sample()` selecciona aleatoriamente un único elemento (`size = 1`) del vector `moneda`. Si se deseara simular más de 2 lanzamientos con esta moneda de 2 caras, no es posible hacerlo sin permitir la repetición de los elementos. Este aspecto se debe a que por defecto la función `sample()` realiza un muestreo sin reemplazo (`replace = FALSE`). Por lo tanto, para simular múltiples lanzamientos independientes es indispensable habilitar el muestreo con reemplazo mediante `replace = TRUE`, tal como se señala en el Listado 19.4:

Listado 19.4 Simulación de múltiples lanzamientos de una moneda con reemplazo mediante `sample()`

```
# Múltiples lanzamientos con reemplazo ----
sample(
  x = moneda,
  size = 3,
  replace = TRUE
)
```

```
[1] "cruz" "cruz" "cruz"
```

Para aplicar a esta rutina una mayor flexibilidad, es posible parametrizar cada una de las opciones del experimento vinculándolas a nombres de variables independientes como `lados`, `n_lanzamientos`, `con_reemplazo`, antes de utilizar `sample()`, tal como se señala en el Listado 19.5:

```
[1] "cara" "cruz" "cara" "cara"
```

Aunque el código presentado en el Listado 19.5 cumple su propósito, si se requiriera simular lanzamientos en distintas partes de un flujo de trabajo sería necesario copiar y pegar este bloque de código una y otra vez, modificando los valores manualmente. Este procedimiento resulta ineficiente y susceptible a inconsistencias donde la solución consiste en encapsular estas instrucciones dentro de una **función**.

Listado 19.5 Parametrización de las instrucciones para simular el lanzamiento de una moneda

```
# Variables auxiliares ----
lados <- c("cara", "cruz")
n_lanzamientos <- 4
con_reemplazo <- TRUE

# Ejecución del experimento ----
lanzamientos <- sample(
  x = lados,
  size = n_lanzamientos,
  replace = con_reemplazo
)

lanzamientos
```

En la Figura 19.2 se ilustran los componentes conceptuales y la anatomía estructural de una función en R (Grolemund, 2014, Capítulo 1) encapsulando estas instrucciones dentro de una función denominada `lanzar_moneda`.

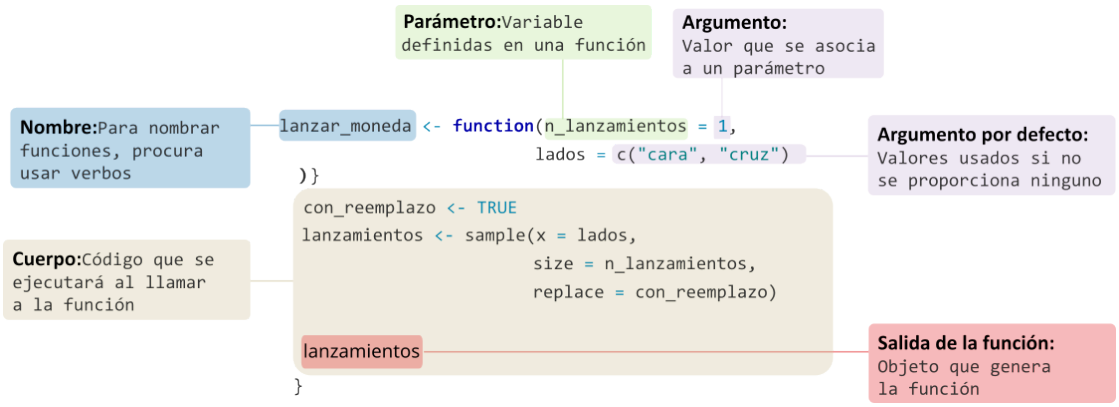


Figura 19.2.: Estructura de una función (Grolemund, 2014, Capítulo 1)

Tal como se detalla en la Figura 19.2, una función en R se compone esencialmente de los siguientes elementos:

1. **Nombre**: el identificador asignado a la función mediante el operador `<-` para almacenarla en el entorno y poder utilizarla posteriormente. Siguiendo las convenciones de estilo de R, los nombres de funciones deben ser verbos concisos que indiquen con claridad la acción realizada como por ejemplo `lanzar_moneda`.
2. **Parámetros**: las variables declaradas dentro del paréntesis de `function(...)`

corresponden a los valores que la función espera recibir para operar.

3. **Argumento**: el valor específico asignado a un parámetro al momento de invocar la función.

4. **Argumento por defecto**: valor predeterminado asignado a un parámetro en la definición de la función como por ejemplo, `= c("cara", "cruz")`. Si al llamar la función no se suministra un argumento para dicho parámetro, R utilizará automáticamente este valor predeterminado.

5. **Cuerpo** (en inglés **body**): el bloque de código encerrado entre llaves `{ ... }` que contiene las instrucciones que se ejecutarán secuencialmente cada vez que se invoque la función.

6. **Salida de la función** (en inglés **return value**): el objeto o resultado que la función entrega a quien la llamó. En R no es obligatorio emplear una sentencia explícita de retorno como `return()` que utilizan otros lenguajes de programación. Por defecto, R retorna automáticamente el resultado de la última expresión evaluada dentro del cuerpo donde en este caso corresponde a `lanzamientos`.

De manera general, la sintaxis para construir una función en R responde al siguiente esquema:

```
nombre <- function(parametros) {
  cuerpo
}
```

A partir de esta estructura, en el Listado 19.6 se encapsulan las instrucciones de simulación dentro de la función `lanzar_moneda()` sin definir parámetros iniciales, tal como se señala a continuación:

Para hacer que `lanzar_moneda()` sea verdaderamente reutilizable y adaptable a diferentes escenarios, en el Listado 19.7 se agregan los parámetros `n_lanzamientos` y `lados` al asignarles argumentos por defecto, tal como se señala a continuación:

Una vez definida la función `lanzar_moneda()` con sus parámetros, es posible invocarla especificando el número de lanzamientos deseado mediante el argumento `n_lanzamientos`, tal como se señala en el Listado 19.8:

```
[1] "cara" "cruz" "cruz" "cruz" "cruz"
```

Listado 19.6 Definición de una función básica sin parámetros en R

```
# Función básica ----
lanzar_moneda <- function() {
  # Variables auxiliares
  lados <- c("cara", "cruz")
  n_lanzamientos <- 4
  con_reemplazo <- TRUE

  lanzamientos <- sample(
    x = lados,
    size = n_lanzamientos,
    replace = con_reemplazo
  )

  # Salida
  lanzamientos
}
```

19.4. Funciones anónimas

En el Listado 19.7 creaste una función formal a la cual se le asignó el nombre `lanzar_moneda`. Nombrar una función es indispensable cuando se trata de una operación compleja o cuando el objetivo es reutilizarla recurrentemente a lo largo de un flujo de trabajo.

Sin embargo, en los flujos de iteración funcional con `purrr` es muy común requerir funciones breves y que se utilizan exclusivamente en un único paso de transformación y que no vuelven a emplearse en el resto del análisis. En tales circunstancias, tener que inventar un nombre para cada función de un solo uso resulta contraproducente ya que sobrecarga el entorno global con variables innecesarias.

Para solucionar este inconveniente se introducen las **funciones anónimas**, que se conocen también como funciones **lambda** en otros lenguajes de programación. Una función anónima es una función que no tiene un nombre asignado (Wickham, 2019, Capítulo 9). A partir de la versión `4.1.0` de R, el lenguaje incorporó una sintaxis nativa y concisa para definir funciones anónimas mediante la barra diagonal inversa `\(...) { ... }`, la cual constituye una forma abreviada y equivalente a escribir `function(...) { ... }`, tal como se señala en el Listado 19.9:

En el Listado 19.9, la expresión define una función anónima que recibe el parámetro `x` y que representa el número de lanzamientos a realizar donde devuelve los resultados

Listado 19.7 Definición de la función `lanzar_moneda()` con parámetros y argumentos por defecto

```
# Función parametrizada ----
lanzar_moneda <- function(
  n_lanzamientos = 1,
  lados = c("cara", "cruz")
) {
  # Variable auxiliar
  con_reemplazo <- TRUE

  lanzamientos <- sample(
    x = lados,
    size = n_lanzamientos,
    replace = con_reemplazo
  )

  # Salida
  lanzamientos
}
```

Listado 19.8 Invocación de la función `lanzar_moneda()` especificando el argumento `n_lanzamientos`

```
# Invocación de la función ----
lanzar_moneda(n_lanzamientos = 5)
```

muestreados con reemplazo. Como no está vinculada a ningún nombre mediante el operador `<-`, esta función existe únicamente en la expresión donde se evalúa.

Si bien es posible ejecutar una función anónima de forma inmediata rodeándola de paréntesis como por ejemplo `(\(x) { sample(c("cara", "cruz"), x, replace = TRUE) })(3)`, su verdadero potencial se despliega al suministrarla como argumento `.f` a las funciones de iteración del paquete `purrr`, tal como se examinará en la Sección 19.5.

19.5. `map()`

La función `map()` es la pieza central del paquete `purrr` (Wickham, 2023, Capítulo 26). Su propósito fundamental es transformar un vector de entrada aplicando una función a cada uno de sus elementos de manera independiente. En la Figura 19.3 se ilustra la representación gráfica del funcionamiento de la función `map()`.

Listado 19.9 Definición de una función anónima en R mediante la sintaxis nativa `(...) { ... }`

```
# Función anónima nativa ----
\(x) {
  sample(
    x = c("cara", "cruz"),
    size = x,
    replace = TRUE
  )
}
```

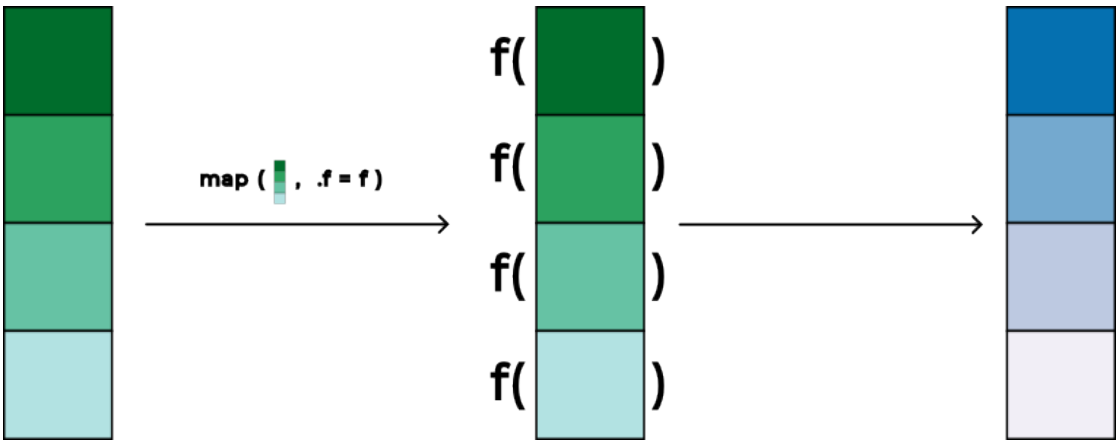


Figura 19.3.: Representación gráfica de la función `map()`

Tal como se aprecia en la Figura 19.3, la función `map()` recibe 2 argumentos principales:

1. `.x`: un vector atómico o lista que contiene los elementos sobre los cuales se iterará.
2. `.f`: la función que se aplicará a cada elemento individual de `.x`.

Durante la ejecución, `map()` recorre ordenadamente cada elemento x_i del vector `.x`, evalúa $f(x_i)$ y almacena el resultado en una posición correspondiente. La propiedad distintiva de `map()` es que **siempre retorna una lista** cuya longitud es exactamente igual a la cantidad de elementos presentes en el vector de entrada `.x`.

Para ilustrar este comportamiento, supongamos que deseamos simular experimentos de lanzamientos de monedas donde la cantidad de lanzamientos varía sucesivamente de 1 a 3, es decir el vector atómico `1:3`. En el Listado 19.10 se utiliza `map()` pasando como argumento `.f` la función nombrada `lanzar_moneda`:

```
[[1]]
```

Listado 19.10 Aplicación de la función `map()` junto con una función nombrada

```
# Iteración con función nombrada ----
map(
  .x = 1:3,
  .f = lanzar_moneda
)
```

```
[1] "cara"

[[2]]
[1] "cruz" "cruz"

[[3]]
[1] "cruz" "cara" "cruz"
```

En el Listado 19.10, `map()` toma el vector `1:3` y pasa sucesivamente el valor `1` a `lanzar_moneda(n_lanzamientos = 1)`, el valor `2` a `lanzar_moneda(n_lanzamientos = 2)` y el valor `3` a `lanzar_moneda(n_lanzamientos = 3)`. El resultado es una lista de 3 elementos señalados con las posiciones `[[1]]`, `[[2]]` y `[[3]]`, donde cada elemento alberga un vector de caracteres con el resultado de sus respectivos lanzamientos.

De forma completamente análoga, es posible alcanzar este mismo resultado sin necesidad de definir previamente la función `lanzar_moneda`, suministrando directamente una función anónima como argumento `.f`, tal como se señala en el Listado 19.11:

Listado 19.11 Aplicación de la función `map()` junto con una función anónima

```
# Iteración con función anónima ----
map(
  .x = 1:3,
  .f = \(x) {
    sample(
      x = c("cara", "cruz"),
      size = x,
      replace = TRUE
    )
  }
)
```

```
[[1]]
```

```
[1] "cruz"

[[2]]
[1] "cruz" "cruz"

[[3]]
[1] "cruz" "cruz" "cruz"
```

Tal como se evidencia al comparar el Listado 19.10 y el Listado 19.11, ambas alternativas generan una lista con idéntica estructura. La ventaja de la función anónima en este contexto radica en que toda la lógica requerida se encuentra autocontenida dentro de la propia invocación de `map()`. Este patrón resulta útil para tareas como la lectura secuencial de múltiples archivos en un directorio, donde el vector de entrada corresponde a las rutas de los archivos y la función anónima ejecuta la importación de cada uno de ellos, tal como se implementará en el Capítulo 20.

19.6. `reduce()`

Mientras que `map()` aplica una transformación independiente a cada elemento y devuelve una lista con la misma cantidad de elementos de entrada, la función `reduce()` del paquete `purrr` tiene como finalidad combinar progresivamente todos los elementos de un vector o lista hasta consolidarlos en un único objeto final (Wickham, 2023, Capítulo 26). En la Figura 19.4 se ilustra la representación gráfica del funcionamiento de la función `reduce()`.

Como se observa en la Figura 19.4, `reduce()` requiere:

1. `.x`: un vector o lista que contiene los elementos a consolidar.
2. `.f`: una **función binaria**, es decir, una función que acepta 2 argumentos y retorna un único valor como por ejemplo operaciones matemáticas de suma o producto o funciones de teoría de conjuntos como `intersect()` o `union()`.

El mecanismo operativo de `reduce()` es acumulativo:

1. Toma el primer elemento (`a`) y el segundo elemento (`b`), y evalúa $f(a, b)$.
2. Toma el resultado intermedio obtenido en el paso anterior y el tercer elemento (`c`), y evalúa $f(f(a, b), c)$.
3. Continúa este proceso de manera recursiva hasta agotar todos los elementos de la colección, devolviendo un único valor consolidado.

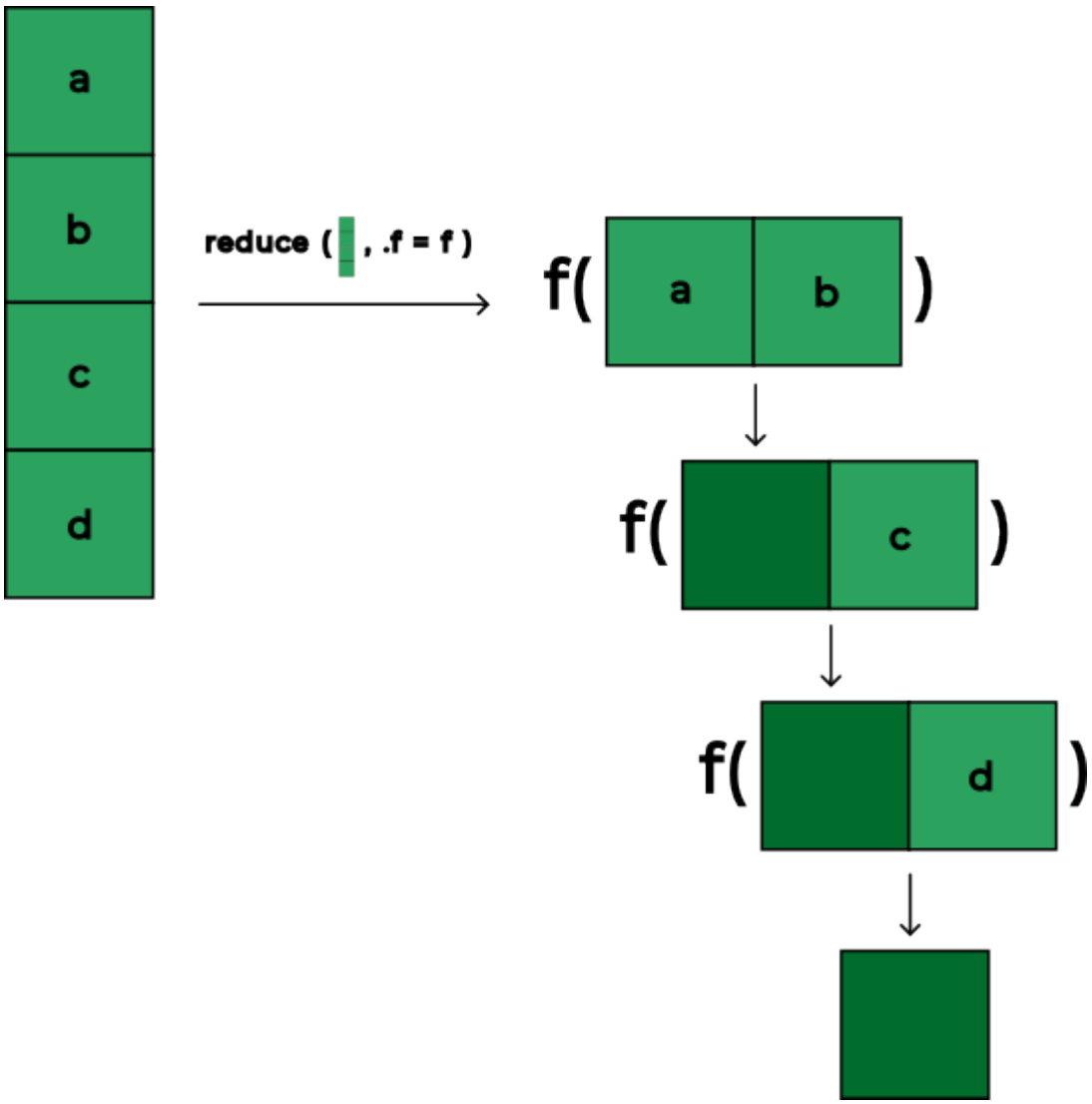


Figura 19.4.: Representación gráfica de la función `reduce()`

Para ilustrar la utilidad práctica de `reduce()`, supongamos que se dispone de una lista con 3 vectores de caracteres denominados `conjuntos` y se desea encontrar cuáles elementos son comunes a los 3 vectores. Es decir, la intersección múltiple entre ellos. Aunque la función base `intersect()` únicamente puede comparar 2 vectores a la vez, combinarla con `reduce()` permite resolver esta operación para cualquier cantidad de conjuntos, tal como se señala en el Listado 19.12:

Listado 19.12 Identificación de elementos comunes en una lista de vectores mediante `reduce()` e `intersect`

```
# Lista de conjuntos ----
conjuntos <- list(
  c("A", "B", "C"),
  c("B", "C", "D"),
  c("C", "D", "E")
)

# Intersección acumulativa ----
reduce(
  .x = conjuntos,
  .f = intersect
)
```

```
[1] "C"
```

En el Listado 19.12, `reduce()` opera secuencialmente de la siguiente manera:

1. Aplica `intersect()` entre el primer y segundo elemento, `intersect(c("A", "B", "C"), c("B", "C", "D"))`, obteniendo como resultado intermedio `c("B", "C")`.
2. Aplica `intersect()` entre dicho resultado intermedio y el tercer elemento, `intersect(c("B", "C"), c("C", "D", "E"))`.

El resultado final devuelto es `"C"`, el único elemento presente simultáneamente en los 3 vectores de la lista. Esta técnica resulta fundamental para el análisis de datos reales. En el Capítulo 20 utilizaremos esta misma lógica combinando `map()` y `reduce()` para identificar con exactitud cuáles columnas se encuentran presentes en la totalidad de los archivos anuales del examen Saber Pro antes de unificarlos.

19.7. `list_rbind()`

Al procesar múltiples archivos mediante `map()`, el resultado obtenido es una lista en la que cada elemento es una tabla de datos independiente dentro de una `tibble`. Sin embargo,

para realizar tareas de exploración, filtrado, modelado o visualización, es necesario reunir todas esas tablas individuales en un único conjunto de datos rectangular consolidado.

Para llevar a cabo esta tarea, el paquete `purrr` proporciona la función `list_rbind()`, la cual toma una lista de tablas de datos y las une o apila verticalmente por filas (Wickham, 2023, Capítulo 26). En la Figura 19.5 se ilustra la representación gráfica del funcionamiento de la función `list_rbind()`.

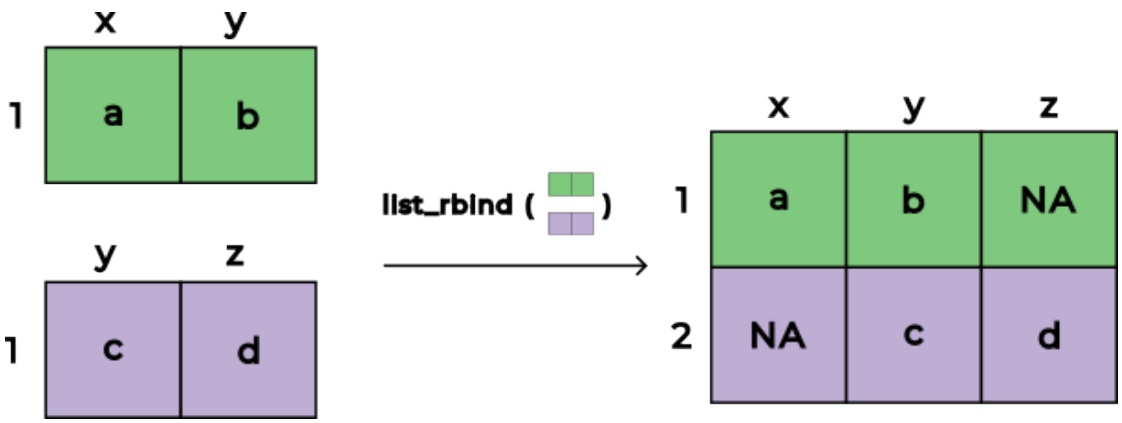


Figura 19.5.: Representación gráfica de la función `list_rbind()`

Tal como se detalla en la Figura 19.5, la ventaja de `list_rbind()` radica en que alinea automáticamente las columnas basándose en sus **nombres** y no en su orden posicional:

- Si una columna está presente en todas las tablas, como la columna `y` en Figura 19.5, sus valores se concatenan verticalmente en una sola columna.
- Si una columna existe únicamente en algunas tablas pero no en otras, como las columnas `x` y `z` en Figura 19.5, `list_rbind()` preserva dichas variables e imputa automáticamente el valor especial de datos faltantes `NA` en las filas de las tablas donde la columna no existía originalmente.

Para comprobar este comportamiento, consideremos una lista denominada `tablas_datos` compuesta por 2 `tibbles` con columnas parcialmente superpuestas, tal como se señala en el Listado 19.13:

```
# A tibble: 2 x 3
  x     y     z
  <chr> <chr> <chr>
1 a     b     <NA>
2 <NA>  c     d
```

En el resultado del Listado 19.13, `list_rbind()` consolida ambas tablas en una única `tibble` de 2 filas y 3 columnas. La primera fila conserva `x = "a"` e `y = "b"` asignando `NA` a `z`, mientras que la segunda fila incorpora `y = "c"` y `z = "d"` asignando `NA` a `x`.

Listado 19.13 Consolidación vertical de una lista de tibbles mediante la función `list_rbind()`

```
# Lista de tablas con columnas parcialmente coincidentes ----
tablas_datos <- list(
  tibble(x = "a", y = "b"),
  tibble(y = "c", z = "d")
)

# Unión por filas ----
list_rbind(tablas_datos)
```

Esta operación completa el ciclo fundamental de importación y consolidación de archivos múltiples que se desarrollará en el Capítulo 20. Primero se empleará `map()` junto con una función de lectura para importar cada archivo en una lista de tablas de datos. Luego se utilizará la función `reduce()` para identificar las variables comunes en las diferentes tablas de datos importadas. Posteriormente se utilizará `list_rbind()` para apilar todas las filas en un único conjunto de datos estructurado para realizar un análisis posterior.

20. Funciones `map()`, `reduce()` y `list_rbind()`

Este capítulo se basa en (Wickham, 2023, Capítulo 26) así como en la documentación oficial de `purrr` donde tiene como propósito introducir la forma de consolidar múltiples archivos que se encuentran en un directorio, utilizando el paquete `purrr` e información de los resultados del examen Saber Pro del grupo de competencias genéricas correspondientes a programas de pregrado de la Universidad Militar Nueva Granada (UMNG).

Exploraremos los siguientes aspectos:

1. ¿Cómo importar múltiples archivos en un directorio utilizando el paquete `purrr`?
2. ¿Cómo consolidar múltiples tablas de datos por filas en un solo archivo utilizando el paquete `purrr`?
3. ¿Cómo utilizar la gramática de manipulación de datos (ver Capítulo 10 y Capítulo 11) para limpiar y exportar un archivo consolidado?

> **Nota 45**
>
> Para desarrollar el contenido de este capítulo es necesario que hayas guardado el archivo `2021-saber_pro-genericas.txt.xz` en la carpeta `data`, tal como se señaló en la Nota 28. En caso que no lo hayas hecho, es necesario que lo descargues y lo incluyas dentro de la carpeta `data`:
>
> **2021-saber_pro-genericas.txt.xz**
>
> Adicionalmente, es necesario que hayas descargado los archivos señalados a continuación y los incluyas dentro de la carpeta `data`, respecto a lo descrito en la Nota 11:
>
> **2022-saber_pro-genericas.txt.xz**
>
> **2023-saber_pro-genericas.txt.xz**
>
> **2024-saber_pro-genericas.txt.xz**

> En caso que hayas seguido estas indicaciones, procede a crear una nueva carpeta llamada `020_funciones-map-reduce-list-rbind-en-r` y dentro de ella genera el archivo `001_funciones-map-reduce-list-rbind.R`, siguiendo las pautas de la Sección 4.4.

20.1. Importación

Cuando se trabaja con datos históricos, es común que la información se encuentre fragmentada en múltiples archivos dentro de un mismo directorio donde cada archivo corresponde a un año o periodo determinado. Para procesar estos archivos sin necesidad de repetir manualmente las instrucciones de importación una y otra vez, es posible combinar herramientas de exploración del sistema de archivos con funciones de iteración del paquete `purrr`.

Para gestionar rutas y archivos en un sistema operativo, utilizarás el paquete `fs`. El paquete `fs` ofrece una interfaz homogénea, consistente e independiente del sistema operativo, que se integra de manera natural con el operador `|>` y las funciones del `tidyverse`.

En particular, la función `dir_ls()` del paquete `fs` lista los contenidos de un directorio aplicando criterios específicos de filtrado (ver Listado 20.1). Sus parámetros principales son:

1. `path`: la ruta del directorio que se desea explorar, como por ejemplo `"data/"`.
2. `type`: el tipo de elemento a buscar. Al especificar por ejemplo `type = "file"` se restringe la búsqueda exclusivamente a archivos, descartando posibles subcarpetas.
3. `glob`: un patrón de texto basado en caracteres comodín (en inglés **wildcards**) para seleccionar los archivos que cumplen con un patrón determinado. Por ejemplo, en el patrón `"*.txt.xz$"`:
    - El asterisco `*` actúa como comodín para representar cualquier secuencia de caracteres de longitud variable como por ejemplo `2021-saber_pro-genericas`.
    - `.txt.xz` señala la extensión compuesta de los archivos comprimidos de texto plano.
    - El símbolo `$` ancla la búsqueda garantizando que únicamente se seleccionen archivos cuya denominación termine con esa extensión exacta.

En el Listado 20.1 se cargan los paquetes necesarios y se identifican las rutas de los 4 archivos del examen Saber Pro, tal como se señala a continuación:

Listado 20.1 Identificación de rutas de archivos en el directorio con `dir_ls()` del paquete `fs`

```
# Cargar paquetes ----
library(tidyverse)
library(fs)

# Rutas ----
paths <- dir_ls(
  path = "data/",
  type = "file",
  glob = "*.txt.xz$"
)

paths
```

```
data/2021-saber_pro-genericas.txt.xz
data/2022-saber_pro-genericas.txt.xz
data/2023-saber_pro-genericas.txt.xz
data/2024-saber_pro-genericas.txt.xz
```

Una vez obtenido el vector atómico de rutas `paths`, se realiza una importación inicial aplicando la función `map()` junto con una función anónima, tal como se estudió en el Capítulo 19. Como se examinó en el Capítulo 13, los microdatos del ICFES utilizan el punto y coma (`;`) como delimitador, por lo que la función adecuada de lectura es `read_csv2()`, tal como se señala en el Listado 20.2:

Listado 20.2 Importación inicial de múltiples archivos mediante `map()` y `read_csv2()`

```
# Importación inicial ----
saber_pro_genericas_2021_2024 <- paths |>
  map(
    .f = \(x) {
      read_csv2(
        file = x
      )
    }
  )
```

Al finalizar la ejecución del Listado 20.2, `saber_pro_genericas_2021_2024` es una lista con 4 **tibbles**, correspondientes a los años 2021, 2022, 2023 y 2024. Sin embargo, si inspeccionas la cantidad de columnas de cada tabla se evidencia que no todas contienen

el mismo número de variables. Por ejemplo, el archivo de 2021 posee 95 variables, el del 2022 tiene 100, el del 2023 contiene 96 y el de 2024 abarca 90.

Estas discrepancias obedecen a actualizaciones o modificaciones en los cuestionarios socioeconómicos aplicados por el ICFES a lo largo del tiempo. Para poder unificar estas tablas en una sola estructura homogénea, es indispensable identificar cuáles columnas están presentes en la totalidad de los archivos anuales.

Aplicando lo aprendido en el Capítulo 19, puedes extraer primero los nombres de las variables de cada tabla con `map(.f = names)` y luego aplicar la función `reduce()` junto con `intersect` para encontrar la intersección de todas las listas de nombres, tal como se señala en el Listado 20.3:

Listado 20.3 Identificación de variables comunes en todos los archivos con `map()` y `reduce()`

```
# Identificación variables completas ----
variables_completas <- saber_pro_genericas_2021_2024 |>
  map(
    .f = names
  ) |>
  reduce(
    .f = intersect
  )
```

Al evaluar el Listado 20.3, se obtiene el vector `variables_completas` que contiene 82 variables comunes a los 4 años. Con este insumo, puedes proceder a realizar una segunda importación seleccionando únicamente las columnas comunes mediante el argumento `col_select = all_of(variables_completas)` de la función `read_csv2()`, tal como se señala en el Listado 20.4:

Listado 20.4 Importación filtrando únicamente las columnas comunes con `col_select`

```
# Importación variables completas ----
saber_pro_genericas_2021_2024 <- paths |>
  map(
    .f = \(x) {
      read_csv2(
        file = x,
        col_select = all_of(variables_completas)
      )
    }
  )
```

En el Listado 20.4 conviene destacar el uso de la función auxiliar `all_of()`, perteneciente al paquete `tidyselect` (integrado en el `tidyverse` y utilizado por `readr` en argumentos como `col_select`). Su empleo cumple dos propósitos fundamentales:

1. **Resolución explícita de variables de entorno:** El argumento `col_select` opera mediante evaluación ordenada (*tidy selection*), lo que permite seleccionar columnas directamente por su nombre o mediante expresiones de selección. Cuando se suministra un vector de caracteres definido en el entorno de trabajo como `variables_completas`, envolverlo en `all_of()` le indica a R de forma inequívoca que debe evaluar el contenido de dicho vector como los nombres de las columnas a seleccionar. Esto evita advertencias y previene ambigüedades entre las variables de los datos (*data-variables*) y las variables del entorno (*env-variables*).

2. **Validación estricta de columnas:** `all_of()` exige que **todas** las columnas especificadas en el vector existan en el archivo que se está leyendo; si falta alguna, la función interrumpe la ejecución y emite un error. Esta rigurosidad garantiza la integridad del proceso de importación frente a su alternativa `any_of()`, que selecciona las columnas coincidentes e ignora silenciosamente las ausentes. Al usar `all_of(variables_completas)`, aseguras que las 82 variables comunes identificadas se carguen efectivamente en la totalidad de los archivos anuales.

Antes de proceder a la unión de las tablas, es necesario revisar también las especificaciones de tipos de datos inferidas por `readr` mediante la función `spec()`, tal como se señala en el Listado 20.5:

Listado 20.5 Inspección de las especificaciones de columnas inferidas con `spec()` y `map()`

```
# Revisión especificación variables ----
saber_pro_genericas_2021_2024 |>
  map(
    .f = spec
  )
```

Al examinar la especificación de las variables con detalle puedes identificar las siguientes necesidades de ajuste:

1. `periodo`: es inferida como una variable numérica continua (`col_double()`), pero corresponde a un código cronológico compuesto, como por ejemplo `"20212"`, que posteriormente debe separarse en año y periodo. Por ende, debe importarse como texto (`col_character()`).
2. Códigos geográficos e institucionales como `estu_cod_depto_presentacion`, `estu_cod_mcpio_presentacion`, `estu_inst_codmunicipio` y `estu_prgm_codmunicipio`: son inferidos como numéricos (`col_double()`), lo cual provoca la pérdida de los

ceros a la izquierda. Por ejemplo, el código "05" de Antioquia se convierte en el número 5, alterando la codificación de la División Político-Administrativa de Colombia (DIVIPOLA) del DANE. En ese sentido deben importarse como texto (col_character()).

3. estu_fechanacimiento: contiene fechas expresadas en formato "día/mes/año" (DD/MM/AAAA). Como se examinó en el Capítulo 13, deben convertirse explícitamente mediante col_date(format = "%d/%m/%Y").
4. estu_snies_prgmacademico e inst_cod_institucion: son identificadores numéricos enteros discretos, por lo que es preferible importarlos como col_integer().

Para ajustar estos tipos de datos de forma compacta y eficiente, la función cols() del paquete readr permite sobreescribir únicamente las 8 columnas de interés, dejando que read_csv2() infiera automáticamente el tipo de las 74 variables restantes. En el Listado 20.6 se realiza la importación definitiva incorporando esta especificación ajustada, tal como se señala a continuación:

Listado 20.6 Importación definitiva con ajuste compacto de tipos de columna mediante cols()

```
# Ajuste especificación variables ----
saber_pro_genericas_2021_2024 <- paths |>
  map(
    .f = \(x) {
      read_csv2(
        file = x,
        col_types = cols(
          periodo = col_character(),
          estu_cod_depto_presentacion = col_character(),
          estu_cod_mcpio_presentacion = col_character(),
          estu_fechanacimiento = col_date(format = "%d/%m/%Y"),
          estu_inst_codmunicipio = col_character(),
          estu_prgm_codmunicipio = col_character(),
          estu_snies_prgmacademico = col_integer(),
          inst_cod_institucion = col_integer()
        ),
        col_select = all_of(variables_completas),
      )
    }
  )
```

Finalmente, se utiliza la función problems() en combinación con map() para verificar que la lectura se haya ejecutado sin advertencias ni inconsistencias de conversión, tal como se señala en el Listado 20.7:

Listado 20.7 Verificación de problemas de importación mediante `problems()` y `map()`

```
# Chequeo problemas ----
saber_pro_genericas_2021_2024 |>
  map(
    .f = problems
  )
```

```
$`data/2021-saber_pro-genericas.txt.xz`
# A tibble: 0 x 5
# i 5 variables: row <int>, col <int>, expected <chr>,
#   actual <chr>, file <chr>

$`data/2022-saber_pro-genericas.txt.xz`
# A tibble: 0 x 5
# i 5 variables: row <int>, col <int>, expected <chr>,
#   actual <chr>, file <chr>

$`data/2023-saber_pro-genericas.txt.xz`
# A tibble: 0 x 5
# i 5 variables: row <int>, col <int>, expected <chr>,
#   actual <chr>, file <chr>

$`data/2024-saber_pro-genericas.txt.xz`
# A tibble: 0 x 5
# i 5 variables: row <int>, col <int>, expected <chr>,
#   actual <chr>, file <chr>
```

Al verificar el resultado del Listado 20.7 puedes confirmar que las 4 tablas retornan **tibbles** vacías de 0 filas. Es decir, cada una de ellas indica que todos los registros se importaron exitosamente con los tipos de datos requeridos.

20.2. Unión por filas

Una vez importados los 4 archivos con idéntica estructura de columnas y tipos de datos correctos, el objeto `saber_pro_genericas_2021_2024` consiste en una lista con 4 tablas independientes. Para poder realizar análisis consolidados sobre el periodo 2021-2024, se deben unir verticalmente todas las filas en un único conjunto de datos rectangular.

Como se expuso en la Sección 19.7 del Capítulo 19, el paquete `purrr` proporciona la función `list_rbind()` para concatenar por filas una lista de tablas de datos. En

el Listado 20.8 se aplica esta función sobre la lista importada, tal como se señala a continuación:

Listado 20.8 Consolidación de la lista de tablas de datos en una sola estructura rectangular con `list_rbind()`

```
# Unión filas ----
saber_pro_genericas_2021_2024 <- saber_pro_genericas_2021_2024 |>
  list_rbind()
```

Al ejecutar `list_rbind()`, la totalidad de observaciones correspondientes a los 4 años quedan consolidadas en una única **tibble** de 82 columnas para que pueda ser manipulada.

20.3. Separación de caracteres en columnas

En la tabla consolidada, la variable `periodo` almacena códigos de 5 dígitos, como por ejemplo `"20212"`, `"20224"`, `"20231"` o `"20241"`, donde los primeros 4 dígitos corresponden al año del examen y el último dígito indica la convocatoria semestral respectiva. Al encontrarse fusionadas estas 2 variables en una sola cadena de caracteres, se dificulta realizar agrupaciones o filtros temporales independientes por año o por periodo.

Aplicando lo estudiado en el Capítulo 18, puedes emplear la función `separate_wider_position()` del paquete `tidyr` para dividir esta variable a partir de anchos fijos de posición con 4 caracteres para el año (`ano`) y 1 carácter para el semestre (`periodo`), tal como se señala en el Listado 20.9:

Listado 20.9 Separación de la columna `periodo` en año y semestre mediante `separate_wider_position()`

```
# Separación periodo ----
saber_pro_genericas_2021_2024 <- saber_pro_genericas_2021_2024 |>
  separate_wider_position(
    cols = periodo,
    width = c(ano = 4, periodo = 1)
  )
```

Al aplicar `separate_wider_position()`, la variable compuesta `periodo` se descompone en 2 columnas independientes: `ano` y `periodo`. De esa manera se facilita posteriormente el análisis de datos.

20.4. Corrección códigos departamentos y municipios

De acuerdo con el estándar de codificación DIVIPOLA establecido por el DANE, los códigos de departamento deben poseer exactamente 2 dígitos, por ejemplo, `"05"` para Antioquia u `"11"` para Bogotá D.C., mientras que los códigos de municipio deben contar exactamente con 5 dígitos, por ejemplo `"05001"` para Medellín.

No obstante, al consultar la longitud en caracteres de las variables geográficas mediante la función `str_length()` del paquete `stringr` y la función `count()` de `dplyr`, se identifica que algunos registros presentan longitudes inferiores a las reglamentarias, tal como se señala en el Listado 20.11:

Listado 20.10 Diagnóstico de longitudes en caracteres para códigos de departamento y municipio

```
# Conteo de caracteres en departamentos ----
saber_pro_genericas_2021_2024 |>
  count(n_depto = str_length(estu_cod_depto_presentacion))
```

```
# A tibble: 3 x 2
  n_depto      n
    <int>  <int>
1       1 114007
2       2 901249
3      NA     39
```

Listado 20.11 Diagnóstico de longitudes en caracteres para códigos de departamento y municipio

```
# Conteo de caracteres en municipios ----
saber_pro_genericas_2021_2024 |>
  count(n_mcpio_pres = str_length(estu_cod_mcpio_presentacion))
```

```
# A tibble: 3 x 2
  n_mcpio_pres      n
         <int>  <int>
1            4 114007
2            5 901249
3           NA     39
```

Al inspeccionar los resultados del Listado 20.11, se observa que existen departamentos con longitud 1 en lugar de 2 y municipios con longitud 4 en lugar de 5. Para confirmar el origen de este comportamiento, en el Listado 20.13 se filtran las primeras filas con longitudes reducidas:

Listado 20.12 Inspección de registros con longitudes reducidas en códigos DIVIPOLA

```
# Inspección departamento con 1 carácter ----
saber_pro_genericas_2021_2024 |>
  filter(str_length(estu_cod_depto_presentacion) == 1) |>
  select(estu_cod_depto_presentacion) |>
  head()
```

```
# A tibble: 6 x 1
  estu_cod_depto_presentacion
  <chr>
1 5
2 8
3 5
4 8
5 5
6 5
```

Listado 20.13 Inspección de registros con longitudes reducidas en códigos DIVIPOLA

```
# Inspección municipio con 4 caracteres ----
saber_pro_genericas_2021_2024 |>
  filter(str_length(estu_cod_mcpio_presentacion) == 4) |>
  select(estu_cod_mcpio_presentacion) |>
  head()
```

```
# A tibble: 6 x 1
  estu_cod_mcpio_presentacion
  <chr>
1 5001
2 8001
3 5001
4 8001
5 5001
6 5001
```

El resultado del Listado 20.13 demuestra que la discrepancia radica en la pérdida del cero a la izquierda en aquellos departamentos con códigos entre `"01"` y `"09"` registrados erróneamente como por ejemplo `5` en vez de `"05"` y municipios asociados registrados como por ejemplo `5001` en vez de `"05001"`.

Para corregir esta anomalía y restablecer los ceros a la izquierda se utiliza la función `str_pad()` del paquete `stringr`. En manipulación de texto y cadenas de caracteres, el término **padding** (en español **relleno**) hace referencia a la acción de añadir caracteres a una cadena hasta alcanzar una longitud o ancho total deseado. Si el texto original ya cuenta con dicha longitud o la supera, `str_pad()` lo deja inalterado sin modificarlo.

La función `str_pad()` se compone de 4 argumentos clave:

1. `string`: el vector o columna de texto que se desea rellenar.
2. `width`: el ancho total mínimo o número de caracteres deseado para la cadena resultante. Por ejemplo, en los códigos de departamento DIVIPOLA se define `width = 2`, mientras que para los códigos municipales se especifica `width = 5`.
3. `side`: el extremo o lado de la cadena donde se insertarán los caracteres de relleno para alcanzar el ancho definido en `width`. Este parámetro contempla 3 opciones:
    - `"left"`: inserta el relleno a la izquierda donde antepone caracteres. Esta opción es la indicada cuando se requiere restituir ceros a la izquierda en variables numéricas o códigos estandarizados, como por ejemplo transformar `"5"` en `"05"` o `"5001"` en `"05001"`.
    - `"right"`: inserta el relleno a la derecha donde añade caracteres al final.
    - `"both"`: distribuye el relleno de manera equilibrada a ambos lados, centrando el texto original.
4. `pad`: el carácter individual con el que se rellenará el espacio faltante. Por defecto, `str_pad()` utiliza un espacio en blanco (`pad = " "`), pero en este contexto se define `pad = "0"` para restituir el formato numérico oficial.

De esta manera, al evaluar `str_pad(string = "5", width = 2, side = "left", pad = "0")` se obtiene `"05"`, mientras que un código como `"11"` permanece intacto porque ya satisface el ancho mínimo de 2 caracteres. Análogamente, para los municipios con `width = 5`, un código incompleto como `"5001"` pasa a ser `"05001"`, mientras que `"11001"` no sufre alteraciones.

Asimismo, para no repetir la misma instrucción en cada una de las columnas de municipio se recurre a la función `across()`, tal como se introdujo en el Capítulo 10. Mediante `across()`, se aplica simultáneamente una función anónima con `str_pad(width = 5, side = "left", pad = "0")` a las 3 variables de municipio (`estu_cod_mcpio_presentacion`, `estu_inst_codmunicipio` y `estu_prgm_codmunicipio`), tal como se señala en el Listado 20.14:

Listado 20.14 Corrección de códigos DIVIPOLA mediante `str_pad()` y `across()`

```
# Corregir códigos departamento y municipio ----
saber_pro_genericas_2021_2024 <- saber_pro_genericas_2021_2024 |>
  mutate(
    estu_cod_depto_presentacion = str_pad(
      string = estu_cod_depto_presentacion,
      width = 2,
      side = "left",
      pad = "0"
    )
  ) |>
  mutate(
    across(
      .cols = c(
        estu_cod_mcpio_presentacion,
        estu_inst_codmunicipio,
        estu_prgm_codmunicipio
      ),
      .fns = \(x) {
        str_pad(
          string = x,
          width = 5,
          side = "left",
          pad = "0"
        )
      }
    )
  )
```

20.5. Caso de estudio: consolidación de resultados Saber Pro para UMNG

Habiendo unificado y normalizado el conjunto de datos para el orden nacional, el propósito de este caso de estudio es construir el subconjunto consolidado correspondiente a los programas de pregrado de la Universidad Militar Nueva Granada (UMNG) aplicando la **gramática de manipulación de datos** (ver Capítulo 10).

20.5.1. Limpiar y filtrar

El primer paso comprende la gestión de los exámenes presentados fuera del país y la delimitación de la población objetivo:

1. **Estudiantes en el exterior**: cuando un estudiante presenta la prueba en el exterior (`estu_exterior == "SI"`), los códigos de departamento y municipio colombianos no resultan aplicables. Mediante la función `replace_when()` en combinación con `across()`, se asigna el valor ausente `NA` a las columnas `estu_cod_depto_presentacion` y `estu_cod_mcpio_presentacion` cuando se cumple dicha condición.

2. **Población agregada oficial**: se aplica `filter(estu_agregado == "S")` para conservar únicamente a los estudiantes evaluados formalmente como parte de la población agregada institucional válida para indicadores de calidad.

3. **Filtro institucional UMNG**: se utiliza `filter(inst_cod_institucion == 1117)` para seleccionar exclusivamente los registros de la Universidad Militar Nueva Granada (UMNG) cuyo código oficial asignado por el SNIES es `1117`.

En el Listado 20.15 se implementan estas operaciones, tal como se señala a continuación:

Listado 20.15 Limpieza de registros y filtrado de la población oficial para la UMNG

```
# Limpiar y filtrar ----
saber_pro_genericas_2021_2024_umng <- saber_pro_genericas_2021_2024 |>
  mutate(
    across(
      .cols = c(
        estu_cod_depto_presentacion,
        estu_cod_mcpio_presentacion
      ),
      .fns = \(x) {
        replace_when(
          x = x,
          estu_exterior == "SI" ~ NA
        )
      }
    )
  ) |>
  filter(estu_agregado == "S") |>
  filter(inst_cod_institucion == 1117)
```

20.5.2. Transformar

El segundo paso consiste en asociar el tipo de datos adecuado a cada variable mediante `mutate()` y la función `factor()`, de forma análoga a lo desarrollado en el Capítulo 10:

- Las variables temporales `ano` y `periodo` se transforman a números enteros utilizando `as.integer()`.
- La variable nominal `estu_genero` se codifica como un factor no ordenado con `ordered = FALSE`.
- Las variables ordinales `estu_semestrecursa`, `fami_estratovivienda`, `fami_educacionmadre`, `fami_educacionpadre` y `mod_ingles_desem` se convierten en factores ordenados con `ordered = TRUE`, especificando de manera jerárquica la secuencia de sus niveles mediante el parámetro `levels`.

En el Listado 20.16 se llevan a cabo estas transformaciones, tal como se señala a continuación:

```
# A tibble: 9,328 x 83
     ano periodo estu_consecutivo estu_estudiante
   <int>   <int> <chr>            <chr>
 1  2021       2 EK202120262469   ESTUDIANTE
 2  2021       2 EK202120101505   ESTUDIANTE
 3  2021       2 EK202120101600   ESTUDIANTE
 4  2021       2 EK202120183856   ESTUDIANTE
 5  2021       2 EK202120163038   ESTUDIANTE
 6  2021       2 EK202120257595   ESTUDIANTE
 7  2021       2 EK202120116785   ESTUDIANTE
 8  2021       2 EK202120140627   ESTUDIANTE
 9  2021       2 EK202120116868   ESTUDIANTE
10  2021       2 EK202120140778   ESTUDIANTE
# i 9,318 more rows
# i 79 more variables: estu_tipodocumento <chr>,
#   estu_nacionalidad <chr>,
#   estu_actividadrefuerzoareas <chr>,
#   estu_actividadrefuerzogeneric <chr>,
#   estu_agregado <chr>, estu_cod_depto_presentacion <chr>,
#   estu_cod_mcpio_presentacion <chr>, ...
```

20.5.3. Exportar

Una vez culminado el procesamiento, la tabla depurada y estructurada contiene los registros de los estudiantes de la Universidad Militar Nueva Granada evaluados entre 2021 y 2024. Para guardar esta información y poder utilizarla en análisis posteriores,

sin necesidad de repetir todo el flujo de importación, el lector puede exportar el objeto resultante en extensión `.rds` mediante la función `write_rds()` del paquete `readr`, tal como se señala a continuación:

Listado 20.16 Transformación de tipos de columna a enteros y factores ordenados para la UMNG

```
# Transformar ----
saber_pro_genericas_2021_2024_umng <- saber_pro_genericas_2021_2024_umng |>
  mutate(
    ano = as.integer(ano),
    periodo = as.integer(periodo)
  ) |>
  mutate(
    estu_genero = factor(
      x = estu_genero,
      ordered = FALSE
    ),
    estu_semestrecursa = factor(
      x = estu_semestrecursa,
      levels = c("02", "03", "06", "07", "08", "09", "10", "11", "12 o más"),
      ordered = TRUE
    ),
    fami_estratovivienda = factor(
      x = fami_estratovivienda,
      levels = c(
        "Sin Estrato",
        "Estrato 1",
        "Estrato 2",
        "Estrato 3",
        "Estrato 4",
        "Estrato 5",
        "Estrato 6"
      ),
      ordered = TRUE
    ),
    fami_educacionmadre = factor(
      x = fami_educacionmadre,
      levels = c(
        "No sabe",
        "No Aplica",
        "Ninguno",
        "Primaria incompleta",
        "Primaria completa",
        "Secundaria (Bachillerato) incompleta",
        "Secundaria (Bachillerato) completa",
        "Técnica o tecnológica incompleta",
        "Técnica o tecnológica completa",
        "Educación profesional incompleta",
        "Educación profesional completa",
        "Postgrado"
      ),
      ordered = TRUE
    ),
    fami_educacionpadre = factor(
      x = fami_educacionpadre,
      levels = c(
        "No sabe",
        "No Aplica",
        "Ninguno",
```

Listado 20.17 Exportación del conjunto consolidado depurado para la UMNG en extensión `.rds`

```
# Exportar datos consolidados ----
saber_pro_genericas_2021_2024_umng |>
  write_rds(
    file = "data/2021-2024_saber-pro-genericas_umng.rds"
  )
```

Parte VII.

Comunicación con Quarto

Pendiente

Referencias

Anderson, A., & Benedetti, R. (2009). How to Use this Book: Intro. En *Head First Networking: A Brain-Friendly Guide* (1.ª ed., pp. xxv-xxxiii). O'Reilly Media.

Bryan, J. (2017). *Project-oriented workflow*. https://www.tidyverse.org/blog/2017/12/workflow-vs-script/

Bryan, J., & Hester, J. (2025). *Happy Git and GitHub for the useR* (1.ª ed.). https://happygitwithr.com/

Bryan, J., Hester, J., Pileggi, S., & Aja, E. D. (2025). *What They Forgot to Teach You About R*. https://rstats.wtf/

Buuren, S. van. (2018). *Flexible imputation of missing data* (Second edition). CRC Press, Taylor & Francis Group. https://stefvanbuuren.name/fimd/

Conway, D. (2010). *The data science venn diagram*. http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram

Faraway, J. J. (2025). *Linear Models with R* (3.ª ed.). Chapman; Hall/CRC. https://doi.org/10.1201/9781003449973

Grolemund, G. (2014). *Hands-on programming with R* (First edition). O'Reilly. https://jjallaire.github.io/hopr/

H2O.ai. (2015). *Intro to Data Science with H2O - Erin LeDell*. https://www.youtube.com/watch?v=Yz5x-YU7pik

Halmos, P. R. (1973). Paul R. Halmos. En *How to Write Mathematics* (1.ª ed., pp. 19-48). American Mathematical Society.

IA-Uniandes, E. proyecto. (2024). *Lineamientos para el uso de inteligencia artificial generativa (IAG) en la Universidad de los Andes*. https://secretariageneral.uniandes.edu.co/images/documents/lineamientos-uso-inteligencia-artificial-generativa-IAG-uniandes.pdf

Ismay, C. (2025). *Statistical inference via data science: a moderndive into r and the tidyverse* (Second edition). CRC Press. https://moderndive.com/v2/index.html

Jachym, C. (2017). *Shapefile must die!* http://switchfromshapefile.org/

Karlton, D. P. (2017). *Naming things is hard | dk*. https://www.karlton.org/2017/12/naming-things-hard/

Lovelace, R., Nowosad, J., & Münchow, J. (2025). *Geocomputation with R* (2.ª ed.). CRC Press, Taylor & Francis Group. https://r.geocompx.org/

Popovic, M. (2025). *Mapping Worlds with R* (1.ª ed.).

Wickham, H. (2010). A Layered Grammar of Graphics. *Journal of Computational and Graphical Statistics*, *19*(1), 3-28. https://doi.org/10.1198/jcgs.2009.07098

Wickham, H. (2014). Tidy Data. *Journal of Statistical Software*, *59*(10). https://doi.org/10.18637/jss.v059.i10

Wickham, H. (2016). *ggplot2: elegant graphics for data analysis* (Second edition). Springer international publishing. https://link-springer-com.ezproxy.umng.edu.co/book/10.1007/978-3-319-24277-4

Wickham, H. (2019). *Advanced R* (Second edition). CRC Press/Taylor & Francis Group. https://adv-r.hadley.nz/

Wickham, H. (2023). *R for Data Science* (2.ª ed.). O'Reilly Media, Incorporated. https://r4ds.hadley.nz/

Wickham, H., & Bryan, J. (2023). *R packages: organize, test, document, and share your code* (Second edition). O'Reilly. https://r-pkgs.org/

Wilke, C. (2019). *Fundamentals of data visualization: a primer on making informative and compelling figures* (First edition). O'Reilly. https://clauswilke.com/dataviz/

Wilkinson, L., & Wills, G. (2005). *The grammar of graphics* (2nd ed). Springer.